Explanatory Note to the Detailed Design (TRP)

Teams Call Records Integration (TCRI) — Unified Engineering Specification

Revision: TRP-1.5.1 for Owner Approval · 21 October 2026. Platform: .NET 8 / C# 12 / EF Core 8 / SQLite. Repository: teams-call-records-integration. Normative basis: approved Technical Requirements Specification (TRS) v1.0 and Preliminary Design (PD) v1.1.

Status: DRAFT FOR OWNER APPROVAL; NOT APPROVED BY THE OWNER. Code presented in this document is not deemed implemented; automated test results are expected outcomes, not actual results. The Digital Polygraph production environment is not modified.

Provenance and Precedence Rules

1. Sections 3.1–3.8 are based on Part 1 of the independently reviewed detailed-design draft and incorporate agreed amendments to the file tree and SQL queue. Golden samples G01–G11 are retained with all JSON documents.
2. Sections 3.9–3.21 have been checked against the authentic Infrastructure v0.2 Markdown (e9425fc7…), adopted as a normative source. Later integration changes are marked [P-26]/[P-27] and listed in Appendix I. RV-01 concerning source availability has been closed; exact byte-for-byte equivalence of the sections is not claimed because registered changes were made.
3. Sections 3.22 onward, including Host, security, deployment, diagrams, and end-to-end tests, were developed independently as proposed design decisions. Until approved, they do not have the status of decisions made by the owner.
4. If sections of this document conflict, the order of precedence is: approved TRS → approved PD → the more detailed normative contract of this TRP. Any discrepancies identified before implementation are entered in the design decision register (appendix).

Completeness and Provenance Control (TRP-1.5.1)

Authentic baseline sources: approved TRS v1.0 and PD v1.1; Part 1 of TRP v0.1 after review; authentic Part 2 Infrastructure v0.2, SHA-256 e9425fc7aae50113be4ecdf46eaf8c8393b5a734f328e9e925739d14d33cb8d3, received in full and compared section by section with v1.4. Verbatim equivalence of passages changed after v0.2 is not asserted; the complete list appears in Appendix I.

Integration design proposals: the inbox index conflict model; clarification of NeedsFetch(force); logging of rejected hashes; handling malformed Graph JSON; revised S14/S15 golden samples; Host configuration; and the test plan. These decisions are submitted for approval; they do not supersede the approved TRS or PD and have not been verified by running C# code.

Register of blocking checks before approval (taking the TRP-1.1 review into account): RV-01 — documentary comparison with authentic v0.2 completed; RV-02 — run restore/build/test for the target .NET 8 solution; RV-03 — perform tenant-contract checks for Microsoft Graph and the Teams app manifest; RV-04 — check transactions and concurrent retries against actual SQLite; RV-05 — settle outstanding owner decisions (P-13, P-14, P-20, P-22, hosting site, tenant). Until these checks have been completed, the document remains a candidate for approval only.

Review Finding Closure Control for TRP-1.2 (TRP-1.5 Revision)

Independent Review Finding	TRP-1.5 Change	Acceptance Check
Figures 11–24 Without Arrows	New exporter reads actual Mermaid ->> / -->> and renders messages and branches	Check ≥1 arrow and messages in every PNG/PDF page
Figures 4–10 Missing from MD / Old Versions in PDF	Seven updated Mermaid diagrams added to the relevant §§3.12–3.17	24 graphic blocks in MD; 24 images in DOCX, all re-rendered
Missing .dockerignore	Full normative file in §3.34; paths included in register	Exclude .env, SQLite, and bin/obj from Docker context
GetStatsAsync Inconsistency	tenantId in interface, Host, diagram, and parameterized SQLite query	SQLite COUNT(DISTINCT call_id) with tenant_id filter
Obsolete D04 in §3.28	One original G01 v3, only session end is missing	S05(b), §3.33, and D04 JSON are consistent
Undefined Bot/Teams SDK	Microsoft.Teams.Apps 2.1.1 (net8.0) and AzureAd schema	restore/build + JWT tests are mandatory and not claimed executed
Webhook mixed batch / unknown subscription	Per-item behaviour defined, 503 for temporarily unknown subscription	K12 plus regression tests
Graph Reprocess Inside HTTP	Durable 202 job, background execution, ≤7-day period, and lease renewal	/api/reprocess regression tests
Incomplete CI Script List	Normative contents/verification rules in §3.34.9a	Artifact scripts checked in CI
Infrastructure v0.2 Source Comparison	RV-01 closed by SHA and section-by-section comparison	Registered deviations in Appendix I

Evidence boundary: this document is a specification. Schema checks, the test register, and visual inspection of the PDF/DOCX build do not establish that .NET builds or Graph/Teams tests have run. [P-25] (DR-16) reproduces the original v0.2 rule and does not require a new decision; proposals [P-26]–[P-28] remain subject to approval. No code has been implemented in production.

Contents and Scope

- 3.1–3.8: Core F6, data schema, G01–G11, Core tests.
- 3.9–3.21: Infrastructure, Graph, inbox/outbox, source state, recovery, tests.
- 3.22–3.31: consolidated file register, Host and routes, authentication, bot, lifecycle, states, sequences.
- 3.32–3.37: operations, security, interfaces, proposed CI, K1–K15 test protocol, decision register.
- 4–5: technical and economic indicators and primary sources.

Readiness rule: after approval, implementation must not change public contracts or the schema without a registered TRP change request; external Microsoft Graph restrictions requiring a live tenant are checked using previously defined procedures.

1. Introduction

TCRI is a standalone Microsoft Teams/Graph application that receives notifications, reads callRecord data, produces one CDR per session, stores versions, and supports REST/bot queries and notifications. It was developed independently of DP (Digital Polygraph). This explanatory note follows the five principal sections of GOST 19.404-79; additional design sections appear in Section 3 and the appendices.

2. Purpose and Scope

A unified specification for building and testing a software product using Microsoft Entra ID app-only authentication and the minimum necessary Microsoft Graph/RSC permissions. The product does not sell call data, does not record call media, and does not use proprietary Nokia/ZTE/Ericsson CDRs.

3. Technical Characteristics

3.0. Approved Architecture and Processing-Flow Diagrams

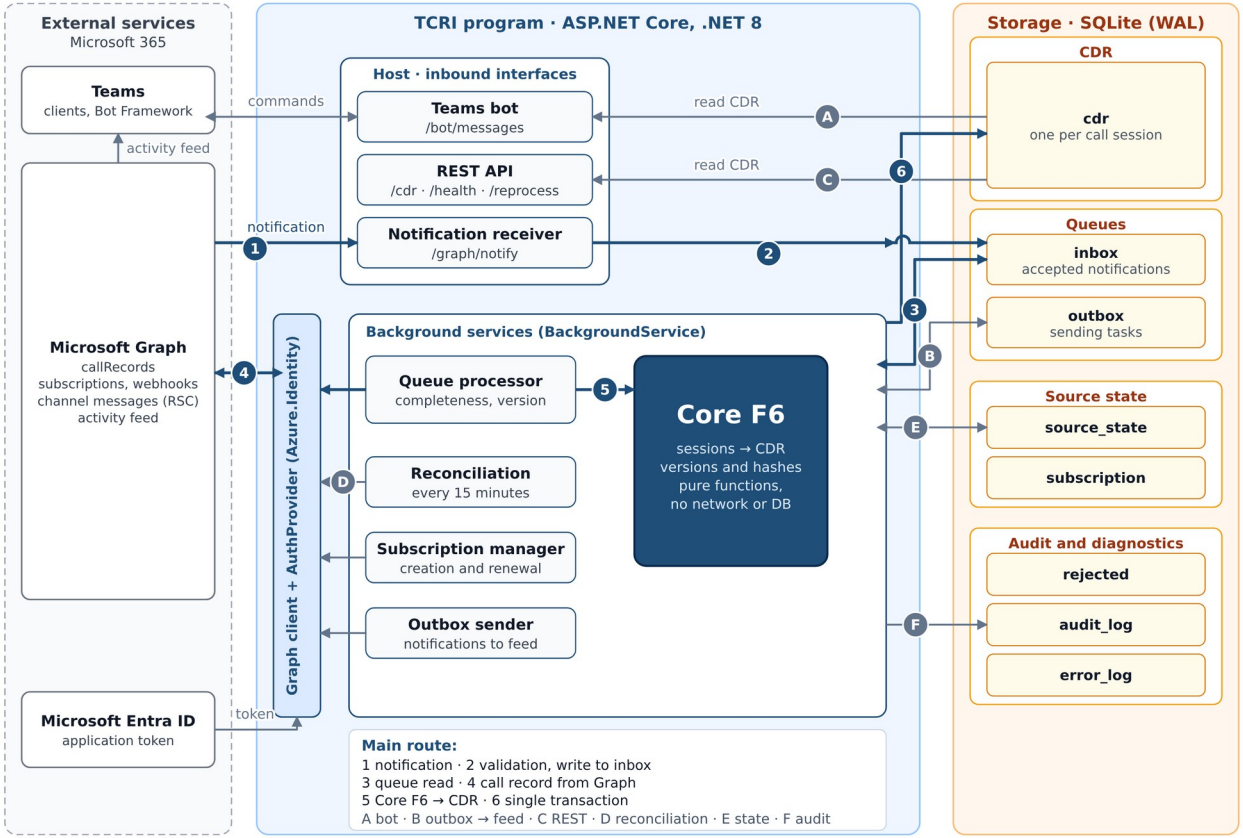
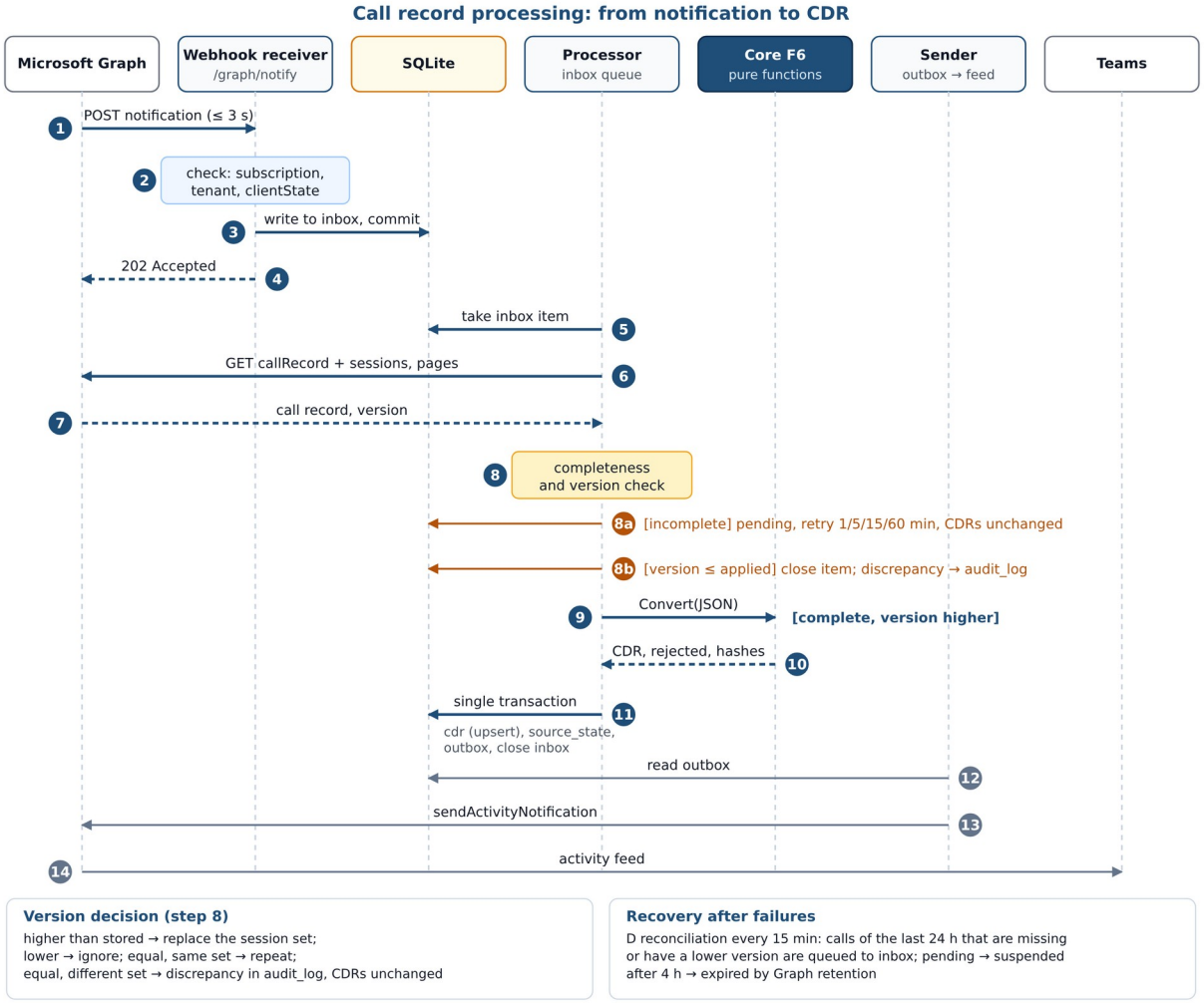


Figure 1. TCRI structural architecture. Original diagram from approved Preliminary Design v1.1: Microsoft 365/Graph/Entra ID → Webhook/Graph Client/Core F6/Bot/REST → SQLite (cdr, source_state, inbox, outbox, audit/diagnostics). The Word/PDF document embeds the image from the approved explanatory note; the same components are represented below in machine-readable form.

Figure 2. Converting a Teams notification into a CDR and a notification. Adopted principle: completeness and version checks precede any CDR update; pending does not delete previously accepted CDRs. Updated Infrastructure clarifications are reflected in the sequences in §§3.12–3.31.



3.1. Conventions and Entity Register

Naming conventions [P]:

Scope	Rule	Example
Root Namespace	TeamsCallRecords + project name	TeamsCallRecords.Core

Scope	Rule	Example
C# File	File name matches the primary public type	CallRecordConverter.cs
Exception	Models.cs contains several small model types	Cdr, PartyRef, Rejection
Interfaces	I prefix	IGraphClient
SQLite Tables and Columns	snake_case; names fixed in 3.5	source_state
Database Row Properties	Column name converted to PascalCase	SourceVersion ↔ source_version
Contract JSON	snake_case	duration_s
Setting Keys	PascalCase, : separator	Inbox:MaxPending
Test Files	<Type>Tests.cs; methods Condition_Expectation	Duration_RoundsHalfUp
Time in Database Text and JSON	ISO 8601 UTC, 7 fractional digits, Z suffix	2026-10-20T09:00:00.5000000Z

Project register.

Code	Project	Namespace	Part
PRJ-CORE	src/TeamsCallRecords.Core	TeamsCallRecords.Core	1
PRJ-INF	src/TeamsCallRecords.Infrastructure	TeamsCallRecords.Infrastructure	2
PRJ-HOST	src/TeamsCallRecords.Host	TeamsCallRecords.Host	3
PRJ-CORE-T	tests/TeamsCallRecords.Core.Tests	TeamsCallRecords.Core.Tests	1
PRJ-INF-T	tests/TeamsCallRecords.Infrastructure.Tests	TeamsCallRecords.Infrastructure.Tests	2
PRJ-HOST-T	tests/TeamsCallRecords.Host.Tests	TeamsCallRecords.Host.Tests	3

SQLite table register (details in 3.5): cdr, source_state, inbox, rejected, audit_log, outbox, subscription, error_log.

Rejection code register (RejectionCodes; rejected.code table):

Constant	Value	Scope
BadJson	CALLRECORD_BAD_JSON	record
NoCallId	CALLRECORD_NO_ID	record
NoVersion	CALLRECORD_NO_VERSION	record
SessionNoId	SESSION_NO_ID	session
SessionBadDate	SESSION_BAD_DATE	session

Constant	Value	Scope
SessionEndBeforeStart	SESSION_END_BEFORE_START	session
SessionDuplicateId	SESSION_DUPLICATE_ID	session, new code [P]

Incomplete-reason register (IncompleteReasons; ConversionResult.IncompleteReasons):

Constant	Value	Class
NoSessions	NO_SESSIONS	transient
SessionDatesMissing	SESSION_DATES_MISSING	transient
SessionInvalid	SESSION_INVALID	permanent for this version

For Bad results (record-level errors), IncompleteReasons receives the rejection code from the register above; the classification is permanent.

3.2. Repository Tree

Complete repository inventory. Column “Part” identifies the part of the detailed design that fully describes the file. The file contents of Parts 2–4 are not repeated here, but their names and purposes are fixed here and must not be changed.

Path	Purpose	Part
TeamsCallRecords.sln	decision	1
global.json	pins SDK 8.0.x, rollForward: latestFeature	4
Directory.Build.props	common properties: TargetFramework=net8.0, Nullable, ImplicitUsings, TreatWarningsAsErrors=true, LangVersion=12	4
Directory.Packages.props	central package versions	4
.editorconfig	formatting rules	4
.gitignore	exceptions	4
LICENSE	MIT license [P-06]	4
README.md	deployment and test scenario	4
HANDOFF.md	handover between implementers	4
.dockerignore	exclude secrets and temporary files from Docker build context	4
Dockerfile	application image	4
docker-compose.yml	launch the application and Caddy	4

Path	Purpose	Part
Caddyfile	reverse proxy, TLS	4
.env.example	example environment variables with placeholders	4
.github/workflows/ci.yml	build and test on .NET 8	4
teams-app/manifest.json	Teams application manifest with RSC	4
teams-app/color.png	color icon	4
teams-app/outline.png	outline icon	4
docs/explanatory-note-ru.docx	explanatory note (Preliminary Design v1.1, RU)	4
docs/explanatory-note-en.docx	the same, EN	4
docs/figures/fig1.svg	diagram 1 source	4
docs/figures/fig2.svg	diagram 2 source	4
docs/figures/fig3.svg	diagram 3 source (CallRecordConverter.Convert sequence)	4
docs/test-protocol.md	test protocol	4
docs/permissions.md	Graph and RSC permissions register	4
src/TeamsCallRecords.Core/TeamsCallRecords.Core.csproj	project with no external packages	1
src/TeamsCallRecords.Core/Models.cs	CallType, CdrStatus, AggregationStatus, PartyRef, Cdr, Rejection, RejectionCodes, IncompleteReasons, ConversionResult	1
src/TeamsCallRecords.Core/Canonical.cs	Canonical: canonical JSON and hashes	1
src/TeamsCallRecords.Core/VersionRules.cs	VersionAction, VersionRules: version decision, rounding	1
src/TeamsCallRecords.Core/CallRecordConverter.cs	CallRecordConverter.Convert	1
src/TeamsCallRecords.Core/CdrJson.cs	CdrJson: JSON representation of CDR and result	1
src/TeamsCallRecords.Infrastructure/TeamsCallRecords.Infrastructure.csproj	project	2
src/TeamsCallRecords.Infrastructure/InfrastructureRegistration.cs	Infrastructure DI registration	2
src/TeamsCallRecords.Infrastructure/Data/TcriDbContext.cs	TcriDbContext	2
src/TeamsCallRecords.Infrastructure/Data/IWriteGate.cs	write-gate contract	2

Path	Purpose	Part
src/TeamsCallRecords.Infrastructure/Data/WriteGate.cs	WriteGate: serialized writes	2
src/TeamsCallRecords.Infrastructure/Data/SqlitePragmaInterceptor.cs	SqlitePragmaInterceptor: PRAGMA upon connection open	2
src/TeamsCallRecords.Infrastructure/Data/UtcText.cs	UtcText: database timestamp format	1
src/TeamsCallRecords.Infrastructure/Data/Cursor.cs	REST cursor encoding and decoding	2
src/TeamsCallRecords.Infrastructure/Data/CdrMapping.cs	CdrMapping: Cdr ↔ CdrRow	1
src/TeamsCallRecords.Infrastructure/Data/Entities/States.cs	state constants	2
src/TeamsCallRecords.Infrastructure/Data/Entities/CdrRow.cs	cdr row	1
src/TeamsCallRecords.Infrastructure/Data/Entities/SourceStateRow.cs	rows for other tables	1
src/TeamsCallRecords.Infrastructure/Data/Entities/InboxRow.cs	rows for other tables	1
src/TeamsCallRecords.Infrastructure/Data/Entities/RejectedRow.cs	rows for other tables	1
src/TeamsCallRecords.Infrastructure/Data/Entities/AuditLogRow.cs	rows for other tables	1
src/TeamsCallRecords.Infrastructure/Data/Entities/OutboxRow.cs	rows for other tables	1
src/TeamsCallRecords.Infrastructure/Data/Entities/SubscriptionRow.cs	rows for other tables	1
src/TeamsCallRecords.Infrastructure/Data/Entities/ErrorLogRow.cs	rows for other tables	1
src/TeamsCallRecords.Infrastructure/Data/Migrations/20261021000000_InitialCreate.cs	schema migration from 3.5	2
src/TeamsCallRecords.Infrastructure/Data/Migrations/ 20261021000000_InitialCreate.Designer.cs	schema migration from 3.5	2
src/TeamsCallRecords.Infrastructure/Data/Migrations/TcriDbContextModelSnapshot.cs	schema migration from 3.5	2
src/TeamsCallRecords.Infrastructure/Data/Migrations/InitialCreate.sql	normative DDL from 3.5	2
src/TeamsCallRecords.Infrastructure/Queue/QueueModels.cs	DTO inbox	2
src/TeamsCallRecords.Infrastructure/Queue/InboxTransitions.cs	atomic transitions	2
src/TeamsCallRecords.Infrastructure/Queue/IInboxStore.cs	incoming queue	2
src/TeamsCallRecords.Infrastructure/Queue/InboxStore.cs	incoming queue	2
src/TeamsCallRecords.Infrastructure/Processing/ProcessingModels.cs	processing outcomes	2
src/TeamsCallRecords.Infrastructure/Processing/IReconciler.cs	reconciliation contract	2

Path	Purpose	Part
src/TeamsCallRecords.Infrastructure/Processing/Reconciler.cs	reconciliation and recovery	2
src/TeamsCallRecords.Infrastructure/Processing/ISourceStateStore.cs	call state, replacement transaction, item processing	2
src/TeamsCallRecords.Infrastructure/Processing/SourceStateStore.cs	call state, replacement transaction, item processing	2
src/TeamsCallRecords.Infrastructure/Processing/ICdrApplier.cs	call state, replacement transaction, item processing	2
src/TeamsCallRecords.Infrastructure/Processing/CdrApplier.cs	call state, replacement transaction, item processing	2
src/TeamsCallRecords.Infrastructure/Processing/ICallProcessor.cs	call state, replacement transaction, item processing	2
src/TeamsCallRecords.Infrastructure/Processing/CallProcessor.cs	call state, replacement transaction, item processing	2
src/TeamsCallRecords.Infrastructure/Outbox/IOutboxSender.cs	sender contract	2
src/TeamsCallRecords.Infrastructure/Outbox/OutboxSender.cs	sender	2
src/TeamsCallRecords.Infrastructure/Outbox/OutboxModels.cs	DTO outbox	2
src/TeamsCallRecords.Infrastructure/Outbox/IOutboxStore.cs	outgoing notifications	2
src/TeamsCallRecords.Infrastructure/Outbox/OutboxStore.cs	outgoing notifications	2
src/TeamsCallRecords.Infrastructure/Graph/GraphModels.cs	Graph DTOs and errors	2
src/TeamsCallRecords.Infrastructure/Graph/GraphException.cs	Graph exceptions	2
src/TeamsCallRecords.Infrastructure/Graph/IGraphClient.cs	Graph client	2
src/TeamsCallRecords.Infrastructure/Graph/GraphClient.cs	Graph client	2
src/TeamsCallRecords.Infrastructure/Graph/GraphOptions.cs	Graph client	2
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionOptions.cs	subscription options	2
src/TeamsCallRecords.Infrastructure/Subscriptions/ISubscriptionStore.cs	subscriptions	2
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionStore.cs	subscriptions	2
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionService.cs	subscriptions	2
src/TeamsCallRecords.Infrastructure/Diagnostics/IErrorLog.cs	database error log	2
src/TeamsCallRecords.Infrastructure/Diagnostics/ErrorLog.cs	database error log	2
src/TeamsCallRecords.Host/TeamsCallRecords.Host.csproj	Host project	3
src/TeamsCallRecords.Host/Program.cs	startup, service registration	3

Path	Purpose	Part
src/TeamsCallRecords.Host/appsettings.json	default settings	3
src/TeamsCallRecords.Host/appsettings.example.json	example settings with secret placeholders	3
src/TeamsCallRecords.Infrastructure/Config/InboxOptions.cs	InboxOptions	3
src/TeamsCallRecords.Infrastructure/Config/ProcessingOptions.cs	ProcessingOptions	3
src/TeamsCallRecords.Infrastructure/Config/ReconcileOptions.cs	ReconcileOptions	3
src/TeamsCallRecords.Infrastructure/Config/OutboxOptions.cs	OutboxOptions	3
src/TeamsCallRecords.Host/Config/BotOptions.cs	BotOptions	3
src/TeamsCallRecords.Host/Config/ApiOptions.cs	ApiOptions	3
src/TeamsCallRecords.Host/Config/WebhookOptions.cs	WebhookOptions	3
src/TeamsCallRecords.Infrastructure/Config/StorageOptions.cs	StorageOptions	3
src/TeamsCallRecords.Host/Web/GraphNotifyEndpoint.cs	HTTP	3
src/TeamsCallRecords.Host/Web/CdrEndpoints.cs	HTTP	3
src/TeamsCallRecords.Host/Web/HealthEndpoint.cs	HTTP	3
src/TeamsCallRecords.Host/Web/ReprocessEndpoint.cs	HTTP	3
src/TeamsCallRecords.Host/Web/ApiKeyFilter.cs	HTTP	3
src/TeamsCallRecords.Host/Bot/TcriBot.cs	Teams bot	3
src/TeamsCallRecords.Host/Bot/BotCommands.cs	Teams bot	3
src/TeamsCallRecords.Host/Workers/InboxWorker.cs	background services	3
src/TeamsCallRecords.Host/Workers/OutboxWorker.cs	background services	3
src/TeamsCallRecords.Host/Workers/ReconcileWorker.cs	background services	3
src/TeamsCallRecords.Host/Workers/SubscriptionWorker.cs	background services	3
tests/TeamsCallRecords.Core.Tests/TeamsCallRecords.Core.Tests.csproj	test project (xunit 2.9.3)	1
tests/TeamsCallRecords.Core.Tests/Samples.cs	synthetic record builder	1
tests/TeamsCallRecords.Core.Tests/ConverterTests.cs	Core tests	1
tests/TeamsCallRecords.Core.Tests/RulesTests.cs	Core tests	1

Path	Purpose	Part
tests/TeamsCallRecords.Core.Tests/HashTests.cs	Core tests	1
tests/TeamsCallRecords.Core.Tests/DuplicateSessionTests.cs	Core tests	1
tests/TeamsCallRecords.Core.Tests/IncompleteReasonsTests.cs	Core tests	1
tests/TeamsCallRecords.Core.Tests/CdrJsonTests.cs	Core tests	1
tests/TeamsCallRecords.Core.Tests/GoldenTests.cs	Core tests	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G01_one_to_one_single_session.json	Golden input G01	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G02_group_two_sessions.json	Golden input G02	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G03_overlapping_segments.json	Golden input G03	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G04_failure_info.json	Golden input G04	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G05_no_caller_no_callee.json	Golden input G05	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/ G06_phone_and_unknown_party.json	Golden input G06	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G07_no_sessions.json	Golden input G07	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G08_session_end_missing.json	Golden input G08	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G09_invalid_and_valid_session.json	Golden input G09	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/ G10_utc_offset_and_half_second.json	Golden input G10	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/ G11_version_higher_content_changed.json	Golden input G11	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/ G01_one_to_one_single_session.json	Golden expected result G01	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G02_group_two_sessions.json	Golden expected result G02	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G03_overlapping_segments.json	Golden expected result G03	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G04_failure_info.json	Golden expected result G04	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G05_no_caller_no_callee.json	Golden expected result G05	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/ G06_phone_and_unknown_party.json	Golden expected result G06	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G07_no_sessions.json	Golden expected result G07	1

Path	Purpose	Part
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G08_session_end_missing.json	Golden expected result G08	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G09_invalid_and_valid_session.json	Golden expected result G09	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G10_utc_offset_and_half_second.json	Golden expected result G10	1
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G11_version_higher_content_changed.json	Golden expected result G11	1
tests/TeamsCallRecords.Infrastructure.Tests/ (directory)	Part 2 tests: file inventory and test project specified in Part 2	2
tests/TeamsCallRecords.Host.Tests/ (directory)	Part 3 tests: file inventory and test project specified in Part 3	3

3.3. Core: Data Types

All types are public and immutable (record or static class) in namespace TeamsCallRecords.Core, in Models.cs unless otherwise stated.

Enumerations.

Type	Values	String in JSON and Database
CallType	Unknown, OneToOne, Group	unknown, oneToOne, group
CdrStatus	Completed, Failed, Unknown	completed, failed, unknown
AggregationStatus	Complete, Incomplete	complete, incomplete

Only CdrJson (for JSON) and CdrMapping (for the database) convert enumerations to strings; no other code performs this conversion.

PartyRef(string Type, string Id) — a normalized call party. Type ∈ {user, application, phone, service, unknown}. For service and unknown, Id is an empty string. The guest type is not distinguished in v1 [P, Appendix A: P-01].

Cdr — one CDR record per session.

Property	C# Type	JSON Field	cdr Column	Required
SchemaVersion	int, default 1	schema_version	schema_version	yes
EventType	string, default "call"	event_type	event_type	yes
TenantId	string	tenant_id	tenant_id	yes
CallId	string	call_id	call_id	yes
SessionId	string	session_id	session_id	yes

Property	C# Type	JSON Field	cdr Column	Required
SourceVersion	long	source_version	source_version	yes
CallType	CallType	call_type	call_type	yes
Caller	PartyRef?	caller ({type,id} or null)	caller_type, caller_id	no
Callee	PartyRef?	callee	callee_type, callee_id	no
StartUtc	DateTimeOffset (zero offset)	start_utc	start_utc	yes
EndUtc	DateTimeOffset (zero offset)	end_utc	end_utc	yes
DurationSeconds	long	duration_s	duration_s	yes
Modalities	IReadOnlyList<string>	modalities	modalities (JSON array)	yes
Status	CdrStatus	status	status	yes
TerminationReason	string?	termination_reason	termination_reason	no
SegmentCount	int	segment_count	segment_count	yes
AggregationStatus	AggregationStatus	aggregation_status	aggregation_status	yes
SourceHash	string (64 lowercase hex characters)	source_hash	source_hash	yes
ExtensionsJson	string?	extensions (object or null)	extensions	no

cdr_seq is not a member of Cdr: the database assigns it; the REST JSON representation adds it as the first field (CdrJson.ToJson(cdr, cdrSeq)).

Rejection(string? CallId, string? SessionId, string Code, string Message) — rejection reason; Code is drawn from RejectionCodes; Message is an English log message and test cases do not assert its exact text.

ConversionResult

Property	Type	Meaning
CallId	string?	call identifier; null if it could not be extracted
SourceVersion	long?	record version; null if not extracted
Cdrs	IReadOnlyList<Cdr>	CDRs for valid sessions
Rejections	IReadOnlyList<Rejection>	rejections
IsComplete	bool	true only if the session set is complete and all sessions are valid; only such a result may replace stored CDRs
IncompleteReasons	IReadOnlyList<string>	distinct incomplete reasons (register 3.1)

Property	Type	Meaning
SessionSetHash	string?	session set hash; null for Bad results

RejectionCodes and IncompleteReasons are static classes of string constants from the register in 3.1. Additionally, IncompleteReasons.IsTransient(string reason) returns true for NO_SESSIONS and SESSION_DATES_MISSING, otherwise false.

3.4. Core: Specifications

3.4.1. Canonical (Canonical.cs)

Member	Signature	Purpose
ExcludedSuffixes	IReadOnlyList<string>: @odata.context, @odata.nextLink, @odata.count	suffixes identifying excluded transport annotation names
ToCanonicalJson	string ToCanonicalJson(JsonElement element)	canonical JSON
Sha256Hex	string Sha256Hex(string text)	SHA-256 of UTF-8 bytes; 64 lowercase characters
HashOf	string HashOf(JsonElement element)	Sha256Hex(ToCanonicalJson(element))
SessionSetHash	string SessionSetHash(IEnumerable<(string Id, string Hash)> sessions)	session set hash

ToCanonicalJson. Precondition: element was obtained from a JsonDocument that has not yet been disposed. Serialization rules: for an object, omit properties whose names end (StringComparison.Ordinal) in any suffix from ExcludedSuffixes; write all other properties in StringComparer.Ordinal order by name. For an array, retain the original element order. Write strings as JSON values using UnsafeRelaxedJsonEscaping. For numbers, write their original lexical representation (GetRawText), so 1.10 and 1.1 are different. Write true/false/null unchanged; write no whitespace. Preserve @odata.type as substantive content. Postcondition: the output does not depend on property order or whitespace in the source JSON.

SessionSetHash. Precondition: identifiers are non-null. Algorithm: sort pairs by Id (Ordinal); append <Id length in characters>:<Id>=<Hash>\n to the buffer for each pair; return Sha256Hex(buffer). Postcondition: input pair order does not matter; changing the identifier set or any hash changes the result; empty input produces the hash of an empty string.

3.4.2. VersionRules (VersionRules.cs)

enum VersionAction { Apply, IgnoreLower, DuplicateSame, Discrepancy }.

VersionAction Decide(long? appliedVersion, string? appliedSetHash, long incomingVersion, string incomingSetHash). Pure function. Decision table:

Condition	Result
appliedVersion = null	Apply
incomingVersion < appliedVersion	IgnoreLower
incomingVersion > appliedVersion	Apply
versions equal and appliedSetHash equals incomingSetHash (Ordinal)	DuplicateSame

Condition	Result
versions equal, hashes differ	Discrepancy

Only this function makes the decision; Host and Infrastructure do not independently compare versions.

long RoundDurationSeconds(DateTimeOffset start, DateTimeOffset end). Precondition: end >= start (for end < start the function would produce a negative value using the same rule, but Core rejects such sessions before calling it). Integer-tick algorithm: ticks = (end − start).Ticks; second = TimeSpan.TicksPerSecond; for ticks >= 0 return (ticks + second/2) / second (integer division); for negative ticks use the symmetric signed calculation. Floating-point arithmetic is prohibited.

3.4.3. CallRecordConverter.Convert (CallRecordConverter.cs)

Signature: public static ConversionResult Convert(string callRecordJson, string tenantId).

Preconditions. Neither argument may be null (violation raises ArgumentNullException: a programmer error, not a data error [P, P-02]). An empty or malformed JSON string is not a precondition violation: return BadJson.

Postconditions. Does not throw exceptions due to JSON content; never accesses the network or database; deterministic (identical input yields identical results, including hashes).

Helper rules (private methods; fixed names):

Method	Rule
Str(obj, name)	property value if present and of string type; otherwise null
Strings(obj, name)	string elements of the array; empty list if not an array
ReadDate(obj, name, out value)	missing property, null, or empty string → Missing; non-string value or a value that cannot be parsed using DateTimeOffset.TryParse(text, InvariantCulture, AssumeUniversal | AdjustToUniversal) → Bad; otherwise Ok with the UTC value (a string without offset is interpreted as UTC)
FailureOf(obj)	no failureInfo object → null; otherwise "<stage>:<reason>"; absent components become unknown
Party(session, name)	no caller/callee object → null; search identity in order user, application, phone; the first object with a nonempty id string becomes PartyRef(kind, id); otherwise, if @odata.type contains serviceEndpoint (case-insensitive), return PartyRef("service", ""); otherwise PartyRef("unknown", "")
SegmentsJson(segments)	no segments → null; otherwise JSON {"segments":[{"id","start","end","failure"}…]}: id, start, end are source segment strings (null when absent); failure = FailureOf(segment); retain source segment order
Bad(callId, code, message)	result: one Rejection entry, IsComplete=false, IncompleteReasons=[code], no CDRs, SessionSetHash=null

Algorithm.

1. Parse callRecordJson as JsonDocument. Parse failure → Bad(null, BadJson). Non-object root → Bad(null, BadJson).
2. callId = Str(root,"id"); empty value → Bad(null, NoCallId).
3. version must be a numeric property that fits Int64; otherwise → Bad(callId, NoVersion).
4. callType: type = peerToPeer → OneToOne; groupCall → Group; otherwise Unknown. recordModalities = Strings(root,"modalities").

5. If sessions is absent, is not an array, or is empty → reason NoSessions; proceed to step 11 (no CDRs).
6. For each session in source order, compute Id = Str(session,"id") and Hash = Canonical.HashOf(session); collect nonempty Id/Hash pairs for SessionSetHash.
7. Identify nonempty identifiers that occur more than once. These are duplicate identifiers [P, P-03].
8. For each session in order:
    a. If Id is empty → Rejection(callId, null, SessionNoId), reason SessionInvalid; then proceed to the next session;
    b. If Id is duplicated → Rejection(callId, Id, SessionDuplicateId) (one per session with this identifier), reason SessionInvalid;
    c. ReadDate for startDateTime and endDateTime: if either is Bad → Rejection(callId, Id, SessionBadDate), reason SessionInvalid;
    d. Otherwise, if either is Missing → no rejection, reason SessionDatesMissing;
    e. Otherwise, if end < start → Rejection(callId, Id, SessionEndBeforeStart), reason SessionInvalid;
    f. Otherwise, build Cdr: TenantId=tenantId; CallId, SessionId, SourceVersion, CallType; Caller, Callee — from Party; StartUtc, EndUtc — UTC; DurationSeconds = RoundDurationSeconds; Modalities — Strings(session,"modalities"), falling back to recordModalities if empty; Segments — elements of the session segments array (zero segments if no array); failure = FailureOf(session), or, if null, the first nonempty FailureOf(segment). Set Status = Failed and TerminationReason = failure if failure is not null; otherwise Status = Completed and TerminationReason = null. SegmentCount = number of segments; SourceHash = Hash; ExtensionsJson = SegmentsJson(segments); set AggregationStatus temporarily to Complete.
9. The v1 Core does not assign status unknown: a session with a start and end and no failureInfo is completed; one with failureInfo is failed; a session without an end produces no CDR [P, P-04].
10. IsComplete = (the reason list is empty).
11. Construct ConversionResult: Cdrs — the generated CDRs, each with AggregationStatus = Complete when IsComplete, otherwise Incomplete; Rejections; IncompleteReasons — distinct reasons in order of occurrence; SessionSetHash = Canonical.SessionSetHash(pairs from step 6).

Errors and handling. All data errors become Rejection entries or incomplete reasons; parse exceptions are caught in step 1. Core has no queues, retries, or transactions.

Host contract (Parts 2–3): if IsComplete=false, the result does not replace stored CDRs; IsTransient=true reasons lead to pending; all other reasons result in rejection entries and rejected_version.

3.4.4. CdrJson (CdrJson.cs, new file)

The only point at which Cdr and ConversionResult are serialized as JSON. Uses System.Text.Json.Nodes.

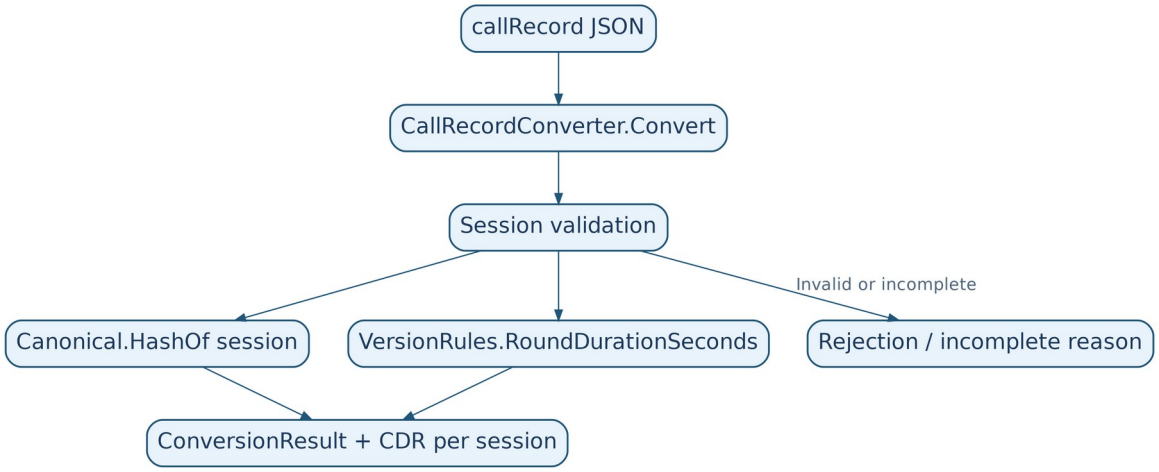
Member	Signature	Result
FormatTime	string FormatTime(DateTimeOffset value)	value.UtcDateTime.ToString("yyyy-MM-dd'T'HH:mm:ss.fffffff'Z'", InvariantCulture)
ToJson(Cdr)	JsonObject ToJson(Cdr cdr)	properties in Table 3.3 order, excluding cdr_seq
ToJson(Cdr, long)	JsonObject ToJson(Cdr cdr, long cdrSeq)	the same, with cdr_seq as the first property
ToJson(ConversionResult)	JsonObject ToJson(ConversionResult result)	golden-sample object: call_id, source_version, is_complete, incomplete_reasons, session_set_hash, cdrs, rejections

Member	Signature	Result
Serialize	string Serialize(JsonNode node)	2-space indentation, UnsafeRelaxedJsonEscaping

Rules: caller and callee are { "type", "id" } objects or null; extensions is the parsed ExtensionsJson object or null; modalities is an array of strings; termination_reason is a string or null; each rejections item contains call_id, session_id, code, and message. Golden samples are compared structurally in tests (JsonNode.DeepEquals), not textually.

3.4.5. Call Sequence

Figure 3. Core F6: Converting a Record into CDRs.



3.4.6. Mapping F6 Rules to Core Methods

F6 Rule (TRS, Section 2)	Implemented in Core
1. One CDR per Session	Convert, step 8f
2. Segments: segment_count, extensions, not summed	step 8f, SegmentsJson
3. Duration	RoundDurationSeconds
4. Version and Hash	Decide, Canonical.HashOf, SessionSetHash
5. Parties	Party
6. Statuses	steps 8f, 11
7. Idempotency	Decide (Core); keys and replacement — Part 2
8. Source Traceability	CallId, SourceVersion, SourceHash fields
9. Rejection	steps 1–3, 8a–8e, RejectionCodes
10. No Participant-Level Records	not generated
11. Incomplete Data	steps 5, 8d, 11, IncompleteReasons
12. Missed Notifications	not part of Core (Part 2: reconciliation)

3.4.7. Rejected Source Material Hash [Proposed Addition]

Canonical.RejectionHash(string rawJson) — SHA-256 of canonical JSON for a successfully parsed document, or SHA-256 of the original UTF-8 text if JSON is malformed. For an individual session error, compute Canonical.HashOf(session) for that session if available; for a whole-record error, compute RejectionHash(rawJson). This hashes the specific rejected material, not session_set_hash. An identical rejection is identified by (tenant_id, call_id, session_id, code, source_hash); registering the same tuple again does not add a row. An empty hash is not permitted when raw data is present. Tests: HashTests.RejectionHash_*, SourceStateStoreTests.Rejection_*.

3.5. SQLite Model

General rules. The tcri.db database file is located in Storage: DataDirectory. Migration 20261021000000_InitialCreate creates the schema; the migration must produce a schema equivalent to the SQL below (names of tables, columns, constraints and indexes). Upon opening every connection, SqlitePragmaInterceptor executes PRAGMA journal_mode=WAL; PRAGMA synchronous=FULL; PRAGMA foreign_keys=ON; PRAGMA busy_timeout=5000; (Part 2). synchronous=FULL was selected because Graph notification acknowledgment 202 is sent only after the inbox insert is committed; with NORMAL under WAL, even a committed transaction can be lost on a power failure or operating-system crash [P-08]. The notification-processing load test and disk-volume reliability requirements are described in Part 2. Tables have no foreign keys between them: relationships are logical, by (tenant_id, call_id).

```
-- TeamsCallRecords: SQLite schema, schema version 1 (migration 20261021000000_InitialCreate)
-- Time: TEXT, ISO 8601 UTC, exactly 7 fractional digits and Z suffix: 2026-10-20T09:00:00.5000000Z
```

```
CREATE TABLE cdr (
    cdr_seq           INTEGER PRIMARY KEY AUTOINCREMENT,
    schema_version    INTEGER NOT NULL,
    event_type        TEXT    NOT NULL,
    tenant_id         TEXT    NOT NULL,
    call_id           TEXT    NOT NULL,
    session_id        TEXT    NOT NULL,
    source_version    INTEGER NOT NULL,
    call_type         TEXT    NOT NULL CHECK (call_type IN ('oneToOne','group','unknown')),
    caller_type       TEXT    NULL,
    caller_id         TEXT    NULL,
    callee_type       TEXT    NULL,
    callee_id         TEXT    NULL,
    start_utc         TEXT    NOT NULL,
    end_utc           TEXT    NOT NULL,
    duration_s        INTEGER NOT NULL CHECK (duration_s >= 0),
    modalities        TEXT    NOT NULL,
    status            TEXT    NOT NULL CHECK (status IN ('completed','failed','unknown')),
    termination_reason TEXT   NULL,
    segment_count     INTEGER NOT NULL CHECK (segment_count >= 0),
    aggregation_status TEXT   NOT NULL CHECK (aggregation_status IN ('complete','incomplete')),
    source_hash       TEXT    NOT NULL,
    extensions        TEXT    NULL,
    created_utc       TEXT    NOT NULL,
    updated_utc       TEXT    NOT NULL,
    CONSTRAINT uq_cdr_key UNIQUE (tenant_id, call_id, session_id)
);
CREATE INDEX ix_cdr_start ON cdr (start_utc, cdr_seq);
CREATE INDEX ix_cdr_call  ON cdr (tenant_id, call_id);

CREATE TABLE source_state (
    tenant_id       TEXT    NOT NULL,
    call_id         TEXT    NOT NULL,
    applied_version  INTEGER NULL,
    rejected_version INTEGER NULL,
    session_set_hash TEXT    NULL,
    state           TEXT    NOT NULL CHECK (state IN ('pending','done','suspended','expired')),
    attempts        INTEGER NOT NULL DEFAULT 0,
    next_attempt_utc TEXT    NULL,
    first_seen_utc   TEXT    NOT NULL,
    last_error      TEXT    NULL,
```

```
    updated_utc      TEXT    NOT NULL,
    PRIMARY KEY (tenant_id, call_id)
);
CREATE INDEX ix_source_state_due ON source_state (state, next_attempt_utc);

CREATE TABLE inbox (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    element_hash     TEXT    NOT NULL,
    kind             TEXT    NOT NULL CHECK (kind IN ('callRecord','channelMessage','reconcile')),
    tenant_id        TEXT    NOT NULL,
    call_id          TEXT    NULL,
    raw              TEXT    NOT NULL,
    state            TEXT    NOT NULL CHECK (state IN ('new','processing','done','failed')),
    attempts         INTEGER NOT NULL DEFAULT 0,
    next_attempt_utc TEXT    NOT NULL,
    locked_until_utc TEXT    NULL,
    received_utc     TEXT    NOT NULL,
    done_utc         TEXT    NULL,
    last_error       TEXT    NULL
);
CREATE UNIQUE INDEX uq_inbox_new_hash ON inbox (element_hash) WHERE state = 'new';
CREATE INDEX ix_inbox_hash ON inbox (element_hash);
CREATE INDEX ix_inbox_call ON inbox (tenant_id, call_id, state);
CREATE INDEX ix_inbox_due ON inbox (state, next_attempt_utc);

CREATE TABLE rejected (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    tenant_id   TEXT NOT NULL,
    call_id     TEXT NULL,
    session_id  TEXT NULL,
    code        TEXT NOT NULL,
    message     TEXT NOT NULL,
    source_hash TEXT NULL,
    created_utc TEXT NOT NULL
);
CREATE INDEX ix_rejected_created ON rejected (created_utc);

CREATE TABLE audit_log (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    ts        TEXT NOT NULL,
    tenant_id TEXT NOT NULL,
```

```
    call_id   TEXT NOT NULL,
    kind      TEXT NOT NULL CHECK (kind IN ('version_discrepancy')),
    detail    TEXT NOT NULL
);

CREATE TABLE outbox (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    kind            TEXT    NOT NULL CHECK (kind IN ('activity')),
    tenant_id       TEXT    NOT NULL,
    call_id         TEXT    NOT NULL,
    session_id      TEXT    NOT NULL,
    recipient       TEXT    NOT NULL,
    payload         TEXT    NOT NULL,
    state           TEXT    NOT NULL CHECK (state IN ('new','sending','sent','failed')),
    attempts        INTEGER NOT NULL DEFAULT 0,
    next_attempt_utc TEXT    NOT NULL,
    locked_until_utc TEXT    NULL,
    created_utc     TEXT    NOT NULL,
    sent_utc        TEXT    NULL,
    last_error      TEXT    NULL,
    CONSTRAINT uq_outbox_key UNIQUE (kind, tenant_id, call_id, session_id, recipient)
);
CREATE INDEX ix_outbox_due ON outbox (state, next_attempt_utc);

CREATE TABLE subscription (
    id              TEXT PRIMARY KEY,
    kind            TEXT NOT NULL CHECK (kind IN ('callRecords','channelMessages')),
    resource        TEXT NOT NULL,
    notification_url  TEXT NOT NULL,
    expires_utc     TEXT NOT NULL,
    client_state_hash TEXT NOT NULL,
    status          TEXT NOT NULL CHECK (status IN ('active','renewing','failed')),
    created_utc     TEXT NOT NULL,
    renewed_utc     TEXT NULL
);

CREATE TABLE error_log (
    id      INTEGER PRIMARY KEY AUTOINCREMENT,
    ts      TEXT NOT NULL,
    component TEXT NOT NULL,
    message   TEXT NOT NULL,
```

```
    details   TEXT NULL
);
CREATE INDEX ix_error_log_ts ON error_log (ts);
```

Row classes (Data/Entities). Each table maps to one <Name>Row class: CdrRow, SourceStateRow, InboxRow, RejectedRow, AuditLogRow, OutboxRow, SubscriptionRow, ErrorLogRow. Property names are column names converted to PascalCase (source_version → SourceVersion). Types: TEXT → string, INTEGER → long (cdr_seq and id use long; counters attempts, segment_count and schema_version use int); columns allowing NULL use nullable types (?). Timestamps are stored as strings in UtcText format.

UtcText (Data/UtcText.cs): string Format(DateTimeOffset value) — a format compatible with CdrJson.FormatTime; DateTimeOffset Parse(string text) — parses using DateTimeStyles.RoundtripKind; string Now(TimeProvider clock) — current time in this format. Lexicographic ordering of timestamps in this format equals chronological ordering.

CdrMapping (Data/CdrMapping.cs):

Method	Purpose
CdrRow ToRow(Cdr cdr, string nowText)	Cdr → row; call_type, status, aggregation_status are strings from 3.3; caller_type/caller_id are both NULL if Caller=null; modalities is a JSON array; start_utc, end_utc use UtcText.Format; created_utc=updated_utc=nowText; cdr_seq is not populated
Cdr ToCdr(CdrRow row)	reverse conversion; ToCdr(ToRow(c)) equals c for all properties
void CopyOnUpdate(CdrRow target, CdrRow source, string nowText)	on replacement copies every column except cdr_seq and created_utc; updated_utc=nowText

Fractional timestamp preservation. Values with tick precision (...fffffff7) are written and read without loss; CdrMappingTests (Part 2) verifies round-trip conversion.

3.6. Core Golden Sample Catalog

The golden samples are synthetic records based on the public Microsoft Graph callRecord resource structure; they contain no real data and must not be presented as genuine Graph records. Tenant identifier: 11111111-1111-4111-8111-111111111111. Expected results were obtained by executing the prepared I1 module and checked against the rules in 3.4.3; in the implementation they are stored as files and compared automatically (GoldenTests, 3.7). The files reside in tests/TeamsCallRecords.Core.Tests/TestData/Golden/in and out. This document includes the summary table and all eleven complete input–expected-result pairs.

№	Scenario	Version	CDR	duration_s	status	aggregation	IsComplete	Incomplete Reasons	Rejections
G01	one_to_one_single_session	4	1	61	completed	complete	yes	—	—
G02	group_two_sessions	7	2	600,450	completed,completed	complete	yes	—	—
G03	overlapping_segments	3	1	300	completed	complete	yes	—	—
G04	failure_info	2	1	4	failed	complete	yes	—	—
G05	no_caller_no_callee	1	1	120	completed	complete	yes	—	—
G06	phone_and_unknown_party	5	1	45	completed	complete	yes	—	—
G07	no_sessions	1	0	—	—	—	no	NO_SESSIONS	—

№	Scenario	Version	CDR	duration_s	status	aggregation	IsComplete	Incomplete Reasons	Rejections
G08	session_end_missing	2	0	—	—	—	no	SESSION_DATES_MISSING	—
G09	invalid_and_valid_session	6	1	300	completed	incomplete	no	SESSION_INVALID	SESSION_END_BEFORE_START
G10	utc_offset_and_half_second	1	1	31	completed	complete	yes	—	—
G11	version_higher_content_changed	5	1	61	completed	complete	yes	—	—

Scenarios: G01 — one-to-one, one session, duration 60.5 s → 61 (half-up); G02 — group call, two sessions, video and audio, fractional seconds (449.876 s → 450); G03 — two overlapping segments (segment_count=2, durations are not summed); G04 — failureInfo → failed; G05 — missing caller and callee → null; G06 — telephone and party with no identity (unknown); G07 — no sessions; G08 — missing session end; G09 — one valid and one invalid session; G10 — offset +03:00 converted to UTC, 30.5 s → 31; G11 — G01 record with version 5 and modified session content (different source_hash and session_set_hash).

Golden sample G01 (one-to-one, one session):

Input (G01_one_to_one_single_session.json):


```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
 "id": "cccccccc-0000-4000-8000-000000000001",
 "version": 4,
 "type": "peerToPeer",
 "modalities": [
  "audio"
 ],
 "startDateTime": "2026-10-20T09:00:00.0000000Z",
 "endDateTime": "2026-10-20T09:05:00.0000000Z",
 "sessions": [
  {
   "id": "dddddddd-0000-4000-8000-000000000001",
   "modalities": [
    "audio"
   ],
   "startDateTime": "2026-10-20T09:00:00.0000000Z",
   "endDateTime": "2026-10-20T09:01:00.5000000Z",
   "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000001",
```

```
      "displayName": "User A"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
    }
   },
   "callee": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000002",
      "displayName": "User B"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
    }
   },
   "segments": [
    {
     "id": "seg-1",
     "startDateTime": "2026-10-20T09:00:00.0000000Z",
     "endDateTime": "2026-10-20T09:01:00.5000000Z",
     "media": [
      {
       "label": "main-audio",
       "streams": [
        {
         "streamId": "s1",
         "streamDirection": "callerToCallee"
        }
       ]
      }
     ]
    }
   ]
```

```
    }
  ]
}
```

Expected result (G01_one_to_one_single_session.json):


```
{
  "call_id": "cccccccc-0000-4000-8000-000000000001",
  "source_version": 4,
  "is_complete": true,
  "incomplete_reasons": [],
  "session_set_hash": "146719861efe9004a28b3e18b1b10301ee5a5dd868aaa6b3ce4ef21ea8195381",
  "cdrs": [
    {
      "schema_version": 1,
      "event_type": "call",
      "tenant_id": "11111111-1111-4111-8111-111111111111",
      "call_id": "cccccccc-0000-4000-8000-000000000001",
      "session_id": "dddddddd-0000-4000-8000-000000000001",
      "source_version": 4,
      "call_type": "oneToOne",
      "caller": {
        "type": "user",
        "id": "aaaaaaaa-0000-4000-8000-000000000001"
      },
      "callee": {
        "type": "user",
        "id": "aaaaaaaa-0000-4000-8000-000000000002"
      },
      "start_utc": "2026-10-20T09:00:00.0000000Z",
      "end_utc": "2026-10-20T09:01:00.5000000Z",
      "duration_s": 61,
      "modalities": [
        "audio"
      ],
      "status": "completed",
      "termination_reason": null,
      "segment_count": 1,
      "aggregation_status": "complete",
      "source_hash": "604b888cbcb6bd1bf3351d5ca29fbe489d11bb936aa1a1eda53071ed7593dcb3",
      "extensions": {
        "segments": [
```

```
        {
          "id": "seg-1",
          "start": "2026-10-20T09:00:00.0000000Z",
          "end": "2026-10-20T09:01:00.5000000Z",
          "failure": null
        }
      ]
    }
  }
 ],
 "rejections": []
}
```

Golden sample G02 (group call, two sessions, fractional seconds):

Input (G02_group_two_sessions.json):

```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
 "id": "cccccccc-0000-4000-8000-000000000002",
 "version": 7,
 "type": "groupCall",
 "modalities": [
  "audio",
  "video"
 ],
 "startDateTime": "2026-10-20T10:00:00.0000000Z",
 "endDateTime": "2026-10-20T10:10:00.0000000Z",
 "sessions": [
  {
   "id": "dddddddd-0000-4000-8000-000000000001",
   "modalities": [
    "audio",
    "video"
   ],
   "startDateTime": "2026-10-20T10:00:00.0000000Z",
   "endDateTime": "2026-10-20T10:10:00.0000000Z",
   "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000001",
```

```
      "displayName": "User A"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
    }
   },
   "callee": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000003",
      "displayName": "User C"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
    }
   },
   "segments": [
    {
     "id": "seg-1",
     "startDateTime": "2026-10-20T10:00:00.0000000Z",
     "endDateTime": "2026-10-20T10:10:00.0000000Z",
     "media": [
      {
       "label": "main-audio",
       "streams": [
        {
         "streamId": "s1",
         "streamDirection": "callerToCallee"
        }
       ]
      }
     ]
    }
   ]
```

```
  },
  {
   "id": "ddddddd-0000-4000-8000-000000000002",
   "modalities": [
    "audio"
   ],
   "startDateTime": "2026-10-20T10:02:30.1230000Z",
   "endDateTime": "2026-10-20T10:09:59.9990000Z",
   "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000002",
      "displayName": "User B"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
    }
   },
   "callee": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000003",
      "displayName": "User C"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
    }
   },
   "segments": [
    {
     "id": "seg-1",
     "startDateTime": "2026-10-20T10:02:30.1230000Z",
     "endDateTime": "2026-10-20T10:09:59.9990000Z",
```

```
        "media": [
          {
            "label": "main-audio",
            "streams": [
              {
                "streamId": "s1",
                "streamDirection": "callerToCallee"
              }
            ]
          }
        ]
      }
    ]
  }
 ]
}
```

Expected result (G02_group_two_sessions.json):

```
{
  "call_id": "cccccccc-0000-4000-8000-000000000002",
  "source_version": 7,
  "is_complete": true,
  "incomplete_reasons": [],
  "session_set_hash": "5522cff43718258e6bcfc7c90c92e6d51e68cfb160deec4e30920749e23e98cf",
  "cdrs": [
    {
      "schema_version": 1,
      "event_type": "call",
      "tenant_id": "11111111-1111-4111-8111-111111111111",
      "call_id": "cccccccc-0000-4000-8000-000000000002",
      "session_id": "dddddddd-0000-4000-8000-000000000001",
      "source_version": 7,
      "call_type": "group",
      "caller": {
        "type": "user",
        "id": "aaaaaaaa-0000-4000-8000-000000000001"
      },
      "callee": {
        "type": "user",
        "id": "aaaaaaaa-0000-4000-8000-000000000003"
      },
```

```
    "start_utc": "2026-10-20T10:00:00.0000000Z",
    "end_utc": "2026-10-20T10:10:00.0000000Z",
    "duration_s": 600,
    "modalities": [
      "audio",
      "video"
    ],
    "status": "completed",
    "termination_reason": null,
    "segment_count": 1,
    "aggregation_status": "complete",
    "source_hash": "c1d4d9ba424314df4c9e0442dfab4d8d37c9e3993c4e3df9a42002cff31fb95b",
    "extensions": {
      "segments": [
        {
          "id": "seg-1",
          "start": "2026-10-20T10:00:00.0000000Z",
          "end": "2026-10-20T10:10:00.0000000Z",
          "failure": null
        }
      ]
    }
  },
  {
    "schema_version": 1,
    "event_type": "call",
    "tenant_id": "11111111-1111-4111-8111-111111111111",
    "call_id": "cccccccc-0000-4000-8000-000000000002",
    "session_id": "dddddddd-0000-4000-8000-000000000002",
    "source_version": 7,
    "call_type": "group",
    "caller": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000002"
    },
    "callee": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000003"
    },
    "start_utc": "2026-10-20T10:02:30.1230000Z",
    "end_utc": "2026-10-20T10:09:59.9990000Z",
```

```
    "duration_s": 450,
    "modalities": [
      "audio"
    ],
    "status": "completed",
    "termination_reason": null,
    "segment_count": 1,
    "aggregation_status": "complete",
    "source_hash": "bbab6a67cc35652b2185bcf345a5a1e89370aceadbd0f300d2be04fda6b21b0f",
    "extensions": {
      "segments": [
        {
          "id": "seg-1",
          "start": "2026-10-20T10:02:30.1230000Z",
          "end": "2026-10-20T10:09:59.9990000Z",
          "failure": null
        }
      ]
    }
  }
 ],
 "rejections": []
}
```


Golden sample G03 (overlapping segments):

Input (G03_overlapping_segments.json):


```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
 "id": "cccccccc-0000-4000-8000-000000000003",
 "version": 3,
 "type": "peerToPeer",
 "modalities": [
   "audio"
 ],
 "startDateTime": "2026-10-20T11:00:00.0000000Z",
 "endDateTime": "2026-10-20T11:05:00.0000000Z",
 "sessions": [
   {
     "id": "dddddddd-0000-4000-8000-000000000001",
     "modalities": [
```

```
    "audio"
  ],
  "startDateTime": "2026-10-20T11:00:00.0000000Z",
  "endDateTime": "2026-10-20T11:05:00.0000000Z",
  "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
      "user": {
        "id": "aaaaaaaa-0000-4000-8000-000000000001",
        "displayName": "User A"
      }
    },
    "userAgent": {
      "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
      "platform": "windows",
      "productFamily": "teams"
    }
  },
  "callee": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
      "user": {
        "id": "aaaaaaaa-0000-4000-8000-000000000002",
        "displayName": "User B"
      }
    },
    "userAgent": {
      "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
      "platform": "windows",
      "productFamily": "teams"
    }
  },
  "segments": [
    {
      "id": "seg-1",
      "startDateTime": "2026-10-20T11:00:00.0000000Z",
      "endDateTime": "2026-10-20T11:03:00.0000000Z",
      "media": [
        {
          "label": "main-audio",
          "streams": [
```

```
            {
              "streamId": "s1",
              "streamDirection": "callerToCallee"
            }
          ]
        }
      ]
    },
    {
      "id": "seg-2",
      "startDateTime": "2026-10-20T11:02:00.0000000Z",
      "endDateTime": "2026-10-20T11:05:00.0000000Z",
      "media": [
        {
          "label": "main-audio",
          "streams": [
            {
              "streamId": "s1",
              "streamDirection": "callerToCallee"
            }
          ]
        }
      ]
    }
  ]
 }
}
```

Expected result (G03_overlapping_segments.json):

```
{
 "call_id": "cccccccc-0000-4000-8000-000000000003",
 "source_version": 3,
 "is_complete": true,
 "incomplete_reasons": [],
 "session_set_hash": "c7b955b9b62d40f403f7377f366b5b967661350e7c70e4e06d82cac7a77de0fd",
 "cdrs": [
  {
   "schema_version": 1,
   "event_type": "call",
   "tenant_id": "11111111-1111-4111-8111-111111111111",
```

```
    "call_id": "cccccccc-0000-4000-8000-000000000003",
    "session_id": "dddddddd-0000-4000-8000-000000000001",
    "source_version": 3,
    "call_type": "oneToOne",
    "caller": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000001"
    },
    "callee": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000002"
    },
    "start_utc": "2026-10-20T11:00:00.0000000Z",
    "end_utc": "2026-10-20T11:05:00.0000000Z",
    "duration_s": 300,
    "modalities": [
      "audio"
    ],
    "status": "completed",
    "termination_reason": null,
    "segment_count": 2,
    "aggregation_status": "complete",
    "source_hash": "caf53e440ba8932ef8390a0f4cfbbab62b7c82acdfa7564070288d38a77fb00a",
    "extensions": {
      "segments": [
        {
          "id": "seg-1",
          "start": "2026-10-20T11:00:00.0000000Z",
          "end": "2026-10-20T11:03:00.0000000Z",
          "failure": null
        },
        {
          "id": "seg-2",
          "start": "2026-10-20T11:02:00.0000000Z",
          "end": "2026-10-20T11:05:00.0000000Z",
          "failure": null
        }
      ]
    }
  }
]
```

```
  "rejections": []
}
```


Golden sample G04 (failure information):

Input (G04_failure_info.json):


```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
  "id": "cccccccc-0000-4000-8000-000000000004",
  "version": 2,
  "type": "peerToPeer",
  "modalities": [
    "audio"
  ],
  "startDateTime": "2026-10-20T12:00:00.0000000Z",
  "endDateTime": "2026-10-20T12:00:04.0000000Z",
  "sessions": [
    {
      "id": "dddddddd-0000-4000-8000-000000000001",
      "modalities": [
        "audio"
      ],
      "startDateTime": "2026-10-20T12:00:00.0000000Z",
      "endDateTime": "2026-10-20T12:00:04.0000000Z",
      "caller": {
        "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
        "identity": {
          "user": {
            "id": "aaaaaaaa-0000-4000-8000-000000000001",
            "displayName": "User A"
          }
        },
        "userAgent": {
          "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
          "platform": "windows",
          "productFamily": "teams"
        }
      },
      "callee": {
        "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
        "identity": {
```

```
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000002",
          "displayName": "User B"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "segments": [
      {
        "id": "seg-1",
        "startDateTime": "2026-10-20T12:00:00.0000000Z",
        "endDateTime": "2026-10-20T12:00:04.0000000Z",
        "media": [
          {
            "label": "main-audio",
            "streams": [
              {
                "streamId": "s1",
                "streamDirection": "callerToCallee"
              }
            ]
          }
        ],
        "failureInfo": {
          "@odata.type": "#microsoft.graph.callRecords.failureInfo",
          "stage": "callSetup",
          "reason": "networkError"
        }
      }
    ]
  }
 ]
}
```

Expected result (G04_failure_info.json):

```
{
  "call_id": "cccccccc-0000-4000-8000-000000000004",
```

```
"source_version": 2,
"is_complete": true,
"incomplete_reasons": [],
"session_set_hash": "bdafe61873c8079c03e7363494082c42932ddb2ed0cc401f66fc6adf0a449979",
"cdrs": [
 {
  "schema_version": 1,
  "event_type": "call",
  "tenant_id": "11111111-1111-4111-8111-111111111111",
  "call_id": "cccccccc-0000-4000-8000-000000000004",
  "session_id": "dddddddd-0000-4000-8000-000000000001",
  "source_version": 2,
  "call_type": "oneToOne",
  "caller": {
   "type": "user",
   "id": "aaaaaaaa-0000-4000-8000-000000000001"
  },
  "callee": {
   "type": "user",
   "id": "aaaaaaaa-0000-4000-8000-000000000002"
  },
  "start_utc": "2026-10-20T12:00:00.0000000Z",
  "end_utc": "2026-10-20T12:00:04.0000000Z",
  "duration_s": 4,
  "modalities": [
   "audio"
  ],
  "status": "failed",
  "termination_reason": "callSetup:networkError",
  "segment_count": 1,
  "aggregation_status": "complete",
  "source_hash": "459f5a417571bc67b15d9b161d8439bf4a5d1b8a2d5a2cde3c6f58e367e8eab4",
  "extensions": {
   "segments": [
    {
     "id": "seg-1",
     "start": "2026-10-20T12:00:00.0000000Z",
     "end": "2026-10-20T12:00:04.0000000Z",
     "failure": "callSetup:networkError"
    }
   ]
```

```
    }
   }
  ],
  "rejections": []
}
```

Golden sample G05 (no caller or callee):

Input (G05_no_caller_no_callee.json):

```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
  "id": "cccccccc-0000-4000-8000-000000000005",
  "version": 1,
  "type": "groupCall",
  "modalities": [
    "audio"
  ],
  "startDateTime": "2026-10-20T13:00:00.0000000Z",
  "endDateTime": "2026-10-20T13:02:00.0000000Z",
  "sessions": [
    {
      "id": "dddddddd-0000-4000-8000-000000000001",
      "modalities": [
        "audio"
      ],
      "startDateTime": "2026-10-20T13:00:00.0000000Z",
      "endDateTime": "2026-10-20T13:02:00.0000000Z",
      "segments": [
        {
          "id": "seg-1",
          "startDateTime": "2026-10-20T13:00:00.0000000Z",
          "endDateTime": "2026-10-20T13:02:00.0000000Z",
          "media": [
            {
              "label": "main-audio",
              "streams": [
                {
                  "streamId": "s1",
                  "streamDirection": "callerToCallee"
                }
              ]
```

```
        }
       ]
      }
     ]
    }
  ]
}
```


Expected result (G05_no_caller_no_callee.json):


```
{
  "call_id": "cccccccc-0000-4000-8000-000000000005",
  "source_version": 1,
  "is_complete": true,
  "incomplete_reasons": [],
  "session_set_hash": "7490850e79c54194ee8cb0524b1913a597a8fca4a40bc846877da69c99c97dd9",
  "cdrs": [
    {
      "schema_version": 1,
      "event_type": "call",
      "tenant_id": "11111111-1111-4111-8111-111111111111",
      "call_id": "cccccccc-0000-4000-8000-000000000005",
      "session_id": "dddddddd-0000-4000-8000-000000000001",
      "source_version": 1,
      "call_type": "group",
      "caller": null,
      "callee": null,
      "start_utc": "2026-10-20T13:00:00.0000000Z",
      "end_utc": "2026-10-20T13:02:00.0000000Z",
      "duration_s": 120,
      "modalities": [
        "audio"
      ],
      "status": "completed",
      "termination_reason": null,
      "segment_count": 1,
      "aggregation_status": "complete",
      "source_hash": "ea52317e0c8770c225cb77101d528779987890c11b54dac2855a2c5ed3c5b98f",
      "extensions": {
        "segments": [
          {
            "id": "seg-1",
```

```
          "start": "2026-10-20T13:00:00.0000000Z",
          "end": "2026-10-20T13:02:00.0000000Z",
          "failure": null
        }
      ]
    }
  }
 ],
 "rejections": []
}
```


Golden sample G06 (telephone and unidentified party):

Input (G06_phone_and_unknown_party.json):


```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
 "id": "cccccccc-0000-4000-8000-000000000006",
 "version": 5,
 "type": "peerToPeer",
 "modalities": [
  "audio"
 ],
 "startDateTime": "2026-10-20T14:00:00.0000000Z",
 "endDateTime": "2026-10-20T14:00:45.0000000Z",
 "sessions": [
  {
   "id": "dddddddd-0000-4000-8000-000000000001",
   "modalities": [
    "audio"
   ],
   "startDateTime": "2026-10-20T14:00:00.0000000Z",
   "endDateTime": "2026-10-20T14:00:45.0000000Z",
   "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "phone": {
      "id": "+10000000001",
      "displayName": "Test number"
     }
    }
   },
```

```
    "callee": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint"
    },
    "segments": []
   }
 ]
}
```

Expected result (G06_phone_and_unknown_party.json):

```
{
 "call_id": "cccccccc-0000-4000-8000-000000000006",
 "source_version": 5,
 "is_complete": true,
 "incomplete_reasons": [],
 "session_set_hash": "5532aef65d20f55018ac857ae27f2b38956634eacaf20b6b02b066aa9b75c561",
 "cdrs": [
  {
    "schema_version": 1,
    "event_type": "call",
    "tenant_id": "11111111-1111-4111-8111-111111111111",
    "call_id": "cccccccc-0000-4000-8000-000000000006",
    "session_id": "dddddddd-0000-4000-8000-000000000001",
    "source_version": 5,
    "call_type": "oneToOne",
    "caller": {
      "type": "phone",
      "id": "+10000000001"
    },
    "callee": {
      "type": "unknown",
      "id": ""
    },
    "start_utc": "2026-10-20T14:00:00.0000000Z",
    "end_utc": "2026-10-20T14:00:45.0000000Z",
    "duration_s": 45,
    "modalities": [
      "audio"
    ],
    "status": "completed",
    "termination_reason": null,
    "segment_count": 0,
```

```
    "aggregation_status": "complete",
    "source_hash": "f857e23ac8e3762d3ac785a053f951765722125e8bac742f9c74756e2a2c654b",
    "extensions": null
   }
  ],
  "rejections": []
}
```


Golden sample G07 (no sessions; result is unsuitable for replacement):

Input (G07_no_sessions.json):


```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
  "id": "cccccccc-0000-4000-8000-000000000007",
  "version": 1,
  "type": "peerToPeer",
  "modalities": [
    "audio"
  ],
  "startDateTime": "2026-10-20T09:00:00.0000000Z",
  "endDateTime": "2026-10-20T09:05:00.0000000Z"
}
```


Expected result (G07_no_sessions.json):


```
{
  "call_id": "cccccccc-0000-4000-8000-000000000007",
  "source_version": 1,
  "is_complete": false,
  "incomplete_reasons": [
    "NO_SESSIONS"
  ],
  "session_set_hash": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "cdrs": [],
  "rejections": []
}
```


Golden sample G08 (missing session end; result is unsuitable for replacement):

Input (G08_session_end_missing.json):


```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
  "id": "cccccccc-0000-4000-8000-000000000008",
  "version": 2,
```

```
"type": "peerToPeer",
"modalities": [
  "audio"
],
"startDateTime": "2026-10-20T15:00:00.0000000Z",
"endDateTime": "2026-10-20T15:05:00.0000000Z",
"sessions": [
  {
    "id": "dddddddd-0000-4000-8000-000000000001",
    "modalities": [
      "audio"
    ],
    "startDateTime": "2026-10-20T15:00:00.0000000Z",
    "caller": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000001",
          "displayName": "User A"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "callee": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000002",
          "displayName": "User B"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
```

```
    "segments": [
      {
        "id": "seg-1",
        "startDateTime": "2026-10-20T15:00:00.0000000Z",
        "endDateTime": null,
        "media": [
          {
            "label": "main-audio",
            "streams": [
              {
                "streamId": "s1",
                "streamDirection": "callerToCallee"
              }
            ]
          }
        ]
      }
    ]
  }
 ]
}
```

Expected result (G08_session_end_missing.json):

```
{
  "call_id": "cccccccc-0000-4000-8000-000000000008",
  "source_version": 2,
  "is_complete": false,
  "incomplete_reasons": [
    "SESSION_DATES_MISSING"
  ],
  "session_set_hash": "2fd1450529819450a1d643ef5e3d3e82bda92ce173a1edd81f69662d4a838053",
  "cdrs": [],
  "rejections": []
}
```

Golden sample G09 (valid and invalid sessions; result is unsuitable for replacement):

Input (G09_invalid_and_valid_session.json):

```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
  "id": "cccccccc-0000-4000-8000-000000000009",
  "version": 6,
```

```
"type": "groupCall",
"modalities": [
  "audio"
],
"startDateTime": "2026-10-20T16:00:00.0000000Z",
"endDateTime": "2026-10-20T16:10:00.0000000Z",
"sessions": [
  {
    "id": "dddddddd-0000-4000-8000-000000000001",
    "modalities": [
      "audio"
    ],
    "startDateTime": "2026-10-20T16:00:00.0000000Z",
    "endDateTime": "2026-10-20T16:05:00.0000000Z",
    "caller": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000001",
          "displayName": "User A"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "callee": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000002",
          "displayName": "User B"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
```

```
    },
    "segments": [
      {
        "id": "seg-1",
        "startDateTime": "2026-10-20T16:00:00.0000000Z",
        "endDateTime": "2026-10-20T16:05:00.0000000Z",
        "media": [
          {
            "label": "main-audio",
            "streams": [
              {
                "streamId": "s1",
                "streamDirection": "callerToCallee"
              }
            ]
          }
        ]
      }
    ]
  },
  {
    "id": "dddddddd-0000-4000-8000-000000000002",
    "modalities": [
      "audio"
    ],
    "startDateTime": "2026-10-20T16:10:00.0000000Z",
    "endDateTime": "2026-10-20T16:09:00.0000000Z",
    "caller": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000003",
          "displayName": "User C"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
```

```
    "callee": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000002",
          "displayName": "User B"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "segments": []
  }
 ]
}
```

Expected result (G09_invalid_and_valid_session.json):

```
{
 "call_id": "cccccccc-0000-4000-8000-000000000009",
 "source_version": 6,
 "is_complete": false,
 "incomplete_reasons": [
  "SESSION_INVALID"
 ],
 "session_set_hash": "740b55f6623a6accc0629a2856934c9ba64456afdd38e3ccc34e15066c4d98dc",
 "cdrs": [
  {
   "schema_version": 1,
   "event_type": "call",
   "tenant_id": "11111111-1111-4111-8111-111111111111",
   "call_id": "cccccccc-0000-4000-8000-000000000009",
   "session_id": "dddddddd-0000-4000-8000-000000000001",
   "source_version": 6,
   "call_type": "group",
   "caller": {
    "type": "user",
    "id": "aaaaaaaa-0000-4000-8000-000000000001"
   },
```

```
    "callee": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000002"
    },
    "start_utc": "2026-10-20T16:00:00.0000000Z",
    "end_utc": "2026-10-20T16:05:00.0000000Z",
    "duration_s": 300,
    "modalities": [
      "audio"
    ],
    "status": "completed",
    "termination_reason": null,
    "segment_count": 1,
    "aggregation_status": "incomplete",
    "source_hash": "7a435bc6b09149a2e4b7932a195690fc587ff12270c1b5da94b64b713ffc66e0",
    "extensions": {
      "segments": [
        {
          "id": "seg-1",
          "start": "2026-10-20T16:00:00.0000000Z",
          "end": "2026-10-20T16:05:00.0000000Z",
          "failure": null
        }
      ]
    }
  }
 ],
 "rejections": [
  {
    "call_id": "cccccccc-0000-4000-8000-000000000009",
    "session_id": "dddddddd-0000-4000-8000-000000000002",
    "code": "SESSION_END_BEFORE_START",
    "message": "Session ends before it starts."
  }
 ]
}
```


Golden sample G10 (+03:00 offset and a half-second):

Input (G10_utc_offset_and_half_second.json):

```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
 "id": "cccccccc-0000-4000-8000-000000000010",
 "version": 1,
 "type": "peerToPeer",
 "modalities": [
  "audio"
 ],
 "startDateTime": "2026-10-20T09:00:00.0000000Z",
 "endDateTime": "2026-10-20T09:05:00.0000000Z",
 "sessions": [
  {
   "id": "dddddddd-0000-4000-8000-000000000001",
   "modalities": [
    "audio"
   ],
   "startDateTime": "2026-10-20T12:00:00.0000000+03:00",
   "endDateTime": "2026-10-20T12:00:30.5000000+03:00",
   "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000001",
      "displayName": "User A"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
    }
   },
   "callee": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000002",
      "displayName": "User B"
     }
    },
    "userAgent": {
```

```
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "segments": [
      {
        "id": "seg-1",
        "startDateTime": "2026-10-20T12:00:00.0000000+03:00",
        "endDateTime": "2026-10-20T12:00:30.5000000+03:00",
        "media": [
          {
            "label": "main-audio",
            "streams": [
              {
                "streamId": "s1",
                "streamDirection": "callerToCallee"
              }
            ]
          }
        ]
      }
    ]
  }
 ]
}
```

Expected result (G10_utc_offset_and_half_second.json):

```
{
  "call_id": "cccccccc-0000-4000-8000-000000000010",
  "source_version": 1,
  "is_complete": true,
  "incomplete_reasons": [],
  "session_set_hash": "052271b811f340d180eb2de4f314003490e99772a41dd067a1fe8a05213ecd6b",
  "cdrs": [
    {
      "schema_version": 1,
      "event_type": "call",
      "tenant_id": "11111111-1111-4111-8111-111111111111",
      "call_id": "cccccccc-0000-4000-8000-000000000010",
      "session_id": "dddddddd-0000-4000-8000-000000000001",
```

```
    "source_version": 1,
    "call_type": "oneToOne",
    "caller": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000001"
    },
    "callee": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000002"
    },
    "start_utc": "2026-10-20T09:00:00.0000000Z",
    "end_utc": "2026-10-20T09:00:30.5000000Z",
    "duration_s": 31,
    "modalities": [
      "audio"
    ],
    "status": "completed",
    "termination_reason": null,
    "segment_count": 1,
    "aggregation_status": "complete",
    "source_hash": "30024d86434689e9d64499e37e2db461af77763074d0b8468ade2973ab0479c5",
    "extensions": {
      "segments": [
        {
          "id": "seg-1",
          "start": "2026-10-20T12:00:00.0000000+03:00",
          "end": "2026-10-20T12:00:30.5000000+03:00",
          "failure": null
        }
      ]
    }
  }
 ],
 "rejections": []
}
```


Golden sample G11 (G01 record with version 5 and changed content):

Input (G11_version_higher_content_changed.json):


```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
```

```
"id": "cccccccc-0000-4000-8000-000000000001",
"version": 5,
"type": "peerToPeer",
"modalities": [
  "audio"
],
"startDateTime": "2026-10-20T09:00:00.0000000Z",
"endDateTime": "2026-10-20T09:05:00.0000000Z",
"sessions": [
  {
    "id": "dddddddd-0000-4000-8000-000000000001",
    "modalities": [
      "audio"
    ],
    "startDateTime": "2026-10-20T09:00:00.0000000Z",
    "endDateTime": "2026-10-20T09:01:00.5000000Z",
    "caller": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000001",
          "displayName": "User A"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "callee": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000002",
          "displayName": "User B"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
```

```
        "productFamily": "teams"
      }
    },
    "segments": [
      {
        "id": "seg-1",
        "startDateTime": "2026-10-20T09:00:00.0000000Z",
        "endDateTime": "2026-10-20T09:01:00.5000000Z",
        "media": [
          {
            "label": "main-audio-2",
            "streams": [
              {
                "streamId": "s1",
                "streamDirection": "callerToCallee"
              }
            ]
          }
        ]
      }
    ]
  }
 ]
}
```

Expected result (G11_version_higher_content_changed.json):

```
{
 "call_id": "cccccccc-0000-4000-8000-000000000001",
 "source_version": 5,
 "is_complete": true,
 "incomplete_reasons": [],
 "session_set_hash": "3d0a0015d9fc1618734c1022f0c4ccb513b4f1e6b2a6aa45816edb18a4581360",
 "cdrs": [
  {
    "schema_version": 1,
    "event_type": "call",
    "tenant_id": "11111111-1111-4111-8111-111111111111",
    "call_id": "cccccccc-0000-4000-8000-000000000001",
    "session_id": "dddddddd-0000-4000-8000-000000000001",
    "source_version": 5,
    "call_type": "oneToOne",
```

```
    "caller": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000001"
    },
    "callee": {
      "type": "user",
      "id": "aaaaaaaa-0000-4000-8000-000000000002"
    },
    "start_utc": "2026-10-20T09:00:00.0000000Z",
    "end_utc": "2026-10-20T09:01:00.5000000Z",
    "duration_s": 61,
    "modalities": [
      "audio"
    ],
    "status": "completed",
    "termination_reason": null,
    "segment_count": 1,
    "aggregation_status": "complete",
    "source_hash": "2acda2272c8fe133b6fb51a37b2b9f26c7f64b0555d8bdd81a83f13ab8c61175",
    "extensions": {
      "segments": [
        {
          "id": "seg-1",
          "start": "2026-10-20T09:00:00.0000000Z",
          "end": "2026-10-20T09:01:00.5000000Z",
          "failure": null
        }
      ]
    }
  }
 ],
 "rejections": []
}
```


3.7. Core Automated Tests

Test infrastructure. Project TeamsCallRecords.Core.Tests, net8.0; packages: xunit 2.9.3, xunit.runner.visualstudio 3.1.4, Microsoft.NET.Test.Sdk 17.14.1, coverlet.collector 6.0.4; implicit Using Xunit. Execute: dotnet test TeamsCallRecords.sln -c Release. Tests do not access the network, clock, or file system except for reading golden samples from TestData/Golden (files are copied to the output directory: <None Update="TestData\Golden***.json" CopyToOutputDirectory="PreserveNewest" />).

Samples (Samples.cs, internal static): constant Tenant = "tenant-test"; Session(id, start = "2026-01-01T10:00:00Z", end = "2026-01-01T10:01:00Z", caller = "user-a", callee = "user-b", failure = null, segments = "") — session JSON (failure as stage:reason; an empty caller/callee string omits that party; end = null produces null); Segment(id, start, end) — segment JSON with a single callerToCallee stream; Record(sessions, version = 3, type = "peerToPeer", id = "call-1") — call record JSON. The builder belongs to the test project and is not included in production code.

Legend. “Input” denotes Samples arguments; “Expected” denotes assertions; K denotes a TRS acceptance criterion.

3.7.1. ConverterTests (ConverterTests.cs, 18 tests from I1)

Method	Input	Expected	K
OneToOne_SingleSession_ProducesOneCdr	Record(Session("s1"))	IsComplete; 1 CDR; CallId="call-1", SessionId="s1", TenantId=Tenant, SourceVersion=3, CallType=OneToOne, Caller=(user,user-a), Callee=user-b, DurationSeconds=60, Completed, Complete, SegmentCount=1, SourceHash length 64	K8
OneToOne_CanHaveSeveralSessions_CountFollowsData	two sessions, peerToPeer	2 CDR	K8
GroupCall_OneCdrPerSession	three sessions, groupCall	3 CDR, CallType=Group	K8
OverlappingSegments_DoNotChangeDuration_AndAreCounted	segments 10:00:00–10:00:40 and 10:00:20–10:01:00	1 CDR; SegmentCount=2; DurationSeconds=60; ExtensionsJson contains "segments"	K8
MissingCallerAndCallee_AreNotRejected	caller="", callee="", groupCall	IsComplete; 1 CDR; Caller=null, Callee=null	K8
FailureInfo_GivesFailedWithReason	failure="callSetup:busy"	Failed; TerminationReason="callSetup:busy"	K8
ShortCallWithoutFailureInfo_IsNotFailed	end 2 seconds after start, no failure	Completed; TerminationReason=null	K8
NoSessions_IsIncomplete_AndCreatesNoCdr	{"id":"call-1","version":2,"type":"peerToPeer"}	not IsComplete; 0 CDR; reason NO_SESSIONS	K8
EmptySessionsArray_IsIncomplete	Record("")	not IsComplete; 0 CDR	K8
MissingEndDate_IsIncomplete_NoZeroDurationInvented	end=null	not IsComplete; 0 CDR; 0 rejections; reason SESSION_DATES_MISSING	K8
IncompleteSet_MarksProducedCdrsIncomplete	s1 is valid, s2 has no end	not IsComplete; 1 CDR marked Incomplete	K8
EndBeforeStart_IsRejected	end precedes start	1 SESSION_END_BEFORE_START rejection; not IsComplete	K14
UnparsableDate_IsRejected	start="not-a-date"	SESSION_BAD_DATE	K14
SessionWithoutId_IsRejected	session without id	SESSION_NO_ID	K14
BrokenJson_ReturnsRejection_NoException	{ this is not json	CALLRECORD_BAD_JSON; 0 CDR; no exception	K14

Method	Input	Expected	K
MissingVersion_IsRejected	{"id":"call-1","sessions":[]}	CALLRECORD_NO_VERSION	K14
ValidSessionSurvives_WhenAnotherIsInvalid	s1 valid, s2 has start="bad"	1 CDR; 1 rejection; not IsComplete	K14
Time_ZoneOffsets_AreNormalizedToUtc	12:00:00+02:00	StartUtc = 10:00:00Z, offset 0	K13

3.7.2. RulesTests (RulesTests.cs, 14 tests from I1)

Method	Input	Expected	K
Duration_RoundsHalfUp (7 InlineData rows)	ticks: 0; 4 990 000; 5 000 000; 5 010 000; 14 999 999; 15 000 000; 600 000 000	seconds: 0; 0; 1; 1; 1; 2; 60	K13
Timestamps_KeepSourcePrecision	start 10:00:00.1234567Z, end 10:00:01.6234567Z	StartUtc = T0 + 1 234 567 ticks; DurationSeconds=2	K13
FirstTime_Applies	Decide(null,null,1,"h")	Apply	K11
HigherVersion_Applies	Decide(2,"h1",3,"h2")	Apply	K11
LowerVersion_IsIgnored	Decide(3,"h1",2,"h2")	IgnoreLower	K11
SameVersion_SameHash_IsDuplicate	Decide(3,"h1",3,"h1")	DuplicateSame	K11
SameVersion_OtherHash_IsDiscrepancy	Decide(3,"h1",3,"h2")	Discrepancy	K11
ReprocessingSameRecord_GivesIdenticalResult	one two-session record converted twice	same SessionSetHash and SourceHash; Decide over both results = DuplicateSame	K11

3.7.3. HashTests (HashTests.cs, 10 tests; in I1 this class is in RulesTests.cs and is moved to a separate file)

Method	Expected
PropertyOrder_DoesNotChangeHash	{"a":1,"b":{"x":1,"y":2}} and {"b":{"y":2,"x":1},"a":1} produce the same hash
Whitespace_DoesNotChangeHash	whitespace and line breaks do not affect the hash
ArrayOrder_ChangesHash	[1,2] and [2,1] are different
TransportAnnotations_AreExcluded	@odata.context, segments@odata.context, b@odata.nextLink do not affect the hash
ODataType_IsContent_AndChangesHash	adding @odata.type changes the hash
MediaChange_ChangesSessionHash	changing callerToCallee to calleeToCaller changes SourceHash
FailureInfoChange_ChangesSessionHash	callSetup:busy and callSetup:noAnswer are different
NumberPrecision_IsKept	1.10 and 1.1 are different

Method	Expected
SessionSetHash_CoversIds	sets {s1} and {s1,s2} are different
SessionSetHash_IgnoresSessionOrder	session order does not affect the result

3.7.4. New Tests (Bring I1 into Conformance with the Design)

GoldenTests.cs (11 cases):

Method	Input	Expected	K
Convert_MatchesGolden(string name), [Theory] + [MemberData] over in/G01… G11 files	contents of in/<name>.json; tenant 11111111-1111-4111-8111-111111111111	CdrJson.ToJson(result) structurally equals out/<name>.json (JsonNode.DeepEquals)	K8, K13, K14
Golden_Reprocessing_IsIdempotent	in/G01 twice	Decide(4, set_hash, 4, set_hash) = DuplicateSame	K11
Golden_HigherVersion_Applies_AndHashes Differ	in/G01 (version 4) and in/G11 (version 5)	Decide(4, hashG01, 5, hashG11) = Apply; source_hash and session_set_hash differ	K11
Golden_LowerVersion_IsIgnored	the same	Decide(5, hashG11, 4, hashG01) = IgnoreLower	K11
Golden_SameVersion_OtherContent_IsDiscr epancy	in/G01 and a copy with version 4 and a changed stream	Discrepancy	K11

DuplicateSessionTests.cs (2 tests):

Method	Input	Expected	K
DuplicateSessionId_RejectsAllOccurre nces	Record(Session("s1") + "," + Session("s1") + "," + Session("s2"))	2 SESSION_DUPLICATE_ID rejections with SessionId="s1"; 1 CDR (s2); not IsComplete; reason SESSION_INVALID	K14
DuplicateSessionId_DoesNotAffectOth erCall	record without duplicates	0 rejections	K14

IncompleteReasonsTests.cs (2 tests): Transient_ReasonsAreNoSessionsAndDatesMissing — IsTransient is true for NO_SESSIONS and SESSION_DATES_MISSING; Permanent_ReasonsAreNotTransient — false for SESSION_INVALID and all RejectionCodes values.

CdrJsonTests.cs (3 tests): FormatTime_HasSevenFractionDigitsAndZ — FormatTime(2026-10-20T09:00:00.5Z) = 2026-10-20T09:00:00.5000000Z; ToJson_NullParties_AreJsonNull — in G05, caller and callee are JsonValueKind.Null; ToJsonWithSeq_PutsCdrSeqFirst — cdr_seq is the first object property.

Total Core tests: 18 + 14 + 10 + 11 + 4 + 2 + 2 + 3 = 64 (42 from I1, 22 new). Counting test methods while accounting for InlineData and MemberData cases: expected result on execution is 64 passed, 0 skipped.

3.7.5. Requirements Coverage

Requirement	Tests
K8	ConverterTests (1–11), GoldenTests G01–G03, G05, G06

Requirement	Tests
K11	RulesTests (versions), GoldenTests.Golden_*; Part 2: database replacement
K13	RulesTests.Duration_RoundsHalfUp, Timestamps_KeepSourcePrecision, Time_ZoneOffsets_AreNormalizedToUtc, G10
K14	ConverterTests (12–17), DuplicateSessionTests, G07–G09; Part 2: rejected-record register
F6-4 (hashes)	HashTests

3.8. Aligning the Prepared I1 Module with the Design

The prepared I1 module is used as a preliminary basis for implementing Core. Its 42 tests were run only on .NET 10. Those results do not demonstrate that the target application works on .NET 8. Mandatory tests are run on .NET 8 after implementation. To align with the design, the following changes, and only these changes, are made:

№	File	Change
1	Models.cs	add RejectionCodes.SessionDuplicateId; add IncompleteReasons class (constants and IsTransient)
2	CallRecordConverter.cs	ArgumentNullException.ThrowIfNull for both arguments; replace literal reason strings with IncompleteReasons constants; add steps 7 and 8b (duplicate session identifiers)
3	CdrJson.cs	new file per 3.4.4
4	Core.Tests	move HashTests into HashTests.cs; add GoldenTests.cs, DuplicateSessionTests.cs, IncompleteReasonsTests.cs, CdrJsonTests.cs, TestData/Golden directory from 3.6; configure sample copying in .csproj
5	.github/workflows/ci.yml	dotnet-version: 8.0.x instead of 10.0.x
6	root	add global.json, Directory.Build.props, Directory.Packages.props (Part 4)

The remainder of the I1 code is unchanged. After these changes, the expected test outcome is 64 passed; this may only be confirmed by running dotnet test with the .NET 8 SDK after implementation, and that run is mandatory. Implementation may start only after the author approves the detailed design.

3.9. Changes to the Part 1 Register

Infrastructure design identified an inconsistency in the Part 1 file tree (3.2): option classes InboxOptions, ProcessingOptions, ReconcileOptions, OutboxOptions, and StorageOptions were placed in Host, although they are used by Infrastructure services; Infrastructure cannot reference Host (dependency direction: Host → Infrastructure → Core). Correction [P-10]:

№	Change	Rationale
1	Remove src/TeamsCallRecords.Host/Config/InboxOptions.cs, ProcessingOptions.cs, ReconcileOptions.cs, OutboxOptions.cs, StorageOptions.cs from Host and create them under src/TeamsCallRecords.Infrastructure/Config/	dependency direction
2	Only BotOptions.cs, ApiOptions.cs, and WebhookOptions.cs stay in Host/Config	used only by Host

№	Change	Rationale
3	Reconcile:Interval and Reconcile:Lookback keys from TRP v0.1 become Reconcile:IntervalMinutes and Reconcile:LookbackHours	units encoded in key names
4	Add SubscriptionOptions (Subscriptions:*), IWriteGate, IReconciler, Reconciler, IOutboxSender, OutboxSender, ICdrReader, CdrReader, model files, and others as listed below	Algorithms in 3.11–3.17 need separate modules; reconciliation, sending, and CDR reading must be testable without Host
5	Processing:Workers key and defaults remain as in TRP v0.1; Processing:MaxAttempts is added	see 3.13.3

Amendment A1 to Part 1 DDL (3.5). The Part 2 review found that making element_hash unique across the entire inbox table prevents a repeated notification from causing a fresh Graph read after an earlier item reaches done (3.12.3). The rule changes: only an item in state new counts as a duplicate [P-22]. Only the inbox table is affected; all other tables, Core, and golden samples G01–G11 remain unchanged. The original Part 1 fragment (before amendment):

```
    last_error      TEXT    NULL,
    CONSTRAINT uq_inbox_hash UNIQUE (element_hash)
);
CREATE INDEX ix_inbox_due ON inbox (state, next_attempt_utc);
```

Fragment after the amendment (final inbox table lines and indexes; all other table columns and constraints remain as in Part 1):

```
    last_error      TEXT    NULL
);
CREATE UNIQUE INDEX uq_inbox_new_hash ON inbox (element_hash) WHERE state = 'new';
CREATE INDEX ix_inbox_hash ON inbox (element_hash);
CREATE INDEX ix_inbox_call ON inbox (tenant_id, call_id, state);
CREATE INDEX ix_inbox_due ON inbox (state, next_attempt_utc);
```

The amendment is applied to InitialCreate.sql before the first deployment, so no separate migration is needed. Part 1 amendments consist of A1 and lines 1–5 of the table above; there are no other Part 1 changes. Changes to Part 2 itself relative to v0.1 are listed in Appendix D.

Complete file inventory for Parts 1–2 covering Infrastructure and its tests. Each file occupies one row with its repository-relative path. Column “Origin”: “from Part 1” means that the name and purpose were fixed in Part 1, 3.2, with contents specified here; “moved from Host” means the file previously belonged to Host; “new” means added in Part 2.

Path	Purpose	Origin
src/TeamsCallRecords.Infrastructure/TeamsCallRecords.Infrastructure.csproj	Infrastructure project: Core reference and packages in 3.10	from Part 1
src/TeamsCallRecords.Infrastructure/InfrastructureRegistration.cs	InfrastructureRegistration: service registration, option validation, startup migration (3.18)	new
src/TeamsCallRecords.Infrastructure/Config/StorageOptions.cs	StorageOptions (Storage:*)	moved from Host
src/TeamsCallRecords.Infrastructure/Config/InboxOptions.cs	InboxOptions (Inbox:*)	moved from Host
src/TeamsCallRecords.Infrastructure/Config/ProcessingOptions.cs	ProcessingOptions (Processing:*)	moved from Host

Path	Purpose	Origin
src/TeamsCallRecords.Infrastructure/Config/ReconcileOptions.cs	ReconcileOptions (Reconcile:*)	moved from Host
src/TeamsCallRecords.Infrastructure/Config/OutboxOptions.cs	OutboxOptions (Outbox:*)	moved from Host
src/TeamsCallRecords.Infrastructure/Data/TcriDbContext.cs	TcriDbContext: table mapping	from Part 1
src/TeamsCallRecords.Infrastructure/Data/TcriDbContextFactory.cs	TcriDbContextFactory: design-time factory for dotnet ef	new
src/TeamsCallRecords.Infrastructure/Data/IWriteGate.cs	IWriteGate: write serialization contract	new
src/TeamsCallRecords.Infrastructure/Data/WriteGate.cs	WriteGate: implementation	from Part 1
src/TeamsCallRecords.Infrastructure/Data/SqlitePragmaInterceptor.cs	SqlitePragmaInterceptor	from Part 1
src/TeamsCallRecords.Infrastructure/Data/UtcText.cs	UtcText	from Part 1
src/TeamsCallRecords.Infrastructure/Data/CdrMapping.cs	CdrMapping	from Part 1
src/TeamsCallRecords.Infrastructure/Data/ICdrReader.cs	ICdrReader, CdrItem, CdrPage, DailyStats	new
src/TeamsCallRecords.Infrastructure/Data/CdrReader.cs	CdrReader: reading CDRs for REST and bot	new
src/TeamsCallRecords.Infrastructure/Data/Cursor.cs	Cursor: page cursor encoding and decoding	new
src/TeamsCallRecords.Infrastructure/Data/Entities/States.cs	state/kind value constants: InboxKinds, InboxStates, SourceStates, OutboxStates, SubscriptionKinds, SubscriptionStatuses, AuditKinds, Components	new
src/TeamsCallRecords.Infrastructure/Data/Entities/CdrRow.cs	cdr row	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Entities/SourceStateRow.cs	source_state row	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Entities/InboxRow.cs	inbox row	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Entities/RejectedRow.cs	rejected row	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Entities/AuditLogRow.cs	audit_log row	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Entities/OutboxRow.cs	outbox row	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Entities/SubscriptionRow.cs	subscription row	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Entities/ErrorLogRow.cs	error_log row	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Migrations/InitialCreate.sql	schema text from 3.5 as embedded resource	new
src/TeamsCallRecords.Infrastructure/Data/Migrations/20261021000000_InitialCreate.cs	migration executes InitialCreate.sql	from Part 1

Path	Purpose	Origin
src/TeamsCallRecords.Infrastructure/Data/Migrations/20261021000000_InitialCreate.Designer.cs	generated by dotnet ef (3.11.6)	from Part 1
src/TeamsCallRecords.Infrastructure/Data/Migrations/TcriDbContextModelSnapshot.cs	generated by dotnet ef (3.11.6)	from Part 1
src/TeamsCallRecords.Infrastructure/Queue/IInboxStore.cs	IInboxStore	from Part 1
src/TeamsCallRecords.Infrastructure/Queue/InboxStore.cs	InboxStore	from Part 1
src/TeamsCallRecords.Infrastructure/Queue/QueueModels.cs	InboxNewItem, InboxEnqueueResult, InboxItem, InboxStats	new
src/TeamsCallRecords.Infrastructure/Queue/InboxTransitions.cs	internal inbox item transitions within a transaction	new
src/TeamsCallRecords.Infrastructure/Processing/ISourceStateStore.cs	ISourceStateStore	from Part 1
src/TeamsCallRecords.Infrastructure/Processing/SourceStateStore.cs	SourceStateStore	from Part 1
src/TeamsCallRecords.Infrastructure/Processing/ICdrApplier.cs	ICdrApplier	from Part 1
src/TeamsCallRecords.Infrastructure/Processing/CdrApplier.cs	CdrApplier	from Part 1
src/TeamsCallRecords.Infrastructure/Processing/ICallProcessor.cs	ICallProcessor	from Part 1
src/TeamsCallRecords.Infrastructure/Processing/CallProcessor.cs	CallProcessor	from Part 1
src/TeamsCallRecords.Infrastructure/Processing/IReconciler.cs	IReconciler	new
src/TeamsCallRecords.Infrastructure/Processing/Reconciler.cs	Reconciler: reconciliation and rechecking suspended calls	new
src/TeamsCallRecords.Infrastructure/Processing/ProcessingModels.cs	ProcessOutcome, ApplyRequest, ApplyStatus, ApplyResult, PendingResult, SourceStateCounts	new
src/TeamsCallRecords.Infrastructure/Outbox/IOutboxStore.cs	IOutboxStore	from Part 1
src/TeamsCallRecords.Infrastructure/Outbox/OutboxStore.cs	OutboxStore	from Part 1
src/TeamsCallRecords.Infrastructure/Outbox/IOutboxSender.cs	IOutboxSender	new
src/TeamsCallRecords.Infrastructure/Outbox/OutboxSender.cs	OutboxSender	new
src/TeamsCallRecords.Infrastructure/Outbox/OutboxModels.cs	OutboxItem, OutboxStats, ActivityPayload	new
src/TeamsCallRecords.Infrastructure/Graph/IGraphClient.cs	IGraphClient	from Part 1
src/TeamsCallRecords.Infrastructure/Graph/GraphClient.cs	GraphClient	from Part 1
src/TeamsCallRecords.Infrastructure/Graph/GraphOptions.cs	GraphOptions (Graph:*)	from Part 1

Path	Purpose	Origin
src/TeamsCallRecords.Infrastructure/Graph/GraphModels.cs	GraphFetchStatus, GraphFetchResult, GraphCallRecordRef, GraphSubscriptionRequest, GraphSubscription, ActivityNotification	new
src/TeamsCallRecords.Infrastructure/Graph/GraphException.cs	GraphException	new
src/TeamsCallRecords.Infrastructure/Subscriptions/ISubscriptionStore.cs	ISubscriptionStore	from Part 1
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionStore.cs	SubscriptionStore	from Part 1
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionService.cs	SubscriptionService	from Part 1
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionOptions.cs	SubscriptionOptions (Subscriptions:*)	new
src/TeamsCallRecords.Infrastructure/Diagnostics/IErrorLog.cs	IErrorLog	from Part 1
src/TeamsCallRecords.Infrastructure/Diagnostics/ErrorLog.cs	ErrorLog	from Part 1
tests/TeamsCallRecords.Infrastructure.Tests/ TeamsCallRecords.Infrastructure.Tests.csproj	test project	new
tests/TeamsCallRecords.Infrastructure.Tests/Support/TestDb.cs	TestDb: temporary database, factory, WriteGate, clock	new
tests/TeamsCallRecords.Infrastructure.Tests/Support/FakeGraphClient.cs	FakeGraphClient: programmable IGraphClient fake	new
tests/TeamsCallRecords.Infrastructure.Tests/Support/FakeCredential.cs	FakeCredential: offline token	new
tests/TeamsCallRecords.Infrastructure.Tests/Support/FakeReconciler.cs	FakeReconciler: IReconciler substitute for CallProcessorTests	new
tests/TeamsCallRecords.Infrastructure.Tests/Support/FakeCdrApplier.cs	FakeCdrApplier: ICdrApplier substitute with predefined results	new
tests/TeamsCallRecords.Infrastructure.Tests/Support/StubHandler.cs	StubHandler: scripted HTTP responses	new
tests/TeamsCallRecords.Infrastructure.Tests/Support/Golden.cs	Golden: reads G01–G11 and derived samples	new
tests/TeamsCallRecords.Infrastructure.Tests/Support/Builders.cs	builders for options, queue items, and records	new
tests/TeamsCallRecords.Infrastructure.Tests/SchemaTests.cs	schema and connection	new
tests/TeamsCallRecords.Infrastructure.Tests/CdrMappingTests.cs	mapping	new
tests/TeamsCallRecords.Infrastructure.Tests/UtcTextTests.cs	timestamp format	new
tests/TeamsCallRecords.Infrastructure.Tests/WriteGateTests.cs	serialized writes	new
tests/TeamsCallRecords.Infrastructure.Tests/InboxStoreTests.cs	inbox queue	new
tests/TeamsCallRecords.Infrastructure.Tests/SourceStateStoreTests.cs	call state	new

Path	Purpose	Origin
tests/TeamsCallRecords.Infrastructure.Tests/CdrApplierTests.cs	replacement transaction	new
tests/TeamsCallRecords.Infrastructure.Tests/CallProcessorTests.cs	item processing	new
tests/TeamsCallRecords.Infrastructure.Tests/ReconcilerTests.cs	reconciliation	new
tests/TeamsCallRecords.Infrastructure.Tests/OutboxStoreTests.cs	outbox queue	new
tests/TeamsCallRecords.Infrastructure.Tests/OutboxSenderTests.cs	sending	new
tests/TeamsCallRecords.Infrastructure.Tests/GraphClientTests.cs	Graph client	new
tests/TeamsCallRecords.Infrastructure.Tests/GraphResilienceTests.cs	retries and circuit breaker	new
tests/TeamsCallRecords.Infrastructure.Tests/SubscriptionStoreTests.cs	subscription store	new
tests/TeamsCallRecords.Infrastructure.Tests/SubscriptionServiceTests.cs	subscriptions	new
tests/TeamsCallRecords.Infrastructure.Tests/CdrReaderTests.cs	CDR reading	new
tests/TeamsCallRecords.Infrastructure.Tests/CursorTests.cs	page cursor	new
tests/TeamsCallRecords.Infrastructure.Tests/ErrorLogTests.cs	error log	new
tests/TeamsCallRecords.Infrastructure.Tests/OptionsTests.cs	options	new
tests/TeamsCallRecords.Infrastructure.Tests/ScenarioTests.cs	S01–S15 scenarios (3.19)	new
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/D01_same_version_other_content.json	derived golden sample (input), Appendix B	new
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/out/D01_same_version_other_content.json	derived golden sample (expected Core output), Appendix B	new
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/D02_group_one_session_v8.json	derived golden sample (input), Appendix B	new
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/out/D02_group_one_session_v8.json	derived golden sample (expected Core output), Appendix B	new
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/D03_session_end_arrived_v3.json	derived golden sample (input), Appendix B	new
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/out/D03_session_end_arrived_v3.json	derived golden sample (expected Core output), Appendix B	new
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/D04_older_version_incomplete.json	derived golden sample (input), Appendix B	new

Path	Purpose	Origin
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/out/D04_older_version_incomplete.json	derived golden sample (expected Core output), Appendix B	new

Infrastructure test fixtures are not duplicated: the test project includes tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/*.json and out/*.json by linked files (<None Include="..\TeamsCallRecords.Core.Tests\TestData\Golden***.json" Link="TestData\Golden\%(RecursiveDir)%(Filename)%(Extension)" CopyToOutputDirectory="PreserveNewest" />), so the fixtures have one storage location.

3.10. General Infrastructure Provisions

3.10.1. Project and Packages. TeamsCallRecords.Infrastructure.csproj: Sdk="Microsoft.NET.Sdk", net8.0, reference to the Core project. Packages (package identifier — version line; exact versions are pinned in Directory.Packages.props, Part 4):

Package	Version Line	Purpose
Microsoft.EntityFrameworkCore.Sqlite	8.0.*	SQLite access
Microsoft.EntityFrameworkCore.Design	8.0.*, PrivateAssets=all	dotnet ef
Azure.Identity	1.*	ClientSecretCredential (F1)
Microsoft.Extensions.Http.Resilience	8.*	retries and circuit breaker
Microsoft.Extensions.Options.ConfigurationExtensions	8.0.*	options binding
Microsoft.Extensions.Hosting.Abstractions	8.0.*	registration interfaces

Test project TeamsCallRecords.Infrastructure.Tests.csproj: net8.0, references to Core and Infrastructure; packages xunit 2.9.3, xunit.runner.visualstudio 3.1.4, Microsoft.NET.Test.Sdk 17.14.1, coverlet.collector 6.0.4, Microsoft.Extensions.TimeProvider.Testing 8., Microsoft.Extensions.Diagnostics.Testing 8. (FakeLogger).

3.10.2. Rules Common to All Classes.

Rule	Contents
Time	TimeProvider is the sole time source (constructor-injected); DateTimeOffset.UtcNow and DateTime.Now are prohibited; time is stored in tables as UtcText.Format(...) strings; time comparisons are performed in SQL query text (FromSqlInterpolated) because string ordering matches UTC chronology
Asynchrony	all database and network operations return Task and take CancellationToken ct as the final argument; blocking .Result/.Wait() calls are prohibited
Writes	all database writes execute within IWriteGate.RunAsync; reads use a separate IDbContextFactory<TcriDbContext> context with AsNoTracking()
Cancellation	OperationCanceledException is neither caught nor written to error_log
Errors	A “record-level error” (exception while processing one item) is caught in CallProcessor/OutboxSender, written to error_log and ILogger, and the service continues (F9, K9); a “process-level error” (database inaccessible at startup) is an exception from InfrastructureRegistration.MigrateAsync, and the process stops.

Rule	Contents
Logging	ILogger<T>; BeginScope includes component, tenant_id, call_id, session_id, inbox_id wherever known; never log secrets (ClientSecret, ClientState, Api:Key, tokens), call record bodies, or notification bodies in ILogger or error_log
Strings	Identifiers are compared using StringComparison.Ordinal; tenant_id and call_id are stored unchanged from the source
Message Size	Truncate error_log.message and rejected.message to 1000 characters, error_log.details and last_error to 4000

3.10.3. Dependencies Between Units (Arrow Means “Uses”).

CallProcessor -> IGraphClient, IInboxStore, ISourceStateStore, ICdrApplier, IReconciler, IErrorLog, CallRecordConverter (Core), VersionRules (Core)
CdrApplier -> IWriteGate, CdrMapping, InboxTransitions, VersionRules (Core)
SourceStateStore -> IWriteGate, IDbContextFactory, InboxTransitions, ProcessingOptions
Reconciler -> IGraphClient, ISourceStateStore, IInboxStore, GraphOptions, ReconcileOptions
OutboxSender -> IOutboxStore, IGraphClient, IErrorLog
SubscriptionService -> IGraphClient, ISubscriptionStore, IInboxStore, IErrorLog
InboxStore, OutboxStore, SubscriptionStore, ErrorLog, CdrReader -> IWriteGate / IDbContextFactory
GraphClient -> HttpClient, TokenCredential, GraphOptions

There are no circular dependencies; CallProcessor and Reconciler have a one-way relationship (the processor invokes reconciliation for an item of kind reconcile without call_id; the reconciler does not invoke the processor).

3.11. Data: Context, Connection, Writing, Migration, and Reading

3.11.1. State Constants (Data/Entities/States.cs)

Static classes of string constants; their values match the CHECK constraints in 3.5. String literals for these values are prohibited elsewhere.

Class	Constant = Value
InboxKinds	CallRecord = "callRecord", ChannelMessage = "channelMessage", Reconcile = "reconcile"
InboxStates	New = "new", Processing = "processing", Done = "done", Failed = "failed"
SourceStates	Pending = "pending", Done = "done", Suspended = "suspended", Expired = "expired"
OutboxStates	New = "new", Sending = "sending", Sent = "sent", Failed = "failed"
OutboxKinds	Activity = "activity"
SubscriptionKinds	CallRecords = "callRecords", ChannelMessages = "channelMessages"
SubscriptionStatuses	Active = "active", Renewing = "renewing", Failed = "failed"
AuditKinds	VersionDiscrepancy = "version_discrepancy"
Components	Inbox = "inbox", Processor = "processor", Reconciler = "reconciler", Outbox = "outbox", Subscriptions = "subscriptions", Graph = "graph", Startup = "startup"

CdrRow.CallType, Status, and AggregationStatus values are assigned by CdrMapping (Part 1, 3.5).

3.11.2. TcriDbContext (Data/TcriDbContext.cs)

public sealed class TcriDbContext : DbContext, constructor TcriDbContext(DbContextOptions<TcriDbContext> options).

Property	Type	Table
Cdrs	DbSet<CdrRow>	cdr
SourceStates	DbSet<SourceStateRow>	source_state
InboxItems	DbSet<InboxRow>	inbox
RejectedItems	DbSet<RejectedRow>	rejected
AuditLog	DbSet<AuditLogRow>	audit_log
OutboxItems	DbSet<OutboxRow>	outbox
Subscriptions	DbSet<SubscriptionRow>	subscription
ErrorLog	DbSet<ErrorLogRow>	error_log

OnModelCreating(ModelBuilder b) — deterministic mapping without attributes on row classes:

1. For each entity type, the table name is derived by removing the Row suffix from the class name and converting to snake_case (SourceStateRow → source_state); each column name is the corresponding property name in snake_case (SourceVersion → source_version). The snake_case rule inserts _ before each uppercase letter except the first and lowercases everything; property names do not use abbreviations.
2. Primary keys: CdrRow.CdrSeq, InboxRow.Id, RejectedRow.Id, AuditLogRow.Id, OutboxRow.Id, ErrorLogRow.Id — ValueGeneratedOnAdd; SourceStateRow — composite (TenantId, CallId); SubscriptionRow.Id — a string without generation.
3. Indexes and unique constraints use the DDL names: uq_cdr_key (TenantId, CallId, SessionId), ix_cdr_start (StartUtc, CdrSeq), ix_cdr_call (TenantId, CallId), ix_source_state_due (State, NextAttemptUtc), uq_inbox_new_hash (ElementHash, unique, filtered on state = 'new': HasFilter("state = 'new'")), ix_inbox_hash (ElementHash), ix_inbox_call (TenantId, CallId, State), ix_inbox_due (State, NextAttemptUtc), ix_rejected_created (CreatedUtc), uq_outbox_key (Kind, TenantId, CallId, SessionId, Recipient), ix_outbox_due (State, NextAttemptUtc), ix_error_log_ts (Ts).
4. CHECK constraints are not configured in the model: only InitialCreate.sql creates them.
5. Properties permitting NULL are declared string? / long? / int?; all other properties do not permit NULL.

Postcondition: the schema created by migration and the model mapping are consistent; SchemaTests (3.20) verifies this.

3.11.3. SqlitePragmaInterceptor (Data/SqlitePragmaInterceptor.cs)

public sealed class SqlitePragmaInterceptor : DbConnectionInterceptor. Overrides: ConnectionOpened(DbConnection, ConnectionEndEventData) and ConnectionOpenedAsync(DbConnection, ConnectionEndEventData, CancellationToken). Action: execute the following command on the open connection:

PRAGMA journal_mode=WAL; PRAGMA synchronous=FULL; PRAGMA foreign_keys=ON; PRAGMA busy_timeout=5000;

ExecuteNonQuery; ignore the result. Precondition: the SQLite connection is open. Postcondition: the connection operates in WAL mode with synchronous=FULL [P-08]. Execution errors propagate as exceptions (the connection fails to open). The interceptor is a singleton registered in AddDbContextFactory (3.18).

3.11.4. IWriteGate and WriteGate (Data/IWriteGate.cs, Data/WriteGate.cs)

public interface IWriteGate
{
  Task<T> RunAsync<T>(Func<TcriDbContext, CancellationToken, Task<T>> work, CancellationToken ct);
  Task RunAsync(Func<TcriDbContext, CancellationToken, Task> work, CancellationToken ct);
}

public sealed class WriteGate : IWriteGate, constructor WriteGate(IDbContextFactory<TcriDbContext> factory); singleton.

RunAsync algorithm (single transaction):

1. await _semaphore.WaitAsync(ct) (SemaphoreSlim(1, 1)). Cancellation while waiting raises OperationCanceledException and changes nothing.
2. await using var db = await factory.CreateDbContextAsync(ct).
3. await using var tx = await db.Database.BeginTransactionAsync(ct).
4. var result = await work(db, ct). The caller invokes db.SaveChangesAsync(ct) itself one or more times; SaveChanges inside the gate participates in the enclosing transaction and does not commit it.
5. await tx.CommitAsync(ct); return result.
6. Any exception during steps 4–5: await tx.RollbackAsync(CancellationToken.None), then rethrow the original exception unchanged.
7. finally: _semaphore.Release() — always, including on cancellation and failure.

Invariants: at most one writing transaction executes in the process at a time; one transaction uses one context; work must not reenter the gate (calling RunAsync recursively from work would deadlock and is prohibited and checked in review). Readers using IDbContextFactory without the gate run concurrently thanks to WAL mode. The gate enforces exclusive writing, so a regular transaction is used instead of BEGIN IMMEDIATE: the process is the only writer and no second writing connection exists [P-09].

3.11.5. TcriDbContextFactory (Data/TcriDbContextFactory.cs)

public sealed class TcriDbContextFactory : IDesignTimeDbContextFactory<TcriDbContext>; CreateDbContext(string[] args) creates a context using UseSqlite("Data Source=design-time.db") without runtime configuration.

3.11.6. InitialCreate Migration

Data/Migrations/InitialCreate.sql contains the Part 1, 3.5 DDL as amended by A1 in 3.9 (the inbox table without uq_inbox_hash, with partial unique index uq_inbox_new_hash and indexes ix_inbox_hash, ix_inbox_call); everything else is unchanged. The .csproj marks it as <EmbeddedResource Include="Data\Migrations\InitialCreate.sql" />; logical resource name TeamsCallRecords.Infrastructure.Data.Migrations.InitialCreate.sql.

InitialCreate : Migration class ([Migration("20261021000000_InitialCreate")], [DbContext(typeof(TcriDbContext))]):

Method	Action
Up(MigrationBuilder)	Read the embedded resource as UTF-8 and call migrationBuilder.Sql(text)
Down(MigrationBuilder)	DROP TABLE for error_log, subscription, outbox, audit_log, rejected, inbox, source_state, cdr, in that order

Generate the supporting files from the repository root by running:

dotnet tool install --global dotnet-ef --version 8.*
dotnet ef migrations add InitialCreate --project src/TeamsCallRecords.Infrastructure --startup-project src/TeamsCallRecords.Infrastructure --output-dir Data/Migrations

The tool generates <stamp>_InitialCreate.cs, <stamp>_InitialCreate.Designer.cs, and TcriDbContextModelSnapshot.cs. The implementer replaces the stamp in file names and the [Migration] attribute with 20261021000000, and the Up and Down bodies with those specified above. The .Designer.cs files and snapshot are not edited manually.

3.11.7. ICdrReader and CdrReader (Data/ICdrReader.cs, Data/CdrReader.cs)

```
public sealed record CdrItem(long CdrSeq, Cdr Cdr);
public sealed record CdrPage(IReadOnlyList<CdrItem> Items, string? Next);
public sealed record DailyStats(int Calls, long TotalDurationSeconds, int Cdrs);

public interface ICdrReader
{
    Task<CdrPage> QueryAsync(DateTimeOffset fromUtc, DateTimeOffset toUtc, int limit, string? cursor, CancellationToken ct);
    Task<DailyStats> GetStatsAsync(string tenantId, DateTimeOffset fromUtc, DateTimeOffset toUtc, CancellationToken ct);
}
```

CdrReader(IDbContextFactory<TcriDbContext> factory); singleton; read-only.

QueryAsync. Preconditions: limit between 1 and 1000 (otherwise ArgumentOutOfRangeException); fromUtc <= toUtc (otherwise ArgumentException). Algorithm:

1. from = UtcText.Format(fromUtc), to = UtcText.Format(toUtc).
2. If cursor is nonempty, (cs, cq) = Cursor.Decode(cursor) (3.11.8); malformed cursor raises ArgumentException("cursor").
3. Without a cursor: SELECT * FROM cdr WHERE tenant_id = {tenantId} AND start_utc >= {from} AND start_utc < {to}; with a cursor append AND (start_utc > {cs} OR (start_utc = {cs} AND cdr_seq > {cq})).
4. ORDER BY start_utc, cdr_seq LIMIT {limit + 1}, AsNoTracking.
5. If the query produces more than limit rows, discard the last extra row and set Next = Cursor.Encode(start_utc and cdr_seq of the last returned row); otherwise Next = null.
6. Rows are converted by CdrMapping.ToCdr; CdrItem.CdrSeq is assigned the cdr_seq value.

Postcondition: pages have no omissions or duplicates under concurrent insertion of rows with greater (start_utc, cdr_seq); use the half-open interval [from, to).

GetStatsAsync. SELECT COUNT(DISTINCT call_id), COALESCE(SUM(duration_s), 0), COUNT(*) FROM cdr WHERE tenant_id = {tenantId} AND start_utc >= {from} AND start_utc < {to} (half-open interval); returns DailyStats(Calls, TotalDurationSeconds, Cdrs). “Calls” is the count of distinct call_id values, and “total duration” is the sum of duration_s over CDRs [P-13, open detailed design question, Appendix B].

Integration clarification [P-26]. Statistics are strictly scoped to tenantId: SQL SELECT COUNT(DISTINCT call_id) ... FROM cdr WHERE tenant_id = @tenantId AND start_utc >= @fromUtc AND start_utc < @toUtc. SQLite-invalid COUNT(DISTINCT (tenant_id,call_id)) is prohibited. Host calls GetStatsAsync(Graph:TenantId, fromUtc, toUtc, ct).

3.11.8. Cursor (Data/Cursor.cs)

public static class Cursor — an opaque page cursor for CdrReader.QueryAsync and the REST API (Part 3). Two methods; no internal state.

Method	Contract
string Encode(string startUtc, long cdrSeq)	Preconditions: startUtc must be canonical UtcText (i.e. UtcText.Format(UtcText.Parse(startUtc)) == startUtc), and cdrSeq >= 1; otherwise ArgumentException. Result: UTF-8 bytes of text <startUtc>|<cdrSeq> (cdrSeq is a decimal number with CultureInfo.InvariantCulture) encoded as URL-safe Base64, with + replaced by -, / by _, and = removed. Example: Encode("2026-10-20T09:00:00.0000000Z",17) encodes the text 2026-10-20T09:00:00.0000000Z|17.
(string StartUtc, long CdrSeq) Decode(string cursor)	Reverse transformation: restore = padding until length is divisible by four, replace - with + and _ with /, decode Base64 and UTF-8 (invalid bytes cause an error), split the text on the single |. Requirements: exactly two parts; the first is a canonical UtcText timestamp; the second consists only of digits with a value from 1 through long.MaxValue. Every violation (empty input, invalid Base64 characters, missing delimiter, extra parts, noncanonical timestamp, nonnumeric or nonpositive value) throws ArgumentException("cursor is malformed", "cursor"). The method throws no other exception types.

Property: Decode(Encode(t, n)) = (t, n) for any valid t and n; Encode produces no +, /, or = characters. The cursor is neither secured nor signed: it only identifies a position, and tampering merely selects another page of the same data.

3.12. Queue: Incoming Inbox

3.12.1. Types (Queue/QueueModels.cs)

public sealed record InboxNewItem(string Kind, string TenantId, string? CallId, string Raw);
public sealed record InboxEnqueueResult(int Added, int Duplicates, bool Overloaded);
public sealed record InboxItem(long Id, string Kind, string TenantId, string? CallId, string Raw, int Attempts, DateTimeOffset ReceivedUtc);
public sealed record InboxStats(int Pending, long OldestPendingAgeSeconds, int Failed);

InboxNewItem.Kind — one of InboxKinds; Raw — the item's JSON (for callRecord, a notification element without clientState; for reconcile, an object per 3.13.6). CallId is required for callRecord and per-call reconcile, null for channelMessage and general reconciliation.

3.12.2. InboxTransitions (Queue/InboxTransitions.cs)

internal static class InboxTransitions — item transitions within an existing write transaction; used by InboxStore, CdrApplier, and SourceStateStore so closing the inbox item is committed atomically with the data change.

Method	Action on InboxRow with identifier id
MarkDone(TcriDbContext db, long id, string nowText)	state='done', done_utc=nowText, locked_until_utc=NULL, last_error=NULL
Reschedule(TcriDbContext db, long id, string nextText, string? error, bool resetAttempts)	state='new', next_attempt_utc=nextText, locked_until_utc=NULL, last_error=error; when resetAttempts, set attempts=0
MarkFailed(TcriDbContext db, long id, string nowText, string error)	state='failed', done_utc=nowText, locked_until_utc=NULL, last_error=error

If no row is found for id, throw InvalidOperationException (a programming error, not a data error). The caller invokes SaveChanges.

3.12.3. IInboxStore and InboxStore (Queue/IInboxStore.cs, Queue/InboxStore.cs)

InboxStore(IWriteGate gate, IDbContextFactory<TcriDbContext> factory, IOptions<InboxOptions> inbox, IOptions<ProcessingOptions> processing, TimeProvider clock); singleton.

Member	Signature	Description
Enqueue	Task<InboxEnqueueResult> EnqueueAsync(IReadOnlyList<InboxNewItem> items, CancellationToken ct)	Enqueue a batch in one transaction (algorithm below)
RenewLease	Task<DateTimeOffset?> RenewLeaseAsync(long id, DateTimeOffset expectedLockedUntilUtc, CancellationToken ct)	CAS renewal of an active processing lease only: UPDATE inbox SET locked_until_utc=@next WHERE id=@id AND state='processing' AND locked_until_utc=@expected AND locked_until_utc>@now; return null if 0 rows are updated. The new expiry is now+LeaseSeconds; the operation runs under IWriteGate.
ClaimNext	Task<InboxItem?> ClaimNextAsync(CancellationToken ct)	Claim one ready item (algorithm below)
Complete	Task CompleteAsync(long id, CancellationToken ct)	InboxTransitions.MarkDone in its own transaction
Reschedule	Task RescheduleAsync(long id, DateTimeOffset nextAttemptUtc, string? error, bool resetAttempts, CancellationToken ct)	InboxTransitions.Reschedule
Fail	Task FailAsync(long id, string error, CancellationToken ct)	InboxTransitions.MarkFailed
GetStats	Task<InboxStats> GetStatsAsync(CancellationToken ct)	Read statistics: Pending is the count of new and processing rows; OldestPendingAgeSeconds is integer seconds between the oldest such row's received_utc and now (0 if none); Failed is the count of failed rows
ElementHash	public static string ElementHash(string kind, string tenantId, string raw)	See below

ElementHash. Sha256Hex(kind + "\n" + tenantId + "\n" + Canonical.HashOf(JsonDocument.Parse(raw).RootElement)), where Sha256Hex is the Core method (Canonical.Sha256Hex); returns 64 lowercase characters. If raw is not JSON, throw ArgumentException. A repeated notification with the same content produces the same hash [P-15].

EnqueueAsync algorithm (one gate transaction):

1. If items is empty, return (0,0,false).
2. pending = number of inbox rows in states new and processing. If pending >= Inbox:MaxPending, write nothing and return (0,0,true).
3. For each item: hash = ElementHash(...); find the row with this element_hash in state new (the query uses partial unique index uq_inbox_new_hash).
   - If no such row exists, insert: state='new', attempts=0, next_attempt_utc=now, received_utc=now; Added++. This also applies when an earlier row with the same hash is already done, failed, or processing: a repeated notification may indicate changed source data and must trigger another Graph read [P-22];
   - If a new row with the same hash already exists, it has not yet fetched Graph and will read the latest data; do not change it; Duplicates++. Items with identical hashes in the same batch count as one (the second increments Duplicates).

4. SaveChanges, then commit.

Postcondition: when Overloaded=false is returned, every batch item has an inbox row in state new with its hash (added now or already present); rows in done, failed, or processing are not changed and do not block repeat notifications; Host responds 202 only in this case. A write error throws (Host responds 5xx and Graph retries notification).

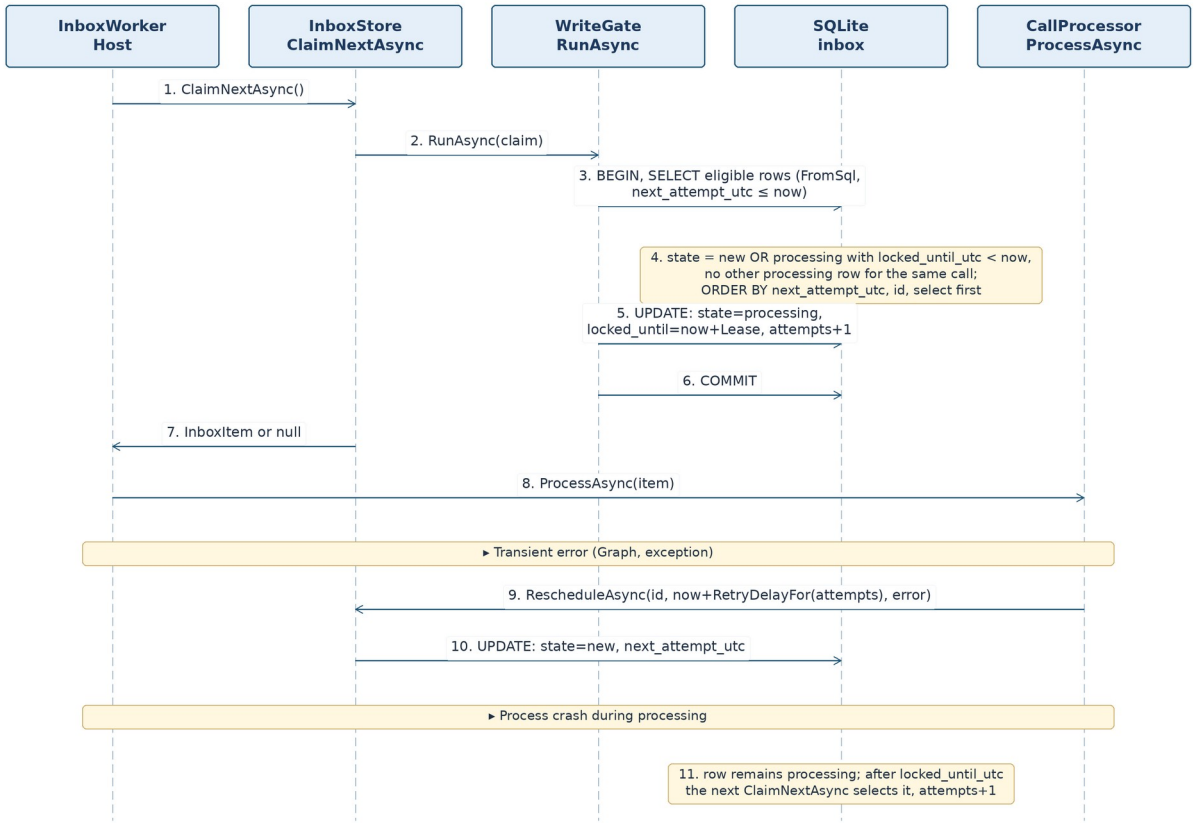
ClaimNextAsync algorithm (one gate transaction):

1. now = UtcText.Now(clock).
2. Select using FromSqlInterpolated: SELECT * FROM inbox WHERE next_attempt_utc <= {now} AND (state = 'new' OR (state = 'processing' AND locked_until_utc < {now})) AND (call_id IS NULL OR NOT EXISTS (SELECT 1 FROM inbox p WHERE p.call_id = inbox.call_id AND p.tenant_id = inbox.tenant_id AND p.id <> inbox.id AND p.state = 'processing' AND p.locked_until_utc >= {now})), then OrderBy(NextAttemptUtc).ThenBy(Id).Take(1) with change tracking.
3. If no row exists, return null.
4. Set state='processing', locked_until_utc = now + Processing:LeaseSeconds, attempts += 1; SaveChanges.
5. Return InboxItem (Attempts reflects the incremented value).

Properties: two concurrent claims cannot take the same item (gate); items from the same call are processed serially — while another item of that call has an active lease, the claim skips this row and takes the next one [P-22]; a repeat notification arriving during processing is queued as a new item and fetches Graph after the current operation completes; an expired processing lease following a failure is reclaimed automatically with attempts incremented; the stored raw body is not changed.

Claim, retry, and crash recovery sequence — Figure 4.

Figure 4. Inbox: Claim, Retries, and Recovery. Mermaid text diagram; steps and numbering match the diagram embedded in the PDF and DOCX.



Errors. All methods propagate database exceptions; the store does not retry internally (the caller schedules retries according to 3.13.2).

Lease and retry addition [P-27]. InboxWorker renews its lease every LeaseSeconds/3 using RenewLeaseAsync, supplying the last obtained expectedLockedUntilUtc value. If null is returned or a database error occurs, the associated Graph processing is cancelled, and the stale worker must not complete the item. The CAS check prevents lease renewal after another worker has reclaimed it. For long-running ReconcileAsync, the heartbeat runs concurrently with Graph page fetching; the server does not treat the work as complete if the lease is lost.

Partial unique-index concurrency conflict [P-22, P-27]. When RescheduleAsync moves an already processing item A to new, another item B with the same element_hash may already be in new state. All changes occur in one write transaction: if B exists, A is marked done as superseded by the repeat, B remains new with a schedule no later than required for A; neither the call state nor the scheduled retry is lost. If B does not exist, A receives state=new and its own next_attempt_utc. The stale A does not change the CDR. This prevents a uq_inbox_new_hash exception and ensures reprocessing. Test: Reschedule_WhenDuplicateNew_MergesWithoutLostRetry, already included in the registry.

3.13. Processing: Item Handling, Call State, CDR Replacement, Reconciliation

3.13.1. Types (Processing/ProcessingModels.cs)

public enum ProcessOutcome { Applied, IgnoredLower, Duplicate, Discrepancy, Rejected, Pending, Suspended, Expired, Retry, Failed, Logged, ReconcileRun }
public sealed record ApplyRequest(string TenantId, ConversionResult Result, long InboxId);
public enum ApplyStatus { Applied, DecisionChanged }
public sealed record ApplyResult(ApplyStatus Status, VersionAction? ChangedAction, int Inserted, int Updated, int Deleted, int OutboxCreated);
public sealed record PendingResult(string State, DateTimeOffset? NextAttemptUtc);
public sealed record SourceStateCounts(int Pending, int Suspended, int Expired);

Final inbox item state based on processing outcome:

ProcessOutcome	Inbox Item	Data Changed
Applied	done	CDR, source_state, outbox — in one transaction
IgnoredLower	done	nothing (event in log)
Duplicate	done	nothing
Discrepancy	done	audit_log row
Rejected	done	rejected rows, source_state.rejected_version
Pending	new, attempts=0, next_attempt_utc according to schedule	source_state (pending)
Suspended	done	source_state (suspended)
Expired	done	source_state (expired), error_log row
Retry	new, attempts preserved, next_attempt_utc according to schedule	—
Failed	failed	error_log row
Logged	done	—
ReconcileRun	done	reconcile items for calls enqueued in inbox

3.13.2. Retry Schedule (Config/ProcessingOptions.cs)

ProcessingOptions: int Workers = 2, int LeaseSeconds = 120, string RetrySchedule = "1,5,15,60", int PendingLimitHours = 4, int SuspendedRecheckHours = 24, int MaxAttempts = 10 [P-11].

Method	Description
IReadOnlyList<int> RetryScheduleMinutes()	Parse RetrySchedule as positive comma-separated integers; spaces allowed
TimeSpan RetryDelayFor(int attempt)	attempt >= 1; minutes at index min(attempt − 1, n − 1), repeating the last interval; attempt < 1 → ArgumentOutOfRangeException
IEnumerable<string> Validate()	configuration errors (3.18)

Example for 1,5,15,60 schedule: attempt 1 → 1 min; 2 → 5; 3 → 15; 4 → 60; 5 onward → 60.

3.13.3. ICallProcessor and CallProcessor (Processing/ICallProcessor.cs, Processing/CallProcessor.cs)

public interface ICallProcessor { Task<ProcessOutcome> ProcessAsync(InboxItem item, CancellationToken ct); }

CallProcessor(IGraphClient graph, IInboxStore inbox, ISourceStateStore states, ICdrApplier applier, IReconciler reconciler, IErrorLog errors, IOptions<ProcessingOptions> options, TimeProvider clock, ILogger<CallProcessor> log); singleton, thread-safe (stateless).

Precondition: item was returned by IInboxStore.ClaimNextAsync and is in state processing. Postcondition: the item is transitioned to a state from Table 3.13.1 or, upon cancellation or an error inside exception handling, remains processing until the lease expires and is reclaimed. The method throws no exceptions except OperationCanceledException [K9].

ProcessAsync algorithm (A2). Diagram — Figure 5.

1. Logging scope: component=processor, tenant_id, call_id, inbox_id.
2. By item.Kind:
   - channelMessage — log a “channel message received” event (without message text), inbox.CompleteAsync, return Logged;
   - reconcile with CallId = null: first parse item.Raw as JSON and determine the work type [P-27]. If any of from_utc, to_utc, or force is present, the item requests manual interval reconciliation; both UTC fields are strictly required; force must be Boolean and true; from_utc < to_utc and the interval must not exceed 7 days. Only when all checks succeed call reconciler.ReconcileAsync(fromUtc,toUtc,force:true,ct). On incomplete, damaged, or invalid input, call inbox.FailAsync(id,"RECONCILE_BAD_RANGE",ct), write safe diagnostics, and return Failed without calling ReconcileAsync or ReconcileLookbackAsync. If all three fields are absent, perform normal reconciler.ReconcileLookbackAsync(ct). On success, inbox.CompleteAsync and return ReconcileRun; a GraphException from reconciliation follows step 12; other errors follow step 13. The job is executed by a background service; the HTTP handler only enqueues it;
   - callRecord or reconcile with CallId ≠ null — step 3; callRecord without CallId — inbox.FailAsync(id,"INBOX_NO_CALL_ID"), errors.WriteAsync(Components.Processor,"inbox item without call_id","inbox_id=<id>"), return Failed.
3. GraphFetchResult r = await graph.GetCallRecordAsync(item.CallId, ct). On GraphException ex, go to step 12.
4. state = await states.GetAsync(item.TenantId, item.CallId, ct) (one read; the following decisions are based on it, and CdrApplier rechecks it within the transaction).
5. Switch on r.Status:
   - NotFound or PagesIncomplete: if state?.AppliedVersion is set, the call has already been applied, and its state must not be degraded: log, inbox.CompleteAsync, return Duplicate. Otherwise proceed to step 11 with reason GRAPH_NOT_FOUND or GRAPH_PAGES_INCOMPLETE, respectively;

- Malformed — the source returned a response that could not be parsed as JSON; its text is in r.Json; proceed to step 6 (the converter returns CALLRECORD_BAD_JSON) [P-24];
- Ok — step 6.

6. result = CallRecordConverter.Convert(r.Json, item.TenantId).
7. If result.Rejections contains a record-level rejection (BadJson, NoCallId, NoVersion): sourceHash = RejectedSourceHash(r.Json); call states.RecordRejectionAsync(item.TenantId, result.CallId ?? item.CallId, result.SourceVersion, result.Rejections, sourceHash, item.Id, ct); return Rejected.
8. version = result.SourceVersion; action = VersionRules.Decide(state?.AppliedVersion, state?.SessionSetHash, version, result.SessionSetHash). If state?.RejectedVersion > version, treat the action as IgnoreLower. The version decision is made before any attempt to apply a result.
9. If action is not Apply:
   - IgnoreLower — log, inbox.CompleteAsync, return IgnoredLower;
   - DuplicateSame — log, inbox.CompleteAsync, return Duplicate;
   - Discrepancy — call states.RecordDiscrepancyAsync(...), return Discrepancy. These branches also apply when result is incomplete: an obsolete or duplicate incomplete version must not change the source_state of a completed call.
10. If action = Apply:

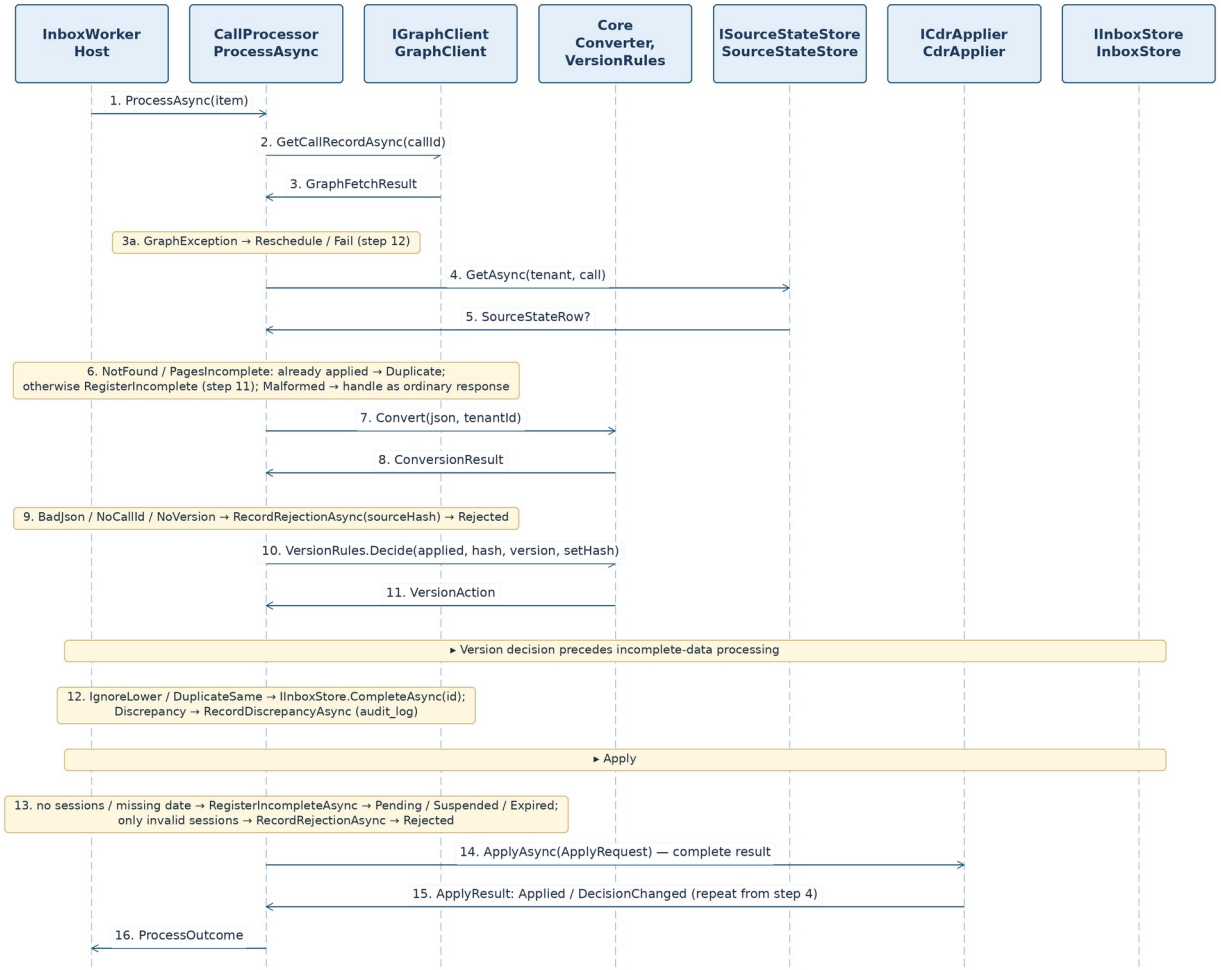
- Transient reasons (result.IncompleteReasons contains NO_SESSIONS or SESSION_DATES_MISSING) — step 11 with a comma-separated list of transient reasons; session-level rejections are not recorded at this stage, as they will be recorded when a higher-quality source version becomes available;
- **Only permanent reasons (result.IsComplete = false and invalid sessions remain):** if state?.RejectedVersion >= version, call inbox.CompleteAsync and return Duplicate; otherwise call states.RecordRejectionAsync(..., RejectedSourceHash(r.Json), ...), return Rejected;
- Complete result — applier.ApplyAsync(new ApplyRequest(...), ct); if Applied, return Applied; if DecisionChanged, return to step 4 (no more than three consecutive attempts; after the third, proceed to step 13 with InvalidOperationException("decision changed"));

11. Incomplete data: PendingResult p = await states.RegisterIncompleteAsync(item.TenantId, item.CallId, reason, item.Id, ct); return Pending, Suspended, or Expired according to p.State.
12. Graph error ex: if ex.IsTransient and item.Attempts < MaxAttempts, call inbox.RescheduleAsync(id, now + RetryDelayFor(item.Attempts), message, resetAttempts: false) and return Retry; otherwise call inbox.FailAsync(id, message), errors.WriteAsync(Components.Graph, "graph request failed", "tenant_id=<...>; call_id=<...>; status=<code or none>; attempts=<count>") and return Failed. Errors 401 and 403 are permanent [P-11]. Reading (GetCallRecordAsync) is safe to retry, so the uncertain outcome in 3.14.4 is not distinguished here.
13. Any other exception (except cancellation): log its type and message; within an exception handler protected by its own try/catch, call errors.WriteAsync(Components.Processor, "unexpected processing error", "tenant_id=<...>; call_id=<...>; exception=<exception type>"). Then, if item.Attempts < MaxAttempts, call inbox.RescheduleAsync(id, now + RetryDelayFor(item.Attempts), ex.GetType().Name + ": " + ex.Message, false) and return Retry; otherwise call inbox.FailAsync and return Failed. If the exception handler itself throws, the error is written only to ILogger; the item remains processing until its lease expires.

RejectedSourceHash(string json) (internal static, CallProcessor): if json can be parsed as JSON, use Canonical.HashOf(root) (the same canonical hash as source_hash in a CDR: independent of whitespace and property order, with transport annotations excluded). If parsing fails (JsonException, empty text), use Canonical.Sha256Hex(json) over the UTF-8 response text. Store this value in rejected.source_hash for every rejection of that record [P-23].

Idempotency: reprocessing one item (reclaim after lease expiry) is safe — version decisions and unique keys prevent duplicates.

Figure 5. CallProcessor: Processing a Graph Record. Mermaid text diagram; steps and numbering match the PDF/DOCX image.



3.13.4. ISourceStateStore and SourceStateStore (Processing/ISourceStateStore.cs, Processing/SourceStateStore.cs)

SourceStateStore(IWriteGate gate, IDbContextFactory<TcriDbContext> factory, IOptions<ProcessingOptions> processing, IOptions<GraphOptions> graph, TimeProvider clock); singleton.

Member	Signature
Get	Task<SourceStateRow?> GetAsync(string tenantId, string callId, CancellationToken ct)

Member	Signature
GetMany	Task<IReadOnlyDictionary<string, SourceStateRow>> GetManyAsync(string tenantId, IReadOnlyCollection<string> callIds, CancellationToken ct) — dictionary key is call_id; queries in batches of 500 identifiers
GetCounts	Task<SourceStateCounts> GetCountsAsync(CancellationToken ct)
ListSuspendedDue	Task<IReadOnlyList<SourceStateRow>> ListSuspendedDueAsync(int limit, CancellationToken ct) — state='suspended' and next_attempt_utc <= now, sorted ascending by next_attempt_utc, at most limit items
RegisterIncomplete	Task<PendingResult> RegisterIncompleteAsync(string tenantId, string callId, string reason, long inboxId, CancellationToken ct)
RecordRejection	Task RecordRejectionAsync(string tenantId, string? callId, long? version, IReadOnlyList<Rejection> rejections, string sourceHash, long inboxId, CancellationToken ct)
RecordDiscrepancy	Task RecordDiscrepancyAsync(string tenantId, string callId, long version, string? appliedSetHash, string incomingSetHash, long inboxId, CancellationToken ct)

All three writing methods execute in a single gate transaction, including the inbox item transition.

RegisterIncompleteAsync (now is the current time; age = now − first_seen_utc):

1. Find source_state row. If no row exists or state = 'done', create or enter a new episode: state='pending', first_seen_utc = now, attempts = 1; preserve applied_version, rejected_version, and session_set_hash. Otherwise increment attempts.
2. last_error = reason, updated_utc = now.
3. By state:
   - pending (including just created): if age >= PendingLimitHours hours, set state='suspended', next_attempt_utc = now + SuspendedRecheckHours, MarkDone the item, and return ("suspended", next). Otherwise compute next = now + RetryDelayFor(attempts); set next_attempt_utc = next; Reschedule(next, reason, resetAttempts: false); return ("pending", next).
   - suspended (recheck): if age > GraphOptions.RetentionDays days, set state='expired', next_attempt_utc = NULL, create an error_log row (component='processor', message="source_state expired", details="tenant_id=<...>; call_id=<...>"), MarkDone the item and return ("expired", null). Otherwise set next_attempt_utc = now + SuspendedRecheckHours, MarkDone the item and return ("suspended", next).
   - expired — leave the state unchanged, MarkDone item, return ("expired", null).
4. SaveChanges, then commit.

RecordRejectionAsync:

1. For each Rejection r, insert a RejectedRow (tenant_id, call_id=r.CallId ?? callId, session_id=r.SessionId, code, message truncated to 1000 characters, source_hash=sourceHash, created_utc=now). Do not add the row if rejected already contains the same tenant_id, call_id, session_id (including NULL), code and source_hash: repeatedly reading the same damaged record must not multiply the rejected list.
2. If callId and version are known: look up source_state; if no row exists, create one (state='done', applied_version=NULL, rejected_version=version, attempts=0, first_seen_utc=now). If a row exists, set rejected_version=max(current, version), and if state is pending or suspended, set state='done', attempts=0, next_attempt_utc=NULL.
3. MarkDone the item. SaveChanges, then commit.

RecordDiscrepancyAsync: insert AuditLogRow (ts=now, tenant_id, call_id, kind='version_discrepancy', detail is JSON {"version":<version>,"applied_set_hash":"<...>","incoming_set_hash":"<...>"}; represent a null hash as JSON null); MarkDone the item. Neither source_state nor cdr is changed.

3.13.5. ICdrApplier and CdrApplier (Processing/ICdrApplier.cs, Processing/CdrApplier.cs)

public interface ICdrApplier { Task<ApplyResult> ApplyAsync(ApplyRequest request, CancellationToken ct); }

CdrApplier(IWriteGate gate, IOptions<OutboxOptions> outbox, TimeProvider clock); singleton.

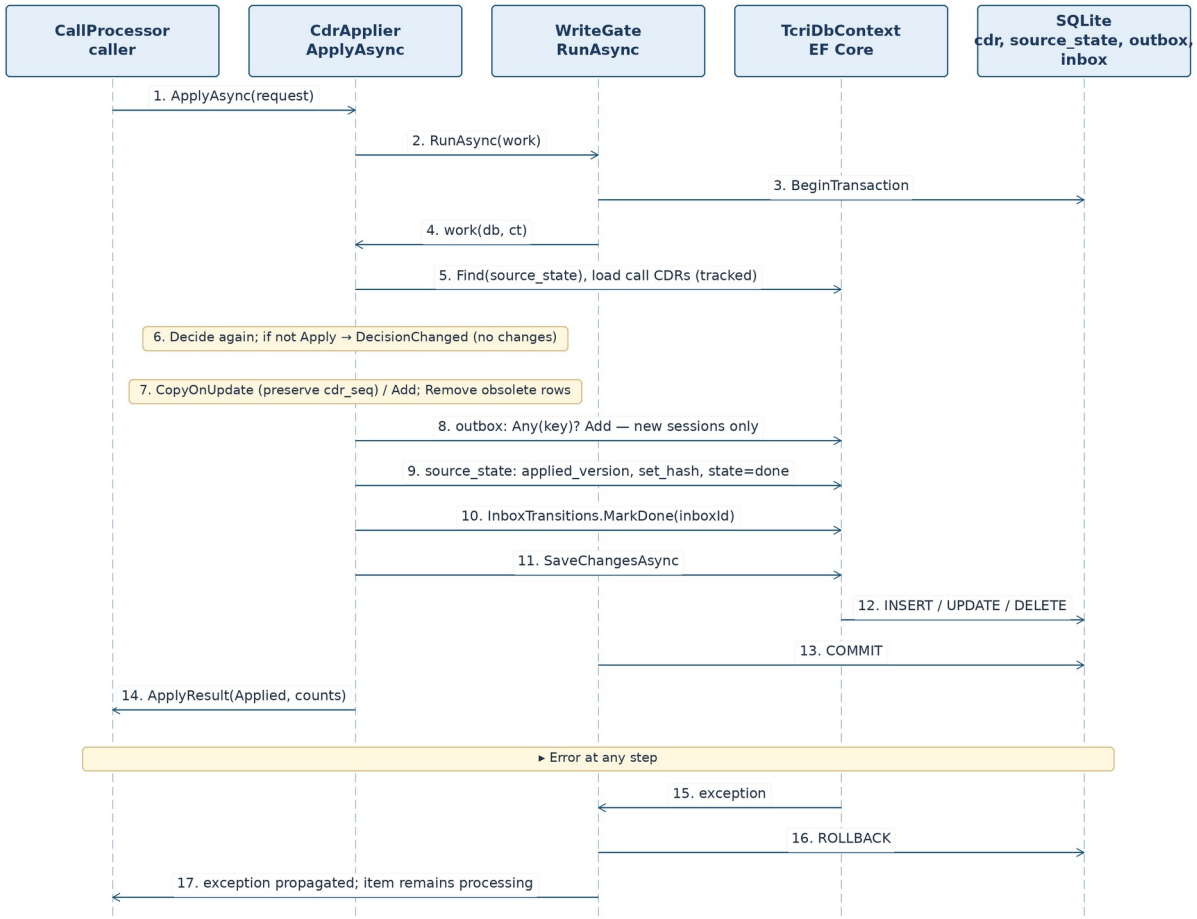
Preconditions: request.Result.IsComplete = true; request.Result.Cdrs is nonempty; every CDR has the same CallId and TenantId = request.TenantId; request.InboxId refers to an item in state processing.

ApplyAsync algorithm (one gate transaction; Figure 6):

1. now; state = db.SourceStates.Find(tenant, call) (with tracking).
2. Reevaluate the decision: VersionRules.Decide(state?.AppliedVersion, state?.SessionSetHash, version, setHash); if state?.RejectedVersion > version, IgnoreLower. If the result is not Apply, return ApplyResult(DecisionChanged, action, 0,0,0,0) without changing data.
3. existing = db.Cdrs.Where(TenantId == tenant && CallId == call).ToList() (with tracking); oldSessions is the set of SessionId values from existing.
4. For each Cdr c in request.Result.Cdrs: row = CdrMapping.ToRow(c, now). If existing contains a row with the same SessionId, call CdrMapping.CopyOnUpdate(existingRow, row, now) (preserve cdr_seq and created_utc), Updated++; otherwise db.Cdrs.Add(row), Inserted++.
5. For each row in existing whose SessionId is absent from the new set, call db.Cdrs.Remove and increment Deleted.
6. Outbound notifications: for every Cdr c whose SessionId is not in oldSessions, provided c.EndUtc >= now − Outbox:MaxAgeHours hours, recipients are the distinct nonempty Id values of c.Caller and c.Callee with Type="user" [P-14]. For each recipient, if no row with key (activity, tenant, call, session, recipient) exists in outbox (checked by query in this same transaction), insert OutboxRow (kind='activity', payload=ActivityPayload.Create(c).ToJson(), state='new', attempts=0, next_attempt_utc=now, created_utc=now); increment OutboxCreated.
7. source_state: if missing, create (first_seen_utc = now); applied_version = version, session_set_hash = setHash, state='done', attempts=0, next_attempt_utc=NULL, last_error=NULL, updated_utc=now; rejected_version is unchanged.
8. InboxTransitions.MarkDone(db, request.InboxId, now).
9. await db.SaveChangesAsync(ct); gate commits. Errors cause a gate rollback and propagate to CallProcessor step 13.

Postconditions when Applied: the call's cdr rows correspond exactly to the result's session set; cdr_seq values of surviving sessions do not change; source_state.applied_version = version, state='done'; inbox item is done; outbox rows are created for new sessions (not updated sessions), subject to recipient rules.

Figure 6. CdrApplier: Atomic CDR Replacement. Mermaid text diagram; steps and numbering match the PDF/DOCX image.



3.13.6. IReconciler and Reconciler (Processing/IReconciler.cs, Processing/Reconciler.cs)

```
public interface IReconciler
{
    DateTimeOffset? LastRunUtc { get; }
    Task<int> ReconcileAsync(DateTimeOffset fromUtc, DateTimeOffset toUtc, bool force, CancellationToken ct);
    Task<int> ReconcileLookbackAsync(CancellationToken ct);
    Task<int> RecheckSuspendedAsync(CancellationToken ct);
}
```

Reconciler(IGraphClient graph, ISourceStateStore states, IInboxStore inbox, IOptions<GraphOptions> graphOptions, IOptions<ReconcileOptions> options, TimeProvider clock, ILogger<Reconciler> log); singleton. ReconcileOptions: int IntervalMinutes = 15, int LookbackHours = 24.

Method	Algorithm
ReconcileAsync(from, to, force)	1. refs = graph.ListCallRecordsAsync(from, to). 2. known = states.GetManyAsync(tenant, refs.Id). 3. For each reference r: evaluate NeedsFetch(known.GetValueOrDefault(r.Id), r, force); if true, create InboxNewItem(Reconcile, tenant, r.Id, raw), with raw JSON {"call_id":"<id>","version":<number or null>}. 4. Call inbox.EnqueueAsync(items). 5. If Overloaded = true, write a warning to the log and return 0. 6. Return Added. Propagate any GraphException to the caller.
ReconcileLookbackAsync()	ReconcileAsync(now − LookbackHours, now, force: false); after successful completion, LastRunUtc = now (stored only in process memory, null again after restart [P-17])
RecheckSuspendedAsync()	rows=states.ListSuspendedDueAsync(200); for each row create InboxNewItem(Reconcile, tenant, row.CallId, raw), where raw={"call_id":"<id>","recheck_at":"<row.next_attempt_utc>"}. The recheck_at value distinguishes scheduled rechecks of the same call; as long as a new row with that hash remains unprocessed, a repeated call does not duplicate it. Call inbox.EnqueueAsync; return Added.

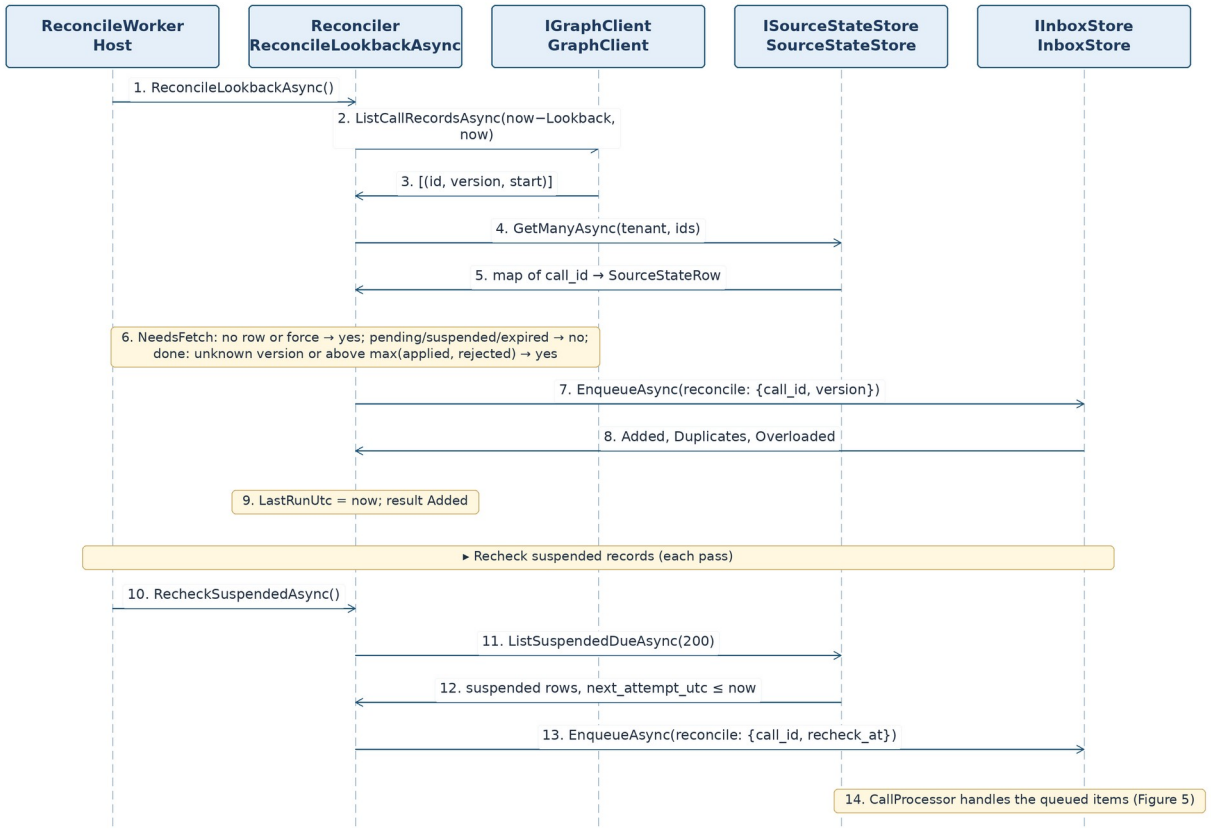
NeedsFetch(SourceStateRow? s, GraphCallRecordRef r, bool force) — conditions are checked from top to bottom; the first match determines the result:

№	Condition	Result
1	s = null	yes
2	force = true (manual /reprocess)	yes, in every state, including pending, suspended, expired, and done at the same version
3	s.State ∈ {pending, suspended}	no (retries and daily checks are handled by item processing)
4	s.State = expired	no (source retention has expired; only a manual force can retry it)
5	s.State = done and r.Version = null	yes (version is unknown and cannot be compared; refetch is safe but occurs at every reconciliation, limited by the LookbackHours window [P-21])
6	s.State = done and r.Version > max(s.applied_version ?? −1, s.rejected_version ?? −1)	yes
7	otherwise	no

Postcondition: after execution, for each call in the period which needs a read according to the NeedsFetch table, inbox contains a new row with that call's hash. A repeat run without new versions and without force does not add items while the new rows have not been processed; once processed it does not add any at all. /reprocess (Part 3) invokes ReconcileAsync(from,to,force:true): every call in the period is reread regardless of state. Repetition is safe because the version decision (3.13.3, step 8) prevents duplicates.

Reconciliation sequence — Figure 7.

Figure 7. Reconciler: Reconciliation with Microsoft Graph. Mermaid text diagram; steps and numbering match the PDF/DOCX image.



Comparison with authentic v0.2 [RV-01]. Under normal NeedsFetch(force:false), pending and suspended states return false: their next retry is determined by next_attempt_utc using the standard mechanisms. With force:true, the requested period is checked independently of next_attempt_utc.

3.14. Graph: Microsoft Graph Client

3.14.1. Types (Graph/GraphModels.cs, Graph/GraphException.cs)

```
public enum GraphFetchStatus { Ok, NotFound, PagesIncomplete, Malformed }
public sealed record GraphFetchResult(GraphFetchStatus Status, string? Json);
public sealed record GraphCallRecordRef(string Id, long? Version, DateTimeOffset? StartUtc);
public sealed record GraphSubscriptionRequest(string Kind, string Resource, string NotificationUrl, DateTimeOffset ExpiresUtc, string ClientState);
public sealed record GraphSubscription(string Id, string Resource, DateTimeOffset ExpiresUtc);
public sealed record GraphSubscriptionInfo(string Id, string Resource, string NotificationUrl, DateTimeOffset ExpiresUtc);
```

```
public sealed record ActivityNotification(string RecipientUserId, string ActivityType, string TopicText, string PreviewText, IReadOnlyDictionary<string, string>
TemplateParameters);

public sealed class GraphException : Exception
{
    public GraphException(string message, int? statusCode, bool isTransient, Exception? inner = null, bool outcomeUnknown = false);
    public int? StatusCode { get; }
    public bool IsTransient { get; }
    public bool OutcomeUnknown { get; }
}
```

GraphFetchResult.Json is nonempty when Ok; for PagesIncomplete, it holds the portion already read but is not used by the processor; for Malformed, it contains the response text that could not be parsed as JSON (used to record the rejection and compute source_hash, 3.13.3 steps 5 and 7).

GraphException.OutcomeUnknown means an uncertain outcome: the server may have performed a side-effecting operation even though no response was received or a server error was received. Set this flag only for POST methods (CreateSubscriptionAsync, SendActivityNotificationAsync); it is always false for reads, PATCH and DELETE, because their retries are safe.

Error classification:

Situation	StatusCode	IsTransient	OutcomeUnknown
network unavailable or request timeout	null	yes	true for POST, false otherwise
open circuit breaker (BrokenCircuitException; request was not sent)	null	yes	no
429 after retries exhausted	429	yes	no (the server rejected without performing the request)
408 and 5xx after retries exhausted	status code	yes	true for POST, false otherwise
400, 401, 403, all other 4xx except 404	status code	no	no
404 for GetCallRecordAsync	—	not an exception: GraphFetchStatus.NotFound	—
404 for other operations	404	no	no
2xx response body cannot be parsed as JSON: GetCallRecordAsync	—	not an exception: GraphFetchStatus.Malformed	—
the same for other operations	null	no	no

Exception messages include the method, path without query string, and status code; the response body and token are excluded.

3.14.2. GraphOptions Settings (Graph/GraphOptions.cs)

Property	Key	Default
TenantId	Graph:TenantId	—
ClientId	Graph:ClientId	—
ClientSecret	Graph:ClientSecret	— (environment)
NotificationUrl	Graph:NotificationUrl	—
ClientState	Graph:ClientState	— (environment)
RetentionDays	Graph:RetentionDays	30
BaseUrl	Graph:BaseUrl	https://graph.microsoft.com/v1.0
Scope	Graph:Scope	https://graph.microsoft.com/.default
RequestTimeoutSeconds	Graph:RequestTimeoutSeconds	30
TotalTimeoutSeconds	Graph:TotalTimeoutSeconds	120
RetryMaxAttempts	Graph:RetryMaxAttempts	3
RetryBaseDelayMilliseconds	Graph:RetryBaseDelayMilliseconds	500
MaxPages	Graph:MaxPages	100
ActivityTopicWebUrl	Graph:ActivityTopicWebUrl	https://teams.microsoft.com/

Validate() rules: TenantId, ClientId, ClientSecret, NotificationUrl and ClientState must be nonempty; NotificationUrl and BaseUrl must be absolute HTTPS addresses; RetentionDays 1–365; RequestTimeoutSeconds 1–300; TotalTimeoutSeconds >= RequestTimeoutSeconds; RetryMaxAttempts 0–10; MaxPages 1–1000.

3.14.3. IGraphClient and GraphClient

```
public interface IGraphClient
{
    Task<GraphFetchResult> GetCallRecordAsync(string callId, CancellationToken ct);
    Task<IReadOnlyList<GraphCallRecordRef>> ListCallRecordsAsync(DateTimeOffset fromUtc, DateTimeOffset toUtc, CancellationToken ct);
    Task<IReadOnlyList<GraphSubscriptionInfo>> ListSubscriptionsAsync(CancellationToken ct);
    Task<GraphSubscription> CreateSubscriptionAsync(GraphSubscriptionRequest request, CancellationToken ct);
    Task<GraphSubscription> RenewSubscriptionAsync(string subscriptionId, DateTimeOffset newExpiresUtc, CancellationToken ct);
    Task DeleteSubscriptionAsync(string subscriptionId, CancellationToken ct);
    Task SendActivityNotificationAsync(ActivityNotification notification, CancellationToken ct);
}
```

GraphClient(HttpClient http, TokenCredential credential, IOptions<GraphOptions> options, ILogger<GraphClient> log); registered as a typed client (3.14.5). http.BaseAddress is GraphOptions.BaseUrl with a trailing /.

General request rules:

1. Token: credential.GetTokenAsync(new TokenRequestContext(new[] { Scope }), ct); Authorization: Bearer <token> header. Token caching and refresh are handled by ClientSecretCredential. Token acquisition failure (AuthenticationFailedException) becomes GraphException(isTransient: false, statusCode: null), with message "token acquisition failed".
2. Add the token only to requests whose address has the same host as BaseUrl; a continuation link with a different host throws GraphException(isTransient: false).
3. GET requests have no side effects; POST requests with uncertain outcomes are not blindly retried.
4. A 2xx response is parsed; all other responses are classified according to the table. HttpRequestException and TaskCanceledException without caller cancellation (ct.IsCancellationRequested = false) become GraphException(isTransient: true), with outcomeUnknown: true additionally for POST methods. BrokenCircuitException becomes GraphException(isTransient: true) (request was not sent).

Method	Request	Algorithm and Result
GetCallRecordAsync(callId)	GET communications/callRecords/{Uri.EscapeDataString(callId)}?$expand=sessions($expand=segments)	1. HTTP 404 → (NotFound,null). 2. Parse body into JsonNode root; if this or any later page is not JSON, return (Malformed,text of that body) without throwing. 3. Session pages: while root contains sessions@odata.nextLink, if MaxPages pages have already been read, return (PagesIncomplete,root.ToJsonString()); otherwise GET that link (validating its host), append its value elements to root["sessions"], replace the link with the page's @odata.nextLink or remove it. 4. Segment pages: for each session, likewise follow segments@odata.nextLink within that session's segments array; MaxPages applies separately to each collection. 5. Remove sessions@odata.nextLink from root and segments@odata.nextLink from sessions. 6. Return (Ok,root.ToJsonString()).
ListCallRecordsAsync(from, to)	GET communications/callRecords?$filter=startDateTime ge {from} and startDateTime lt {to} (values use UtcText.Format; URI query is encoded by Uri.EscapeDataString for each component)	Follow @odata.nextLink until absent or MaxPages exceeded (then GraphException(isTransient: false)); from each value item obtain id (required; otherwise skip with warning), version (number or null), startDateTime (time), and required metadata
CreateSubscriptionAsync(request)	POST subscriptions	body from table below; result is id, resource, expirationDateTime from response
RenewSubscriptionAsync(id, newExpires)	PATCH subscriptions/{id}	body {"expirationDateTime":"<UtcText.Format(newExpires)>"}; result is id, resource, expirationDateTime from response (use empty string if resource absent)
DeleteSubscriptionAsync(id)	DELETE subscriptions/{id}	HTTP 204 or 404 — success (subscription already absent)
SendActivityNotificationAsync(n)	POST users/{n.RecipientUserId}/teamwork/sendActivityNotification	body from table below; HTTP 204 indicates success

Method	Request	Algorithm and Result
ListSubscriptionsAsync()	GET subscriptions	Follow @odata.nextLink pages up to MaxPages (if exceeded, throw GraphException(isTransient:false)); from each value item extract id, resource, notificationUrl and expirationDateTime (skip with warning when any field is missing or expiration time cannot be parsed); return IReadOnlyList<GraphSubscriptionInfo>. This finds subscriptions created when an earlier request had an unknown outcome.

POST subscriptions body [P-proposed]:

Field	Value
changeType	"created"
notificationUrl	request.NotificationUrl
lifecycleNotificationUrl	the same address
resource	request.Resource
expirationDateTime	UtcText.Format(request.ExpiresUtc)
clientState	request.ClientState

sendActivityNotification body [P-proposed, P-19]:

```
{
 "topic": { "source": "text", "value": "<n.TopicText>", "webUrl": "<GraphOptions.ActivityTopicWebUrl>" },
 "activityType": "<n.ActivityType>",
 "previewText": { "content": "<n.PreviewText>" },
 "templateParameters": [ { "name": "<key>", "value": "<value>" } ]
}
```

templateParameters entries are pairs from n.TemplateParameters in enumeration order. The callCompleted activity type is declared in the Teams app manifest (Part 4).

3.14.4. Resilience

Registration (AddTcriGraphClient, 3.18) adds AddResilienceHandler to HttpClient with policies configured by GraphOptions [P-18]:

Policy	Parameters
Total Timeout	TotalTimeoutSeconds
Retry	MaxRetryAttempts=RetryMaxAttempts; BackoffType=Exponential; UseJitter=true; Delay=RetryBaseDelayMilliseconds; ShouldRetryAfterHeader=true (honor Retry-After). ShouldHandle depends on the request method (args.Outcome.Result.RequestMessage.Method): for GET, PATCH and DELETE, retry 408, 429, 5xx and network exceptions; for POST, retry only 429 (the server refused without executing the request). Retrying a POST after timeout, network fault or 5xx is prohibited: the request may have succeeded and repeating could create a second subscription or notification [P-20].

Policy	Parameters
Attempt Timeout	RequestTimeoutSeconds
Circuit Breaker	MinimumThroughput = 5, FailureRatio = 0.5, SamplingDuration = 60 s, BreakDuration = 30 s

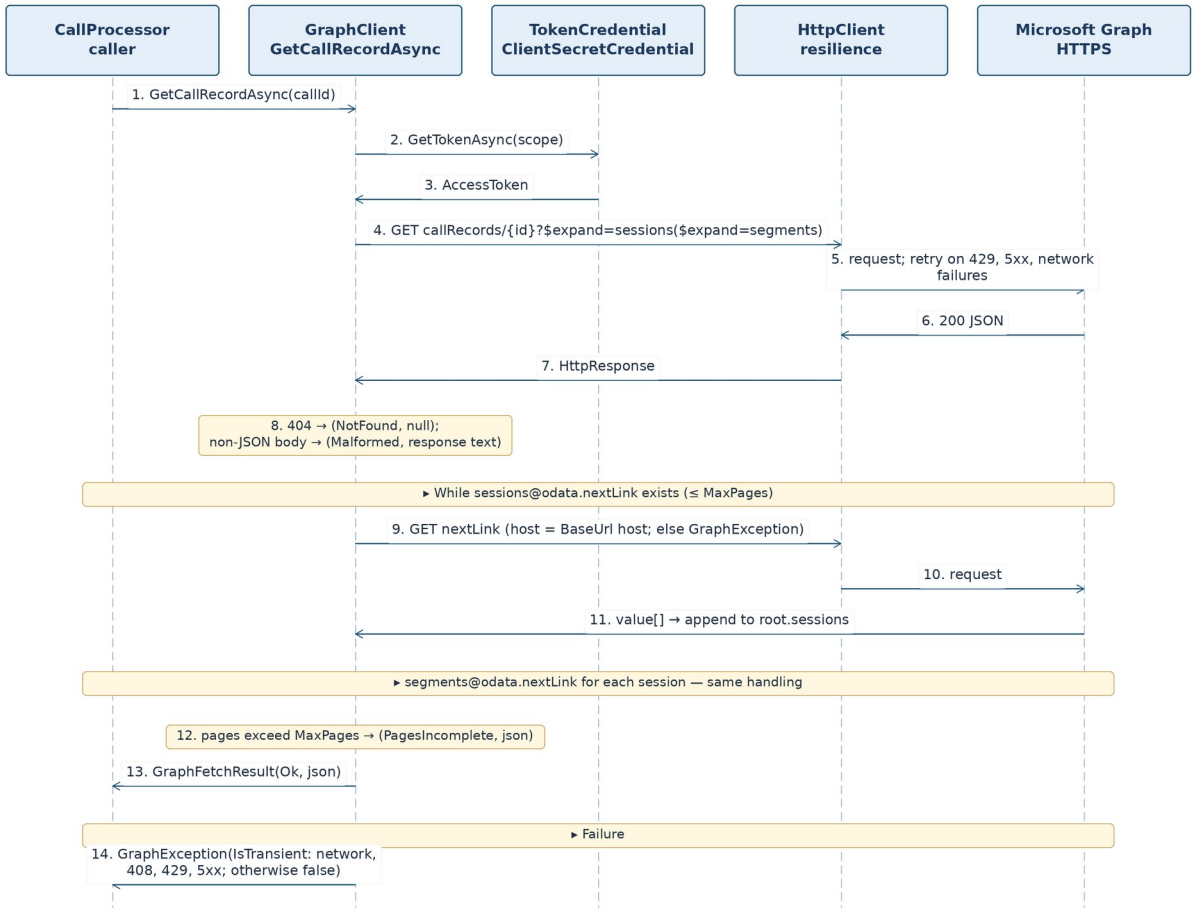
After retries are exhausted, the last status (429/5xx) is returned for GraphClient to classify according to Table 3.14.1 (with OutcomeUnknown set for POST).

Delivery guarantees [P-20]. Reading, renewing, and deleting a subscription use “at least once” (retry is safe). Subscription creation uses “at most once” at request level; possible duplicates after an unknown outcome are detected and removed by SubscriptionService using ListSubscriptionsAsync (3.15.3). Activity feed notification is “at most once” on an unknown outcome (the outbox item is closed as failed with OUTCOME_UNKNOWN and is not automatically resent), and “at least once” only after a definite 429 refusal where no delivery occurred. The owner may manually resend while accepting the duplicate risk.

3.14.5. Diagram Reference

The multi-page call-record request sequence appears in Figure 8.

Figure 8. GraphClient: Paginated callRecord Retrieval. Mermaid text diagram; steps and numbering match the PDF/DOCX image.



3.15. Subscriptions: Graph Subscriptions

3.15.1. SubscriptionOptions (Subscriptions/SubscriptionOptions.cs)

Property	Key	Default	Purpose
CallRecordsLifetimeMinutes	Subscriptions:CallRecordsLifetimeMinutes	4230	call record subscription lifetime [P-as: Graph limit must be tested in tenant] [P-16]
ChannelMessagesLifetimeMinutes	Subscriptions:ChannelMessagesLifetimeMinutes	60	channel message subscription lifetime [P-as]

Property	Key	Default	Purpose
TeamId	Subscriptions:TeamId	empty	team for F3
ChannelId	Subscriptions:ChannelId	empty	channel for F3; create message subscription only when both values are specified
RenewCheckMinutes	Subscriptions:RenewCheckMinutes	5	check interval (used by Host service)

Validation rules: lifetime between 10 and 43,200 minutes; RenewCheckMinutes between 1 and 60; TeamId and ChannelId must both be specified or both empty.

3.15.2. ISubscriptionStore and SubscriptionStore

SubscriptionStore(IWriteGate gate, IDbContextFactory<TcriDbContext> factory); singleton.

Member	Signature	Description
List	Task<IReadOnlyList<SubscriptionRow>> ListAsync(CancellationToken ct)	all rows, sorted by created_utc
Get	Task<SubscriptionRow?> GetAsync(string id, CancellationToken ct)	row or null
Upsert	Task UpsertAsync(SubscriptionRow row, CancellationToken ct)	insert or replace all columns by id
SetStatus	Task SetStatusAsync(string id, string status, CancellationToken ct)	change status; do nothing if row absent
Delete	Task DeleteAsync(string id, CancellationToken ct)	delete row; do nothing if absent
HashClientState	public static string HashClientState(string clientState)	SHA-256 of UTF-8 text, lowercase hex, 64 characters
Matches	public static bool Matches(string storedHash, string clientState)	constant-time comparison of storedHash and HashClientState(clientState) (CryptographicOperations.FixedTimeEquals on UTF-8 bytes; false on different lengths)

ClientState is not stored in the database: only a hash is written to client_state_hash.

3.15.3. SubscriptionService (Subscriptions/SubscriptionService.cs)

SubscriptionService(IGraphClient graph, ISubscriptionStore store, IInboxStore inbox, IErrorLog errors, IOptions<GraphOptions> graphOptions, IOptions<SubscriptionOptions> options, TimeProvider clock, ILogger<SubscriptionService> log); class is stateless and thread-safe.

Member	Signature
Ensure	Task<bool> EnsureAsync(CancellationToken ct)
HandleLifecycle	Task HandleLifecycleAsync(string subscriptionId, string lifecycleEvent, CancellationToken ct)

Desired subscriptions desired: (1) kind callRecords, resource communications/callRecords, lifetime CallRecordsLifetimeMinutes; (2) when TeamId and ChannelId are provided, kind channelMessages, resource teams/{TeamId}/channels/{ChannelId}/messages, lifetime ChannelMessagesLifetimeMinutes.

EnsureAsync algorithm (invoked at startup and every RenewCheckMinutes; Figure 9). Return true when an active row exists for each desired subscription:

1. rows = store.ListAsync; allOk = true.

2. For each d in desired:
    1. mine — rows of the same kind; best — the row among mine with state active or renewing, matching resource and expires_utc > now, with the greatest expires_utc.
    2. For all other rows of kind d.Kind except best, call graph.DeleteSubscriptionAsync (ignore errors but log them), then store.DeleteAsync.
    3. Inside try: if best = null, Create(d); otherwise, if best.expires_utc − now < lifetime/2, RenewOrReplace(best, d).
    4. GraphException ex — errors.WriteAsync(Components.Subscriptions, ...), allOk = false.
3. Rows of kinds absent from desired are not deleted by the service (the kind is disabled by configuration; its row expires naturally).

Create(d) (create with duplicate protection after an uncertain outcome [P-20]):

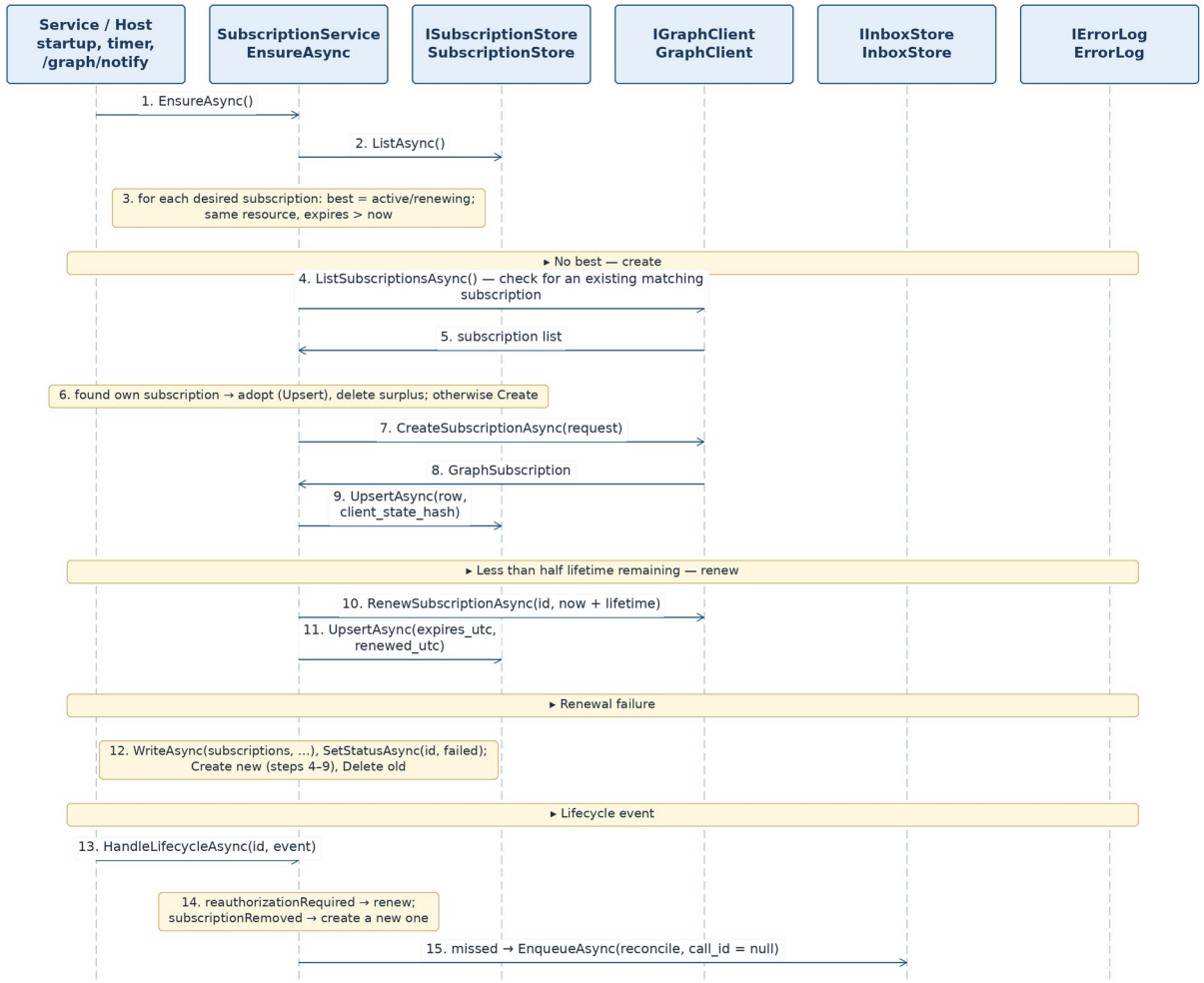
1. known = graph.ListSubscriptionsAsync; found — known items with Resource = d.Resource, NotificationUrl = GraphOptions.NotificationUrl, and ExpiresUtc > now.
2. If found is nonempty, a subscription already exists in Graph (previously created but the response or saved row was lost): choose keep with the greatest ExpiresUtc; delete the others using DeleteSubscriptionAsync (log errors); call store.UpsertAsync for keep (client_state_hash=HashClientState(ClientState), status='active', other fields as below, created_utc=now); create no new subscription; finish. Limitation: a subscription is considered ours based on its resource and notification URL [P-as].
3. Otherwise call graph.CreateSubscriptionAsync(new(d.Kind, d.Resource, GraphOptions.NotificationUrl, now + lifetime, GraphOptions.ClientState)); upon GraphException with OutcomeUnknown, write error_log (subscription create outcome unknown), set allOk=false, and do not retry in this pass: the next EnsureAsync starts with steps 1–2 and discovers the created subscription. Then call store.UpsertAsync for a row: id and expires_utc from the Graph response, resource=d.Resource, notification_url=GraphOptions.NotificationUrl, client_state_hash=HashClientState(ClientState), status='active', created_utc=now, renewed_utc=NULL.

RenewOrReplace(row,d): newExp=now + lifetime; call graph.RenewSubscriptionAsync(row.Id,newExp); on success, call store.UpsertAsync on the same row with expires_utc from the response, status='active', renewed_utc=now. On any GraphException: call errors.WriteAsync(Components.Subscriptions, "subscription renewal failed", ...), store.SetStatusAsync(row.Id,failed), Create(d), then graph.DeleteSubscriptionAsync(row.Id) (ignore errors) and store.DeleteAsync(row.Id). If Create fails again, the old row remains failed, the exception is handled under step 2.4, and the next pass retries Create (a failed row is not considered best).

HandleLifecycleAsync(id, event) algorithm:

1. row = store.GetAsync(id); if no row exists, log and return.
2. Switch on lifecycleEvent:
    - reauthorizationRequired — immediately RenewOrReplace(row, d) (d is the desired subscription of row.Kind; if this kind is not desired, only log);
    - subscriptionRemoved — store.SetStatusAsync(id, failed), Create(d), store.DeleteAsync(id); errors during Create are handled as in EnsureAsync step 2.4;
    - missed — inbox.EnqueueAsync([InboxNewItem(Reconcile, GraphOptions.TenantId, null, raw)]), with raw = {"reason":"missed","subscription_id":"<id>","at":"<UtcText.Format(now)>"}; the queue worker runs a general reconciliation (3.13.3, step 2);
    - Any other value — log a warning.

Figure 9. SubscriptionService: Creation, Renewal, and Unknown POST Outcome. Mermaid text diagram; steps and numbering match the PDF/DOCX image.



Graph errors inside the method are caught and written to error_log; they are not rethrown. Known limitation: if the process stops between creating a Graph subscription and saving the row, the subscription remains unknown to the database until the next discovery pass.

3.16. Outbox: Outbound Notifications

3.16.1. Types (Outbox/OutboxModels.cs) and Settings (Config/OutboxOptions.cs)

public sealed record OutboxItem(long Id, string TenantId, string CallId, string SessionId, string Recipient, string Payload, int Attempts);
public sealed record OutboxStats(int Pending, int Failed);
public sealed record ActivityPayload(string ActivityType, string CallType, string Status, long DurationSeconds, string StartUtc)
{
  public const string CallCompleted = "callCompleted";
  public static ActivityPayload Create(Cdr cdr); // ActivityType=CallCompleted; CallType/Status are values from CdrMapping; StartUtc is UtcText.Format(cdr.StartUtc)
  public string ToJson(); // {"activity_type":...,"call_type":...,"status":...,"duration_s":...,"start_utc":...}, with fixed property order
  public static ActivityPayload Parse(string json);// FormatException for invalid JSON or missing field
}

OutboxStats.Pending counts new and sending rows.

OutboxOptions: int MaxAttempts = 8, int LeaseSeconds = 120, string BackoffSeconds = "30,120,600,1800,3600", int MaxAgeHours = 24 [P-14]. Methods: IReadOnlyList<int> BackoffScheduleSeconds(), TimeSpan BackoffFor(int attempt) (index min(attempt − 1, n − 1); attempt < 1 throws ArgumentOutOfRangeException), IEnumerable<string> Validate().

3.16.2. IOutboxStore and OutboxStore

OutboxStore(IWriteGate gate, IDbContextFactory<TcriDbContext> factory, IOptions<OutboxOptions> options, TimeProvider clock); singleton. CdrApplier creates outbox rows (3.13.5); the store operates on existing rows.

Member	Signature	Description
ClaimNext	Task<OutboxItem?> ClaimNextAsync(CancellationToken ct)	One gate transaction. 1) Rows state='sending' AND locked_until_utc < now are moved to failed (last_error="OUTCOME_UNKNOWN: lease expired", locked_until_utc=NULL); log a warning through ILogger [P-20]. 2) Select the next row state='new' AND next_attempt_utc <= now ordered by next_attempt_utc,id; set state='sending', locked_until_utc=now + LeaseSeconds, attempts+=1.
MarkSent	Task MarkSentAsync(long id, CancellationToken ct)	state='sent', sent_utc = now, locked_until_utc = NULL, last_error = NULL
Reschedule	Task<bool> RescheduleAsync(long id, string error, CancellationToken ct)	If attempts >= MaxAttempts, set state='failed', last_error=error, locked_until_utc=NULL, return false; otherwise state='new', next_attempt_utc=now+BackoffFor(attempts), last_error=error, return true
Fail	Task FailAsync(long id, string error, CancellationToken ct)	state='failed', last_error = error
GetStats	Task<OutboxStats> GetStatsAsync(CancellationToken ct)	Read

3.16.3. IOutboxSender and OutboxSender

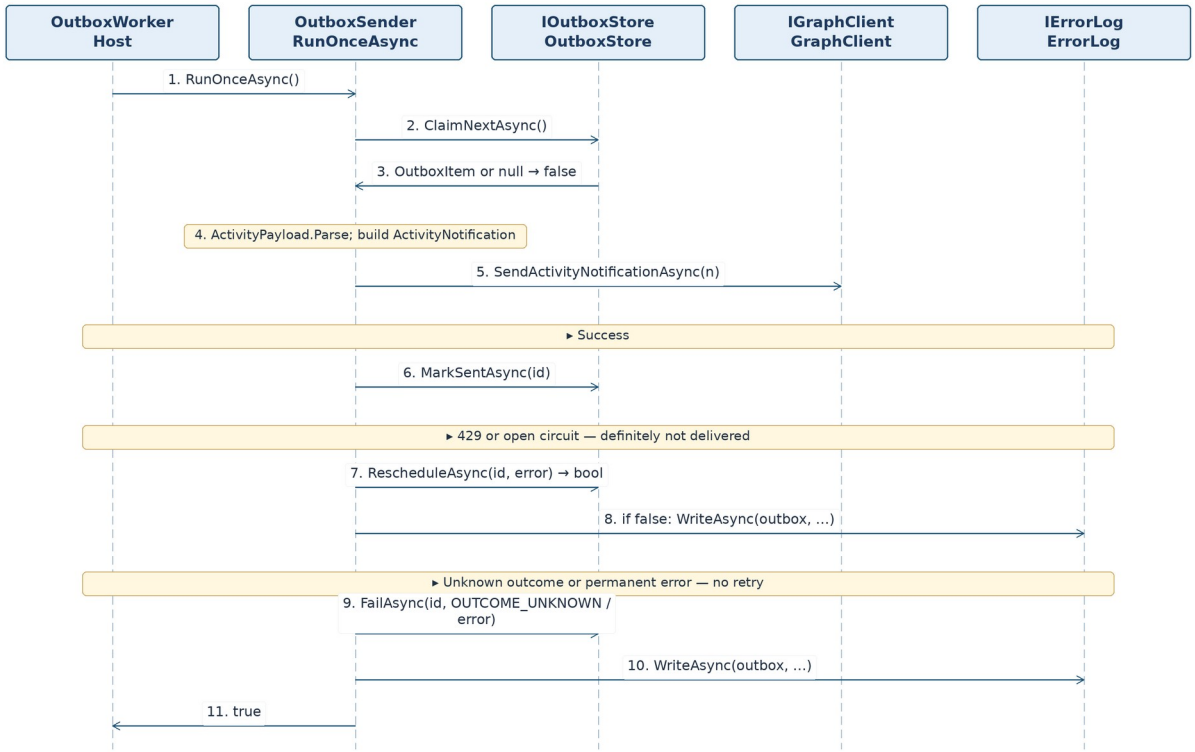
public interface IOutboxSender { Task<bool> RunOnceAsync(CancellationToken ct); }

OutboxSender(IOutboxStore store, IGraphClient graph, IErrorLog errors, TimeProvider clock, ILogger<OutboxSender> log); singleton. Returns true if an item was claimed (Host calls the service again without delay after true; after false it waits for the configured interval).

RunOnceAsync algorithm (Figure 10):

1. item = store.ClaimNextAsync; if null, return false.
2. ActivityPayload.Parse(item.Payload); on FormatException, call store.FailAsync(id, "bad payload"), errors.WriteAsync(Components.Outbox, "outbox item failed", "id=<id>; call_id=<...>; session_id=<...>; attempts=<number>"), return true.
3. Construct ActivityNotification: RecipientUserId = item.Recipient; ActivityType = payload.ActivityType; TopicText = "Teams call"; PreviewText = "Call " + payload.CallType + ", " + payload.DurationSeconds + " s, " + payload.Status; TemplateParameters in this order: callType, status, duration (seconds as a string), startUtc [P-19].
4. await graph.SendActivityNotificationAsync(n, ct); on success, store.MarkSentAsync and return true.
5. GraphException ex [P-20]:
   - ex.OutcomeUnknown (timeout, network failure, or 5xx: the server may have delivered the notification) — do not automatically retry: store.FailAsync(id, "OUTCOME_UNKNOWN: " + message), errors.WriteAsync(Components.Outbox, "outbox item failed", "id=<id>; call_id=<...>; session_id=<...>; attempts=<number>"), return true;
   - Otherwise, if ex.IsTransient (429 or an open circuit breaker: delivery definitely did not occur), ok = store.RescheduleAsync(id, message); if ok=false, call errors.WriteAsync(Components.Outbox, "outbox item failed", "id=<id>; call_id=<...>; session_id=<...>; attempts=<number>"); return true;
   - Otherwise (permanent error, such as 403 or recipient 404), call store.FailAsync(id, message) and errors.WriteAsync with the same message and details; return true.
6. Any other exception (except cancellation) is handled as an unknown outcome (step 5, first branch, using OUTCOME_UNKNOWN: <exception type>); return true.

Figure 10. Outbox: Sending Policy and Unknown Outcome. Mermaid text diagram; steps and numbering match the PDF/DOCX image.



Properties: send failure does not modify cdr, source_state, or inbox; repeat sending to one recipient for one session is prevented by uq_outbox_key and sent state. After a process crash, a sending item with expired lease is not automatically resent (outcome unknown): ClaimNextAsync closes it as failed with last_error="OUTCOME_UNKNOWN: lease expired". Adopted property: “at most once” on an uncertain outcome and “at least once” on definite refusal (429) [P-20].

3.17. Diagnostics: Error Log

IErrorLog: Task WriteAsync(string component, string message, string? details, CancellationToken ct). ErrorLog(IWriteGate gate, TimeProvider clock, ILogger<ErrorLog> log); singleton.

Algorithm: truncate message to 1000 characters and details to 4000; insert an ErrorLogRow (ts=now, component, message, details) inside a gate transaction. Catch every exception except OperationCanceledException and report only to ILogger (the error logger must not itself cause new errors). component is a constant from Components. Never pass secrets or source-record bodies into message or details (verified through review and ErrorLogTests).

Exception: when SourceStateStore.RegisterIncompleteAsync changes a call to expired, it writes error_log directly in the same transaction (one commit with the state transition).

3.18. Registration, Settings, and Startup

3.18.1. InfrastructureRegistration (InfrastructureRegistration.cs)

public static class InfrastructureRegistration:

Method	Description
IServiceCollection AddTcriInfrastructure(this IServiceCollection services, IConfiguration configuration)	Registration per table below
IHttpClientBuilder AddTcriGraphClient(this IServiceCollection services)	Typed IGraphClient → GraphClient; BaseAddress from GraphOptions.BaseUrl; resilience handler from 3.14.4
Task MigrateAsync(this IServiceProvider services, CancellationToken ct)	Create Storage:DataDirectory; call Database.MigrateAsync through context factory; then PRAGMA journal_mode must return wal, otherwise InvalidOperationException; log applied migration version

AddTcriInfrastructure registrations:

Service	Implementation	Lifetime
TimeProvider	TimeProvider.System (TryAddSingleton)	singleton
SqlitePragmaInterceptor	—	singleton
IDbContextFactory<TcriDbContext>	AddDbContextFactory: UseSqlite("Data Source=<DataDirectory>/<DatabaseFileName>;Mode=ReadWriteCreate;Pooling=True"), AddInterceptors(SqlitePragmaInterceptor)	singleton
IWriteGate	WriteGate	singleton
IInboxStore, IOutboxStore, ISourceStateStore, ISubscriptionStore, IErrorLog, ICdrReader	InboxStore, OutboxStore, SourceStateStore, SubscriptionStore, ErrorLog, CdrReader	singleton
ICdrApplier, ICallProcessor, IReconciler, IOutboxSender	CdrApplier, CallProcessor, Reconciler, OutboxSender	singleton
SubscriptionService	—	singleton
TokenCredential	new ClientSecretCredential(TenantId, ClientId, ClientSecret) (TryAddSingleton)	singleton
IGraphClient	GraphClient through AddTcriGraphClient	typed client

Bind settings sections Storage, Inbox, Processing, Reconcile, Outbox, Graph, Subscriptions; register IValidateOptions<T> for each class, calling Validate(); ValidateOnStart() aborts startup on invalid settings.

3.18.2. Settings Validation Rules

Class	Validate() Rules
StorageOptions (DataDirectory = "./data", DatabaseFileName = "tcri.db")	both values must be nonempty; file name must not contain path separators
InboxOptions (MaxPending = 10000)	1 to 1,000,000
ProcessingOptions	Workers 1–16; LeaseSeconds 10–3600; RetrySchedule must be a nonempty list of positive integers; PendingLimitHours 1–72; SuspendedRecheckHours 1–168; MaxAttempts 1–100
ReconcileOptions	IntervalMinutes 1–1440; LookbackHours 1–720
OutboxOptions	MaxAttempts 1–100; LeaseSeconds 10–3600; BackoffSeconds must be a nonempty list of positive integers; MaxAgeHours 1–720
GraphOptions, SubscriptionOptions	See 3.14.2 and 3.15.1

3.18.3. Infrastructure Startup Order

Settings (validation) → MigrateAsync → SubscriptionService.EnsureAsync → inbox workers → sender → reconciliation. After a restart, expired processing leases on inbox and outbox items are reclaimable; committed notifications are retained across restarts.

3.19. Golden Database State Scenarios

Scenarios specify expected table states after processing. They are run by ScenarioTests (3.20) and serve as worked validation examples for the implementer.

Common conditions. Clean database after migration; FakeTimeProvider initial time T0=2026-10-20T18:00:00.0000000Z; tenant 11111111-1111-4111-8111-111111111111; configuration uses defaults from 3.18 (Processing:LeaseSeconds=120, RetrySchedule=1,5,15,60, PendingLimitHours=4, SuspendedRecheckHours=24, MaxAttempts=10; Outbox:MaxAttempts=8, BackoffSeconds=30,120,600,1800,3600, MaxAgeHours=24; Graph:RetentionDays=30). FakeGraphClient returns the named fixture as GraphFetchResult(Ok,<file>) or the configured error. Times on 2026-10-20 appear as HH:MM:SS in scenario tables; in the database they have the complete form 2026-10-20T12:00:00.0000000Z. Hashes are displayed using their first 12 characters; full values are in golden files from 3.6 and Appendix B. cdr_seq starts at 1. Processing time (T0=18:00) is later than the end of all fixture calls (latest is G09 at 16:05), so no scenario processes a record before its call ends. Scenarios S01–S05 run in sequence using one database; the others use a fresh database each, unless specified otherwise.

Inbox element for callRecord: {"subscriptionId":"11111111-aaaa-4aaa-8aaa-111111111111","changeType":"created","resource":"communications/callRecords/<call_id>","resourceData":{"id":"<call_id>"},"tenantId":"11111111-1111-4111-8111-111111111111"}; for reconcile: {"call_id":"<call_id>","version":<version>}. Column element_hash equals InboxStore.ElementHash(kind, tenant, raw) and is omitted from the scenario tables.

S01. First Application of Record G01

Initial state. Database is empty. Inbox item 1 is queued (callRecord, call_id = cccccccc-0000-4000-8000-000000000001). Graph returns G01 (version 4).

Action. At T0, ClaimNextAsync() → ProcessAsync(item).

Expected. ProcessOutcome.Applied; ApplyResult(Applied, ChangedAction=null, Inserted=1, Updated=0, Deleted=0, OutboxCreated=2).

cdr table:

cdr_seq	session (…)	source_version	duration_s	status	source_hash	created_utc	updated_utc
1	01	4	61	completed	604b888cbcb6…	18:00:00	18:00:00

First row in full: schema_version=1, event_type='call', tenant_id, call_id and session_id as in the golden sample, call_type='oneToOne', caller_type='user', caller_id='aaaaaaaa-0000-4000-8000-000000000001', callee_type='user', callee_id='aaaaaaaa-0000-4000-8000-000000000002', start_utc='2026-10-20T09:00:00.0000000Z', end_utc='2026-10-20T09:01:00.5000000Z', modalities='["audio"]', termination_reason=NULL, segment_count=1, aggregation_status='complete', extensions — the JSON extensions property from the golden sample ({"segments":[{"id":"seg-1",…}]}); all other values are as in out/G01_one_to_one_single_session.json.

source_state table:

call_id	applied_version	rejected_version	session_set_hash	state	attempts	next_attempt_utc	first_seen_utc
…0001	4	NULL	146719861efe…	done	0	NULL	18:00:00

outbox table (recipients are parties of type user, in caller-then-callee order):

id	recipient	state	attempts	next_attempt_utc	payload
1	aaaaaaaa-…0001	new	0	18:00:00	{"activity_type":"callCompleted","call_type":"oneToOne","status":"completed","duration_s":61,"start_utc":"2026-10-20T09:00:00.0000000Z"}
2	aaaaaaaa-…0002	new	0	18:00:00	the same

inbox table: row 1 — state='done', attempts=1, done_utc=18:00:00, locked_until_utc=NULL. The rejected, audit_log, and error_log tables are empty.

S02. Repeat of the Same Version (Continuation of S01)

Action. At 18:05:00, inbox item 2 is queued (reconcile, call_id = cccccccc-0000-4000-8000-000000000001, version = 4); Graph returns G01; ClaimNextAsync() → ProcessAsync.

Expected. ProcessOutcome.Duplicate; cdr still has one row (updated_utc=18:00:00), outbox has two rows, source_state is unchanged; inbox row 2 — done, done_utc=18:05:00.

S03. Same Version with Different Content (Continuation of S02)

Action. At 18:10:00, item 3 (reconcile, version 4); Graph returns D01 (version 4, stream direction changed in the first session; Appendix B).

Expected. ProcessOutcome.Discrepancy. audit_log table:

id	ts	call_id	kind	detail
1	18:10:00	…0001	version_discrepancy	{"version":4,"applied_set_hash":"146719861efe9004a28b3e18b1b10301ee5a5dd868aaa6b3ce4ef21ea8195381","incoming_set_hash":"27fdd2c1c47948bf811635d4b2e0118da39b64facf4637d46e0618f7acdcc8d4"}

cdr, source_state, and outbox are unchanged; item 3 — done.

S04. Higher Version Replaces CDR (Continuation of S03)

Action. At 18:15:00, item 4 (reconcile, version 5); Graph returns G11.

Expected. ProcessOutcome.Applied; ApplyResult(Applied, null, Inserted=0, Updated=1, Deleted=0, OutboxCreated=0).

cdr_seq	session (…)	source_version	duration_s	status	source_hash	created_utc	updated_utc
1	01	5	61	completed	2acda2272c8f…	18:00:00	18:15:00

cdr_seq and created_utc are preserved. source_state: applied_version=5, session_set_hash=3d0a0015d9fc…, state='done'. outbox still has two rows (no new notification is created for a session that already existed). Item 4 — done.

S05. Lower Version Is Ignored (Continuation of S04)

Action. At 18:20:00, item 5 (reconcile, version 4); Graph returns G01.

Expected. ProcessOutcome.IgnoredLower; cdr, source_state, outbox, and audit_log unchanged; item 5 — done, done_utc=18:20:00.

Continuation (b): obsolete incomplete version. At 18:25:00, item 6 (reconcile, version 3); Graph returns D04 (version 3, missing session end: SESSION_DATES_MISSING; Appendix B). The version decision precedes the completeness check (3.13.3, steps 8–9): version 3 is lower than applied version 5.

Expected (b). ProcessOutcome.IgnoredLower; source_state remains state='done', applied_version=5, attempts=0, next_attempt_utc=NULL (not moved to pending); cdr, outbox, and audit_log unchanged; item 6 — done, done_utc=18:25:00.

S06. Replacing the Set and Removing a Session (Clean Database)

Action. (1) At T0, a callRecord item for cccccccc-0000-4000-8000-000000000002 is processed; Graph returns G02 (version 7, two sessions). (2) At 18:10:00, reconcile item (version 8) is processed; Graph returns D02 (version 8, only the first session remains).

Expected after (1). Applied, Inserted=2, OutboxCreated=4. cdr table:

cdr_seq	session (…)	source_version	duration_s	status	source_hash	updated_utc
1	01	7	600	completed	c1d4d9ba4243…	18:00:00
2	02	7	450	completed	bbab6a67cc35…	18:00:00

outbox table: rows 1–2 — session …01 (recipients aaaaaaaa-…0001, aaaaaaaa-…0003); rows 3–4 — session …02 (recipients aaaaaaaa-…0002, aaaaaaaa-…0003); all state='new'.

Expected after (2). Applied, Inserted=0, Updated=1, Deleted=1, OutboxCreated=0. cdr table:

cdr_seq	session (…)	source_version	duration_s	source_hash	updated_utc
1	01	8	600	c1d4d9ba4243…	18:10:00

Session …02 row (cdr_seq=2) was deleted; source_state.applied_version=8, session_set_hash=3bc4de956866…. All four outbox rows remain (notification history).

S07. Incomplete Data: Pending, Suspended (Clean Database)

Initial state. Item 1 (callRecord, call_id = cccccccc-0000-4000-8000-000000000007); Graph returns G07 (no sessions, NO_SESSIONS) on every attempt. Each attempt occurs when the item's next_attempt_utc is due; processing is instantaneous.

Expected after each attempt.

attempt	time	source_state .attempts	source_state. state	next_attempt_utc	inbox: state	inbox.next_a ttempt_utc
1	18:00:00	1	pending	18:01:00	new, attempts=0	18:01:00
2	18:01:00	2	pending	18:06:00	new, attempts=0	18:06:00
3	18:06:00	3	pending	18:21:00	new, attempts=0	18:21:00
4	18:21:00	4	pending	19:21:00	new, attempts=0	19:21:00
5	19:21:00	5	pending	20:21:00	new, attempts=0	20:21:00
6	20:21:00	6	pending	21:21:00	new, attempts=0	21:21:00
7	21:21:00	7	pending	22:21:00	new, attempts=0	22:21:00
8	22:21:00	8	suspended	2026-10-21 22:21:00	done	—

first_seen_utc=18:00:00; last_error='NO_SESSIONS'. At attempt 7, age is 3 h 21 min, below PendingLimitHours; at attempt 8 (22:21:00), age is 4 h 21 min, so state='suspended', next_attempt_utc=2026-10-21 22:21:00 and the item is closed (done). Final ProcessOutcome for attempts 1–7 is Pending; for attempt 8 it is Suspended. cdr, outbox, and rejected are empty.

S08. Invalid Session (Clean Database)

Action. Item 1 (callRecord, call_id = cccccccc-0000-4000-8000-000000000009); Graph returns G09 (version 6; session ...02 ends before it starts).

Expected. ProcessOutcome.Rejected. rejected table:

id	call_id	session_id	code	message	source_hash	created_utc
1	...0009	dddddddd-... 0002	SESSION_END_BEFORE_START	Session ends before it starts.	199e254b3e8 0...	18:00:00

source_state: applied_version=NULL, rejected_version=6, session_set_hash=NULL, state='done', attempts=0, first_seen_utc=18:00:00. cdr and outbox tables are empty (no replacement was performed; valid session ...01 was not stored); item 1 — done.

The rejected source_hash is the canonical hash of the entire G09 record (RejectedSourceHash, 3.13.3): 199e254b3e80b676c852b2bc8a03dd6d8af198a8750d94c51b6c79aad94c460a. It is insensitive to whitespace and property ordering.

Repeat. At 18:05:00, item 2 (reconcile, version 6); Graph returns G09 again → ProcessOutcome.Duplicate; rejected still contains one row.

Malformed record (b). Clean database; item 1 (callRecord, call_id = cccccccc-0000-4000-8000-0000000000b1); FakeGraphClient returns GraphFetchResult(Malformed, "{ not json"). Expected: ProcessOutcome.Rejected; one rejected row: call_id='cccccccc-0000-4000-8000-0000000000b1', session_id=NULL, code='CALLRECORD_BAD_JSON', source_hash='0228f717792ffbc02f7fa058c934c10d7e91d0de743e365ebd78bfebf019254e' (SHA-256 of the UTF-8 text { not json because the JSON is not parseable); source_state is not created (version unknown); item done. Reading the same malformed record again at 18:05:00 (item 2) adds no rejected row (same call_id, code, and source_hash).

S09. Transient Graph Error: Retries and Failure (Clean Database)

Initial state. Item 1 (callRecord, call_id = cccccccc-0000-4000-8000-000000000001); FakeGraphClient throws GraphException("HTTP 503", 503, isTransient: true) on every GetCallRecordAsync call.

Expected after each attempt (attempts is the count after claim and is retained on reschedule):

attempt	time	inbox.attempts	inbox.state	next_attempt_utc	last_error
1	18:00:00	1	new	18:01:00	HTTP 503
2	18:01:00	2	new	18:06:00	HTTP 503
3	18:06:00	3	new	18:21:00	HTTP 503
4	18:21:00	4	new	19:21:00	HTTP 503
5	19:21:00	5	new	20:21:00	HTTP 503
6	20:21:00	6	new	21:21:00	HTTP 503
7	21:21:00	7	new	22:21:00	HTTP 503
8	22:21:00	8	new	23:21:00	HTTP 503
9	23:21:00	9	new	2026-10-21 00:21:00	HTTP 503
10	2026-10-21 00:21:00	10	failed	—	HTTP 503

Attempts 1–9 yield ProcessOutcome.Retry; attempt 10 (attempts = MaxAttempts) yields Failed: done_utc=2026-10-21 00:21:00; one error_log row (component='graph', message='graph request failed', details='tenant_id=11111111-1111-4111-8111-111111111111; call_id=cccccccc-0000-4000-8000-000000000001; status=503; attempts=10'). No cdr or source_state data is created.

S10. Recovery After Process Crash (Clean Database)

Action. Item 1 (callRecord) is claimed at T0 (attempts=1, locked_until_utc=18:02:00) and processing is not completed. (1) At 18:01:59, ClaimNextAsync() returns null. (2) At 18:02:01, ClaimNextAsync() returns the same item with attempts=2, locked_until_utc=18:04:01, state='processing'; ProcessAsync then completes it normally.

S11. Source Data Completed in a Later Version (Clean Database)

Action. (1) At T0, Graph returns G08 (version 2, missing session end, SESSION_DATES_MISSING). (2) At 18:01:00, Graph returns D03 (version 3, session ends at 21:05:00).

Expected after (1). Pending; source_state: state='pending', attempts=1, next_attempt_utc=18:01:00, first_seen_utc=18:00:00, applied_version=NULL; cdr is empty; item returns to new (next_attempt_utc=18:01:00, attempts=0).

Expected after (2). Applied, Inserted=1, OutboxCreated=2; cdr has row cdr_seq=1 (duration_s=300, source_version=3, source_hash 99fc67b0658f…); source_state: state='done', applied_version=3, session_set_hash 8ba282b50823…, attempts=0, next_attempt_utc=NULL, first_seen_utc=18:00:00; item — done.

S12. Notification Delivery: Retries and Failure (Clean Database)

Initial state. One outbox row 1 (state='new', attempts=0, next_attempt_utc=18:00:00); FakeGraphClient.SendActivityNotificationAsync always throws GraphException("HTTP 429", 429, isTransient: true) (failure without execution: delivery definitely did not occur). Each RunOnceAsync() is invoked at the due next_attempt_utc.

attempt	time	outbox.attempts	outbox.state	next_attempt_utc
1	18:00:00	1	new	18:00:30
2	18:00:30	2	new	18:02:30
3	18:02:30	3	new	18:12:30
4	18:12:30	4	new	18:42:30
5	18:42:30	5	new	19:42:30
6	19:42:30	6	new	20:42:30
7	20:42:30	7	new	21:42:30
8	21:42:30	8	failed	—

RunOnceAsync() returns true on each attempt. After the eighth attempt, last_error='HTTP 429'; one error_log row (component='outbox', message='outbox item failed', details='id=1; call_id=<...>; session_id=<...>; attempts=8'). cdr, source_state, and inbox are untouched.

Continuation (b): uncertain outcome. Clean database with outbox row 1 as above; SendActivityNotificationAsync throws GraphException("timeout", null, isTransient: true, outcomeUnknown: true). Expected after one RunOnceAsync(): return true; state='failed', attempts=1, last_error='OUTCOME_UNKNOWN: timeout', next_attempt_utc unchanged; one error_log row (component='outbox', message='outbox item failed', details='id=1; call_id=<...>; session_id=<...>; attempts=1; outcome=unknown'); no retries. If the process stops between claiming and receiving a response, on the next ClaimNextAsync() a sending row with an expired lease becomes failed with last_error='OUTCOME_UNKNOWN: lease expired' and is not sent.

S13. Suspended Call Expires (Continuation of S07)

Initial state. After S07, source_state (cccccccc-0000-4000-8000-000000000007) has state='suspended', attempts=8, first_seen_utc=18:00:00, next_attempt_utc=2026-10-21 22:21:00. Clock advances to 2026-11-20T22:21:00Z; Graph returns G07.

Action. Reconciler.RecheckSuspendedAsync() returns 1 and enqueues an item (reconcile, raw={"call_id":"cccccccc-0000-4000-8000-000000000007","recheck_at":"2026-10-21T22:21:00.0000000Z"}); ClaimNextAsync() → ProcessAsync.

Expected. Age from 2026-10-20 18:00:00 to 2026-11-20 22:21:00 is 31 days 4 h 21 min > Graph:RetentionDays (30 days): ProcessOutcome.Expired; source_state: state='expired', attempts=9, next_attempt_utc=NULL; one error_log row (component='processor', message='source_state expired', details='tenant_id=11111111-1111-4111-8111-111111111111; call_id=cccccccc-0000-4000-8000-000000000007'); item done.

S14. Repeated Webhook After Done and Collision During Processing

Initial state: in a clean database, event A with element_hash=H, kind=callRecord, tenant_id=T, call_id=C, received_utc=T0. SQLite assigns all IDs. Process A to done, then accept the identical JSON as B at T0 + 1 min. Verify every state transition in sequence.

Check	inbox.id	element_hash	state	attempts	last_error	Expected
A after completion	1	H	done	1	NULL	G01 applied, one CDR
B after receiving the same notification	2	H	new	0	NULL	Added=1, not a duplicate; a subsequent Graph GET is mandatory
B claimed	2	H	processing	1	NULL	Graph GET executed again; unchanged version yields Duplicate

Collision: in a separate clean database, A(id=1, processing, attempts=1) exists and B(id=2, new, attempts=0) arrives with H. A receives a transient error and calls RescheduleAsync for T0+5 minutes.

id	state	attempts	next_attempt_utc	last_error
1	done	1	as before	MERGED_INTO:2
2	new	1	2026-10-20T18:01:00.0000000Z (B's earlier scheduled due time)	GRAPH_TRANSIENT

Strict postcondition: SELECT count(*) FROM inbox WHERE state='new' AND element_hash=H returns 1; the transaction does not violate uq_inbox_new_hash; subsequent processing of B invokes Graph. A later callRecord version may be present upon repeated notification. Tests: InboxStoreTests.Reschedule_Collision_MergesAtomically, ScenarioTests.S14_RepeatedWebhookRechecksGraph.

S15. Manual Forced Period Reconciliation (K12)

Initial state: source_state has four rows for tenant=T and distinct call_id C1–C4, with states done, pending, suspended, and expired. C1 has applied version 4; Graph list returns Version=null for all four. cdr contains only the previously accepted C1 CDR (cdr_seq=1). Time is T0.

call_id	state before	source_state.applied_version before	GraphCallRecordRef.Version	NeedsFetch(..., force:true)	New inbox
C1	done	4	NULL	true	id=1, kind=reconcile, call_id=C1, state=new
C2	pending	NULL	NULL	true	id=2, kind=reconcile, call_id=C2, state=new
C3	suspended	NULL	NULL	true	id=3, kind=reconcile, call_id=C3, state=new
C4	expired	NULL	NULL	true	id=4, kind=reconcile, call_id=C4, state=new

ReconcileAsync(from,to,force:true) returns 4 if no matching new notifications exist already. source_state and cdr remain unchanged until GET/Convert/Apply actually occurs. For force:false and Version=null, the rule also requires GET; with a proven older version and done it may ignore. Any failed fetch preserves the old C1 CDR. Out-of-range calls are not enqueued.

A second call before claiming: if all four items remain new, EnqueueAsync returns Added=0, Duplicates=4; the original IDs remain. A write failure propagates and the manual API must not report success. Tests: ReconcilerTests.Force_AllStatesEnqueued, ScenarioTests.S15_ForceRechecksAllStates.

3.20. Infrastructure Automated Tests

Project tests/TeamsCallRecords.Infrastructure.Tests, net8.0; packages per 3.10.1. Run dotnet test TeamsCallRecords.sln -c Release (together with Core tests). Tests do not contact the network or a real tenant: FakeGraphClient substitutes for Graph, and StubHandler supplies HTTP responses for GraphClient. The database is actual SQLite in a temporary directory (not :memory:) because WAL, CHECK constraints, and restart behaviour are tested. FakeTimeProvider controls time.

Common conditions. Initial test time T0 = 2026-10-20T18:00:00.0000000Z; tenant 11111111-1111-4111-8111-111111111111; configuration defaults unless a test row states otherwise. Each test creates its own database and is independent of execution order; test classes may run in parallel. Column “K/F” refers to TRS acceptance criterion (K) or TRS feature (F) covered; P denotes a proposal in Appendix A. “State as S0x” means checking all tables and rows specified by scenario 3.19.

Support Classes.

Class	Components and Behaviour
TestDb	static TestDb Create(DateTimeOffset? start=null): creates directory Path.Combine(Path.GetTempPath(),"tcri-tests",Guid.NewGuid().ToString("N")), file tcri.db, FakeTimeProvider initialized to start ?? T0, and IDbContextFactory<TcriDbContext> using SqlitePragmaInterceptor; applies Database.MigrateAsync. Members: Factory, Gate (WriteGate), Clock (FakeTimeProvider), DbPath, Task<T> ReadAsync<T>(Func<TcriDbContext,Task<T>> read) (reader without gate), TestDb Reopen() (new instance using same file after closing the prior one), Dispose() (SqliteConnection.ClearAllPools() and delete directory).
FakeGraphClient	Implements IGraphClient; has per-method response queues: EnqueueFetch(GraphFetchResult), EnqueueGolden(string name) (reads in/<name> as GraphFetchResult(Ok,text)), EnqueueThrow(GraphException), EnqueueList(IReadOnlyList<GraphCallRecordRef>). When a queue becomes empty, the last response is repeated (to model "every call throws"). Includes Calls log (method and arguments) and lists CreatedSubscriptions, RenewedSubscriptions, DeletedSubscriptions, SentNotifications.
FakeCredential	TokenCredential: returns test-token valid for +1 hour; Scopes is the list of requested scopes; ThrowNext(Exception)
StubHandler	HttpMessageHandler: Enqueue(HttpStatusCode, string? body, params (string, string)[] headers), Respond(Func<HttpRequestMessage, HttpResponseMessage>); records Requests (method, URI, headers, body); supports response delay through TimeProvider
FakeReconciler	IReconciler: call counters for ReconcileLookbackAsync, RecheckSuspendedAsync, ReconcileAsync
FakeCdrApplier	ICdrApplier: returns the configured sequence of ApplyResult values
Golden	const string Tenant; string Input(string name) and string Expected(string name) read from TestData/Golden or TestData/Derived; ConversionResult Convert(string name) calls CallRecordConverter.Convert(Input(name), Tenant); call IDs are taken from fixture content
Builders	Options builders for each class (with per-field overrides), InboxNewItem (CallRecord, Reconcile, ChannelMessage), Cdr with configured party/time/duration, and GraphException (Transient(code), Permanent(code))

Golden samples. G01–G11 inputs and expected results are linked from the Core test project (3.9); derived D01–D04 reside in TestData/Derived (Appendix B). All files are copied to the build output directory.

Test count. 285 methods in 20 classes (one Theory counts as one method; Theory data sets are excluded). All counts are design expectations; the tests have not been executed.

3.20.1. SchemaTests (SchemaTests.cs, 8 tests): Schema, Connection, Restart

Method	Input	Expected	K/F
Migration_CreatesEightTables	TestDb.Create() (runs MigrateAsync)	sqlite_master contains exactly cdr, source_state, inbox, rejected, audit_log, outbox, subscription, error_log (and internal __EFMigrationsHistory)	F7
Migration_CreatesNamedIndexesAndKeys	the same	Indexes present: uq_cdr_key, ix_cdr_start, ix_cdr_call, ix_source_state_due, uq_inbox_new_hash, ix_inbox_hash, ix_inbox_call, ix_inbox_due, ix_rejected_created, uq_outbox_key, ix_outbox_due, ix_error_log_ts; uq_inbox_hash absent	F7
Migration_IsIdempotent	MigrateAsync invoked a second time	no exception; schema and __EFMigrationsHistory unchanged (one 20261021000000_InitialCreate entry)	F7
Connection_UsesWalFullForeignKeysBusyTimeout	new context-factory connection opened	PRAGMA journal_mode=wal; PRAGMA synchronous=2; PRAGMA foreign_keys=1; PRAGMA busy_timeout=5000	F7, P-08
CheckConstraint_RejectsUnknownState	INSERT INTO inbox with state='bogus' through raw SQL	SqliteException (CHECK violation); row not inserted	F7
Inbox_UniqueIndexIsPartial	three direct-SQL inserts sharing element_hash: (new, new); (done, new); (done, done)	first pair raises SqliteException (uq_inbox_new_hash violation); second and third succeed	K12, P-22
Model_ColumnsMatchMigratedTables	Compare all *Row properties with PRAGMA table_info	column names equal property snake_case; NOT NULL matches nonnullable properties; no extra or missing columns	F7
Data_SurvivesRestart	write error_log row; close TestDb; open another TestDb over the same file	row is read with the same values	F7, K7

3.20.2. CdrMappingTests (CdrMappingTests.cs, 6 tests): Cdr ↔ CdrRow Mapping

Method	Input	Expected	K/F
RoundTrip_AllGoldenCdrs (Theory, files G01–G06, G09, G10, G11, D02, D03)	for each golden Cdr: ToRow → save → read → ToCdr	obtained Cdr equals source in all fields, including ExtensionsJson, Modalities, SourceHash	K8, K13
Ticks_ArePreservedWithoutLoss	StartUtc = 2026-10-20T09:00:00.1234567Z	database text 2026-10-20T09:00:00.1234567Z; ticks match on reading	K13
NullCaller_WritesBothColumnsNull	G05 (Caller=null, Callee=null)	caller_type, caller_id, callee_type, callee_id are NULL; readback produces null	K8
Modalities_AreJsonArray	Modalities = [audio, video]	modalities = ["audio","video"]; readback returns the same values in the same order	K8

Method	Input	Expected	K/F
CopyOnUpdate_KeepsSeqAndCreated	existing row (cdr_seq=1, created_utc=18:00:00) and a new row with different fields	result: cdr_seq=1, created_utc=18:00:00, other fields updated, updated_utc equals supplied time	K11
ToRow_SetsCreatedAndUpdated	ToRow(cdr, "...18:00:00")	created_utc = updated_utc = supplied value	K11

3.20.3. UtcTextTests (UtcTextTests.cs, 5 tests): Database Time Format

Method	Input	Expected	K/F
Format_SevenDigitsAndZ	2026-10-20T12:00:00Z	2026-10-20T12:00:00.0000000Z	K13, P-05
Format_ConvertsOffsetToUtc	2026-10-20T12:00:00+02:00	2026-10-20T10:00:00.0000000Z	K13
Parse_RoundTripKeepsTicks	Format → Parse for a seven-fractional-digit value	original ticks preserved, Offset = 0	K13
Format_OrderEqualsLexicographicOrder	five timestamps in arbitrary order	sorting strings with string.CompareOrdinal yields chronological order	P-05
Now_UsesInjectedClock	FakeTimeProvider at 18:00:00; then advance +90 s	2026-10-20T18:00:00.0000000Z, then 2026-10-20T18:01:30.0000000Z	P-05

3.20.4. WriteGateTests (WriteGateTests.cs, 7 tests): Write Gate

Method	Input	Expected	K/F
Run_Commits	add an ErrorLogRow within RunAsync	after return, the row is visible to the reader	F7
Run_ReturnsValue	RunAsync<int> returns 42	result is 42	F7
Run_RollsBackOnException	work inserts a row and throws InvalidOperationException	exception propagates; table has 0 rows	K9
Run_ReleasesGateAfterException	a successful call follows a failed call	second call completes (gate is not blocked)	K9
Run_ExecutesOneAtATime	20 parallel calls, each increments the concurrent-inside counter, waits 5 ms, and decrements it	maximum concurrent-inside value is 1; all 20 calls complete	K12, F7
Run_CancelledBeforeEntry	cancellation token was cancelled before the call	OperationCanceledException; work was not called	K9
Readers_DoNotWaitForGate	gate held by long-running work; reader requests COUNT(*) through the factory	read completes without waiting for the gate (WAL)	F7

3.20.5. InboxStoreTests (InboxStoreTests.cs, 27 tests): Incoming Queue

Method	Input	Expected	K/F
Enqueue_Empty_ReturnsZeros	empty list	(0,0,false); table empty	K12
Enqueue_NewItem_IsAdded	one callRecord; time 18:00:00	(Added=1); row: state=new, attempts=0, next_attempt_utc=received_utc=18:00:00, raw unchanged	K12
Enqueue_SameContent_IsDuplicate	two calls with identical raw; first row still new	second: Duplicates=1; table has one row	K12
Enqueue_PropertyOrderDoesNotMatter	{"a":1,"b":2} and {"b":2,"a":1}	hashes equal; second enqueue — Duplicates=1	K12
Enqueue_SameBatchDuplicates	batch of two identical items	Added=1, Duplicates=1	K12
Enqueue_AfterFailed_AddsNewItem	row with this hash has state failed, attempts=10	Added=1; previous row unchanged; a new row added with state=new, attempts=0	K12, P-22
Enqueue_AfterDone_AddsNewItem	row with this hash has state done (another notification about the same call)	Added=1, Duplicates=0; previous row unchanged; new row added with state=new (S14a)	K11, K12, P-22
Enqueue_WhileProcessing_AddsNewItem	row with this hash has state processing	Added=1; new row has state new (S14b)	K11, K12, P-22
Enqueue_Overloaded_WritesNothing	Inbox:MaxPending=2, queue has 2 new rows	(0,0,true); row count remains 2	K12
ElementHash_Is64LowerHex	any valid raw	64 characters [0-9a-f]	K12, P-15
ElementHash_DependsOnKindAndTenant	one raw; different kind, then different tenantId	four hashes are pairwise distinct	K12
ElementHash_BadJson_Throws	raw = "{ not json"	ArgumentException	K12
ClaimNext_ReturnsOldestDue	two new rows (next_attempt_utc 17:59:00 and 17:58:00); time 18:00:00	returns row at 17:58:00; state=processing, locked_until_utc=18:02:00, Attempts=1	K12
ClaimNext_SameTimeOrderedById	two rows with equal timestamps	returns row with smaller id	K12
ClaimNext_SameCallInProcessing_IsSkipped	A (processing, active lease, call X), B (new, call X, earlier due time), C (new, call Y)	first call returns C, not B; after CompleteAsync(A), next returns B; rows with call_id=NULL do not block	K11, K12, P-22
ClaimNext_SameCallExpiredLease_DoesNotBlock	A (processing, expired lease, call X), B (new, call X)	returns row with earlier next_attempt_utc, id; no blocking	K12, P-22
ClaimNext_NothingDue_ReturnsNull	next_attempt_utc = 18:00:01, current time 18:00:00	null; row unchanged	K12
ClaimNext_SkipsLockedProcessing	processing, locked_until_utc=18:02:00, time 18:01:59	null	K12

Method	Input	Expected	K/F
ClaimNext_RecoversExpiredLease	same, time 18:02:01	same row returned: Attempts=2, locked_until_utc=18:04:01 (scenario S10)	K12
ClaimNext_ParallelCallers_NoDuplicates	5 ready rows, 12 concurrent calls	each row is returned to exactly one caller; 5 non-null results, 7 null	K12
Complete_SetsDone	claimed item	state=done, done_utc=now, locked_until_utc=NULL, last_error=NULL	K12
Reschedule_ResetAttemptsFlag	two calls: resetAttempts=true and false	state=new, next_attempt_utc=specified, last_error=text; attempts=0 / previous value	K12
Fail_SetsFailed	item being processed	state=failed, done_utc=now, last_error=text	K9
Transition_MissingId_Throws	CompleteAsync(999)	InvalidOperationException	K9
GetStats_CountsPendingOldestAndFailed	rows: 2 new (received at 17:50:00 and 17:55:00), 1 processing, 1 failed, 1 done; time 18:00:00	Pending=3, OldestPendingAgeSeconds=600, Failed=1; without rows — 0	K12
RenewLease_ExtendsOnlyProcessing	processing, expected locked_until_utc; lease still valid	returns new deadline; row updated atomically using CAS; for new, done, failed returns null	K9, K12, P-27
RenewLease_AfterReclaim_Fails	A claimed row; lease expired; B claimed it; A attempts to renew with stale deadline	null; B's deadline and state remain unchanged; A cancels work	K9, K12, P-27

3.20.6. SourceStateStoreTests (SourceStateStoreTests.cs, 22 tests): Call State

Method	Input	Expected	K/F
RegisterIncomplete_First_SetsPending	new row, reason NO_SESSIONS; time 18:00:00	("pending", 18:01:00); source_state: pending, attempts=1, first_seen_utc=18:00:00, last_error=NO_SESSIONS; new item with next_attempt_utc=18:01:00, attempts=0	K8
RegisterIncomplete_Schedule_1_5_15_60_60	7 sequential calls advancing to next_attempt_utc	next after 1, 5, 15, 60, 60, 60, 60 minutes (table S07)	K8
RegisterIncomplete_BecomesSuspendedAtLimit	first_seen_utc=18:00:00, attempts=7, time 22:00:00 (age exactly 4 h)	("suspended", 2026-10-21T22:00:00Z); item done	K8
RegisterIncomplete_BelowLimit_StaysPending	age 3 h 59 min 59 s	pending	K8
RegisterIncomplete_SuspendedRecheck_Reschedules	suspended, age 5 days	suspended, next_attempt_utc=now + 24 h; item done; error_log empty	K8
RegisterIncomplete_BecomesExpiredAfterRetention	suspended, age 30 days 1 min, RetentionDays=30	expired, next_attempt_utc=NULL; one error_log row (processor, source_state expired, tenant_id=…; call_id=…) in same commit	K8, F9

Method	Input	Expected	K/F
RegisterIncomplete_AtExactRetention_NotExpired	age exactly 30 days	suspended (strict condition: age > RetentionDays)	K8
RegisterIncomplete_Expired_StaysExpired	expired	state unchanged; item done	K8
RegisterIncomplete_NewEpisodeAfterDone	done, applied_version=4	pending, attempts=1, first_seen_utc=now, applied_version=4 preserved	K8, K11
RegisterIncomplete_MissingInbox_RollsBack	nonexistent inboxId	InvalidOperationException; source_state not created	K9
RecordRejection_G09_WritesRejectedAndState	G09 rejections; version 6; sourceHash = RejectedSourceHash(G09)	one rejected row (SESSION_END_BEFORE_START, dddddddd-...0002, source_hash=199e254b3e80b676c852b2bc8a03dd6d8af198a8750d94c51b6c79aad94c460a); source_state: done, applied_version=NULL, rejected_version=6; item done	K14, P-23
RecordRejection_KeepsMaxRejectedVersion	existing rejected_version=7; version 6 arrives	rejected_version=7	K14
RecordRejection_SameRowTwice_NotDuplicated	two calls with identical call_id, session_id, code, source_hash	one rejected row	K14, P-23
RecordRejection_OtherHash_AddsRow	same call_id and code, different source_hash	two rows	K14, P-23
RecordRejection_ClosesPendingEpisode	pending, attempts=3	done, attempts=0, next_attempt_utc=NULL	K14
RecordRejection_WithoutCallId_WritesOnlyRejected	callId=null, rejection CALLRECORD_BAD_JSON, sourceHash=SHA-256 of text { not json	rejected row with source_hash=0228f717792ffbc02f7fa058c934c10d7e91d0de743e365ebd78bfebf019254e; source_state not created	K14, P-23
RecordRejection_MessageTruncatedTo1000	message of 1500 characters	1000 characters stored	K14
RecordDiscrepancy_WritesAuditRow	version 4, hashes from D01/G01	audit_log row as in S03; cdr and source_state unchanged; item done	K11
RecordDiscrepancy_NullAppliedHash_WritesJsonNull	appliedSetHash=null	detail contains "applied_set_hash":null	K11
GetMany_ChunksOf500	1200 rows, request for 1200 identifiers	dictionary with 1200 entries (querying in batches of 500 loses none)	K12
ListSuspendedDue_OrdersAndLimits	three suspended rows with next_attempt_utc 18:03, 18:01, 18:09; time 18:05; limit=1	one row at 18:01; with limit=10, two rows (18:01, 18:03)	K8
GetCounts_GroupsByState	rows pending×2, suspended×1, expired×1, done×3	corresponding counts in SourceStateCounts	F9

3.20.7. CdrApplierTests (CdrApplierTests.cs, 20 tests): CDR Replacement Transaction

Method	Input	Expected	K/F
Apply_G01_InsertsCdrStateOutbox	ApplyRequest from G01; item processing	ApplyResult(Applied, null, 1,0,0,2); table contents as in S01	K7, K8, K11
Apply_G02_TwoSessions_FourOutboxRows	G02	Inserted=2, OutboxCreated=4; cdr_seq 1 and 2	K8
Apply_G03_OverlappingSegments_OneCdr	G03	Inserted=1; segment_count=2; duration_s as in out/G03…	K8, K13
Apply_G04_Failed_StoresReason	G04	status=failed, termination_reason as in golden fixture	K8
Apply_G05_NoParties_NoOutbox	G05	Inserted=1, OutboxCreated=0; caller_* and callee_* are NULL	K8
Apply_G06_PhoneAndUnknown_NoOutbox	G06 (phone and unknown parties)	OutboxCreated=0	K8
Apply_G10_KeepsSubSecondTime	G10	start_utc and end_utc in database equal golden fixture values (7 digits)	K13
Apply_HigherVersion_UpdatesInPlace	after S01 — G11	Updated=1; cdr_seq and created_utc unchanged; updated_utc new; outbox — 2 rows (scenario S04)	K11
Apply_ReplaceSet_DeletesMissingSession	G02, then D02	Updated=1, Deleted=1; cdr has one row; outbox — 4 rows (S06)	K11
Apply_ExistingSession_NoNewOutbox	repeated replacement with same session set, higher version	OutboxCreated=0	K11
Apply_OldEndedCall_NoOutbox	end_utc 25 h earlier than now; Outbox:MaxAgeHours=24	OutboxCreated=0; CDR persisted	F5
Apply_OutboxBoundary_ExactlyMaxAge	end_utc exactly 24 h before now	OutboxCreated equals recipient count (condition >=)	F5
Apply_SameCaller_AndCallee_OneRecipient	Caller.Id = Callee.Id, type user	one outbox row	F5
Apply_OnlyUserParties_AreRecipients	Caller type application, Callee type user	one outbox row — for callee	F5, P-14
Apply_DecisionChanged_ChangesNothing	source_state.applied_version=5; request version 4	ApplyResult(DecisionChanged, IgnoreLower, 0,0,0,0); tables unchanged	K11
Apply_SameVersionTwice_SecondIsDecisionChanged	two successive calls with same result	second: DecisionChanged (DuplicateSame); no duplicate CDR or outbox	K11
Apply_Failure_RollsBackEverything	CDR with DurationSeconds=-1 (CHECK violation) as second item in set	exception; no cdr, outbox, source_state data; item remains processing	K9

Method	Input	Expected	K/F
Apply_ClosesInboxInSameTransaction	normal apply	inbox.state=done, done_utc=now; on rollback (previous test) — processing	K12
Apply_PreservesRejectedVersion	source_state.rejected_version=3, apply version 5	rejected_version=3, applied_version=5	K14
Payload_Format_IsExact	G01	outbox.payload equals {"activity_type":"callCompleted","call_type":"oneToOne","status":"completed","duration_s":61,"start_utc":"2026-10-20T09:00:00.0000000Z"}	F5

3.20.8. CallProcessorTests (CallProcessorTests.cs, 35 tests): Queue Item Processing

Method	Input	Expected	K/F
ChannelMessage_IsLoggedAndCompleted	channelMessage item	Logged; item done; message body absent from log	F3
ReconcileWithoutCallId_RunsLookback	reconcile item without call_id	ReconcileLookbackAsync called once; ReconcileRun; item done	K12
CallRecordWithoutCallId_Fails	callRecord with call_id=NULL	Failed; last_error=INBOX_NO_CALL_ID; error_log row "inbox item without call_id"	K9
G01_Applied	S01	Applied; table states as in S01	K7, K8
SameVersion_Duplicate	S02	Duplicate; tables unchanged	K11
SameVersionOtherContent_Discrepancy	S03 (D01)	Discrepancy; audit_log row as in S03	K11
HigherVersion_ReplacesCdr	S04 (G11)	Applied, Updated=1	K11
LowerVersion_Ignored	S05	IgnoredLower; tables unchanged	K11
OlderIncompleteVersion_DoesNotReopenDone	after S04 (applied_version=5): Graph returns D04 (version 3, no session end)	IgnoredLower; source_state remains done, attempts=0; pending not created (S05b)	K8, K11, P-23
SameVersionIncomplete_AfterApplied_Discrepancy	applied_version=4; Graph returns version 4 with a different session set and incomplete record	Discrepancy (version decision precedes completeness check); source_state unchanged	K11, P-23
G07_NoSessions_Pending	G07	Pending; source_state.pending; cdr empty	K8
G08_SessionEndMissing_Pending	G08	Pending, reason SESSION_DATES_MISSING; session rejections not stored	K8, K14
G08_ThenD03_Applied	S11	after D03 — Applied; Inserted=1, duration_s=300	K8

Method	Input	Expected	K/F
G09_InvalidSession_Rejected	S08	Rejected; rejected and source_state as S08; cdr empty	K14
G09_Repeat_Duplicate	S08, repeated	Duplicate; still one rejected row	K14
G09_SourceHash_IsCanonical	G09 in original and compact representation (no whitespace, different property order)	both have source_hash=199e254b3e80b676c852b2bc8a03dd6d8af198a8750d94c51b6c79aad94c460a	K14, P-23
G09_NewerVersionValid_Applies	after S08, version 7 with corrected session	Applied; rejected_version=6 preserved	K14
BadJson_Malformed_RejectedWithRawHash	FakeGraphClient returns GraphFetchResult(Malformed, "{ not json")	Rejected, CALLRECORD_BAD_JSON, source_hash=0228f717792ffbc02f7fa058c934c10d7e91d0de743e365ebd78bfebf019254e; source_state not created; item done; service continues (S08b)	K14, K9, P-23, P-24
BadJson_Repeated_NoSecondRejectedRow	repeat same read (reconcile item)	Rejected; still one rejected row	K14, P-23
BadJson_FromGraphClientStub_IsRejectedNotFailed	StubHandler responds 200 with body { bad; real GraphClient	Rejected (not Failed); error_log empty	K14, K9, P-24
GraphNotFound_RegistersIncomplete	GraphFetchStatus.NotFound, no call state	Pending, reason GRAPH_NOT_FOUND	K12
GraphPagesIncomplete_RegistersIncomplete	PagesIncomplete	Pending, reason GRAPH_PAGES_INCOMPLETE; CDR not created	K8
GraphNotFoundOrIncomplete_AlreadyApplied_Duplicate (Theory, 2 data sets)	source_state: done, applied_version=4; Graph returns NotFound or PagesIncomplete	Duplicate; source_state unchanged (done, not pending); cdr unchanged; item done	K8, K11, P-23
TransientGraphError_Reschedules	GraphException(503, transient), attempt 1	Retry; next_attempt_utc=+1 min; attempts not reset (S09)	K9
TransientGraphError_RetryScheduleTable	10 successive attempts	delays 1, 5, 15, 60, 60, 60, 60, 60, 60 min; 10th — Failed (S09)	K9
TransientError_ExhaustedAttempts_Fails	attempts=MaxAttempts	Failed; one error_log row "graph request failed" with status=503; attempts=10	K9, F9
PermanentGraphError_FailsImmediately	GraphException(403, permanent), attempt 1	Failed; error_log: status=403; attempts=1	K9

Method	Input	Expected	K/F
UnexpectedException_Reschedules	FakeGraphClient throws InvalidOperationException	Retry; error_log "unexpected processing error"; service does not crash	K9
Cancellation_PropagatesAndKeepsProcessing	token cancelled during request	OperationCanceledException; item remains processing	K9
DecisionChangedRepeatedly_ThreeTimes_Fails	FakeCdrApplier returns DecisionChanged three times	InvalidOperationException("decision changed repeatedly") handled at step 12 → Retry	K9
SessionLevelRejectionsNotWrittenWhileTemporaryReasonsPresent	record: one session without end, another invalid	Pending; no rejected rows (P-12)	K14
SuspendedRecheck_ExpiresAfterRetention	S13	Expired; error_log as in S13	K8, F9
ReprocessSameElement_AfterLeaseLoss_IsIdempotent	processing performed twice in succession for one item (simulating lease loss)	CDR and outbox have no duplicates	K12
ReconcileWithRange_UsesRawPeriod	reconcile, CallId=null, raw from_utc/to_utc/force:true within 7 days	one ReconcileAsync(from,to,true) call, 0 ReconcileLookbackAsync calls, inbox done on success	K12, F2, P-27
ReconcileBadRange_FailsNoLookback	reconcile, CallId=null; cases: missing endpoint, interval >7 days, or force!=true	inbox failed; last_error=RECONCILE_BAD_RANGE; 0 ReconcileAsync and ReconcileLookbackAsync calls; one Theory method, 3 data sets	K9, K12, F2, P-27

3.20.9. ReconcilerTests (ReconcilerTests.cs, 15 tests): Graph Reconciliation

Method	Input	Expected	K/F
Reconcile_UnknownCall_IsEnqueued	Graph: one call version 3; source_state empty	Added=1; raw={"call_id":"<id>","version":3}; result 1	K12, F2
Reconcile_KnownSameVersion_NotEnqueued	done, applied_version=3, Graph version 3	0	K11, F2
Reconcile_KnownHigherVersion_IsEnqueued	done, applied_version=3, Graph version 4	1	K11, F2
Reconcile_VersionLowerThanRejected_NotEnqueued	rejected_version=6, Graph version 6	0	K14, F2
Reconcile_VersionNull_DoneState_IsEnqueued	Graph version null, state done	1 (NeedsFetch #5); repeat after processing item — again 1; repeat before processing — 0 (existing new row)	K11, P-21, F2
Reconcile_PendingSuspendedExpired_NotEnqueued	states pending, suspended, expired	0 (NeedsFetch table)	K8, F2

Method	Input	Expected	K/F
Reconcile_Force_EnqueuesEveryState (Theory, 5 data sets)	force=true; states: absent row, done with same version, pending, suspended, expired	1 in each case (NeedsFetch #1–2); raw={"call_id":"<id>","version":<version>} (S15)	K11, F8, P-21, F2
Reconcile_ForceFalse_SameVersionDone_NotEnqueued	force=false, done, version equals applied	0 (S15, action 1)	K11, P-21, F2
Reconcile_SecondRun_AddsNothing	two consecutive runs with unchanged Graph	second: 0	K12, F2
Reconcile_AfterFailedItem_AddsNewItem	item with same hash is failed	1; new row inserted; previous unchanged	K12, P-22, F2
Reconcile_Overloaded_ReturnsZero	Inbox:MaxPending exhausted	0; log warning; no exception	K12, F2
Reconcile_GraphError_Propagates	GraphException	exception propagates; no items inserted	K9, F2
Lookback_UsesNowMinusLookback_AndSetsLastRun	time 18:00:00; LookbackHours=24	ListCallRecordsAsync(2026-10-19T18:00:00Z, 2026-10-20T18:00:00Z); call ReconcileAsync(..., force=false); LastRunUtc=18:00:00; on error LastRunUtc unchanged	K12, F2
RecheckSuspended_EnqueuesDueRows	two suspended items: one past due, one future due	result 1; raw={"call_id":"<id>","recheck_at":"<next_attempt_utc>"}	K8, F2
RecheckSuspended_SameRowTwice_NotDuplicated	two calls without next_attempt_utc change	second: 0 (same hash)	K12, F2

3.20.10. OutboxStoreTests (OutboxStoreTests.cs, 11 tests): Outgoing Queue

Method	Input	Expected	K/F
ClaimNext_ReturnsDueNew	new row, due	state=sending, locked_until_utc=+120 s, attempts=1	F5
ClaimNext_SkipsFutureAndSent	rows new in future, sent, failed	null	F5
ClaimNext_ExpiredSending_BecomesFailedUnknown	sending, lease expired	row becomes failed, last_error="OUTCOME_UNKNOWN: lease expired", locked_until_utc=NULL; is not returned again; next new row (if any) returned	F5, K12, P-20
ClaimNext_Parallel_NoDuplicates	3 rows, 8 parallel calls	each row returned to one call	F5
MarkSent_SetsSentUtc	claimed row	state=sent, sent_utc=now, last_error=NULL	F5
Reschedule_BackoffTable	row with attempts 1…7 and HTTP 503 error	next_attempt_utc=+30, +120, +600, +1800, +3600, +3600, +3600 s	F5
Reschedule_AtMaxAttempts_Fails	attempts=8	state=failed, result false, locked_until_utc=NULL	F5
Fail_SetsFailedWithoutBackoff	FailAsync	state=failed, last_error=text	F5

Method	Input	Expected	K/F
UniqueKey_PreventsSecondRowForSameRecipient	two inserts with same key (kind, tenant, call, session, recipient)	second — DbUpdateException	F5, K12
GetStats_CountsByState	rows in all states	corresponding counts	F9
BackoffFor_IndexIsClamped	attempt = 1, 5, 6, 100	30, 3600, 3600, 3600 s; attempt=0 — ArgumentOutOfRangeException	F5

3.20.11. OutboxSenderTests (OutboxSenderTests.cs, 10 tests): Notification Delivery

Method	Input	Expected	K/F
RunOnce_NoItems_ReturnsFalse	empty queue	false; no Graph calls	F5
RunOnce_Success_MarksSent	row from S01; FakeGraphClient successful	true; state=sent; exactly one ActivityNotification received: recipient, callCompleted, TopicText="Teams call", PreviewText="Call oneToOne, 61 s, completed", parameters in this order: callType=oneToOne, status=completed, duration=61, startUtc=2026-10-20T09:00:00.0000000Z.	F5, K6
RunOnce_TransientError_Reschedules	GraphException(429, transient)	true; state=new; due in +30 s; error_log empty	F5
RunOnce_TransientError_EightAttempts_Fails	S12	8 calls with 429 response; 8th failed with error_log "outbox item failed"; all calls return true	F5, F9
RunOnce_PermanentError_FailsImmediately	GraphException(403, permanent)	state=failed, attempts=1; one error_log row	F5
RunOnce_OutcomeUnknown_FailsWithoutRetry	GraphException("timeout", null, true, outcomeUnknown: true)	true; state=failed, attempts=1, last_error="OUTCOME_UNKNOWN: timeout"; error_log: …; attempts=1; outcome=unknown; no retries (S12b)	F5, K9, P-20
RunOnce_UnexpectedException_TreatedAsOutcomeUnknown	FakeGraphClient throws InvalidOperationException	state=failed, last_error="OUTCOME_UNKNOWN: InvalidOperationException"	F5, K9, P-20
RunOnce_BadPayload_Fails	payload="{ bad"	state=failed, last_error="bad payload"; error_log written; no Graph call	F5, K9
RunOnce_SendFailure_DoesNotTouchCdr	any send error	cdr, source_state, inbox unchanged	K9
RunOnce_Cancellation_KeepsSending	token cancelled during send	OperationCanceledException; row remains sending	K9

3.20.12. GraphClientTests (GraphClientTests.cs, 31 tests): Graph Client (StubHandler Supplies HTTP Responses)

Method	Input	Expected	K/F
GetCallRecord_Ok_ReturnsJson	response 200 containing G01 body (input)	(Ok, json); request GET {BaseUrl}/communications/callRecords/<id>?$expand=sessions($expand=segments); headers Authorization: Bearer test-token, Accept: application/json	K2, F2
GetCallRecord_404_ReturnsNotFound	response 404	(NotFound, null); no exception	K2, F2
GetCallRecord_EscapesCallId	call_id = a/b c	path contains a%2Fb%20c	K2, F2
GetCallRecord_MergesSessionPages	first page has sessions@odata.nextLink, second has one session and no link	result contains all sessions in original order; no sessions@odata.nextLink key	K8, F2
GetCallRecord_MergesSegmentPages	session has segments@odata.nextLink	segments merged; segments@odata.nextLink removed	K8, F2
GetCallRecord_PageLimit_ReturnsPagesIncomplete	endless page chain, MaxPages=3	PagesIncomplete; exactly 3 page fetches	K8, F2
GetCallRecord_NextLinkOtherHost_Throws	@odata.nextLink points to different host	GraphException(isTransient=false); token not sent to foreign host	K10, F2
GetCallRecord_BadJson_ReturnsMalformed	200 body { bad	(Malformed, "{ bad"); no exception	K14, P-24, F2
GetCallRecord_BadJsonOnNextPage_ReturnsMalformed	first page valid, second { bad	(Malformed, "{ bad")	K14, P-24, F2
ListCallRecords_BadJson_Throws	200 body { bad	GraphException(isTransient=false, statusCode=null)	K9, F2
Error_Classification	codes 400, 401, 403, 404 (not for GetCallRecord), 408, 429, 500, 503	permanent: 400, 401, 403, 404; transient: 408, 429, 500, 503 (after retries exhausted)	K9, F2
Error_Network_IsTransient	HttpRequestException	GraphException(isTransient=true, statusCode=null)	K9, F2
Error_Post_OutcomeUnknown (Theory, 6 data sets)	POST: network error, timeout, 408, 500, 503, 504	IsTransient=true, OutcomeUnknown=true	K9, P-20, F2
Error_Post_DefiniteRefusal_NotUnknown (Theory, 3 data sets)	POST: 429 (after retries), 403, BrokenCircuitException	OutcomeUnknown=false; for 429 and open circuit IsTransient=true, for 403 — false	K9, P-20, F2
Error_NonPost_NeverUnknown	GET, PATCH, DELETE with network fault and 503	OutcomeUnknown=false	K9, P-20, F2
Error_Timeout_IsTransient	TaskCanceledException with ct not cancelled	isTransient=true	K9, F2
Error_CallerCancellation_Propagates	ct cancelled	OperationCanceledException without wrapper	K9, F2

Method	Input	Expected	K/F
Error_TokenFailure_IsPermanent	FakeCredential throws AuthenticationFailedException	GraphException(isTransient=false, statusCode=null), message "token acquisition failed"	K1, F2, F1
Error_Message_HasNoSecretsOrBody	500 response with "secret-body"; token "test-token"	message contains method, path without query, and status; not "secret-body" or "test-token"	K10, F2
Token_RequestedWithGraphScope	any call	FakeCredential received scope https://graph.microsoft.com/.default	K1, F2, F1
ListCallRecords_ParsesPages	two pages of 2 items each	4 links in fetched order; Version is number or null; StartUtc is time or null	K2, F2
ListCallRecords_Filter	from, to	request includes $filter=startDateTime ge 2026-10-19T18:00:00.0000000Z and startDateTime lt 2026-10-20T18:00:00.0000000Z encoded	K2, F2
ListCallRecords_SkipsItemWithoutId	item missing id	skipped; other items returned	K12, F2
ListCallRecords_PageLimit_Throws	chain exceeds MaxPages	GraphException(isTransient=false)	K9, F2
ListSubscriptions_ParsesPages	two pages, 3 subscriptions	3 GraphSubscriptionInfo items (Id, Resource, NotificationUrl, ExpiresUtc); GET subscriptions request	K3, P-20, F2
ListSubscriptions_SkipsIncompleteItems	item without id and item with invalid expirationDateTime	skipped with warning; others returned	K3, P-20, F2
CreateSubscription_Body	call subscription request	POST subscriptions; body: changeType=created, notificationUrl, lifecycleNotificationUrl=same URL, resource=communications/callRecords, expirationDateTime in UtcText format, clientState; response parsed	K3, F2
RenewSubscription_Body	renewal	PATCH subscriptions/<id>; body {"expirationDateTime":"..."}; if resource absent from response, empty string	K3, F2
DeleteSubscription_404And204_AreSuccess	responses 204 and 404	no exception in either case	K3, F2
SendActivityNotification_Body	notification from OutboxSenderTests	POST users/<id>/teamwork/sendActivityNotification; body: topic{source=text, value="Teams call", webUrl}, activityType, previewText.content, templateParameters in listed order; response 204 — success	K6, F2
SendActivityNotification_403_IsPermanent	response 403	GraphException(403, isTransient=false)	K6, F2

3.20.13. GraphResilienceTests (GraphResilienceTests.cs, 12 tests): Retries, Delays, and Circuit Breaker (Registered Pipeline, FakeTimeProvider Clock)

Method	Input	Expected	K/F
Retry_503_ThenOk_Succeeds	responses 503, 503, 200	success; 3 requests total	K9, F2
Retry_Exhausted_Returns5xx_AsTransient	503 on each request, RetryMaxAttempts=3	4 requests; GraphException(503, isTransient=true)	K9, F2
Retry_429_HonorsRetryAfter	429 response with Retry-After: 7, then 200	delay before retry no less than 7 s (FakeTimeProvider)	K9, F2
Retry_400_NotRetried	response 400	one request	K9, F2
Retry_Post_503_NotRetried	503 response to POST subscriptions	one request; GraphException(503, IsTransient=true, OutcomeUnknown=true)	K3, P-20, F2
Retry_Post_Timeout_NotRetried	handler does not respond longer than RequestTimeoutSeconds on POST	one request; OutcomeUnknown=true	K3, P-20, F2
Retry_Post_429_IsRetried	POST: 429 (Retry-After: 2), then 201	success; 2 requests; delay at least 2 s	K3, P-20, F2
Retry_Patch_503_IsRetried	PATCH: 503, then 200	success; 2 requests	K3, P-20, F2
Retry_Delete_503_IsRetried	DELETE: 503, then 204	success; 2 requests	K3, P-20, F2
Retry_BackoffGrowsExponentially	503 on every request with RetryMaxAttempts=4, RetryBaseDelayMilliseconds=500	total delay of last two retries greater than first two (growth regardless of jitter)	K9, F2
Timeout_Attempt_IsTransient	handler takes longer than RequestTimeoutSeconds	GraphException(isTransient=true)	K9, F2
CircuitBreaker_OpensAfterFailures	10 consecutive 503 responses (MinimumThroughput=5, FailureRatio=0.5)	after circuit opens, request ends with GraphException(isTransient=true, statusCode=null) without calling handler; after 30 s (clock +30 s) it passes again	K9, F2

3.20.14. SubscriptionStoreTests (SubscriptionStoreTests.cs, 8 tests): Subscription Store

Method	Input	Expected	K/F
Upsert_InsertsAndReplaces	two records with same id	one row with latest values	K3, F2
List_OrderedByCreated	three rows in arbitrary order	ordered by created_utc	K3, F2
Get_Missing_ReturnsNull	nonexistent id	null	K3, F2
SetStatus_Changes_AndMissingIgnored	existing and nonexistent id	status changed; nonexistent id does not throw	K3, F2
Delete_RemovesRow_AndMissingIgnored	the same	row deleted; no error	K3, F2
HashClientState_IsSha256Lower64	"secret-state"	64 lowercase characters; equals SHA-256 UTF-8 (expected value independently calculated in test)	K12, F2

Method	Input	Expected	K/F
Matches_ConstantTimeSemantics	valid, invalid, empty, different length	true, false, false, false	K12, F2
ClientState_IsNeverStored	Upsert through service	no plaintext ClientState value in database (search all text columns)	K10, K12, F2

3.20.15. SubscriptionServiceTests (SubscriptionServiceTests.cs, 17 tests): Creation, Renewal, Lifecycle Events

Method	Input	Expected	K/F
Ensure_NoRows_CreatesCallRecordsSubscription	empty table; TeamId empty	true; first ListSubscriptionsAsync (no subscriptions), then CreateSubscriptionAsync (resource communications/callRecords, duration 4230 min); row active, nonempty client_state_hash, expires_utc from response	K3, F2
Ensure_CreateOutcomeUnknown_NextEnsureAdoptsExisting	first EnsureAsync: CreateSubscriptionAsync throws GraphException(..., outcomeUnknown: true); Graph actually created subscription; second EnsureAsync	first: false, error_log row "subscription create outcome unknown"; second: ListSubscriptionsAsync finds subscription, CreateSubscriptionAsync not called, subscription row recorded (active); true	K3, P-20, F2
Ensure_DuplicatesInGraph_KeepsLatestDeletesOthers	two Graph subscriptions with our resource and URL (different expiry), database empty	keep one with greatest ExpiresUtc; second deleted (DeleteSubscriptionAsync); no creation	K3, P-20, F2
Ensure_ForeignNotificationUrl_IsNotAdopted	Graph subscription with same resource but different NotificationUrl	not adopted or deleted; new subscription created	K3, P-20, F2
Ensure_WithTeamAndChannel_CreatesBoth	TeamId and ChannelId specified	two subscriptions created, second — teams/{TeamId}/channels/{ChannelId}/messages, duration 60 min	K4, F2
Ensure_ActiveFarFromExpiry_DoesNothing	more than half duration remains	no Graph calls; true	K3, F2
Ensure_LessThanHalfLeft_Renews	less than half duration remains	RenewSubscriptionAsync with new expiration; expires_utc updated, renewed_utc=now	K3, F2
Ensure_RenewFails_CreatesNewAndDeletesOld	renewal — GraphException	old row marked failed → new created → old deleted; error_log entry "subscription renewal failed"	K3, F9, F2
Ensure_ExpiredRow_IsReplaced	expires_utc in the past	new subscription created; old deleted (DeleteSubscriptionAsync called)	K3, F2
Ensure_CreateFails_ReturnsFalse_AndLogs	Create — GraphException	false; error_log entry; no exception	K9, F9, F2
Ensure_ExtraRowsOfKind_AreRemoved	two active rows of same kind	one with latest expires_utc remains; other deleted from Graph and database	K3, F2
Lifecycle_ReauthorizationRequired_Renews	reauthorizationRequired event	immediate renewal; status active	K3, F2

Method	Input	Expected	K/F
Lifecycle_SubscriptionRemoved_Recreates	subscriptionRemoved event	row failed → new subscription → old removed	K3, F2
Lifecycle_Missed_EnqueuesFullReconcile	missed event	inbox reconcile item without call_id, raw={"reason":"missed","subscription_id":"<id>","at":"<time>"}	K12, F2
Lifecycle_UnknownSubscription_OnlyLogs	unknown id	tables unchanged; log entry	K12, F2
Lifecycle_UnknownEvent_OnlyLogs	other event	no changes; warning	K12, F2
Lifecycle_GraphError_IsLoggedNotThrown	renewal fails	method finishes; error_log row	K9, F2

3.20.16. CdrReaderTests (CdrReaderTests.cs, 12 tests): CDR Reading for REST API and Bot

Method	Input	Expected	K/F
Query_Interval_IsHalfOpen	CDR with start_utc exactly from, inside range, exactly to	first and middle returned; last (equal to to) excluded	K7, F8
Query_OrderedByStartThenSeq	three rows with identical start_utc	ordered by cdr_seq	K7, F8
Query_Paging_NoGapsNoRepeats	25 rows, limit=10	pages of 10, 10, 5; Next nonempty for first two and null for last; combined equal to all 25 without repeats	K7, F8
Query_Paging_StableUnderInsert	row with greater (start_utc, cdr_seq) inserted between pages	no skips or duplicates on second page	K7, F8
Query_LimitOutOfRange_Throws	limit=0 and limit=1001	ArgumentOutOfRangeException	K7, F8
Query_FromAfterTo_Throws	from > to	ArgumentException	K7, F8
Query_BadCursor_Throws	cursor="###"	ArgumentException("cursor")	K7, F8
Query_NextCursor_IsDecodable	last row of first page	Next decodes using Cursor.Decode into this row's (start_utc, cdr_seq)	K7, F8, F8
Query_ReturnsCdrSeq	any row	CdrItem.CdrSeq equals cdr_seq	K7, F8
GetStats_CountsDistinctCallsAndSumsDuration	G02 (2 CDRs, 600 and 450 s) and G01 (61 s) in window	Calls=2, TotalDurationSeconds=1111, Cdrs=3	F4, K5, F8
GetStats_EmptyWindow_ReturnsZeros	window with no CDRs	DailyStats(0,0,0)	F4, K5, F8
GetStats_HalfOpenWindow	CDR exactly on to boundary	not counted	F4, F8

3.20.17. ErrorLogTests (ErrorLogTests.cs, 5 tests): Error Log

Method	Input	Expected	K/F
Write_AddsRow	component graph, message, details; time 18:00:00	row with these values, ts=18:00:00	F9, K9

Method	Input	Expected	K/F
Write_TruncatesMessageAndDetails	message 1500 characters, details 5000 characters	1000 and 4000 characters	F9
Write_DatabaseFailure_DoesNotThrow	database unavailable (file deleted/error context)	method completes; ILogger entry (checked using FakeLogger)	K9
Write_Cancellation_Propagates	token cancelled	OperationCanceledException	K9
Write_NullDetails_Allowed	details=null	row with NULL	F9

3.20.18. OptionsTests (OptionsTests.cs, 8 tests): Settings and Registration

Method	Input	Expected	K/F
Defaults_MatchSpecification	creation of all options classes	defaults as in 3.14.2, 3.15.1, 3.16.1, 3.18.2	F9
RetryDelayFor_Table	RetrySchedule="1,5,15,60", attempts 1–6	1, 5, 15, 60, 60, 60 min; attempt 0 — ArgumentOutOfRangeException	K8
BackoffFor_Table	BackoffSeconds="30,120,600,1800,3600"	see OutboxStoreTests.BackoffFor_IndexIsClamped	F5
Validate_InvalidValues (Theory, 21 data sets)	one violation per data set: empty TenantId; NotificationUrl using http; RetentionDays=0 and 366; TotalTimeoutSeconds<RequestTimeoutSeconds; MaxPages=0; Workers=0 and 17; RetrySchedule="" and "1,0"; LeaseSeconds=9; PendingLimitHours=0; MaxAttempts=0 and 101; Reconcile:IntervalMinutes=0; Outbox:BackoffSeconds="a"; Inbox:MaxPending=0; Storage:DatabaseFileName="a/b.db"; TeamId without ChannelId; subscription lifetime 9 and 43201.	Validate() returns nonempty list of violations with option key names	F9
ValidateOnStart_ListsViolations_WithoutValues	host with ClientSecret="" and RetentionDays=0	OptionsValidationException lists key names; no option values in message	K10
AddTcriInfrastructure_ResolvesAllServices	valid configuration	all services of table 3.18.1 can be resolved; IGraphClient is typed client	F9
MigrateAsync_CreatesDirectory_AndWal	DataDirectory does not exist	directory created; journal_mode=wal; migration number logged	F7
Registration_UsesFakeTokenCredential	TokenCredential override before AddTcriInfrastructure	ClientSecretCredential not created (TryAdd)	K1, F1

3.20.19. ScenarioTests (ScenarioTests.cs, 16 tests): Entire Section 3.19 Scenarios

Method	Input	Expected	K/F
S01_FirstApply	S01	table state as in 3.19	K7, K8
S02_Duplicate	S01 → S02	as in 3.19	K11
S03_Discrepancy	S01 → S03	as in 3.19	K11
S04_HigherVersionReplaces	S01 → S04	as in 3.19	K11
S05_LowerVersionIgnored	S01 → S05 (including continuation "b" with D04)	as in 3.19	K11
S06_ReplaceWithDeletion	S06	as in 3.19	K8, K11
S07_PendingThenSuspended	S07 (8 attempts)	attempt table 3.19	K8
S08_InvalidSession_Rejected	S08 (including malformed record "b")	as in 3.19	K14
S09_TransientErrors_ThenFailed	S09 (10 attempts)	attempt table 3.19	K9
S10_LeaseRecovery	S10	as in 3.19	K12
S11_DataCompleted_Later	S11	as in 3.19	K8
S12_OutboxFailure	S12 (8 attempts, 429) and continuation "b" (unknown outcome)	attempt table 3.19	F5, K9
S13_SuspendedExpires	S07 → S13	as in 3.19	K8
S14_RepeatedNotification_RefetchesGraph	S01 → S14 (a) and (b)	as in 3.19	K11, K12
S15_ReprocessForce	S01, S07 → S15 (1)–(4)	as in 3.19	K11, F8
Restart_BetweenSteps_KeepsState	S01, then new TestDb against same file, then S02	S02 result identical; data preserved	K7, K12

3.20.20. CursorTests (CursorTests.cs, 10 tests): Page Cursor (Cursor, 3.11.8)

Method	Input	Expected	K/F
Encode_Decode_RoundTrip	(2026-10-20T09:00:00.0000000Z, 17) and 100 other pairs with different times and cdr_seq from 1 to long.MaxValue	Decode(Encode(t, n)) = (t, n)	K7, F8, F8
Encode_KnownValue	Encode("2026-10-20T09:00:00.0000000Z", 17)	MjAyNi0xMC0yMFQwOTowMDowMC4wMDAwMDAwWnwxNw	K7, F8, F8
Encode_IsUrlSafe_NoPadding	1000 pairs with varying value lengths	result contains none of +, /, =	K7, F8, F8
Encode_InvalidArguments_Throw	time not canonical (2026-10-20T09:00:00Z); cdrSeq=0; cdrSeq=-5	ArgumentException	K7, F8
Decode_EmptyOrWhitespace_Throws	"", " "	ArgumentException (cursor parameter)	K7, F8

Method	Input	Expected	K/F
Decode_InvalidBase64_Throws	"###", "abc!"	ArgumentException	K7, F8
Decode_NoSeparatorOrExtraParts_Throws	encoded texts 2026-10-20T09:00:00.0000000Z17 and 2026-10-20T09:00:00.0000000Z|17|1	ArgumentException	K7, F8
Decode_NonCanonicalTime_Throws	encoded text 2026-10-20T09:00:00Z|17	ArgumentException	K7, F8
Decode_BadSequence_Throws (Theory, 5 data sets)	encoded texts with cdr_seq=abc, 0, -1, 1e3, 5	ArgumentException	K7, F8
Decode_InvalidUtf8_Throws	base64 of FF FE bytes	ArgumentException	K7, F8

3.21. Mapping TRS Requirements to Infrastructure Automated Tests

TRP-1.5 recount method. Results shown are from a programmatic count of 285 unique methods in Tables 3.20.1–3.20.20 by the K/F column (one Theory is one method). This is a proposed-test matrix, not an execution report. The original v0.2 matrix profile is preserved in the RV-01 protocol (Appendix I).

3.21.1. Acceptance Criteria K1–K15

Criterion	Number of methods	Test classes (method count)	External verification
K1	3	GraphClientTests (2), OptionsTests (1)	live-tenant verification required
K2	5	GraphClientTests (5)	live-tenant verification required
K3	26	GraphClientTests (5), GraphResilienceTests (5), SubscriptionServiceTests (11), SubscriptionStoreTests (5)	live-tenant verification required
K4	1	SubscriptionServiceTests (1)	live-tenant verification required
K5	2	CdrReaderTests (2)	live-tenant verification required
K6	3	GraphClientTests (2), OutboxSenderTests (1)	live-tenant verification required
K7	24	CallProcessorTests (1), CdrApplierTests (1), CdrReaderTests (9), CursorTests (10), ScenarioTests (2), SchemaTests (1)	Host / CI / acceptance tests
K8	38	CallProcessorTests (8), CdrApplierTests (6), CdrMappingTests (3), GraphClientTests (3), OptionsTests (1), ReconcilerTests (2), ScenarioTests (5), SourceStateStoreTests (10)	Host / CI / acceptance tests
K9	49	CallProcessorTests (11), CdrApplierTests (1), ErrorLogTests (3), GraphClientTests (9), GraphResilienceTests (7), InboxStoreTests (4), OutboxSenderTests (5), ReconcilerTests (1), ScenarioTests (2), SourceStateStoreTests (1), SubscriptionServiceTests (2), WriteGateTests (3)	Host / CI / acceptance tests
K10	4	GraphClientTests (2), OptionsTests (1), SubscriptionStoreTests (1)	Host / CI / acceptance tests

Criterion	Number of methods	Test classes (method count)	External verification
K11	33	CallProcessorTests (7), CdrApplierTests (6), CdrMappingTests (2), InboxStoreTests (3), ReconcilerTests (5), ScenarioTests (7), SourceStateStoreTests (3)	Host / CI / acceptance tests
K12	52	CallProcessorTests (5), CdrApplierTests (1), GraphClientTests (1), InboxStoreTests (25), OutboxStoreTests (2), ReconcilerTests (6), ScenarioTests (3), SchemaTests (1), SourceStateStoreTests (1), SubscriptionServiceTests (3), SubscriptionStoreTests (3), WriteGateTests (1)	Host / CI / acceptance tests
K13	7	CdrApplierTests (2), CdrMappingTests (2), UtcTextTests (3)	Host / CI / acceptance tests
K14	21	CallProcessorTests (9), CdrApplierTests (1), GraphClientTests (2), ReconcilerTests (1), ScenarioTests (1), SourceStateStoreTests (7)	Host / CI / acceptance tests
K15	0	—	live-tenant verification required

3.21.2. Features F1–F9

Function	Methods	Test classes (method count)
F1	3	GraphClientTests (2), OptionsTests (1)
F2	85	CallProcessorTests (2), GraphClientTests (31), GraphResilienceTests (12), ReconcilerTests (15), SubscriptionServiceTests (17), SubscriptionStoreTests (8)
F3	1	CallProcessorTests (1)
F4	3	CdrReaderTests (3)
F5	25	CdrApplierTests (5), OptionsTests (1), OutboxSenderTests (8), OutboxStoreTests (10), ScenarioTests (1)
F6	0	—
F7	12	OptionsTests (1), SchemaTests (7), WriteGateTests (4)
F8	24	CdrReaderTests (12), CursorTests (10), ReconcilerTests (1), ScenarioTests (1)
F9	14	CallProcessorTests (2), ErrorLogTests (3), OptionsTests (3), OutboxSenderTests (1), OutboxStoreTests (1), SourceStateStoreTests (2), SubscriptionServiceTests (2)

Limitation: K15 cannot be considered met without real calls. Host test results (3.29) and Core test results are counted separately.

3.22. Normative Register of Host Contracts and Project Files

Build dependency graph: TeamsCallRecords.Core has no project dependencies; TeamsCallRecords.Infrastructure → TeamsCallRecords.Core; TeamsCallRecords.Host → TeamsCallRecords.Infrastructure + TeamsCallRecords.Core. Test projects reference only the required production projects. Infrastructure does not use the Host namespace. Service registration lives in InfrastructureRegistration; the Host assembly invokes it from Program.cs.

Host structure: final proposed inventory. Each row gives a physical path relative to the repository root and one primary type. Auxiliary internal nested types, where present, are described in that file's specification; hidden files are prohibited.

File path	Public types / contents	Dependencies	Criteria
src/TeamsCallRecords.Host/ TeamsCallRecords.Host.csproj	Web SDK, net8.0, references to Core/Infrastructure, Microsoft.Teams.Apps 2.1.1 package for .NET 8; command class separated from SDK (DR-13)	Core, Infrastructure	K1–K14
src/TeamsCallRecords.Host/Program.cs	composition root: DI, middleware, routes, configuration, hosted services	all registered services	K1, K7, K9, K10
src/TeamsCallRecords.Host/Config/BotOptions.cs	BotOptions: TimeZoneId; bot credentials in AzureAd:* per Teams SDK 2.1	MicrosoftBotConnection	K5
src/TeamsCallRecords.Host/Config/ApiOptions.cs	ApiOptions: Key, MaxPageSize, AllowedOrigins; secret only in env	Http pipeline	K7, K10
src/TeamsCallRecords.Host/Config/WebhookOptions.cs	WebhookOptions: MaxBodyBytes=1048576, MaxBatchItems=100, ValidationTimeoutSeconds=10, SubscriptionCreationGraceSeconds=120 (allowed 1..300)	Http pipeline	K3, K12
src/TeamsCallRecords.Host/Config/ HostOptionsValidator.cs	HostOptionsValidator: validate Bot/Api/Webhook, diagnostic startup failure	IOptions	K9, K10
src/TeamsCallRecords.Host/Web/GraphNotifyEndpoint.cs	GraphNotifyEndpoint.Map(IEndpointRouteBuilder); ValidateAsync, NotifyAsync, LifecycleAsync	IInboxStore, ISubscriptionStore, GraphOptions	K3, K4, K12
src/TeamsCallRecords.Host/Web/ GraphNotificationParser.cs	GraphNotificationParser.Parse(ReadOnlyMemory<byte>), notification DTOs; strict validation of size, fields, and tenant	System.Text.Json	K12
src/TeamsCallRecords.Host/Web/ GraphNotificationValidator.cs	GraphNotificationValidator.ValidateAsync; explicit classification valid, temporarilyUnknownSubscription, permanentlyInvalid	ISubscriptionStore, PendingSubscriptionRegistry , GraphOptions	K12
src/TeamsCallRecords.Host/Web/ PendingSubscriptionRegistry.cs	PendingSubscriptionRegistry: local in-flight IDs of subscriptions successfully created through Graph, TTL, tenant/resource/hash; no secrets	TimeProvider, WebhookOptions	K10, K12
src/TeamsCallRecords.Host/Web/CdrEndpoints.cs	CdrEndpoints.Map(IEndpointRouteBuilder); GET /api/cdr	ICdrReader, ApiKeyFilter	K7
src/TeamsCallRecords.Host/Web/HealthEndpoint.cs	HealthEndpoint.Map; GET /health/live, /health/ready, /api/status	queue, source, outbox	K9
src/TeamsCallRecords.Host/Web/ReprocessEndpoint.cs	ReprocessEndpoint.Map; POST /api/reprocess only durable InboxNewItem(reconcile)	IInboxStore, ApiKeyFilter	K7, K12
src/TeamsCallRecords.Host/Web/ApiKeyFilter.cs	ApiKeyFilter: IEndpointFilter with constant-time header comparison	IOptions	K7, K10
src/TeamsCallRecords.Host/Web/ErrorContracts.cs	ApiError, ApiRequestTrace; public error JSON without tokens	ASP.NET Core	K7, K9

File path	Public types / contents	Dependencies	Criteria
src/TeamsCallRecords.Host/Bot/TcriBot.cs	TcriBot.RegisterHandlers(TeamsBotApplication teams, BotCommands commands); teams.OnMessage handler	BotCommands, SDK	K5
src/TeamsCallRecords.Host/Bot/BotCommands.cs	BotCommands.ExecuteAsync, Help, CallsToday	ICdrReader, TimeProvider	K5
src/TeamsCallRecords.Host/Bot/BotAdapter.cs	BotAdapter registers AddTeamsBotApplication()/UseTeamsBotApplication() in Host and Teams SDK 2.1 JWT mechanism for /api/messages	Bot SDK	K5, K10
src/TeamsCallRecords.Host/Workers/InboxWorker.cs	InboxWorker: BackgroundService; heartbeat for long-running reconciliation, cancellation on lease loss	IInboxStore, ICallProcessor, TimeProvider	K9, K12
src/TeamsCallRecords.Host/Workers/OutboxWorker.cs	OutboxWorker : BackgroundService	IOutboxSender	K6, K9
src/TeamsCallRecords.Host/Workers/ReconcileWorker.cs	ReconcileWorker : BackgroundService	IReconciler	K11, K12
src/TeamsCallRecords.Host/Workers/ SubscriptionWorker.cs	SubscriptionWorker : BackgroundService	SubscriptionService	K3, K4
src/TeamsCallRecords.Host/Workers/StartupBarrier.cs	StartupBarrier and IStartupBarrier: readiness after DB, HTTP, checks	IHealthCheck	K9
src/TeamsCallRecords.Host/Properties/ launchSettings.json	local HTTP/HTTPS profiles, development only	Kestrel	K10
src/TeamsCallRecords.Host/appsettings.json	safe nonsecret default values	IOptions	K9
src/TeamsCallRecords.Host/appsettings.example.json	demonstration of all keys with placeholders	IOptions	K10
tests/TeamsCallRecords.Host.Tests/ TeamsCallRecords.Host.Tests.csproj	xUnit + Microsoft.AspNetCore.Mvc.Testing + Bot test adapter	Host	K3–K12
tests/TeamsCallRecords.Host.Tests/Support/ TcriWebFactory.cs	test Host factory with replaceable services and SQLite temp file	WebApplicationFactory	K3–K12
tests/TeamsCallRecords.Host.Tests/Support/ FakeTeamsAdapter.cs	incoming signed test Bot activities	SDK mock	K5
tests/TeamsCallRecords.Host.Tests/Support/ WebhookFixtures.cs	valid/invalid notifications, lifecycle, validationToken	fixtures	K3, K12
tests/TeamsCallRecords.Host.Tests/ WebhookEndpointTests.cs	Graph callback acceptance	test host	K3, K12
tests/TeamsCallRecords.Host.Tests/ WebhookSecurityTests.cs	subscription, tenant, clientState, size, HTTPS proxy	test host	K10, K12
tests/TeamsCallRecords.Host.Tests/ LifecycleEndpointTests.cs	reauthorizationRequired, missed, removed events	test host	K3, K12

File path	Public types / contents	Dependencies	Criteria
tests/TeamsCallRecords.Host.Tests/CdrEndpointTests.cs	API key, boundaries, cursor, errors	test host	K7
tests/TeamsCallRecords.Host.Tests/ ReprocessEndpointTests.cs	durable reconcile range 202, interval ≤7 days, no Graph in HTTP	test host	K7, K12
tests/TeamsCallRecords.Host.Tests/ HealthEndpointTests.cs	live/ready/status, DB failure	test host	K9
tests/TeamsCallRecords.Host.Tests/ BotCommandsTests.cs	help, calls today, timezone, no data	fake clock	K5
tests/TeamsCallRecords.Host.Tests/ BotAuthenticationTests.cs	invalid JWT, wrong aud/iss, expiry	Bot SDK	K5, K10
tests/TeamsCallRecords.Host.Tests/ WorkerRecoveryTests.cs	graceful shutdown, reclaim lease, retry	SQLite	K9, K12
tests/TeamsCallRecords.Host.Tests/ ConfigurationTests.cs	missing secret, defaults, options	host factory	K10
tests/TeamsCallRecords.Host.Tests/ EndToEndSyntheticTests.cs	webhook→inbox→Graph fake→Core→CDR→REST/outbox fake	temp SQLite	K7–K14
tests/TeamsCallRecords.Host.Tests/TestData/ webhook_call.json	synthetic input signal; no clientState in logs	fixture	K12
tests/TeamsCallRecords.Host.Tests/TestData/ webhook_lifecycle_missed.json	missed signal	fixture	K12
tests/TeamsCallRecords.Host.Tests/TestData/ bot_calls_today.json	test bot message	fixture	K5

Inventory of other files: the exact Core tree is retained in §3.2 and Infrastructure in §3.9. Section 3.22 supplements those trees; Infrastructure/Config replaces the previously listed Host/Config names rather than creating duplicates. The consolidated docs/file-inventory.tsv is generated from the unified row list in §§3.2, 3.9, and 3.22; columns are path,project,primary_type,requirement,test; CI rejects duplicate physical paths. The register also includes Data/Cursor.cs, tests/.../CursorTests.cs, D04, and deployment files detailed in §3.34.

3.23. Host Application HTTP Contracts

3.23.1. Public and Protected Routes

Method	Path	Access	Expected responses	Implementation
POST	/graph/notify? validationToken=...	public over HTTPS only for Graph challenge (text/plain)	200 text/plain, decoded token	GraphNotifyEndpoint.ValidateAsync
POST	/graph/notify	validate clientState, subscription, tenantId	202, 400, 403, 413, 503	GraphNotifyEndpoint.NotifyAsync

Method	Path	Access	Expected responses	Implementation
POST	/graph/lifecycle	validate subscription, tenantId, and clientState	202, 403, 503	GraphNotifyEndpoint.LifecycleAsync
POST	/api/messages	Bot Framework JWT, not API key	200/401 and SDK response body	BotAdapter
GET	/api/cdr?from=...&to=...&limit=...&cursor=...	X-TCRI-Key header	200/400/401	CdrEndpoints
POST	/api/reprocess	X-TCRI-Key	202, 400, 401, 503	ReprocessEndpoint
GET	/api/status	X-TCRI-Key	200, 401, 503	HealthEndpoint
GET	/health/live	public HTTP/HTTPS behind proxy	200 with no environment leakage	HealthEndpoint
GET	/health/ready	public to infrastructure health monitor, no tenant/data in body	200 or 503	HealthEndpoint

Host serves exactly these paths; MapFallback returns 404. Production has no Swagger/debug UI; in development it may be enabled only on loopback with a separate flag. /api/cdr, /api/reprocess, and /api/status do not accept keys through the query string, cookies, or URL. Production startup fails when ApiOptions.Key is empty.

3.23.2. GraphNotifyEndpoint.ValidateAsync — challenge

Task<IResult> ValidateAsync(HttpContext http, CancellationToken ct).

1. Read only the validationToken query parameter; if absent, return 400. ASP.NET Core query parsing performs decoding (one URL decode). Do not decode again because % may be part of the token.
2. Require exactly one value with a UTF-8 length no greater than 2048 bytes; otherwise return 400.
3. Return Results.Text(token, "text/plain", Encoding.UTF8, 200), without JSON quotes or HTML. Do not query Graph or wait for SQLite.
4. Target response time is under 10 seconds; proposed target ≤2 s; measure with WebhookEndpointTests.Validation_UnderDeadline.

3.23.3. GraphNotificationParser.Parse and GraphNotificationValidator.ValidateAsync

Input contract: { "value": [{ "subscriptionId": "...", "tenantId": "...", "clientState": "...", "changeType": "created", "resource": "communications/callRecords/{id}" }] }. The JSON signal is not trusted as a callRecord; after validation all data is fetched anew through IGraphClient. Only kind, tenantId, callId, and Raw enter InboxNewItem; clientState is not stored in Raw.

Algorithm:

1. Check Content-Type application/json (allow charset=utf-8), Webhook:MaxBodyBytes request size, MaxBatchItems, and JSON nesting depth ≤32. On limit exceeded return 413; do not store the body in error_log.
2. Check required subscriptionId, tenantId, resource, clientState (lifecycleEvent for lifecycle) and an allowed event shape. Return exactly 400 for syntactically invalid outer JSON; 413 when limits are exceeded. Valid elements in a mixed batch must not be lost: classify each element as valid, permanentlyInvalid, or temporarilyUnknownSubscription; never log secrets. Commit valid elements in one durable inbox transaction. If a temporarily unknown subscription has a verified in-flight marker owned by this process, return 503 for the batch after committing valid elements: Graph may retry; new rows are deduplicated. If

there is no transient problem and at least one valid element, return 202 after COMMIT; ignore permanently invalid elements with safe diagnostics. If there are no valid elements and only a confirmed tenant/clientState mismatch, return 403. Repetition must not grant access. Under [P-28], tests separately cover mixed batches, known mismatches, and eventual consistency of a new subscriptionId.

3. Deterministically classify each entry [P-28]. valid: subscription.id exists, its status allows notifications, tenantId = Graph:TenantId, resource matches, and the hash of supplied clientState matches in constant time. temporarilyUnknownSubscription: the subscription row is not yet present, but PendingSubscriptionRegistry contains that exact ID from a trusted successful CreateSubscriptionAsync response no more than 120 seconds ago; supplied tenantId/resource/clientStateHash must match the marker in constant time; return 503 after committing other valid items. permanentlyInvalid: ID is absent with no marker or an expired marker, or tenant/resource/clientState is different; return 403 if no other valid elements exist; ignore such records in mixed batches with safe diagnostics. Do not treat all unknown IDs as transient based solely on server age or a submitted timestamp: an attacker must not be able to induce endless 503 responses. Delete the marker after successful UpsertAsync. TTL is Webhook:SubscriptionCreationGraceSeconds=120 (allowed 1..300); TimeProvider controls it, and expiry is cleaned up upon read. Because the marker is in memory, a crash between remote 201 and local commit may cause 403 on retry: this is an explicit residual risk DR-19/RV-05 requiring a live test and periodic Graph adoption. Do not log clientState or its value.
4. Create InboxNewItem: for callRecords, derive callId from resource only for the exact /communications/callRecords/{id} path; for channel messages, callId=null and Raw includes resource and message ID but not message text; for lifecycle missed enqueue reconcile.
5. await inbox.EnqueueAsync(batch, ct); Overloaded ⇒ 503; disk/commit failure ⇒ 503; only after commit return empty-body 202. Graph target response time ≤3 seconds; overload must not produce a false 202.

In production, trust forwarded HTTPS headers only from explicitly configured proxies; prohibit UseForwardedHeaders with all hosts trusted. No debug/bypass endpoints.

Transactions: the HTTP handler never writes CDRs; durable notification acceptance is the single transaction in IInboxStore.EnqueueAsync, completed before 202 is returned.

3.23.4. CdrEndpoints.QueryAsync and ReprocessEndpoint.ReprocessAsync

GET /api/cdr requires from and to as ISO 8601 with time zone, checks from < to and a period ≤90 days (design limit), limit between 1 and Api:MaxPageSize (default 100; maximum 1000), and parses cursor only through Cursor.Decode.

200 JSON contract:


```
{
  "items": [
    {
      "cdr_seq": 17,
      "schema_version": 1,
      "event_type": "call",
      "tenant_id": "11111111-1111-4111-8111-111111111111",
      "call_id": "cccccccc-0000-4000-8000-000000000001",
      "session_id": "dddddddd-0000-4000-8000-000000000001",
      "source_version": 1,
      "call_type": "oneToOne",
      "caller": null,
      "callee": null,
```

```
    "start_utc": "2026-10-20T09:00:00.0000000Z",
    "end_utc": "2026-10-20T09:01:00.0000000Z",
    "duration_s": 60,
    "modalities": ["audio"],
    "status": "completed",
    "termination_reason": null,
    "segment_count": 1,
    "aggregation_status": "complete",
    "source_hash": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
    "extensions": null
   }
  ],
  "next": null
}
```


Note: this REST JSON is a synthetic shape illustration, not G01; its source_hash made of a characters is explicitly a placeholder. REST tests use actual golden hashes from G01–G11.

POST /api/reprocess body: {"from_utc":"2026-10-20T00:00:00.0000000Z","to_utc":"2026-10-21T00:00:00.0000000Z"}. Host does not call Graph from the HTTP handler: it validates the period (from<to, maximum 7 days per job [P-27]), then durably enqueues one InboxNewItem(kind=reconcile, tenantId=Graph:TenantId, callId=null, raw={"from_utc":"...","to_utc":"...","force":true}) using IInboxStore.EnqueueAsync. Two identical requests while the first reconcile row is new yield the same canonical element_hash and are coalesced by uq_inbox_new_hash under [P-22]: repeated request returns 202 {"accepted":true,"queued":true}, Added=0, Duplicates=1; after done, repetition enqueues again. See ReprocessEndpointTests.DuplicateWhileNew_Collapsed. Return 202 {"accepted":true,"queued":true} only after commit; call count is not known and not returned. Return 503 on overload or durable commit failure, 400 for invalid period. InboxWorker/CallProcessor reads period parameters from raw and invokes IReconciler.ReconcileAsync(from,to,force:true,ct) in the background, paging through Graph; Graph errors trigger scheduled retries, so HTTP 202 is not a false confirmation that calls were processed. Caller splits periods exceeding 7 days into multiple requests; 90 days is allowed only for GET /api/cdr. A long-running reconcile item requires lease heartbeat per §3.12.3: IInboxStore.RenewLeaseAsync(long id, DateTimeOffset expectedLockedUntilUtc, CancellationToken ct) returns a new deadline or null when ownership is lost. InboxWorker runs heartbeat in parallel with Graph pagination every LeaseSeconds/3, updates the expected deadline and cancels the linked processing on null or database error; CallProcessor stops work and the old worker must not mark the lost lease done. Test both successful renewal and reclamation between two workers. Tests: ReprocessEndpointTests.EnqueuesReconcileRaw, ReprocessEndpointTests.DuplicateWhileNew_Collapsed, ReprocessEndpointTests.NoSynchronousGraph, ReprocessEndpointTests.Period_Over7Days400, ReprocessEndpointTests.CommitFailure503, WorkerRecoveryTests.Reconcile_LeaseRenewsDuringPaging, WorkerRecoveryTests.Reconcile_LeaseLossCancelsAndDoesNotComplete, CallProcessorTests.ReconcileWithRange_UsesRawPeriod. All of this is proposed decision P-27, not a claim of implemented code.

API errors use {"error":{"code":"INVALID_PERIOD","message":"..."},"trace_id":"..."}, without SQL, stack traces, or tokens.

3.24. Teams Bot and F4 Call Summary

Signature validation: /api/messages is handled by Teams SDK 2.1 (Microsoft.Teams.Apps) with built-in Authorization Bearer validation according to the documented chain: JWKS, iss, aud, time, and channel msteams. A hand-written “three-line JWT check” is prohibited. On failure, return 401 and do not call BotCommands. Do not log secrets or claim values. Validate BotOptions at startup.

UNVERIFIED SDK signatures (design pseudocode, NOT compile-ready before RV-02): AddTeamsBotApplication, UseTeamsBotApplication, teams.OnMessage, and context.SendAsync must be checked with Microsoft.Teams.Apps 2.1.1 during dotnet restore/build. Do not regard the identifiers below as confirmed API methods.

Teams SDK 2.1 integration contract [P-26; candidate DR-13]: Program.cs calls builder.Services.AddTeamsBotApplication() and, after app.Build(), TeamsBotApplication teams = app.UseTeamsBotApplication(); then TcriBot.RegisterHandlers(teams, commands). RegisterHandlers configures teams.OnMessage(async (context, ct) => { string message = context.Activity.Text ?? string.Empty; string answer = await commands.ExecuteAsync(message, clock.GetUtcNow(), configuredZone, ct); await context.SendAsync(answer, ct); });. TcriBot does not derive from the deprecated ActivityHandler; /api/messages is created and authenticated by the SDK. AzureAd:TenantId, AzureAd:ClientId, AzureAd:ClientCredentials[0].SourceType=ClientSecret, and AzureAd:ClientCredentials[0].ClientSecret are bot settings separate from Graph:*. AzureAd:DangerouslyAllowUnauthenticatedRequests is absent or false in production; unauthenticated mode is prohibited. Exact method signatures are verified by compiling against net8.0 (RV-02); current status remains proposed, not restore-verified.

BotCommands.ExecuteAsync(string text, DateTimeOffset nowUtc, TimeZoneInfo timeZone, CancellationToken ct) contract: returns Task<string>; normalizes text using Trim() and case-insensitive invariant comparison; accepts help and calls today; all other commands return Unknown command. Type help. Responses are consistently in English for the Bliscore demo.

help → Commands: help; calls today.

calls today: localNow = TimeZoneInfo.ConvertTime(nowUtc, timeZone); obtain local startOfDay and startOfNextDay and convert these boundaries to UTC with TimeZoneInfo.ConvertTimeToUtc(DateTime.SpecifyKind(localMidnight, Unspecified), zone), checking ambiguous/nonexistent local times. For midnight affected by a DST gap/overlap, define the fallback in advance as “the first valid instant of the day” and test it separately. Call ICdrReader.GetStatsAsync(_graphOptions.TenantId,fromUtc,toUtc,ct).

Decision P-13: call count = COUNT(DISTINCT call_id) after mandatory WHERE tenant_id = @tenantId filtering within the period; total duration = SUM(duration_s) over CDR sessions. This is the sum of session durations, not the elapsed time of a multi-party call; the bot labels it Total session duration. Coding a different interpretation requires changing the TRP.

With data: Today (Europe/Kyiv): 3 calls, total session duration 00:12:30.; with no data: Today (Europe/Kyiv): no calls recorded.. Time zone comes from Bot:TimeZoneId, default Europe/Kyiv for the demo; Windows mapping uses TimeZoneInfo.FindSystemTimeZoneById with a supported identifier. Startup fails if the zone cannot be resolved. Time comes from TimeProvider, not DateTime.Now.

Contract consistency: Host always passes Graph:TenantId to GetStatsAsync(tenantId,fromUtc,toUtc,ct), and SQL filters by tenant_id = @tenantId. The method belongs to Infrastructure, and its interface change is reflected in Figure 18, CdrReaderTests, and the contract table. Cross-tenant mixing is prevented. Only the public Infrastructure interface changes; Core is unchanged.

3.25. Background Services and Application Lifecycle

Service	Period and loop	What it processes	Readiness condition	Failure
InboxWorker	loop; when no item, Task.Delay(1s) through TimeProvider; heartbeat each LeaseSeconds/3	ClaimNextAsync → ICallProcessor.ProcessAsync + CAS RenewLeaseAsync during Graph pagination	migration applied	log error, delay, continue

Service	Period and loop	What it processes	Readiness condition	Failure
OutboxWorker	loop; when no item, 2s	IOutboxSender.RunOnceAsync	migration applied	log error, delay, continue
ReconcileWorker	every Reconcile:IntervalMinutes (15)	ReconcileLookbackAsync(false) + RecheckSuspendedAsync()	Graph configured	errors/health without stopping others
SubscriptionWorker	Subscriptions:RenewCheckMinutes (5)	SubscriptionService.EnsureAsync	token available	retries only after remote list, not blind POST

InboxWorker.ExecuteAsync starts Processing:Workers parallel loops; one IWriteGate serializes writes. All BackgroundService instances honor CancellationToken. After SIGTERM no new ClaimNext begins; the current operation has a bounded 30-second grace period for transaction commit. If the process dies during a transaction, SQLite rolls it back; the lease is reclaimed later. On shutdown, do not return 202 to a webhook before the actual commit.

StartupBarrier: (1) validate configuration and secrets; (2) create the data directory with appropriate permissions; (3) apply migration and inspect the journal; (4) enable /health/ready; (5) start background workers. Missing Graph token does not prevent /health/live, but readiness may be degraded under the chosen policy; if the database is unavailable, /health/ready must not falsely return HTTP 200.

One application / one process per SQLite database: do not scale to replicas>1 against one database; a second instance is prohibited until a shared coordination mechanism is introduced. Docker restart policy: unless-stopped; state persists in a volume.

3.26. Normative Sequence Diagrams

Markdown contains Mermaid sources; DOCX/PDF must contain corresponding diagrams with legible captions. Every arrow corresponds to a method in §§3.11–3.25, and failure logic appears under alt/opt. These are proposed diagrams, not records of actual code execution traces.

Figure 11. Graph: validationToken and Webhook Verification.

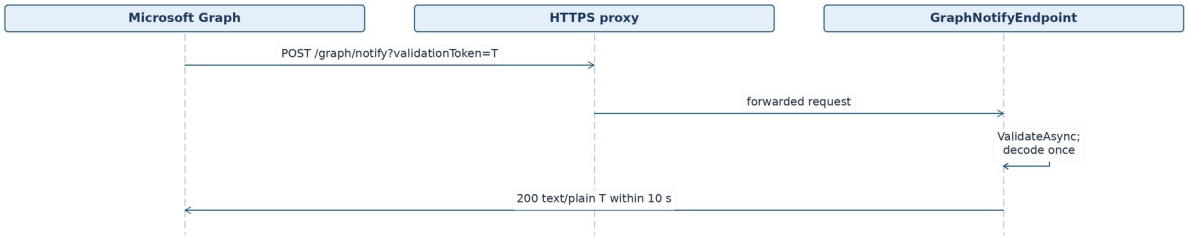
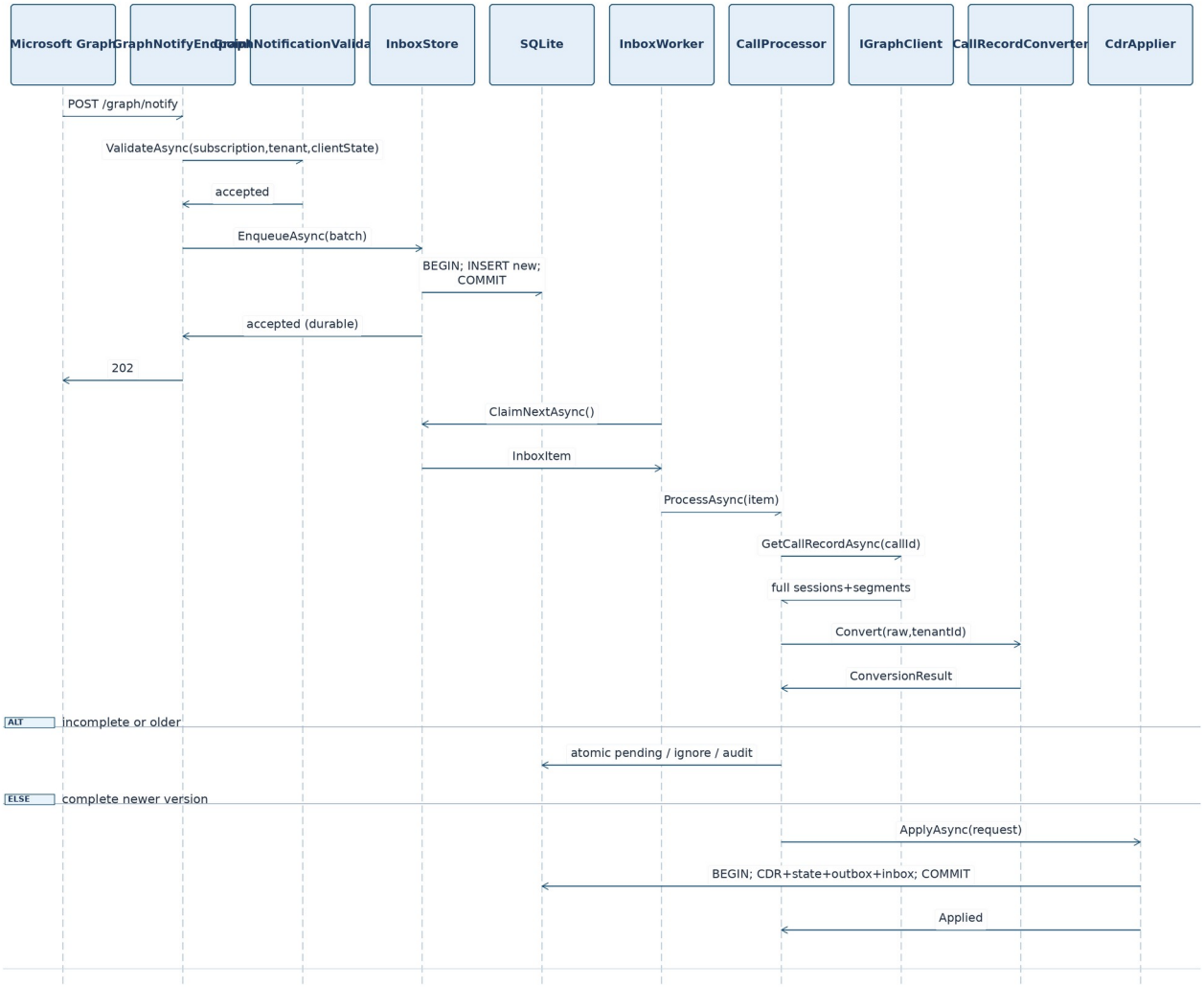


Figure 12. Webhook: Durable Inbox → Core → Outbox.



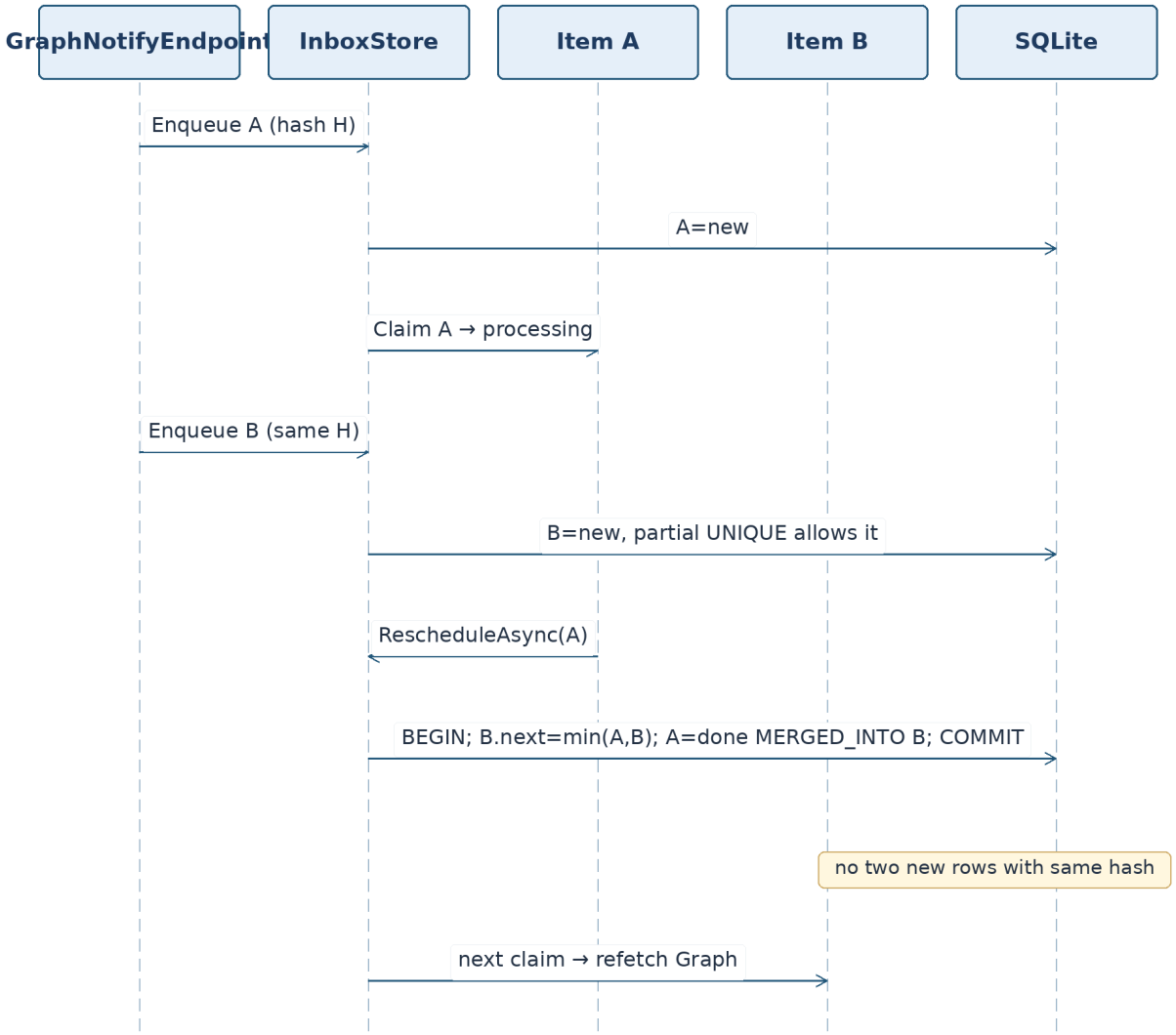
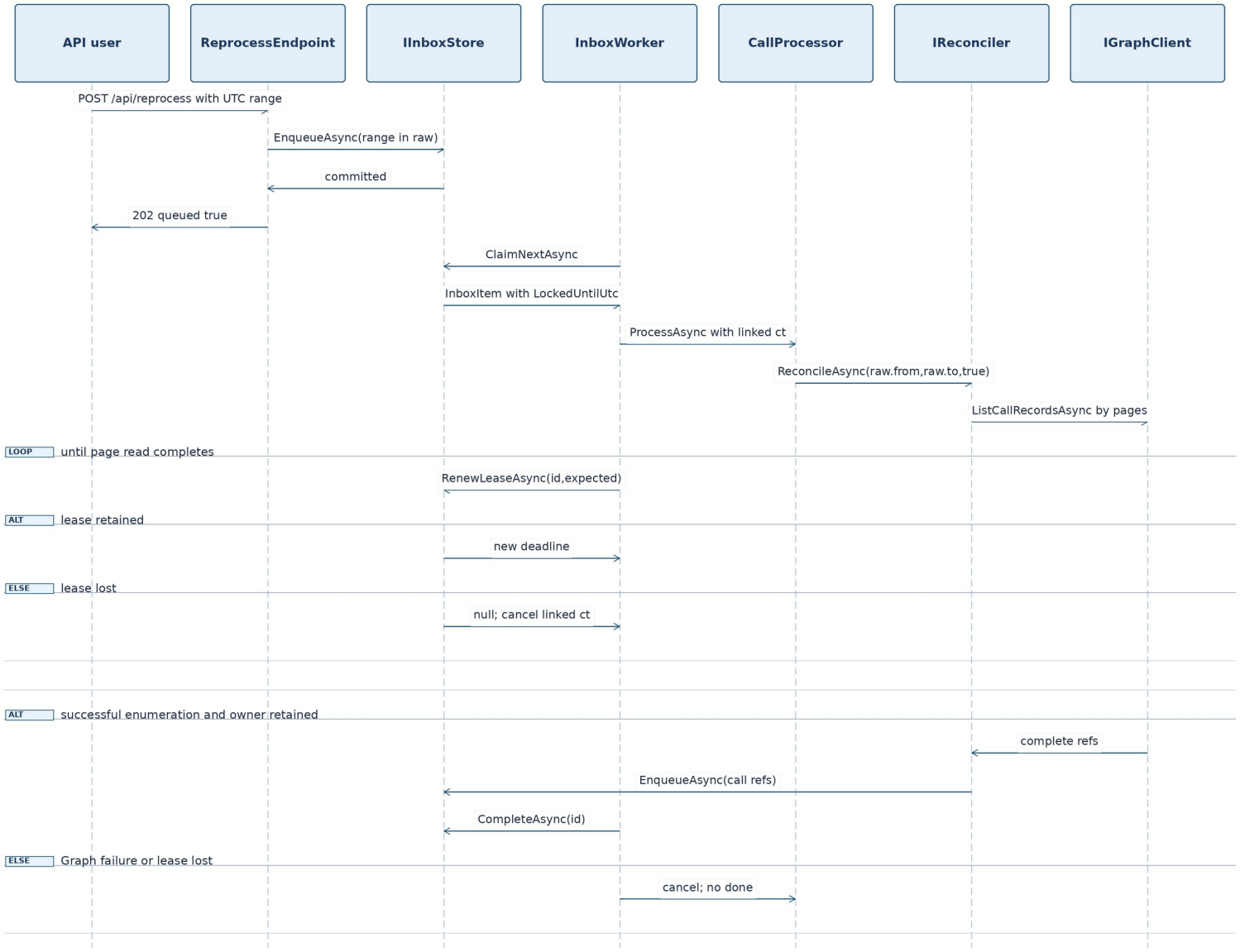


Figure 13. Repeated Webhook During Processing.

Figure 14. Manual /api/reprocess Reconciliation: Queue and Lease Heartbeat.



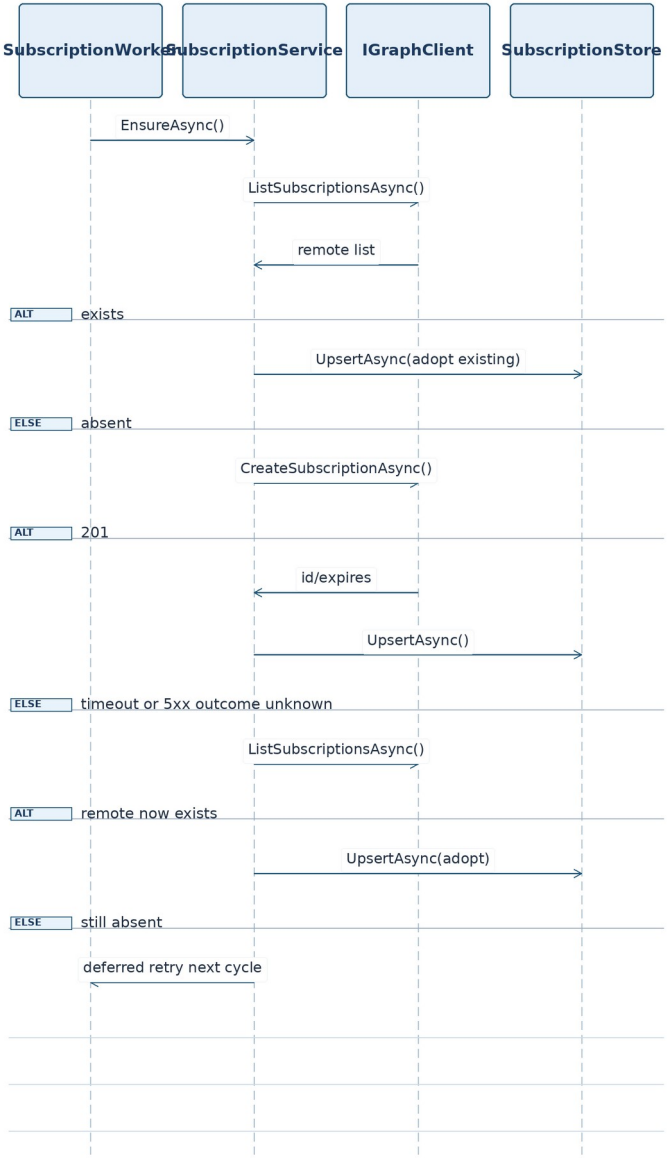


Figure 15. Graph Subscription: Safe Recovery After POST.

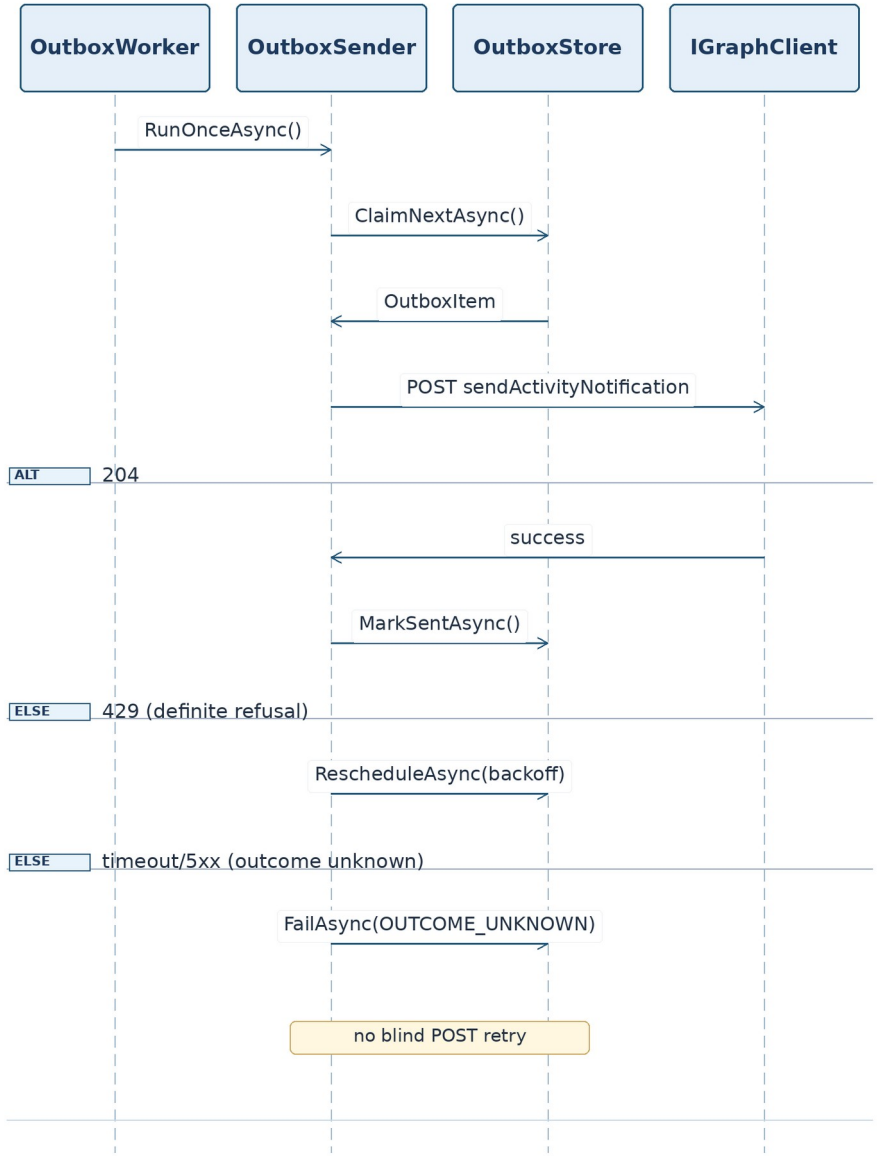


Figure 16. Activity Feed: At-Most-Once Policy.

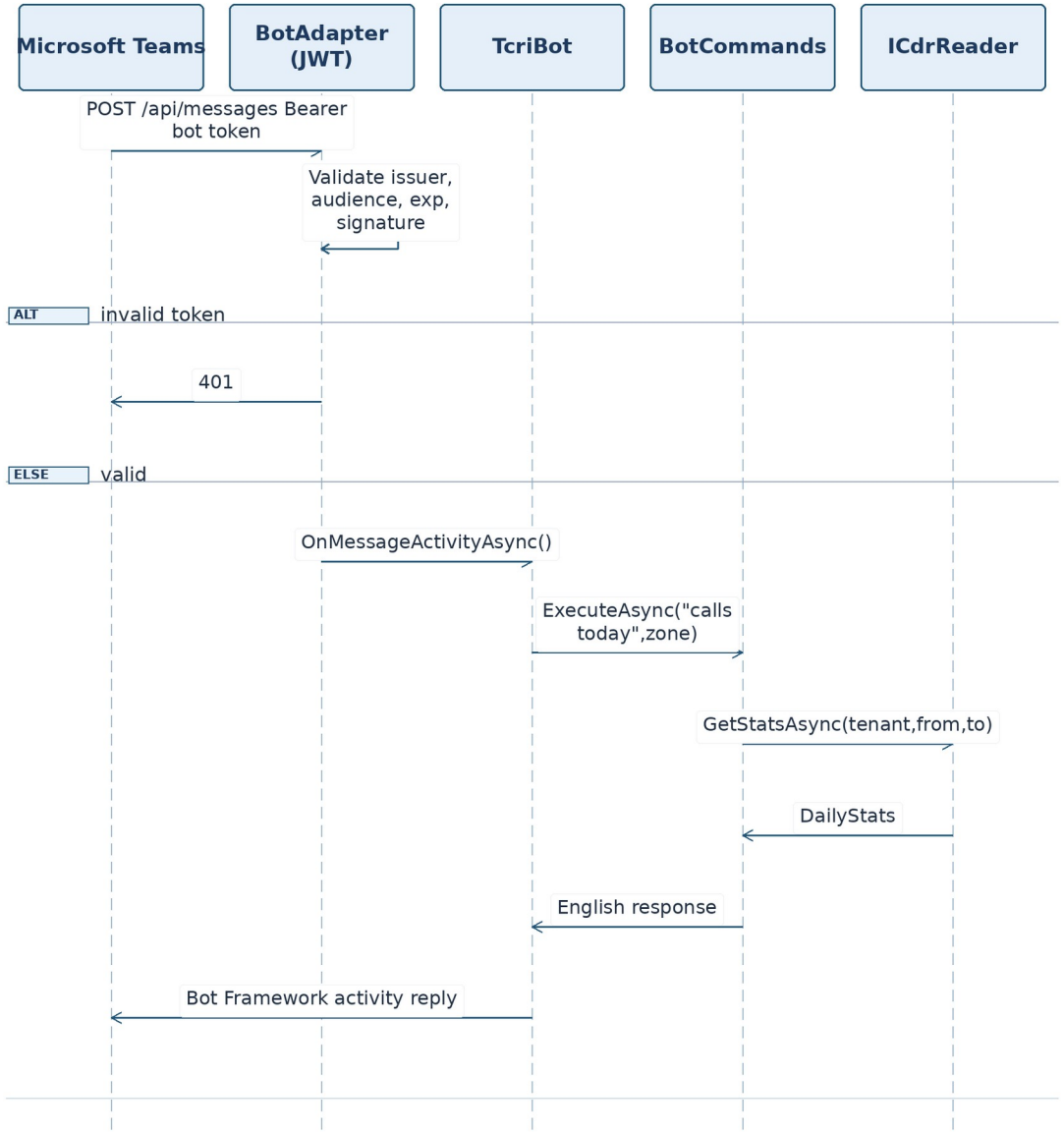


Figure 17. Teams Bot: Authentication and calls today.

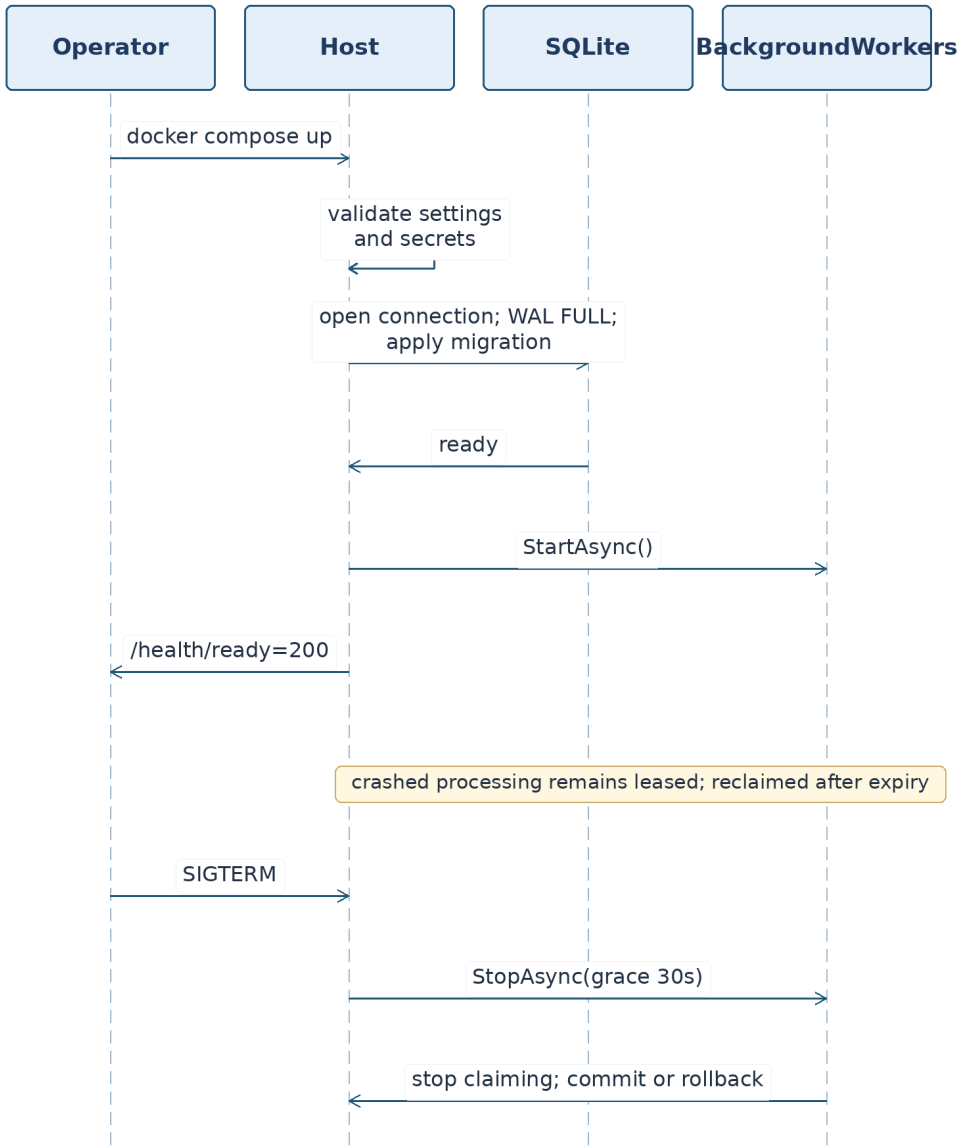


Figure 18. Application Startup and Recovery.

3.27. Complete State Model and Cross-Module Invariants

Object	State	Allowed transitions	On abnormal termination
inbox	new	processing; on duplicate remains new	durable
inbox	processing	done, failed, new (retry), merged into another new	lease timeout → reclaim
inbox	done	terminal for physical item	new webhook = new row
source_state	pending	done, suspended, expired	retry due date preserved
source_state	suspended	done, expired, suspended	periodic recheck
source_state	expired	done only via manual force with actual complete data	audit/diagnostics; previous CDR preserved
outbox	new	sending	eligible for sending
outbox	sending	sent (204), new (429), failed (unknown)	after lease → failed unknown; no blind retry
subscription	active	renewing, failed, active (adopt)	GET/list reconciliation

System invariants (mandatory in SQL and tests):

I-01. UNIQUE(cdr.tenant_id,cdr.call_id,cdr.session_id) always holds; cdr_seq remains stable when updating a session.

I-02. No incomplete callRecord may advance source_state.applied_version or delete an existing CDR.

I-03. A lower version changes neither pending/done nor a CDR. Same version with a different hash → audit only.

I-04. HTTP 202 is not returned before committing inbox. 503 on overload or database unavailability allows Graph to retry notification delivery.

I-05. There cannot be two inbox rows with state='new' and identical element_hash, but there may be one processing and one new with the same hash. A→new collisions are merged atomically.

I-06. Outbox does not change the CDR record; under the chosen policy an uncertain POST outcome is not automatically retried.

I-07. Secrets, access tokens, clientState, and actual Graph payload bodies are neither logged nor published in the repository.

I-08. Moving from one version to the next is transactional: cdr + source_state + inbox + outbox are all written together or remain unchanged.

I-09. RSC is not replaced by tenant-wide protected API access and is not expanded without another owner decision.

I-10. Infrastructure does not depend on Host, and no project code affects the Digital Polygraph production system.

3.28. Reference Integration JSON Contracts

Graph notification (synthetic transport signal).

```
{
  "value": [
    {
      "subscriptionId": "00000000-1111-4222-8333-444444444444",
```

```
    "tenantId": "11111111-1111-4111-8111-111111111111",
    "clientState": "REPLACE_IN_TEST_WITH_FIXED_FAKE_STATE",
    "changeType": "updated",
    "resource": "communications/callRecords/cccccccc-0000-4000-8000-000000000001"
   }
  ]
}
```


CDR: the canonical output for all G01–G11 derived events is in §3.6 and remains the sole Core golden set; REST JSON in §3.23.4 illustrates the shape but does not override G01–G11 hashes. Additional D04 is derived strictly from G01: retain call ID cccccccc-0000-4000-8000-000000000001 and session ID dddddddd-0000-4000-8000-000000000001; change version=4 to 3; delete sessions[0].endDateTime only, leaving root endDateTime intact. This is the input to S05(b), in which complete G01 version 4 was already applied: the outcome is IgnoredLower, with no CDR change, no pending transition, and no new rejected rows. The standalone Core output (when invoked for diagnostic comparison) has IsComplete=false with SESSION_DATES_MISSING. Complete JSON appears in Appendix B.

Outbox payload:


```
{"activity_type":"callCompleted","call_type":"oneToOne","status":"completed","duration_s":60,"start_utc":"2026-10-20T09:00:00.0000000Z"}
```


API error:


```
{"error":{"code":"UNAUTHORIZED","message":"API key required"},"trace_id":"synthetic-trace-001"}
```


Diagnostics (key-protected response, available only after the API key):


```
{
  "status":"degraded",
  "database":"ready",
  "queues":{"inbox_pending":2,"inbox_failed":0,"outbox_pending":1,"outbox_failed":1},
  "sources":{"pending":1,"suspended":1,"expired":0},
  "last_reconcile_utc":null,
  "reason_codes":["OUTBOX_OUTCOME_UNKNOWN"]
}
```


Teams SDK signatures not verified by compilation (RV-02). Names AddTeamsBotApplication, UseTeamsBotApplication, OnMessage and context.SendAsync are pseudocode for integration points. Their existence and parameters are not confirmed by dotnet restore/build for Microsoft.Teams.Apps 2.1.1. Before implementation, a minimal compile probe against the installed package is required. Do not transfer these names into C# as a proven API.

Error contract — no false precision: without a tenant, Graph health may be degraded; this does not mean integration succeeded. The example CDRs in this document are synthetic, not data from a real tenant.

3.29. Host Automated Test Plan

Planned environment: .NET SDK 8.0.x, xUnit 2.9.3, ASP.NET Core WebApplicationFactory, SQLite in a temporary directory, FakeTimeProvider, StubHttpMessageHandler, fake Graph, and a dedicated Bot test adapter if needed. Expected HTTP responses are given directly in test names/descriptions; counts are planned and must be verified after implementation.

Class/file (tests/TeamsCallRecords.Host.Tests/)	Methods — condition and expected result	Count	Criteria
WebhookEndpointTests.cs	Validation_UrlDecoded_Plain200, Validation_Missing400, Validation_DoubleEncodedPreserved, Notification_Valid202AfterCommit, Notification_Overloaded503, Notification_DatabaseError503, Notification_EmptyBatch400, Notification_TooLarge413, Notification_MixedBatch_ValidCommittedInvalidIgnored, Notification_DuplicateDoneEnqueued, Notification_MixedBatch_ValidCommittedTemporaryUnknown503, Notification_MixedBatch_AllInvalid403	12	K3, K12
WebhookSecurityTests.cs	UnknownSubscription_NoPendingMarker403, WrongClientState403, WrongTenant403, WrongResource403, CorrectHashAccepted, NoClientStateInInbox, NoSecretsLogged, BadContentType415, BodyDepthRejected, TrustedForwardedHeadersOnly, UnknownSubscription_PendingMarker503, UnknownSubscription_ExpiredPendingMarker403	12	K10, K12
LifecycleEndpointTests.cs	Missed_ReconcileEnqueued, Removed_EnsureTriggered, Reauthorization_Refresh, UnknownSubscription_NotPending403, DuplicateLifecycleIdempotent, UnknownSubscription_PendingMarker503	6	K3, K12
CdrEndpointTests.cs	MissingKey401, WrongKey401, ValidKey200, FromMissing400, InvalidTime400, InvalidCursor400, PageSizeLimit400, CorrectPagingNoDuplicates, TenantFiltered, EmptyResult200	10	K7
ReprocessEndpointTests.cs	WithoutKey401, WrongPeriod400, ForceDoneEnqueued, ForceSuspendedEnqueued, ForceExpiredEnqueued, Period_Over7Days400, EnqueuesReconcileRaw, Overloaded503, ResponseHasNoCount, DuplicateWhileNew_Collapsed, NoSynchronousGraph, CommitFailure503	12	K7, K12
HealthEndpointTests.cs	LiveWithoutDb200, ReadyDatabaseUnavailable503, StatusNeedsKey, StatusCountsCorrect, StatusHidesSecrets	5	K9, K10
BotCommandsTests.cs	HelpListsCommands, CallsTodayEmpty, CallsTodayThreeCalls, CallsTodayTimezoneBoundary, CallsTodayDSTBoundary, UnknownCommand, CaseInsensitive, DurationIsSessionSum	8	K5
BotAuthenticationTests.cs	MissingBearer401, BadSignature401, WrongAudience401, Expired401, ValidSignedActivityHandled	5	K5, K10
WorkerRecoveryTests.cs	InboxLeaseRecover, OutboxUnknownNotRetried, ShutdownStopsClaim, RollbackPreservesInbox, SecondInstanceProhibited, NoCrashOnSingleGraphFailure, Reconcile_LeaseRenewsDuringPaging, Reconcile_LeaseLossCancelsAndDoesNotComplete	8	K6, K9, K12
ConfigurationTests.cs	MissingApiKeyFailsStartup, InvalidTimezoneFailsStartup, NoGraphSecretInSettingsFile, InvalidMaxPendingFails, DevConfigDoesNotBypassJwt	5	K9, K10
EndToEndSyntheticTests.cs	WebhookG01CreatesOneCdr, GroupG02CreatesTwo, OverlappingG03NotSummed, MalformedGraphRejected, SameVersionDiffHashAudit, NewVersionUpdates, PendingNoFakeCdr, ForcedReprocessRecovers, OutboxFailureKeepsCdr, RestartRetainsData	10	K7–K14

A total of 92 named proposed Host tests (sum of rows). These are not execution results. Every test includes a representative HTTP body, expected status, expected SQLite rows, and absence of forbidden side effects; a real Microsoft tenant test cannot be replaced by a fixture.

3.30. Requirements → Implementation → Test Matrix

Requirements	Layer / key file	Automated verification	Live verification
F1 / K1	GraphClient, GraphOptions, Entra app	GraphClientTests.Token*, ConfigurationTests	App-only token through real Entra
F2 / K2	GraphClient, SubscriptionService	GraphClientTests, SubscriptionServiceTests	users/teams/channels/callRecord in tenant
F3 / K4	GraphNotifyEndpoint, manifest.json	WebhookEndpointTests, LifecycleEndpointTests	channel message without @mention under RSC
F4 / K5	BotAdapter, TcriBot, BotCommands	BotCommandsTests, BotAuthenticationTests	Teams bot responds to help, calls today
F5 / K6	OutboxSender, GraphClient	OutboxSenderTests, EndToEndSyntheticTests	activity feed notification
F6 / K8,K11,K13,K15	CallRecordConverter, VersionRules, CdrApplier	Golden G01–G11, D01–D04, ScenarioTests	real callRecord, manual time comparison
F7 / K7	TcriDbContext, migrations, CdrReader	SchemaTests, CdrEndpointTests	restart preserves CDR
F8 / K7,K12	CdrEndpoints, ReprocessEndpoint, Reconciler	ReprocessEndpointTests	reprocess for selected period
F9 / K9,K10,K14	Diagnostics, error_log, rejected, logs	WebhookSecurityTests, HealthEndpointTests	failure of one record, secret masking
K3	GraphNotifyEndpoint, SubscriptionWorker	WebhookEndpointTests	Graph validationToken challenge
K12	GraphNotificationValidator, InboxStore	WebhookSecurityTests, InboxStoreTests	foreign clientState rejected

Final acceptance criteria: automated tests verify the implemented properties, but K8, K11, K13, and K15 require processing records from an actual tenant. Synthetic-only demonstrations are allowed with the explicit statement “live F6 acceptance is not complete.”

3.31. Atlas of Additional Sequences and Signatures

Scenario	Calls / components	Main branches	Figure
Token F1	GraphClient → ClientSecretCredential.GetTokenAsync	token cache, expired, 401/403	R19
Read callRecord	GraphClient.GetCallRecordAsync → sessions/segments page loaders	@odata.nextLink, MaxPages, malformed, retry	R20
Version transition	CallProcessor → VersionRules.Decide → CdrApplier	lower, equal/same, equal/different, higher	R21
CDR replacement	CdrApplier → IWriteGate → SQLite	update/insert/delete/rollback	R22
Subscription renew	SubscriptionService → IGraphClient → ISubscriptionStore	204, unknown outcome/list, missed	R23
Channel message RSC	Graph → Endpoint → Inbox → CallProcessor	validate team/channel; never use protected tenant-wide API	R24

Figure 19. Entra ID Token for Graph.

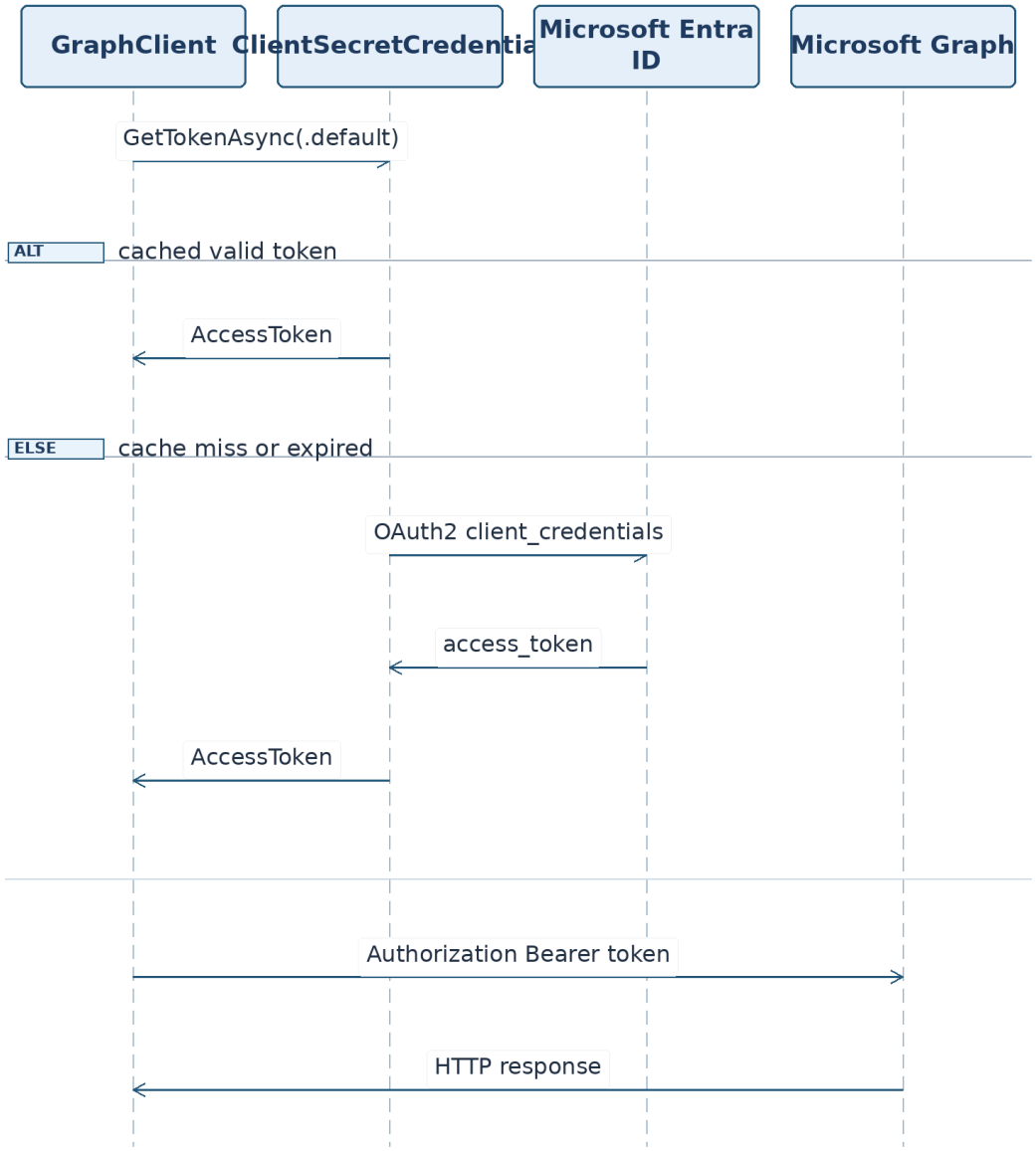


Figure 20. Loading callRecord Pages.

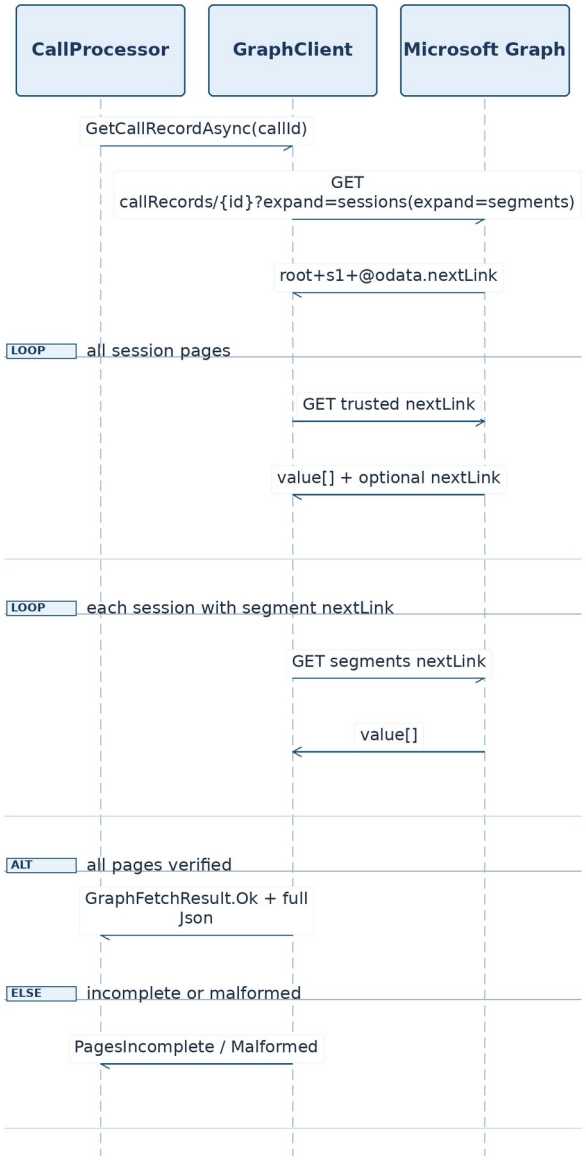


Figure 21. Version and State Check.

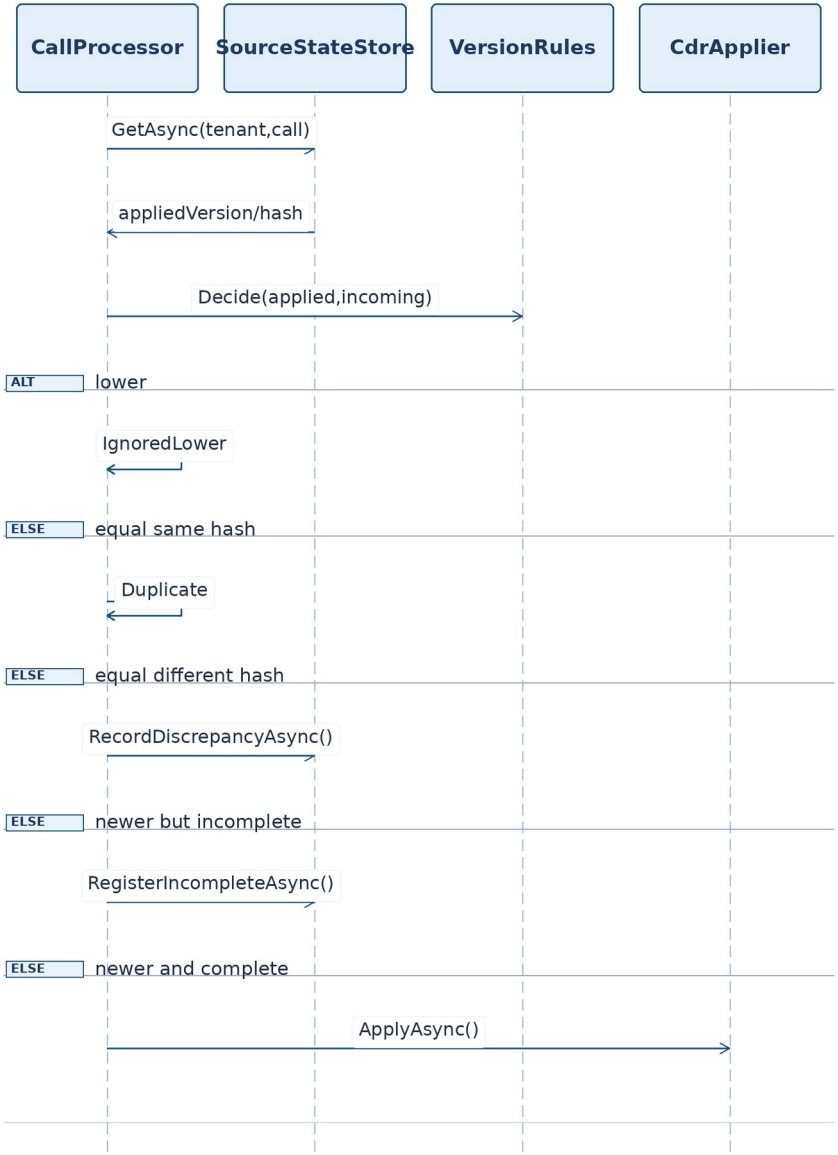


Figure 22. Atomic Update of the CDR Set.

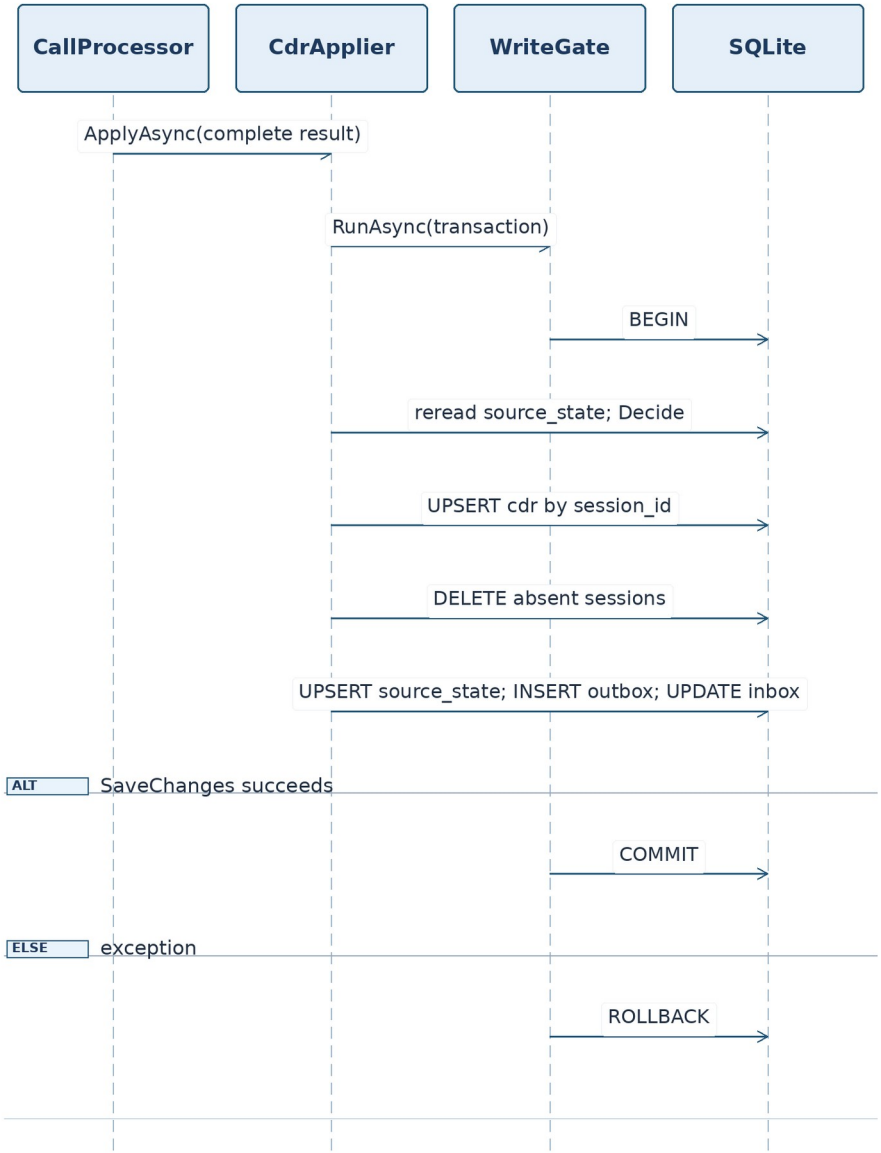


Figure 23. Renewing a Graph Subscription.

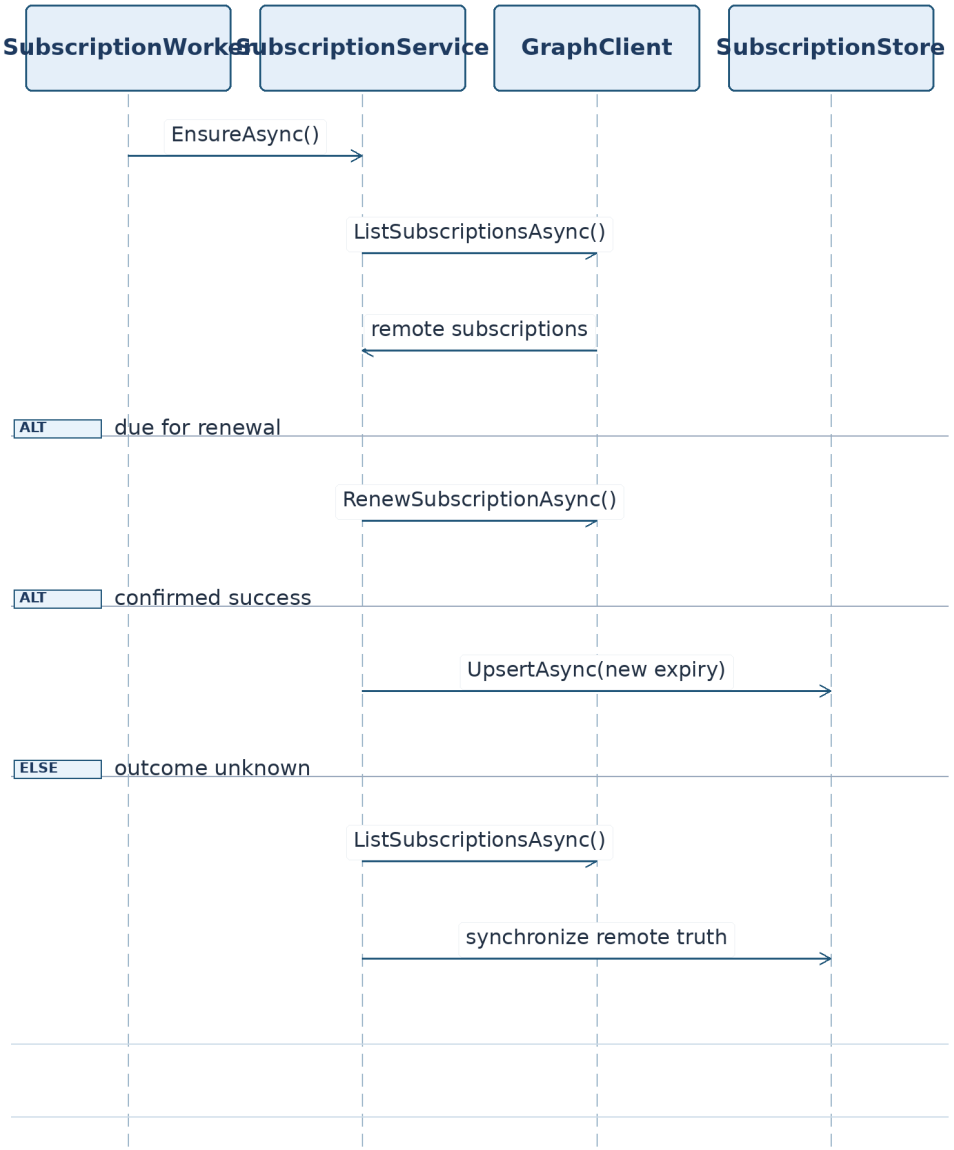
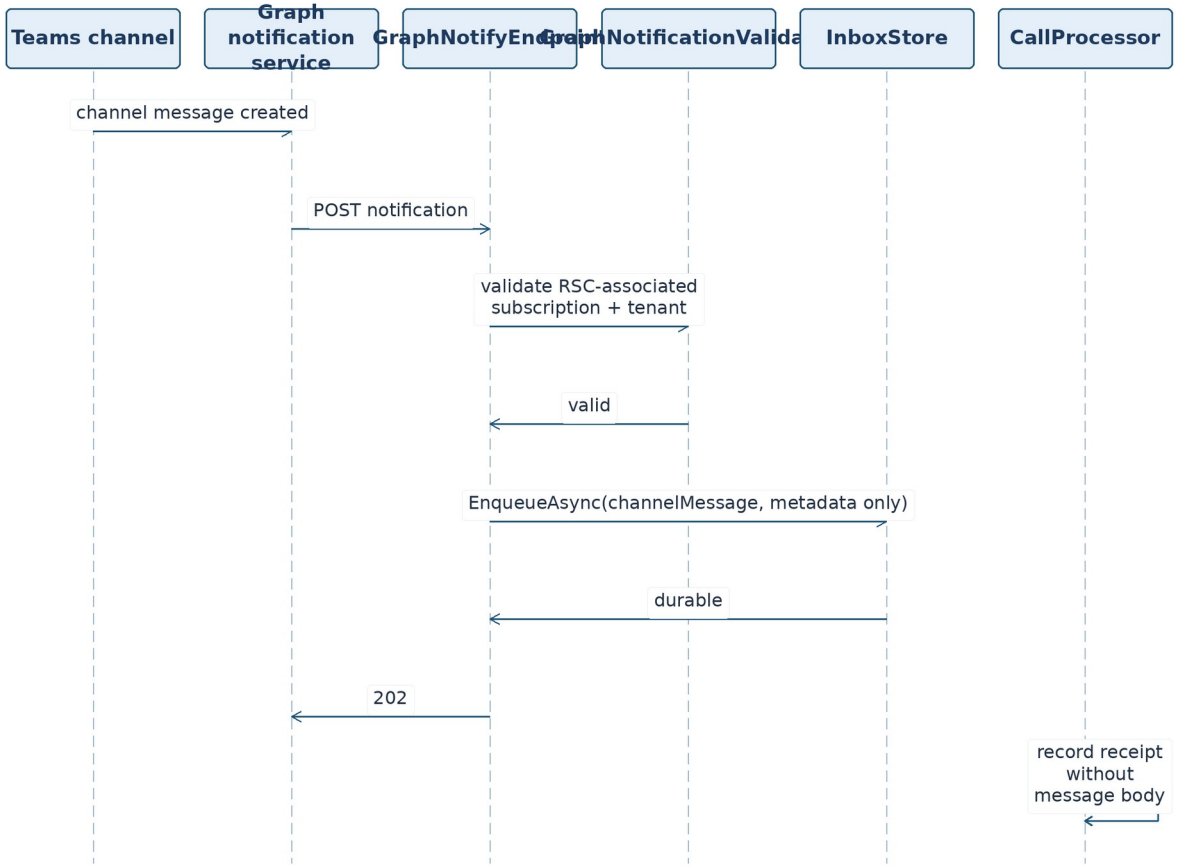


Figure 24. Channel Message Without @mention.



3.32. Preliminary Verification of Contracts and .NET 8 Compatibility

global.json pins the .NET 8 SDK and does not require any change to the Digital Polygraph runtime. TeamsCallRecords.sln is used rather than .slnx. Directory.Build.props sets TargetFramework=net8.0, LangVersion=12, Nullable=enable, ImplicitUsings=enable, TreatWarningsAsErrors=true, deterministic build, and ContinuousIntegrationBuild in CI.

Compatibility must be verified by compiling under .NET 8 and running all tests (Core + Infrastructure + Host), not by choosing a version by inspection. No .NET 10-only code is used. IHostedService, TimeProvider, HttpClient, IOptions, and EF Core 8 are available in the target release. NuGet package versions must be pinned centrally in Directory.Packages.props; unverified versions are not represented as restore-tested.

The design does not assert perfect idempotency of Microsoft Graph POST. Uncertain operation outcomes are treated as a separate state, using remote reads where possible.

3.33. D04 Golden Sample and Its Links to S05(b), S14, and S15

D04 is G01 v3 with the session end time missing. This is a synthetic golden sample, not one retrieved from Microsoft Graph. Original G01 has source_version=4, call_id=...0001, session_id=...0001. D04 sets the lower version 3 and removes only sessions[0].endDateTime. The root callRecord.endDateTime remains: it is not the session end timestamp and does not justify generating a CDR.

D04/in — complete JSON:


```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
 "id": "cccccccc-0000-4000-8000-000000000001",
 "version": 3,
 "type": "peerToPeer",
 "modalities": [
  "audio"
 ],
 "startDateTime": "2026-10-20T09:00:00.0000000Z",
 "endDateTime": "2026-10-20T09:05:00.0000000Z",
 "sessions": [
  {
   "id": "dddddddd-0000-4000-8000-000000000001",
   "modalities": [
    "audio"
   ],
   "startDateTime": "2026-10-20T09:00:00.0000000Z",
   "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000001",
      "displayName": "User A"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
    }
   },
   "callee": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
```

```
        "id": "aaaaaaaa-0000-4000-8000-000000000002",
        "displayName": "User B"
       }
      },
      "userAgent": {
       "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
       "platform": "windows",
       "productFamily": "teams"
      }
     },
     "segments": [
      {
       "id": "seg-1",
       "startDateTime": "2026-10-20T09:00:00.0000000Z",
       "endDateTime": "2026-10-20T09:01:00.5000000Z",
       "media": [
        {
         "label": "main-audio",
         "streams": [
          {
           "streamId": "s1",
           "streamDirection": "callerToCallee"
          }
         ]
        }
       ]
      }
     ]
    }
  ]
}
```


D04/out — complete expected Core JSON (exact hashes per §3.4.1):


```
{
  "call_id": "cccccccc-0000-4000-8000-000000000001",
  "source_version": 3,
  "is_complete": false,
  "incomplete_reasons": [
    "SESSION_DATES_MISSING"
  ],
  "session_set_hash": "b76cf8e6d4dc200613dad12fe14fca9280c08f968aa7961e9ca9026291dfc2b4",
```

```
  "cdrs": [],
  "rejections": []
}
```

Computed Canonical.HashOf(session) = 18f5ecee18c3e04be1c6256840a4422753573c5133caf5895fd9d63896de785f; Canonical.SessionSetHash = b76cf8e6d4dc200613dad12fe14fca9280c08f968aa7961e9ca9026291dfc2b4. The expected CallRecordConverter.Convert(D04) result is incomplete (SESSION_DATES_MISSING), but Infrastructure classifies it as an older version and ignores it before moving to pending. Therefore S05(b)'s main assertion is checked after applying G01 version 4 with the same call_id.

Specific states and numeric expectations for S14 appear in §3.19; S15's forced reconciliation of four states is described there too. Appendix B duplicates D04 in the D01–D04 catalog because the standalone golden sample is required both in the file tree and in the TRP text.

3.34. Exact Build and Deployment Files (Normative Set)

The design includes the following additional files and directories at exact paths: global.json, Directory.Build.props, Directory.Packages.props, NuGet.Config, .editorconfig, .gitignore, .dockerignore, .env.example, Dockerfile, docker-compose.yml, Caddyfile, .github/workflows/ci.yml, teams-app/manifest.json, teams-app/color.png, teams-app/outline.png, README.md, HANDOFF.md, LICENSE, docs/file-inventory.tsv, docs/test-protocol.md, docs/permissions.md, docs/diagrams/sequence-04.mmd … sequence-24.mmd, scripts/check-artifacts.py, scripts/check-secrets.sh, scripts/verify-schema.py. Every ellipsis in this row denotes a range of numbers: the physical register contains one docs/diagrams/sequence-N.mmd file for each N=04,...,24; inventory generation expands this range explicitly. Additional details remain as specified in the file register.

All texts below are given in full or have exactly one specifically marked item to be checked after an owner decision. Loading secrets from .env, local dev tunnels, and public TLS do not require installation of a different .NET SDK on the DP server. Reading this TRP does not run commands on the owner's computer.

3.34.1. global.json

```
{
  "sdk": {
    "version": "8.0.100",
    "rollForward": "latestFeature",
    "allowPrerelease": false
  }
}
```

Use a .sln solution (not .slnx). A later installed SDK in the same version-8 line is allowed with rollForward=latestFeature; .NET 10 need not be installed.

3.34.2. Directory.Build.props

```
<Project>
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <LangVersion>12</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <Deterministic>true</Deterministic>
```

```
  <EnableNETAnalyzers>true</EnableNETAnalyzers>
 </PropertyGroup>
</Project>
```

3.34.3. Directory.Packages.props

The following file consolidates proposed NuGet version lines and versions. Some are inherited from the reviewed source documents; others require verification by actual dotnet restore --locked-mode with the target SDK. Until verified, it is a proposed input, not a proven restorable lockfile.

```
<Project>
 <PropertyGroup>
  <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
 </PropertyGroup>
 <ItemGroup>
  <PackageVersion Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.0.11" />
  <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.11" />
  <PackageVersion Include="Azure.Identity" Version="1.13.2" />
  <PackageVersion Include="Microsoft.Extensions.Http.Resilience" Version="8.10.0" />
  <PackageVersion Include="Microsoft.Extensions.Options.ConfigurationExtensions" Version="8.0.0" />
  <PackageVersion Include="Microsoft.Extensions.Hosting.Abstractions" Version="8.0.0" />
  <PackageVersion Include="Microsoft.Extensions.TimeProvider.Testing" Version="8.10.0" />
  <PackageVersion Include="Microsoft.Extensions.Diagnostics.Testing" Version="8.10.0" />
  <PackageVersion Include="xunit" Version="2.9.3" />
  <PackageVersion Include="xunit.runner.visualstudio" Version="3.1.4" />
  <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
  <PackageVersion Include="coverlet.collector" Version="6.0.4" />
  <PackageVersion Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.11" />
  <PackageVersion Include="Microsoft.Teams.Apps" Version="2.1.1" />
 </ItemGroup>
</Project>
```

Explicit Host dependency for DR-13: TeamsCallRecords.Host.csproj contains <PackageReference Include="Microsoft.Teams.Apps" /> (without Version, because CPM is enabled). Directory.Packages.props specifies Microsoft.Teams.Apps 2.1.1 (NuGet, supports net8.0). The bot implements TeamsBotApplication.OnMessage, not deprecated ActivityHandler. Route /api/messages must use SDK JWT validation without skipAuth. Verify the class/method signatures and MSAL subpackages using dotnet restore, dotnet build, and BotAuthenticationTests under .NET 8; executing these commands is not claimed. Microsoft documentation: https://microsoft.github.io/teams-sdk/csharp/getting-started/quickstart/ ; https://microsoft.github.io/teams-sdk/csharp/migrations/v2-dotnet/ . If the API or packages differ, update DR-13 and the corresponding tests before approval; do not silently switch to legacy Bot Framework 4.x.

3.34.4. src/TeamsCallRecords.Host/appsettings.example.json

```
{
 "Graph": {
  "TenantId": "REPLACE_WITH_TENANT_GUID",
  "ClientId": "REPLACE_WITH_ENTRA_APP_GUID",
```

```
    "ClientSecret": "",
    "ClientState": "",
    "BaseUrl": "https://graph.microsoft.com/v1.0/",
    "Scope": "https://graph.microsoft.com/.default",
    "NotificationUrl": "https://tcri.example.org/graph/notify",
    "RetentionDays": 30,
    "RequestTimeoutSeconds": 30,
    "TotalTimeoutSeconds": 120,
    "RetryMaxAttempts": 3,
    "MaxPages": 100
  },
  "Storage": {"DataDirectory": "/var/lib/tcri"},
  "Inbox": {"MaxPending": 10000},
  "Processing": {"Workers": 2, "LeaseSeconds": 120, "RetrySchedule": "1,5,15,60", "PendingLimitHours": 4, "SuspendedRecheckHours": 24, "MaxAttempts": 10},
  "Reconcile": {"IntervalMinutes": 15, "LookbackHours": 24},
  "Outbox": {"MaxAttempts": 8, "LeaseSeconds": 120, "BackoffSeconds": "30,120,600,1800,3600", "MaxAgeHours": 24},
  "Subscriptions": {"TeamId": "", "ChannelId": "", "CallRecordsLifetimeMinutes": 4230, "ChannelMessagesLifetimeMinutes": 60, "RenewCheckMinutes": 5},
  "Bot": {"TimeZoneId": "Europe/Kyiv"},
  "AzureAd": {"TenantId": "REPLACE_WITH_TENANT_GUID", "ClientId": "REPLACE_WITH_BOT_APP_GUID", "ClientCredentials":
[{"SourceType":"ClientSecret","ClientSecret":""}]},
  "Api": {"Key": "", "MaxPageSize": 100, "AllowedOrigins": []},
  "Webhook": {"MaxBodyBytes": 1048576, "MaxBatchItems": 100, "ValidationTimeoutSeconds": 10},
  "AllowedHosts": "tcri.example.org"
}
```


appsettings.json contains only nonsecret parts of this structure. Graph:ClientSecret, Graph:ClientState, AzureAd:ClientCredentials:0:ClientSecret, and Api:Key are supplied through Graph__ClientSecret, Graph__ClientState, AzureAd__ClientCredentials__0__ClientSecret, and Api__Key environment variables; no file containing actual values is committed.

3.34.5. .env.example

```
# Copy to .env on isolated TCRI host (never commit a populated file).
TCRI_HOSTNAME=tcri.example.org
GRAPH_TENANT_ID=REPLACE_WITH_TENANT_GUID
GRAPH_CLIENT_ID=REPLACE_WITH_APP_GUID
GRAPH_CLIENT_SECRET=
GRAPH_CLIENT_STATE=
BOT_APP_ID=REPLACE_WITH_BOT_GUID
BOT_APP_PASSWORD=
API_KEY=
```

Name mapping: Compose passes GRAPH_TENANT_ID into Graph__TenantId and so on. ASP.NET Core's configuration binder directly reads keys containing double underscores.

3.34.5a. .dockerignore — Required File [K10]

Dockerfile uses COPY . ., so Docker build context must exclude the local .env containing Graph/Bot/API secrets, SQLite files, and build directories. Run scripts/check-secrets.sh before docker compose build; do not create .env inside the image or use ARG for secrets.

```
.git
.env
.env.*
!.env.example
**/bin/
**/obj/
**/TestResults/
**/.vs/
**/.vscode/
**/*.db
**/*.db-wal
**/*.db-shm
**/*.sqlite
**/*.sqlite-wal
**/*.sqlite-shm
coverage/
_tcri_master_work/
```

Dockerfile reads only files present in build context. DockerBuild_EnvIsAbsentFromContext puts a unique test string in .env, builds from a temporary copy of the tree, checks that the string is absent from the image and its layers (without publishing the string in CI), and separately checks that .env.example remains available. CI does not depend on an .env file in the repository.

3.34.6. Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:8.0-bookworm-slim AS build
WORKDIR /src
COPY . .
RUN dotnet restore TeamsCallRecords.sln
RUN dotnet publish src/TeamsCallRecords.Host/TeamsCallRecords.Host.csproj \
    -c Release -o /out --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:8.0-bookworm-slim AS runtime
WORKDIR /app
RUN addgroup --system tcri && adduser --system --ingroup tcri tcri \
    && mkdir -p /var/lib/tcri && chown -R tcri:tcri /var/lib/tcri
COPY --from=build /out/ .
USER tcri
ENV ASPNETCORE_URLS=http://+:8080
```

```
EXPOSE 8080
ENTRYPOINT ["dotnet", "TeamsCallRecords.Host.dll"]
```

Exact image digests must be pinned during release preparation. A Debian Bookworm-derived 8.0-bookworm-slim base image provides the system time zone database for Europe/Kyiv (do not use Alpine without installing tzdata separately); pin its digest on release. The net8 base image includes the application runtime and does not modify existing hosts or DP.

3.34.7. docker-compose.yml

```
services:
  tcri:
    build: .
    restart: unless-stopped
    init: true
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      Graph__TenantId: ${GRAPH_TENANT_ID}
      Graph__ClientId: ${GRAPH_CLIENT_ID}
      Graph__ClientSecret: ${GRAPH_CLIENT_SECRET}
      Graph__ClientState: ${GRAPH_CLIENT_STATE}
      Graph__NotificationUrl: https://${TCRI_HOSTNAME}/graph/notify
      AzureAd__TenantId: ${GRAPH_TENANT_ID}
      AzureAd__ClientId: ${BOT_APP_ID}
      AzureAd__ClientCredentials__0__SourceType: ClientSecret
      AzureAd__ClientCredentials__0__ClientSecret: ${BOT_APP_PASSWORD}
      Api__Key: ${API_KEY}
      Storage__DataDirectory: /var/lib/tcri
    volumes:
      - tcri-data:/var/lib/tcri
    networks: [private]
    read_only: true
    tmpfs:
      - /tmp
    security_opt:
      - no-new-privileges:true
  proxy:
    image: caddy:2
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    environment:
```

```
    TCRI_HOSTNAME: ${TCRI_HOSTNAME}
  volumes:
    - ./Caddyfile:/etc/caddy/Caddyfile:ro
    - caddy-data:/data
    - caddy-config:/config
  depends_on: [tcri]
  networks: [private]
networks:
  private:
volumes:
  tcri-data:
  caddy-data:
  caddy-config:
```

TCRI does not expose port 8080 publicly; Caddy publishes only 80/443. The volume holds SQLite WAL/SHM and is writable by the container user with the UID that owns it. Some Docker hosts need a one-time secure volume ownership initialization (document the exact UID/GID in deployment instructions; never chmod 777). With read_only, only /var/lib/tcri and /tmp are writable.

3.34.8. Caddyfile

```
{$TCRI_HOSTNAME} {
    encode zstd gzip
    reverse_proxy tcri:8080
    header {
        X-Content-Type-Options nosniff
        Referrer-Policy no-referrer
        -Server
    }
    log {
        output stdout
        format json
    }
}
```

Host configures KnownProxies with Caddy's internal address and rejects X-Forwarded-Proto from untrusted clients. Proxy and TCRI communicate over a private bridge; API access requires a separate key, and bot access requires a validated JWT. Caddy manages certificates using ACME, which requires public DNS and accessible ports 80/443.

3.34.9. .github/workflows/ci.yml

```
name: tcri-ci
on:
  push:
  pull_request:
```

```yaml
jobs:
  build-test:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'
      - name: Restore
        run: dotnet restore TeamsCallRecords.sln
      - name: Build
        run: dotnet build TeamsCallRecords.sln -c Release --no-restore
      - name: Tests
        run: dotnet test TeamsCallRecords.sln -c Release --no-build --logger 'trx;LogFileName=tcri.trx'
      - name: Check repository
        run: |
          python3 scripts/check-artifacts.py
          bash scripts/check-secrets.sh
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: tcri-test-results
          path: '**/TestResults/*.trx'
```

CI does not use real client secrets or access other people's tenants. Publication readiness requires a separate manual procedure in §3.36.

*3.34.9a. Normative Contents of CI Checks in scripts/**

scripts/check-secrets.sh (POSIX bash, set -euo pipefail; examines only git-tracked files, does not print contents of the local .env, exits with code 1 upon match):

```
#!/usr/bin/env bash
set -euo pipefail
if git ls-files -z | grep -azE '(^|/)\.env($|\.)' | grep -avzE '(^|/)\.env\.example$' >/dev/null; then
  echo 'FAIL: tracked .env file' >&2; exit 1
fi
if git grep -IlE '-----BEGIN (RSA |EC |OPENSSH )?PRIVATE KEY-----|("?(ClientSecret|Api__Key|Bot__AppPassword)"?[[:space:]]*[:=][[:space:]]*"[^" ]{12,}")' -- ':!*.md' ':!
*.pdf' 2>/dev/null | grep -q .; then
  echo 'FAIL: potential committed credential; inspect locally' >&2; exit 1
fi
printf 'tracked secrets scan: no matches (heuristic; manual review required)\n'
```

Neither file contents nor secret values are printed. Regular expressions provide an additional heuristic, not a substitute for manual K10 checks; CI also checks that sensitive artifacts are absent from Docker context.

scripts/check-artifacts.py — a static design check, not represented as dotnet test:

```
from pathlib import Path
import csv,sys
root=Path(__file__).resolve().parent.parent
with (root/'docs/file-inventory.tsv').open(encoding='utf-8', newline='') as f:
    rows=list(csv.DictReader(f, delimiter='\t'))
paths=[r['path'] for r in rows]
if len(paths)!=len(set(paths)):
    sys.exit('FAIL: duplicate physical path')
missing=[p for p in paths if not (root/p).is_file()]
if missing:
    sys.exit('FAIL: missing files: '+', '.join(missing[:15]))
required={'.dockerignore','Dockerfile','Directory.Packages.props','NuGet.Config',
          'scripts/check-secrets.sh','scripts/check-artifacts.py','scripts/verify-schema.py'}
if required-set(paths):
    sys.exit('FAIL: required files absent in inventory')
print(f'checked {len(paths)} declared files')
```

scripts/verify-schema.py reads src/TeamsCallRecords.Infrastructure/Data/Migrations/InitialCreate.sql, opens sqlite3.connect(':memory:'), runs executescript, compares the exact set of eight expected tables against sqlite_master, checks index names uq_inbox_new_hash, ix_inbox_hash, ix_inbox_call, ix_inbox_due and the WHERE state='new' SQL filter (normalizing whitespace and quotes), and exits 1 on discrepancies. It then inserts two records with the same element_hash in done and new (must succeed) and a second new record with the same hash (must raise sqlite3.IntegrityError). Only the Python 3 standard library is used, with no Graph or secrets. Mandatory inbox column values come directly from the §3.5 DDL without fictitious tenant data. Passing CI cannot be claimed without executing it.

3.34.10. teams-app/manifest.json — Proposed Template Requiring Schema Validation

```
{
  "$schema": "https://developer.microsoft.com/json-schemas/teams/v1.16/MicrosoftTeams.schema.json",
  "manifestVersion": "1.16",
  "version": "1.0.0",
  "id": "REPLACE_WITH_TEAMS_APP_GUID",
  "packageName": "org.example.tcri",
  "developer": {
    "name": "TCRI",
    "websiteUrl": "https://tcri.example.org",
    "privacyUrl": "https://tcri.example.org/privacy",
    "termsOfUseUrl": "https://tcri.example.org/terms"
  },
```

```
 "name": {"short": "TCRI", "full": "Teams Call Records Integration"},
 "description": {"short": "Call records", "full": "Teams call record summaries and notifications."},
 "icons": {"outline": "outline.png", "color": "color.png"},
 "accentColor": "#1456A0",
 "bots": [{
  "botId": "REPLACE_WITH_BOT_APP_GUID",
  "scopes": ["personal", "team"],
  "isNotificationOnly": false,
  "supportsFiles": false
 }],
 "permissions": ["identity"],
 "validDomains": ["tcri.example.org"],
 "webApplicationInfo": {"id": "REPLACE_WITH_ENTRA_APP_GUID", "resource": "api://tcri.example.org/REPLACE_WITH_ENTRA_APP_GUID"},
 "authorization": {"permissions": {"resourceSpecific": [
  {"name": "ChannelMessage.Read.Group", "type": "Application"}
 ]}},
 "activities": {"activityTypes": [{
  "type": "callCompleted",
  "description": "New Teams call recorded",
  "templateText": "Call {callType}: {status}, {duration} seconds at {startUtc}"
 }]}
}
```

External checks: manifestVersion, schema URL, permission shape, activityTypes, and webApplicationInfo.resource are proposed assumptions. Run the official Teams manifest validator before packaging. Microsoft documentation requires the Teams app to be installed for the activity notification recipient and its app ID included in webApplicationInfo. HTTPS privacy/terms URLs require real public pages before public deployment, unless a subsequent owner decision changes the addresses; these URLs are placeholders, not claims that pages exist.

3.35. Operating and Recovery Procedures

New tenant / installation. First obtain an authorized Microsoft 365 tenant, App Registration, and Bot registration. Assign app-only CallRecords.Read.All for Graph callRecord, minimum actual API permissions to read users/teams/channels, and RSC ChannelMessage.Read.Group for the relevant team to receive channel messages. Activity feed needs the permission for the chosen app mechanism (TeamsActivity.Send.User with RSC, or broader permission only upon separate decision). Install the Teams app for users/team and grant required consent. Do not replace RSC with tenant-wide protected getAllMessages API access.

Preparing an independent host: (1) separate machine/container host, not DP; (2) public subdomain DNS; (3) access to ports 80/443; (4) secrets in .env mode 0600 outside git; (5) Docker Compose; (6) readiness; (7) Graph webhook challenge; (8) subscription check; (9) synthetic unit/integration tests; (10) test Teams call and subsequent callRecord retrieval.

Backup: under WAL use SQLite Online Backup API (Microsoft.Data.Sqlite backup), or stop TCRI and atomically copy all tcri.db, -wal, and -shm files; copying only tcri.db during active WAL is prohibited. Plan: daily backups in separate protected storage, retention of seven daily copies, weekly restore test. Secrets are stored separately from the database. When restoring a volume into a new environment, update webhook URLs and recreate subscriptions through remote reconciliation.

Graph failure: does not delete CDRs; recorded in error_log and health. Database failure: readiness 503, webhook 503 (not 202). Power loss: WAL+FULL; after restart, run integrity checks and recover leases; durability depends on the storage volume's actual fsync semantics. External POST failures: audit uncertain outcomes and never promise “exactly once” without Graph confirmation. Tenant change: never reuse SQLite with another tenant's secrets; plan migration separately.

Cleanup: remove inbox done/failed items older than 30 days and outbox sent/failed items older than 30 days using a maintenance command while intake is closed or under a single WriteGate; do not automatically purge CDR and audit_log. Changes to retention are governed by the public privacy policy and project owner. Do not enable regular rejected/error_log cleanup without a separate decision.

3.36. Acceptance Tests and K1–K15 Protocol

Criterion	Procedure	Evidence	Status before execution
K1	register app, execute client credentials flow	HTTP 200 token metadata without exposing token itself	not performed
K2	GET users/teams/channels/callRecord in authorized tenant	request, status code, and duration recorded without PII	not performed
K3	create subscription, accept validationToken, renew subscription	HTTP challenge traces, masked id/expires	not performed
K4	post channel message without @mention under RSC	evidence of inbox event without channel text	not performed
K5	bot commands help and calls today	response screenshots without PII	not performed
K6	create new CDR and deliver activity feed notification	masked timestamp/recipient, 204 from Graph	not performed
K7	read CDR with key; expect 401 without key	REST / JSON responses	not performed
K8	compare session count with saved CDR count on live callRecord	N↔N table	not performed
K9	corrupt one JSON; verify service continues	rejected/error_log, health, next record processed	not performed
K10	audit repository for secrets, tokens, real CDRs	machine scan, manual audit	not performed
K11	repeated and higher version, same/different hash	SQLite snapshots and audit log	not performed
K12	foreign tenant/clientState, repeated webhook, crash, recovery	HTTP 403/503/202, evidence SQLite	not performed
K13	7 fractional digits, round half seconds up, do not sum segments	G03/G10/live examples, timestamp differences	not performed
K14	invalid record in rejected with time, code and SHA-256	row dump without PII	not performed
K15	real tenant calls and manual time verification within requirements tolerance	live test call protocol	not performed

Two readiness levels: Synthetic verified (unit/integration tests and G01–G11 passed) and Live verified (Graph app-only, subscriptions, RSC, bot, feed, real CDRs, K8/K11/K13/K15). Documentation and README must not claim complete production acceptance on synthetic-only evidence.

Test protocol docs/test-protocol.md has fixed columns: criterion_id,run_utc,environment,commit_sha,SDK_version,result,artifact_path,reviewer,note. Publication must attach actual execution results, not proposed test-count figures.

3.37. Design Decision Register and Remaining Dependencies

ID	Technical decision	Status at drafting	Before implementation
DR-01	.NET 8 / C# 12 / .sln, no changes to DP	**fixed by owner**	do not change
DR-02	One CDR per session; segments not summed	**fixed by requirements**	do not change
DR-03	WAL + synchronous=FULL, 8 tables	proposed design specification	approve/reject upon TRP approval
DR-04	inbox UNIQUE(element_hash) WHERE state='new' and merge collision	proposed design specification, amendment A1 [P-22]	approve/reject
DR-05	one processing per call, VersionRules before incompleteness	proposed design specification	approve/reject
DR-06	force=true for manual reprocess regardless of source_state	follows requirements; technical solution proposed	approve/reject
DR-07	Unknown POST outcome → no automatic feed retry [P-20]	**owner decision required**	accept risk of a missed notification or specify another contract
DR-08	bot sums duration_s (not call elapsed time) [P-13]	**owner decision required**	approve label "total session duration"
DR-09	Activity recipient caller/callee user + installed app [P-14]	**owner decision required**	determine recipient for test activity feed
DR-10	Keep last reconciliation time in memory [P-17]	proposed design specification	approve/reject
DR-11	Hosted separately from DP; one app/one SQLite	protection of DP fixed; hosting location open	specify physical host/DNS
DR-12	Tenant, consent, app manifest URL, actual Microsoft schema validation	external dependency	verify in live tenant and update document upon differences
DR-13	specific Bot/Teams SDK NuGet and JWT adapter	external dependency and compilation verification	pin version after .NET 8 restore
DR-14	Public demonstration deadline 22 Oct 2026	previously set by owner; not guaranteed by this document	report actual readiness honestly
DR-15	Maximum API interval 90 days, MaxPageSize=100	design choice	verify / approve
DR-16	[P-25] pending/suspended retries determined by next_attempt_utc due time; force unrestricted	**confirmation of original Infrastructure v0.2 rule**	no new owner decision needed; rule in §3.13.6 already active
DR-17	[P-26] Teams SDK 2.1.1 / AzureAd / OnMessage instead of deprecated Bot Builder 4.x	external Microsoft specification; design choice	.NET 8 restore/build and JWT test mandatory
DR-18	[P-27] /api/reprocess durable enqueue 202, maximum 7 days, worker + lease renewal	proposed Host amendment	approve SLA and range bounds

ID	Technical decision	Status at drafting	Before implementation
DR-19	[P-28] Mixed Graph batch: 503 only for trusted in-flight marker (≤120 s), otherwise 403 for unknown ID	proposed Host amendment; crash window remains RV-05	verify on real Graph callback and approve TTL threshold

TRP approval criterion: the owner approves the entire specification and explicitly resolves DR-07–DR-13 and DR-17–DR-19. A specific parameter value may be fixed and verified in a live tenant during implementation, provided it does not leave an architectural fork unresolved. A change to Microsoft API requires a separate ADR recording traceability to the affected contract and test.

4. Expected Technical and Economic Indicators

The technology requires no dedicated database server: SQLite resides in a secured local volume, EF Core 8 provides transactions, and the container carries its own .NET 8 runtime. Additional mandatory external costs depend on actual Microsoft 365 licensing, hosting, and domain conditions; no specific prices are approved until verified. The demonstration environment imposes no costs or risks on DP production. The design uses a single-process host; horizontal scaling is outside v1 scope.

Quantitative design estimates: 8 SQLite tables; one Graph client; four background workers; one CDR per session; eleven G01–G11 golden samples plus D04; S01–S15 scenarios; and Core, Infrastructure, and Host test catalogs whose actual counts will be confirmed by xUnit discovery after implementation (estimated counts are not passing-test reports).

5. Sources Used in Development

1. Approved TCRI Technical Requirements Specification v1.0 dated 20–21 October 2026 — primary list of F1–F9 and K1–K15.
2. Approved TCRI Preliminary Design v1.1 in Russian and English, 21 October 2026 — system architecture and processing routes.
3. TCRI Detailed Design, Part 1 v0.1, original explanatory note containing Core, G01–G11, and DDL, 21 October 2026.
4. TCRI Detailed Design, Part 2 Infrastructure v0.2 — verified primary source (SHA-256 e9425fc7aae50113be4ecdf46eaf8c8393b5a734f328e9e925739d14d33cb8d3); later changes are reflected in Appendix I. Version v0.1 is used only as archival material.
5. GOST 19.404-79. Unified System for Program Documentation. Explanatory Note: Requirements for Content and Presentation.
6. Microsoft Learn, Get callRecord: https://learn.microsoft.com/en-us/graph/api/callrecords-callrecord-get?view=graph-rest-1.0
7. Microsoft Learn, List sessions: https://learn.microsoft.com/en-us/graph/api/callrecords-callrecord-list-sessions?view=graph-rest-1.0
8. Microsoft Learn, Teams change notifications: https://learn.microsoft.com/en-us/graph/teams-changenotifications-chatmessage
9. Microsoft Learn, Teams resource-specific consent: https://learn.microsoft.com/en-us/microsoftteams/platform/graph-api/rsc/resource-specific-consent
10. Microsoft Learn, Teams activity notifications: https://learn.microsoft.com/en-us/graph/teams-send-activityfeednotifications
11. Microsoft Learn, Send user activity notification: https://learn.microsoft.com/en-us/graph/api/userteamwork-sendactivitynotification?view=graph-rest-1.0
12. Microsoft Learn, Entity Framework Core / SQLite: https://learn.microsoft.com/en-us/ef/core/providers/sqlite/
13. Microsoft Learn, Teams SDK 2.1 C# quickstart: https://microsoft.github.io/teams-sdk/csharp/getting-started/quickstart/
14. Microsoft Learn, Teams SDK 2.1 migration/auth: https://microsoft.github.io/teams-sdk/csharp/migrations/v2-dotnet/
15. NuGet, Microsoft.Teams.Apps 2.1.1: https://www.nuget.org/packages/Microsoft.Teams.Apps/2.1.1

16. Microsoft Learn, .NET 8 lifecycle policy: https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core

Normative caveat: exact Microsoft Graph schemas and limits are verified in a live tenant; discrepancies must not be concealed by automated tests using synthetic data.

End of the unified design explanatory note. This document is submitted for engineering review and an owner decision. It does not assert that C# code, a Graph tenant, or live tests already exist.

17. Microsoft Learn, webhook validation uses POST: https://learn.microsoft.com/en-us/graph/change-notifications-delivery-webhooks

Appendix A. Proposal Register for Author Approval

Part 2 proposals [P] continue the numbering from Part 1 (P-01…P-08). They are not author decisions until approved.

Code	Proposal	Rationale
P-09	Writes use ordinary EF Core transaction under WriteGate (not BEGIN IMMEDIATE)	single process; no second writing connection; gate provides exclusivity
P-10	Options classes InboxOptions, ProcessingOptions, ReconcileOptions, OutboxOptions, StorageOptions moved from Host to Infrastructure/Config; reconciliation keys include units (Reconcile:IntervalMinutes, Reconcile:LookbackHours)	Infrastructure cannot reference Host; corrects Part 1 (3.2) and technical design v0.1
P-11	Retry schedule 1,5,15,60 min (then 60 min), Processing:MaxAttempts=10, Processing:Workers=2; errors 401 and 403 permanent	retry without permissions change would not help; values refined during testing
P-12	While a record has temporary incompleteness reasons (NO_SESSIONS, SESSION_DATES_MISSING), session-level rejections are not stored	otherwise each retry creates transient rejected entries; session may be corrected
P-13	Bot "calls" means distinct call_id per day; "total duration" means sum of duration_s over all CDRs (sessions)	requirements do not define it; session sum overestimates call duration with parallel sessions — owner decides
P-14	Notification recipients are caller/callee parties of type user; sessions ending earlier than Outbox:MaxAgeHours (24 h) create no notifications	activity feed is addressed to user; old notifications unneeded. Demonstration recipient requires owner decision
P-15	Inbound deduplication key is SHA-256 of kind, tenant and canonical body hash	redelivery of same notification does not create second item
P-16	Call subscription lifetime 4230 min (slightly under three days)	Graph limit checked on tenant [P-as]
P-17	Last reconciliation time (IReconciler.LastRunUtc) kept in process memory, not database	value only used for diagnostics; reconciliation runs immediately on restart
P-18	Graph resilience: overall timeout 120 s, attempt timeout 30 s, 3 retries with exponential backoff and jitter from 500 ms, honoring Retry-After; circuit breaker: minimum 5 requests, 50% failure, 60 s window, 30 s break	typical library settings; refine via testing

Code	Proposal	Rationale
P-19	Activity notification: type callCompleted, title "Teams call", parameters callType, status, duration, startUtc	minimum fields; template declared in manifest (Part 4)
P-20	Graph POST requests (subscription creation and activity-feed notification) are automatically retried only after HTTP 429; after timeout, network failure, or 5xx, the outcome is uncertain and no retry occurs. Subscription creation is protected by reconciliation through ListSubscriptionsAsync; the notification is closed as failed (OUTCOME_UNKNOWN) when the outcome is uncertain. Guarantee: “at most once” for an uncertain outcome, not exactly once.	retrying after lost response could create duplicate subscription/notification; missing one feed notification is less harmful than duplication. "At least once" alternative requires owner decision
P-21	NeedsFetch for done with unknown version in Graph listing → fetch (cost bounded by reconciliation window); manual /reprocess (force) fetches all period calls regardless of state, including pending, suspended, expired	reconciliation must not skip calls it is invoked to recover
P-22	An inbound item is duplicate only if a new row with same hash exists; done, failed and processing do not suppress new notifications; same-call items processed sequentially (lease); unique element_hash in DDL replaced with partial index uq_inbox_new_hash; Requeued removed	repeated signal may mean source version changed (requirements); version rules prevent duplicate CDRs. Old inbox row cleanup belongs to Host (Part 3)
P-23	Version decision precedes incompleteness check; NotFound and PagesIncomplete do not change already applied call; rejected.source_hash is canonical hash of record JSON, or SHA-256 of raw text when unparseable; duplicate rejections are suppressed	old incomplete version must not return completed call to pending; requirements mandate rejected record hash
P-24	Graph 2xx response with non-JSON call record is rejection CALLRECORD_BAD_JSON (GraphFetchStatus.Malformed), not queue item failure	K14: malformed source record enters rejection list; transport failure (network, 5xx) remains Graph error

External-system assumptions [P-as] (to be checked in a live tenant, Part 4):

Assumption	Used in
POST subscriptions body with changeType, notificationUrl, lifecycleNotificationUrl, resource, expirationDateTime, clientState accepted as described	3.14.3
Maximum subscription lifetime for communications/callRecords is at least 4230 min; channel messages — 60 min	3.15.1
sendActivityNotification body (text topic, previewText, templateParameters) accepted; activity type declared in manifest	3.14.3, 3.16.3
A guest cannot be distinguished from a user in identity	Part 1, P-01
POST response 429 means request was not executed	3.14.4, P-20
GET subscriptions returns app subscriptions with resource, notificationUrl, expirationDateTime; subscription matching our resource and notification URL was created by this service	3.15.3

Appendix B. Derived Golden Samples D01–D04

The fixtures were derived from G01, G02 and G08 with minimal changes and are used in the scenarios of 3.19 and tests of 3.20. Output files were computed by the Core module (as were G01–G11) and contain the expected CallRecordConverter.Convert result for tenant 11111111-1111-4111-8111-111111111111. Repository location: tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/ and out/ (full paths in table 3.9).

Fixture	Derived from	Change	Purpose
D01	G01	in first session streamDirection changed from callerToCallee to calleeToCaller; version 4 unchanged	S03: same version, changed content (discrepancy)
D02	G02	second session removed, version 7 → 8	S06: session set replacement with deletion
D03	G08	session gained endDateTime=2026-10-20T15:05:00.0000000Z, version 2 → 3	S11: data completed in later version
D04	G01	version 4 → 3, endDateTime removed from first session	S05(b): older incomplete version does not move completed call to pending

Input (D01_same_version_other_content.json):


```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
 "id": "cccccccc-0000-4000-8000-000000000001",
 "version": 4,
 "type": "peerToPeer",
 "modalities": [
  "audio"
 ],
 "startDateTime": "2026-10-20T09:00:00.0000000Z",
 "endDateTime": "2026-10-20T09:05:00.0000000Z",
 "sessions": [
  {
   "id": "dddddddd-0000-4000-8000-000000000001",
   "modalities": [
    "audio"
   ],
   "startDateTime": "2026-10-20T09:00:00.0000000Z",
   "endDateTime": "2026-10-20T09:01:00.5000000Z",
   "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000001",
      "displayName": "User A"
```

```
      }
     },
     "userAgent": {
      "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
      "platform": "windows",
      "productFamily": "teams"
     }
    },
    "callee": {
     "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
     "identity": {
      "user": {
       "id": "aaaaaaaa-0000-4000-8000-000000000002",
       "displayName": "User B"
      }
     },
     "userAgent": {
      "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
      "platform": "windows",
      "productFamily": "teams"
     }
    },
    "segments": [
     {
      "id": "seg-1",
      "startDateTime": "2026-10-20T09:00:00.0000000Z",
      "endDateTime": "2026-10-20T09:01:00.5000000Z",
      "media": [
       {
        "label": "main-audio",
        "streams": [
         {
          "streamId": "s1",
          "streamDirection": "calleeToCaller"
         }
        ]
       }
      ]
     }
    ]
   }
```

```
 ]
}
```

Expected result (D01_same_version_other_content.json):


```
{
 "call_id": "cccccccc-0000-4000-8000-000000000001",
 "source_version": 4,
 "is_complete": true,
 "incomplete_reasons": [],
 "session_set_hash": "27fdd2c1c47948bf811635d4b2e0118da39b64facf4637d46e0618f7acdcc8d4",
 "cdrs": [
  {
   "schema_version": 1,
   "event_type": "call",
   "tenant_id": "11111111-1111-4111-8111-111111111111",
   "call_id": "cccccccc-0000-4000-8000-000000000001",
   "session_id": "dddddddd-0000-4000-8000-000000000001",
   "source_version": 4,
   "call_type": "oneToOne",
   "caller": {
    "type": "user",
    "id": "aaaaaaaa-0000-4000-8000-000000000001"
   },
   "callee": {
    "type": "user",
    "id": "aaaaaaaa-0000-4000-8000-000000000002"
   },
   "start_utc": "2026-10-20T09:00:00.0000000Z",
   "end_utc": "2026-10-20T09:01:00.5000000Z",
   "duration_s": 61,
   "modalities": [
    "audio"
   ],
   "status": "completed",
   "termination_reason": null,
   "segment_count": 1,
   "aggregation_status": "complete",
   "source_hash": "582ce756b512d7fd57c11d2250451eacf9bb07076e6dd7985e81eda4814397e3",
   "extensions": {
    "segments": [
     {
```

```
          "id": "seg-1",
          "start": "2026-10-20T09:00:00.0000000Z",
          "end": "2026-10-20T09:01:00.5000000Z",
          "failure": null
        }
       ]
     }
    }
  ],
  "rejections": []
}
```

Input (D02_group_one_session_v8.json):

```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
  "id": "cccccccc-0000-4000-8000-000000000002",
  "version": 8,
  "type": "groupCall",
  "modalities": [
    "audio",
    "video"
  ],
  "startDateTime": "2026-10-20T10:00:00.0000000Z",
  "endDateTime": "2026-10-20T10:10:00.0000000Z",
  "sessions": [
    {
      "id": "dddddddd-0000-4000-8000-000000000001",
      "modalities": [
        "audio",
        "video"
      ],
      "startDateTime": "2026-10-20T10:00:00.0000000Z",
      "endDateTime": "2026-10-20T10:10:00.0000000Z",
      "caller": {
        "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
        "identity": {
          "user": {
            "id": "aaaaaaaa-0000-4000-8000-000000000001",
            "displayName": "User A"
          }
        },
```

```
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "callee": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000003",
          "displayName": "User C"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "segments": [
      {
        "id": "seg-1",
        "startDateTime": "2026-10-20T10:00:00.0000000Z",
        "endDateTime": "2026-10-20T10:10:00.0000000Z",
        "media": [
          {
            "label": "main-audio",
            "streams": [
              {
                "streamId": "s1",
                "streamDirection": "callerToCallee"
              }
            ]
          }
        ]
      }
    ]
  }
]
}
```

Expected result (D02_group_one_session_v8.json):


```
{
  "call_id": "cccccccc-0000-4000-8000-000000000002",
  "source_version": 8,
  "is_complete": true,
  "incomplete_reasons": [],
  "session_set_hash": "3bc4de956866f6cd6c58d29c99700989ad92114b5486baa739b0eeb33cd50d31",
  "cdrs": [
    {
      "schema_version": 1,
      "event_type": "call",
      "tenant_id": "11111111-1111-4111-8111-111111111111",
      "call_id": "cccccccc-0000-4000-8000-000000000002",
      "session_id": "dddddddd-0000-4000-8000-000000000001",
      "source_version": 8,
      "call_type": "group",
      "caller": {
        "type": "user",
        "id": "aaaaaaaa-0000-4000-8000-000000000001"
      },
      "callee": {
        "type": "user",
        "id": "aaaaaaaa-0000-4000-8000-000000000003"
      },
      "start_utc": "2026-10-20T10:00:00.0000000Z",
      "end_utc": "2026-10-20T10:10:00.0000000Z",
      "duration_s": 600,
      "modalities": [
        "audio",
        "video"
      ],
      "status": "completed",
      "termination_reason": null,
      "segment_count": 1,
      "aggregation_status": "complete",
      "source_hash": "c1d4d9ba424314df4c9e0442dfab4d8d37c9e3993c4e3df9a42002cff31fb95b",
      "extensions": {
        "segments": [
          {
            "id": "seg-1",
```

```
          "start": "2026-10-20T10:00:00.0000000Z",
          "end": "2026-10-20T10:10:00.0000000Z",
          "failure": null
        }
      ]
    }
  }
 ],
 "rejections": []
}
```


Input (D03_session_end_arrived_v3.json):


```
{
 "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
 "id": "cccccccc-0000-4000-8000-000000000008",
 "version": 3,
 "type": "peerToPeer",
 "modalities": [
  "audio"
 ],
 "startDateTime": "2026-10-20T15:00:00.0000000Z",
 "endDateTime": "2026-10-20T15:05:00.0000000Z",
 "sessions": [
  {
   "id": "dddddddd-0000-4000-8000-000000000001",
   "modalities": [
    "audio"
   ],
   "startDateTime": "2026-10-20T15:00:00.0000000Z",
   "caller": {
    "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
    "identity": {
     "user": {
      "id": "aaaaaaaa-0000-4000-8000-000000000001",
      "displayName": "User A"
     }
    },
    "userAgent": {
     "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
     "platform": "windows",
     "productFamily": "teams"
```

```
      }
     },
     "callee": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
       "user": {
        "id": "aaaaaaaa-0000-4000-8000-000000000002",
        "displayName": "User B"
       }
      },
      "userAgent": {
       "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
       "platform": "windows",
       "productFamily": "teams"
      }
     },
     "segments": [
      {
       "id": "seg-1",
       "startDateTime": "2026-10-20T15:00:00.0000000Z",
       "endDateTime": null,
       "media": [
        {
         "label": "main-audio",
         "streams": [
          {
           "streamId": "s1",
           "streamDirection": "callerToCallee"
          }
         ]
        }
       ]
      }
     ],
     "endDateTime": "2026-10-20T15:05:00.0000000Z"
    }
   ]
}
```

Expected result (D03_session_end_arrived_v3.json):

```
{
 "call_id": "cccccccc-0000-4000-8000-000000000008",
 "source_version": 3,
 "is_complete": true,
 "incomplete_reasons": [],
 "session_set_hash": "8ba282b5082377cd19ba35b0df2e47863bedd5635c0038a284f146c53332a221",
 "cdrs": [
  {
   "schema_version": 1,
   "event_type": "call",
   "tenant_id": "11111111-1111-4111-8111-111111111111",
   "call_id": "cccccccc-0000-4000-8000-000000000008",
   "session_id": "dddddddd-0000-4000-8000-000000000001",
   "source_version": 3,
   "call_type": "oneToOne",
   "caller": {
    "type": "user",
    "id": "aaaaaaaa-0000-4000-8000-000000000001"
   },
   "callee": {
    "type": "user",
    "id": "aaaaaaaa-0000-4000-8000-000000000002"
   },
   "start_utc": "2026-10-20T15:00:00.0000000Z",
   "end_utc": "2026-10-20T15:05:00.0000000Z",
   "duration_s": 300,
   "modalities": [
    "audio"
   ],
   "status": "completed",
   "termination_reason": null,
   "segment_count": 1,
   "aggregation_status": "complete",
   "source_hash": "99fc67b0658f03a05604111332692d02a69b38440dfa0b539aae2a7ce26ab54d",
   "extensions": {
    "segments": [
     {
      "id": "seg-1",
      "start": "2026-10-20T15:00:00.0000000Z",
      "end": null,
      "failure": null
```

```
        }
       ]
     }
    }
  ],
  "rejections": []
}
```

Input (D04_older_version_incomplete.json):

```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#communications/callRecords/$entity",
  "id": "cccccccc-0000-4000-8000-000000000001",
  "version": 3,
  "type": "peerToPeer",
  "modalities": [
    "audio"
  ],
  "startDateTime": "2026-10-20T09:00:00.0000000Z",
  "endDateTime": "2026-10-20T09:05:00.0000000Z",
  "sessions": [
    {
      "id": "dddddddd-0000-4000-8000-000000000001",
      "modalities": [
        "audio"
      ],
      "startDateTime": "2026-10-20T09:00:00.0000000Z",
      "endDateTime": null,
      "caller": {
        "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
        "identity": {
          "user": {
            "id": "aaaaaaaa-0000-4000-8000-000000000001",
            "displayName": "User A"
          }
        },
        "userAgent": {
          "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
          "platform": "windows",
          "productFamily": "teams"
        }
      },
```

```
    "callee": {
      "@odata.type": "#microsoft.graph.callRecords.participantEndpoint",
      "identity": {
        "user": {
          "id": "aaaaaaaa-0000-4000-8000-000000000002",
          "displayName": "User B"
        }
      },
      "userAgent": {
        "@odata.type": "#microsoft.graph.callRecords.clientUserAgent",
        "platform": "windows",
        "productFamily": "teams"
      }
    },
    "segments": [
      {
        "id": "seg-1",
        "startDateTime": "2026-10-20T09:00:00.0000000Z",
        "endDateTime": "2026-10-20T09:01:00.5000000Z",
        "media": [
          {
            "label": "main-audio",
            "streams": [
              {
                "streamId": "s1",
                "streamDirection": "callerToCallee"
              }
            ]
          }
        ]
      }
    ]
  }
 ]
}
```

Expected result (D04_older_version_incomplete.json):

```
{
 "call_id": "cccccccc-0000-4000-8000-000000000001",
 "source_version": 3,
 "is_complete": false,
```

```
  "incomplete_reasons": [
    "SESSION_DATES_MISSING"
  ],
  "session_set_hash": "4f6eb30bf13ac34cf101f8b13c3c19825a519d68d32137ea8841ea33a98d2ccf",
  "cdrs": [],
  "rejections": []
}
```


Appendix C. Next Steps

Part 3 — Host: endpoints (webhook, REST API, status), Teams bot, background services (queue workers, sender, reconciliation, subscription renewal, cleanup), Host settings, Host-level sequence diagrams, and Host automated tests. Part 4 — deployment, CI, README, Teams app manifest, end-to-end tenant verification, and consolidated [P] register.

Outstanding author decisions that do not affect Parts 2–3: hosting location (A or B), subdomain, tenant availability date, demonstration date, whether to accept prepared I1, notification recipient [P-14], and the meaning of “total duration” [P-13].

Programming has not started and is not authorized until the author approves the design.

Appendix D. List of Changes in Version 0.2 After Review of Version 0.1

№	Review comment	Change made	Location
1	Repeated webhook did not trigger another Graph fetch	duplicate only if item is new; done, failed, processing do not block; partial unique index (amendment A1 to Part 1 DDL); same-call items processed sequentially; Requeued removed; scenario S14	3.9, 3.11.2, 3.12, 3.19 S14, P-22
2	Reconciliation skipped calls with unknown version; /reprocess did not check range	NeedsFetch with force parameter: fetch done if version unknown, force reads any state; ReconcileAsync(from,to,force); scenario S15	3.13.6, 3.19 S15, P-21
3	Retrying POST is unsafe when response is lost	retry POST only for 429; OutcomeUnknown flag; subscription creation reconciled through ListSubscriptionsAsync (adopt existing); unknown notification outcome marked failed; stalled sending closed instead of resending; test Retry_PostIsRetriedToo replaced	3.14.1, 3.14.4, 3.15.3, 3.16, 3.19 S12, P-20
4	Cursor type undefined	Data/Cursor.cs file, Encode/Decode methods, contracts, CursorTests test class	3.11.8, 3.20
5	rejected.source_hash always NULL	sourceHash parameter to RecordRejectionAsync; RejectedSourceHash rule (canonical hash or SHA-256 text); S08 values calculated by Core; duplicate rejections suppressed	3.13.3, 3.13.4, 3.19 S08, P-23
6	Invalid Graph JSON did not enter rejected	GraphFetchStatus.Malformed; processor passes text to converter, yielding CALLRECORD_BAD_JSON; scenario S08(b)	3.14.1, 3.13.3, P-24
7	Completeness check preceded version check	ProcessAsync steps reordered: version before completeness; NotFound/PagesIncomplete leave already applied call unchanged; fixture D04, scenario S05(b)	3.13.3, 3.19 S05, P-23

№	Review comment	Change made	Location
8	S11 time contradicted call timestamps	base time T0 for scenarios moved to 18:00, after all fixture calls end (G09 — 16:05); S01–S15 and tests recalculated	3.19, 3.20
9	Markdown had figure placeholders instead of diagrams	Figures 4–10 inserted into Markdown as Mermaid diagrams built from same data as PDF/DOCX figures	3.12–3.16

Appendix E. Consolidated Exact File Register (No Path Abbreviations)

This register combines the fixed paths in §§3.2, 3.9, 3.22, and 3.34. Each row denotes one file. Generated .Designer.cs files, Snapshot, and diagram sources are listed explicitly. Each path becomes a verifiable expectation for the future project implementation.

Full path from root	Purpose
.editorconfig	formatting rules
.env.example	example environment variables with placeholders
.github/workflows/ci.yml	build and test on .NET 8
.gitignore	exceptions
Caddyfile	reverse proxy, TLS
Directory.Build.props	common properties: TargetFramework=net8.0, Nullable, ImplicitUsings, TreatWarningsAsErrors=true, LangVersion=12
Directory.Packages.props	central package versions
.dockerignore	Exclude secrets from Docker build context; K10
Dockerfile	application image
HANDOFF.md	handover between implementers
LICENSE	MIT license [P-06]
NuGet.Config	NuGet source configuration; package origin
README.md	deployment and test scenario
TeamsCallRecords.sln	decision
docker-compose.yml	launch the application and Caddy
docs/diagrams/sequence-04.mmd	Original Infrastructure diagram 4 (Mermaid)
docs/diagrams/sequence-05.mmd	Original Infrastructure diagram 5 (Mermaid)
docs/diagrams/sequence-06.mmd	Original Infrastructure diagram 6 (Mermaid)

Full path from root	Purpose
docs/diagrams/sequence-07.mmd	Original Infrastructure diagram 7 (Mermaid)
docs/diagrams/sequence-08.mmd	Original Infrastructure diagram 8 (Mermaid)
docs/diagrams/sequence-09.mmd	Original Infrastructure diagram 9 (Mermaid)
docs/diagrams/sequence-10.mmd	Original Infrastructure diagram 10 (Mermaid)
docs/diagrams/sequence-11.mmd	Source Mermaid for Figure 11
docs/diagrams/sequence-12.mmd	Source Mermaid for Figure 12
docs/diagrams/sequence-13.mmd	Source Mermaid for Figure 13
docs/diagrams/sequence-14.mmd	Source Mermaid for Figure 14
docs/diagrams/sequence-15.mmd	Source Mermaid for Figure 15
docs/diagrams/sequence-16.mmd	Source Mermaid for Figure 16
docs/diagrams/sequence-17.mmd	Source Mermaid for Figure 17
docs/diagrams/sequence-18.mmd	Source Mermaid for Figure 18
docs/diagrams/sequence-19.mmd	Source Mermaid for Figure 19
docs/diagrams/sequence-20.mmd	Source Mermaid for Figure 20
docs/diagrams/sequence-21.mmd	Source Mermaid for Figure 21
docs/diagrams/sequence-22.mmd	Source Mermaid for Figure 22
docs/diagrams/sequence-23.mmd	Source Mermaid for Figure 23
docs/diagrams/sequence-24.mmd	Source Mermaid for Figure 24
docs/explanatory-note-en.docx	the same, EN
docs/explanatory-note-ru.docx	explanatory note (Preliminary Design v1.1, RU)
docs/figures/fig1.svg	diagram 1 source
docs/figures/fig2.svg	diagram 2 source
docs/figures/fig3.svg	diagram 3 source (CallRecordConverter.Convert sequence)
docs/file-inventory.tsv	Canonical file registry generated from TWP
docs/permissions.md	Graph and RSC permissions register

Full path from root	Purpose
docs/tcri-trp-master.docx	This complete technical-working specification
docs/tcri-trp-master.md	Source Markdown for this specification
docs/test-protocol.md	test protocol
global.json	pins SDK 8.0.x, rollForward: latestFeature
scripts/check-artifacts.py	Validate declared files and naming
scripts/check-secrets.sh	Check tracked source for keys and identifiers
scripts/verify-schema.py	Compare SQLite schema to normative DDL
src/TeamsCallRecords.Core/CallRecordConverter.cs	CallRecordConverter.Convert
src/TeamsCallRecords.Core/Canonical.cs	Canonical: canonical JSON and hashes
src/TeamsCallRecords.Core/CdrJson.cs	CdrJson: JSON representation of CDR and result
src/TeamsCallRecords.Core/Models.cs	CallType, CdrStatus, AggregationStatus, PartyRef, Cdr, Rejection, RejectionCodes, IncompleteReasons, ConversionResult
src/TeamsCallRecords.Core/TeamsCallRecords.Core.csproj	project with no external packages
src/TeamsCallRecords.Core/VersionRules.cs	VersionAction, VersionRules: version decision, rounding
src/TeamsCallRecords.Host/Bot/BotAdapter.cs	BotAdapter registers verified SDK JWT adapter and /api/messages endpoint
src/TeamsCallRecords.Host/Bot/BotCommands.cs	Teams bot
src/TeamsCallRecords.Host/Bot/TcriBot.cs	Teams bot
src/TeamsCallRecords.Host/Config/ApiOptions.cs	ApiOptions
src/TeamsCallRecords.Host/Config/BotOptions.cs	BotOptions
src/TeamsCallRecords.Host/Config/HostOptionsValidator.cs	HostOptionsValidator: validate Bot/Api/Webhook, diagnostic startup failure
src/TeamsCallRecords.Host/Config/WebhookOptions.cs	WebhookOptions
src/TeamsCallRecords.Host/Program.cs	startup, service registration
src/TeamsCallRecords.Host/Properties/launchSettings.json	local HTTP/HTTPS profiles, development only
src/TeamsCallRecords.Host/TeamsCallRecords.Host.csproj	Host project

Full path from root	Purpose
src/TeamsCallRecords.Host/Web/ApiKeyFilter.cs	HTTP
src/TeamsCallRecords.Host/Web/CdrEndpoints.cs	HTTP
src/TeamsCallRecords.Host/Web/ErrorContracts.cs	ApiError, ApiRequestTrace; public error JSON without tokens
src/TeamsCallRecords.Host/Web/GraphNotificationParser.cs	GraphNotificationParser.Parse(ReadOnlyMemory), notification DTOs; strict checking of size, fields, tenant
src/TeamsCallRecords.Host/Web/GraphNotificationValidator.cs	GraphNotificationValidator.ValidateAsync; subscription, tenant, clientState validation
src/TeamsCallRecords.Host/Web/PendingSubscriptionRegistry.cs	in-flight markers from trusted Graph POST 201 with 120 s TTL
src/TeamsCallRecords.Host/Web/GraphNotifyEndpoint.cs	HTTP
src/TeamsCallRecords.Host/Web/HealthEndpoint.cs	HTTP
src/TeamsCallRecords.Host/Web/ReprocessEndpoint.cs	HTTP
src/TeamsCallRecords.Host/Workers/InboxWorker.cs	background services
src/TeamsCallRecords.Host/Workers/OutboxWorker.cs	background services
src/TeamsCallRecords.Host/Workers/ReconcileWorker.cs	background services
src/TeamsCallRecords.Host/Workers/StartupBarrier.cs	StartupBarrier and IStartupBarrier: readiness after DB, HTTP, checks
src/TeamsCallRecords.Host/Workers/SubscriptionWorker.cs	background services
src/TeamsCallRecords.Host/appsettings.example.json	example settings with secret placeholders
src/TeamsCallRecords.Host/appsettings.json	default settings
src/TeamsCallRecords.Infrastructure/Config/InboxOptions.cs	InboxOptions
src/TeamsCallRecords.Infrastructure/Config/OutboxOptions.cs	OutboxOptions
src/TeamsCallRecords.Infrastructure/Config/ProcessingOptions.cs	ProcessingOptions
src/TeamsCallRecords.Infrastructure/Config/ReconcileOptions.cs	ReconcileOptions
src/TeamsCallRecords.Infrastructure/Config/StorageOptions.cs	StorageOptions
src/TeamsCallRecords.Infrastructure/Data/CdrMapping.cs	CdrMapping: Cdr ↔ CdrRow
src/TeamsCallRecords.Infrastructure/Data/CdrReader.cs	CdrReader: reading CDRs for REST and bot
src/TeamsCallRecords.Infrastructure/Data/Cursor.cs	REST cursor encoding and decoding

Full path from root	Purpose
src/TeamsCallRecords.Infrastructure/Data/Entities/AuditLogRow.cs	rows for other tables
src/TeamsCallRecords.Infrastructure/Data/Entities/CdrRow.cs	cdr row
src/TeamsCallRecords.Infrastructure/Data/Entities/ErrorLogRow.cs	rows for other tables
src/TeamsCallRecords.Infrastructure/Data/Entities/InboxRow.cs	rows for other tables
src/TeamsCallRecords.Infrastructure/Data/Entities/OutboxRow.cs	rows for other tables
src/TeamsCallRecords.Infrastructure/Data/Entities/RejectedRow.cs	rows for other tables
src/TeamsCallRecords.Infrastructure/Data/Entities/SourceStateRow.cs	rows for other tables
src/TeamsCallRecords.Infrastructure/Data/Entities/States.cs	state constants
src/TeamsCallRecords.Infrastructure/Data/Entities/SubscriptionRow.cs	rows for other tables
src/TeamsCallRecords.Infrastructure/Data/ICdrReader.cs	ICdrReader, CdrItem, CdrPage, DailyStats
src/TeamsCallRecords.Infrastructure/Data/IWriteGate.cs	write-gate contract
src/TeamsCallRecords.Infrastructure/Data/Migrations/20261021000000_InitialCreate.Designer.cs	schema migration from 3.5
src/TeamsCallRecords.Infrastructure/Data/Migrations/20261021000000_InitialCreate.cs	schema migration from 3.5
src/TeamsCallRecords.Infrastructure/Data/Migrations/InitialCreate.sql	normative DDL from 3.5
src/TeamsCallRecords.Infrastructure/Data/Migrations/TcriDbContextModelSnapshot.cs	schema migration from 3.5
src/TeamsCallRecords.Infrastructure/Data/SqlitePragmaInterceptor.cs	SqlitePragmaInterceptor: PRAGMA upon connection open
src/TeamsCallRecords.Infrastructure/Data/TcriDbContext.cs	TcriDbContext
src/TeamsCallRecords.Infrastructure/Data/TcriDbContextFactory.cs	TcriDbContextFactory: design-time factory for dotnet ef
src/TeamsCallRecords.Infrastructure/Data/UtcText.cs	UtcText: database timestamp format
src/TeamsCallRecords.Infrastructure/Data/WriteGate.cs	WriteGate: serialized writes
src/TeamsCallRecords.Infrastructure/Diagnostics/ErrorLog.cs	database error log
src/TeamsCallRecords.Infrastructure/Diagnostics/IErrorLog.cs	database error log
src/TeamsCallRecords.Infrastructure/Graph/GraphClient.cs	Graph client
src/TeamsCallRecords.Infrastructure/Graph/GraphException.cs	Graph exceptions
src/TeamsCallRecords.Infrastructure/Graph/GraphModels.cs	Graph DTOs and errors

Full path from root	Purpose
src/TeamsCallRecords.Infrastructure/Graph/GraphOptions.cs	Graph client
src/TeamsCallRecords.Infrastructure/Graph/IGraphClient.cs	Graph client
src/TeamsCallRecords.Infrastructure/InfrastructureRegistration.cs	Infrastructure DI registration
src/TeamsCallRecords.Infrastructure/Outbox/IOutboxSender.cs	sender contract
src/TeamsCallRecords.Infrastructure/Outbox/IOutboxStore.cs	outgoing notifications
src/TeamsCallRecords.Infrastructure/Outbox/OutboxModels.cs	DTO outbox
src/TeamsCallRecords.Infrastructure/Outbox/OutboxSender.cs	sender
src/TeamsCallRecords.Infrastructure/Outbox/OutboxStore.cs	outgoing notifications
src/TeamsCallRecords.Infrastructure/Processing/CallProcessor.cs	call state, replacement transaction, item processing
src/TeamsCallRecords.Infrastructure/Processing/CdrApplier.cs	call state, replacement transaction, item processing
src/TeamsCallRecords.Infrastructure/Processing/ICallProcessor.cs	call state, replacement transaction, item processing
src/TeamsCallRecords.Infrastructure/Processing/ICdrApplier.cs	call state, replacement transaction, item processing
src/TeamsCallRecords.Infrastructure/Processing/IReconciler.cs	reconciliation contract
src/TeamsCallRecords.Infrastructure/Processing/ISourceStateStore.cs	call state, replacement transaction, item processing
src/TeamsCallRecords.Infrastructure/Processing/ProcessingModels.cs	processing outcomes
src/TeamsCallRecords.Infrastructure/Processing/Reconciler.cs	reconciliation and recovery
src/TeamsCallRecords.Infrastructure/Processing/SourceStateStore.cs	call state, replacement transaction, item processing
src/TeamsCallRecords.Infrastructure/Queue/IInboxStore.cs	incoming queue
src/TeamsCallRecords.Infrastructure/Queue/InboxStore.cs	incoming queue
src/TeamsCallRecords.Infrastructure/Queue/InboxTransitions.cs	atomic transitions
src/TeamsCallRecords.Infrastructure/Queue/QueueModels.cs	DTO inbox
src/TeamsCallRecords.Infrastructure/Subscriptions/ISubscriptionStore.cs	subscriptions
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionOptions.cs	subscription options
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionService.cs	subscriptions
src/TeamsCallRecords.Infrastructure/Subscriptions/SubscriptionStore.cs	subscriptions

Full path from root	Purpose
src/TeamsCallRecords.Infrastructure/TeamsCallRecords.Infrastructure.csproj	project
teams-app/color.png	color icon
teams-app/manifest.json	Teams application manifest with RSC
teams-app/outline.png	outline icon
tests/TeamsCallRecords.Core.Tests/CdrJsonTests.cs	Core tests
tests/TeamsCallRecords.Core.Tests/ConverterTests.cs	Core tests
tests/TeamsCallRecords.Core.Tests/DuplicateSessionTests.cs	Core tests
tests/TeamsCallRecords.Core.Tests/GoldenTests.cs	Core tests
tests/TeamsCallRecords.Core.Tests/HashTests.cs	Core tests
tests/TeamsCallRecords.Core.Tests/IncompleteReasonsTests.cs	Core tests
tests/TeamsCallRecords.Core.Tests/RulesTests.cs	Core tests
tests/TeamsCallRecords.Core.Tests/Samples.cs	synthetic record builder
tests/TeamsCallRecords.Core.Tests/TeamsCallRecords.Core.Tests.csproj	test project (xunit 2.9.3)
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G01_one_to_one_single_session.json	Golden input G01
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G02_group_two_sessions.json	Golden input G02
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G03_overlapping_segments.json	Golden input G03
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G04_failure_info.json	Golden input G04
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G05_no_caller_no_callee.json	Golden input G05
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G06_phone_and_unknown_party.json	Golden input G06
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G07_no_sessions.json	Golden input G07
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G08_session_end_missing.json	Golden input G08
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G09_invalid_and_valid_session.json	Golden input G09
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G10_utc_offset_and_half_second.json	Golden input G10
tests/TeamsCallRecords.Core.Tests/TestData/Golden/in/G11_version_higher_content_changed.json	Golden input G11
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G01_one_to_one_single_session.json	Golden expected result G01

Full path from root	Purpose
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G02_group_two_sessions.json	Golden expected result G02
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G03_overlapping_segments.json	Golden expected result G03
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G04_failure_info.json	Golden expected result G04
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G05_no_caller_no_callee.json	Golden expected result G05
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G06_phone_and_unknown_party.json	Golden expected result G06
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G07_no_sessions.json	Golden expected result G07
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G08_session_end_missing.json	Golden expected result G08
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G09_invalid_and_valid_session.json	Golden expected result G09
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G10_utc_offset_and_half_second.json	Golden expected result G10
tests/TeamsCallRecords.Core.Tests/TestData/Golden/out/G11_version_higher_content_changed.json	Golden expected result G11
tests/TeamsCallRecords.Host.Tests/BotAuthenticationTests.cs	invalid JWT, wrong aud/iss, expiry
tests/TeamsCallRecords.Host.Tests/BotCommandsTests.cs	help, calls today, timezone, no data
tests/TeamsCallRecords.Host.Tests/CdrEndpointTests.cs	API key, boundaries, cursor, errors
tests/TeamsCallRecords.Host.Tests/ConfigurationTests.cs	missing secret, defaults, options
tests/TeamsCallRecords.Host.Tests/EndToEndSyntheticTests.cs	webhook→inbox→Graph fake→Core→CDR→REST/outbox fake
tests/TeamsCallRecords.Host.Tests/HealthEndpointTests.cs	live/ready/status, DB failure
tests/TeamsCallRecords.Host.Tests/LifecycleEndpointTests.cs	reauthorizationRequired, missed, removed events
tests/TeamsCallRecords.Host.Tests/ReprocessEndpointTests.cs	durable reconcile range 202, interval ≤7 days, no Graph in HTTP
tests/TeamsCallRecords.Host.Tests/Support/FakeTeamsAdapter.cs	incoming signed test Bot activities
tests/TeamsCallRecords.Host.Tests/Support/TcriWebFactory.cs	test Host factory with replaceable services and SQLite temp file
tests/TeamsCallRecords.Host.Tests/Support/WebhookFixtures.cs	valid/invalid notifications, lifecycle, validationToken
tests/TeamsCallRecords.Host.Tests/TeamsCallRecords.Host.Tests.csproj	xUnit + Microsoft.AspNetCore.Mvc.Testing + Bot test adapter
tests/TeamsCallRecords.Host.Tests/TestData/bot_calls_today.json	test bot message
tests/TeamsCallRecords.Host.Tests/TestData/webhook_call.json	synthetic input signal; no clientState in logs

Full path from root	Purpose
tests/TeamsCallRecords.Host.Tests/TestData/webhook_lifecycle_missed.json	missed signal
tests/TeamsCallRecords.Host.Tests/WebhookEndpointTests.cs	Graph callback acceptance
tests/TeamsCallRecords.Host.Tests/WebhookSecurityTests.cs	subscription, tenant, clientState, size, HTTPS proxy
tests/TeamsCallRecords.Host.Tests/WorkerRecoveryTests.cs	graceful shutdown, reclaim lease, retry
tests/TeamsCallRecords.Infrastructure.Tests/CallProcessorTests.cs	item processing
tests/TeamsCallRecords.Infrastructure.Tests/CdrApplierTests.cs	replacement transaction
tests/TeamsCallRecords.Infrastructure.Tests/CdrMappingTests.cs	mapping
tests/TeamsCallRecords.Infrastructure.Tests/CdrReaderTests.cs	CDR reading
tests/TeamsCallRecords.Infrastructure.Tests/CursorTests.cs	10 cursor unit tests
tests/TeamsCallRecords.Infrastructure.Tests/ErrorLogTests.cs	error log
tests/TeamsCallRecords.Infrastructure.Tests/GraphClientTests.cs	Graph client
tests/TeamsCallRecords.Infrastructure.Tests/GraphResilienceTests.cs	retries and circuit breaker
tests/TeamsCallRecords.Infrastructure.Tests/InboxStoreTests.cs	inbox queue
tests/TeamsCallRecords.Infrastructure.Tests/OptionsTests.cs	options
tests/TeamsCallRecords.Infrastructure.Tests/OutboxSenderTests.cs	sending
tests/TeamsCallRecords.Infrastructure.Tests/OutboxStoreTests.cs	outbox queue
tests/TeamsCallRecords.Infrastructure.Tests/ReconcilerTests.cs	reconciliation
tests/TeamsCallRecords.Infrastructure.Tests/ScenarioTests.cs	S01–S15 scenarios (3.19)
tests/TeamsCallRecords.Infrastructure.Tests/SchemaTests.cs	schema and connection
tests/TeamsCallRecords.Infrastructure.Tests/SourceStateStoreTests.cs	call state
tests/TeamsCallRecords.Infrastructure.Tests/SubscriptionServiceTests.cs	subscriptions
tests/TeamsCallRecords.Infrastructure.Tests/SubscriptionStoreTests.cs	subscription store
tests/TeamsCallRecords.Infrastructure.Tests/Support/Builders.cs	builders for options, queue items, and records
tests/TeamsCallRecords.Infrastructure.Tests/Support/FakeCdrApplier.cs	FakeCdrApplier: ICdrApplier substitute with predefined results
tests/TeamsCallRecords.Infrastructure.Tests/Support/FakeCredential.cs	FakeCredential: offline token

Full path from root	Purpose
tests/TeamsCallRecords.Infrastructure.Tests/Support/FakeGraphClient.cs	FakeGraphClient: programmable IGraphClient fake
tests/TeamsCallRecords.Infrastructure.Tests/Support/FakeReconciler.cs	FakeReconciler: IReconciler substitute for CallProcessorTests
tests/TeamsCallRecords.Infrastructure.Tests/Support/Golden.cs	Golden: reads G01–G11 and derived samples
tests/TeamsCallRecords.Infrastructure.Tests/Support/StubHandler.cs	StubHandler: scripted HTTP responses
tests/TeamsCallRecords.Infrastructure.Tests/Support/TestDb.cs	TestDb: temporary database, factory, WriteGate, clock
tests/TeamsCallRecords.Infrastructure.Tests/TeamsCallRecords.Infrastructure.Tests.csproj	test project
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/D01_same_version_other_content.json	derived golden sample (input), Appendix B
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/D02_group_one_session_v8.json	derived golden sample (input), Appendix B
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/D03_session_end_arrived_v3.json	derived golden sample (input), Appendix B
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/in/D04_older_incomplete_version.json	D04 synthetic test input
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/out/D01_same_version_other_content.json	derived golden sample (expected Core output), Appendix B
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/out/D02_group_one_session_v8.json	derived golden sample (expected Core output), Appendix B
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/out/D03_session_end_arrived_v3.json	derived golden sample (expected Core output), Appendix B
tests/TeamsCallRecords.Infrastructure.Tests/TestData/Derived/out/D04_older_incomplete_version.json	D04 expected Core conversion result
tests/TeamsCallRecords.Infrastructure.Tests/UtcTextTests.cs	timestamp format
tests/TeamsCallRecords.Infrastructure.Tests/WriteGateTests.cs	serialized writes

Appendix F. Verifiable Named Infrastructure Test Register (TRP-1.5)

Source: verbatim register of 281 Infrastructure v0.2 test methods plus four proposed P-27 extensions: RenewLease_ExtendsOnlyProcessing, RenewLease_AfterReclaim_Fails, ReconcileWithRange_UsesRawPeriod, ReconcileBadRange_FailsNoLookback. There are 285 unique named test methods in total.

F.1. Summary by 20 Classes

Class	From original v0.2	New	Total
SchemaTests	8	0	8
CdrMappingTests	6	0	6
UtcTextTests	5	0	5
WriteGateTests	7	0	7

Class	From original v0.2	New	Total
InboxStoreTests	25	2	27
SourceStateStoreTests	22	0	22
CdrApplierTests	20	0	20
CallProcessorTests	33	2	35
ReconcilerTests	15	0	15
OutboxStoreTests	11	0	11
OutboxSenderTests	10	0	10
GraphClientTests	31	0	31
GraphResilienceTests	12	0	12
SubscriptionStoreTests	8	0	8
SubscriptionServiceTests	17	0	17
CdrReaderTests	12	0	12
ErrorLogTests	5	0	5
OptionsTests	8	0	8
ScenarioTests	16	0	16
CursorTests	10	0	10
Overall total	**281**	**4**	**285**

F.2. Named Test Register

№	Class	Method
1	SchemaTests	Migration_CreatesEightTables
2	SchemaTests	Migration_CreatesNamedIndexesAndKeys
3	SchemaTests	Migration_IsIdempotent
4	SchemaTests	Connection_UsesWalFullForeignKeysBusyTimeout
5	SchemaTests	CheckConstraint_RejectsUnknownState
6	SchemaTests	Inbox_UniqueIndexIsPartial

№	Class	Method
7	SchemaTests	Model_ColumnsMatchMigratedTables
8	SchemaTests	Data_SurvivesRestart
9	CdrMappingTests	RoundTrip_AllGoldenCdrs
10	CdrMappingTests	Ticks_ArePreservedWithoutLoss
11	CdrMappingTests	NullCaller_WritesBothColumnsNull
12	CdrMappingTests	Modalities_AreJsonArray
13	CdrMappingTests	CopyOnUpdate_KeepsSeqAndCreated
14	CdrMappingTests	ToRow_SetsCreatedAndUpdated
15	UtcTextTests	Format_SevenDigitsAndZ
16	UtcTextTests	Format_ConvertsOffsetToUtc
17	UtcTextTests	Parse_RoundTripKeepsTicks
18	UtcTextTests	Format_OrderEqualsLexicographicOrder
19	UtcTextTests	Now_UsesInjectedClock
20	WriteGateTests	Run_Commits
21	WriteGateTests	Run_ReturnsValue
22	WriteGateTests	Run_RollsBackOnException
23	WriteGateTests	Run_ReleasesGateAfterException
24	WriteGateTests	Run_ExecutesOneAtATime
25	WriteGateTests	Run_CancelledBeforeEntry
26	WriteGateTests	Readers_DoNotWaitForGate
27	InboxStoreTests	Enqueue_Empty_ReturnsZeros
28	InboxStoreTests	Enqueue_NewItem_IsAdded
29	InboxStoreTests	Enqueue_SameContent_IsDuplicate
30	InboxStoreTests	Enqueue_PropertyOrderDoesNotMatter
31	InboxStoreTests	Enqueue_SameBatchDuplicates

№	Class	Method
32	InboxStoreTests	Enqueue_AfterFailed_AddsNewItem
33	InboxStoreTests	Enqueue_AfterDone_AddsNewItem
34	InboxStoreTests	Enqueue_WhileProcessing_AddsNewItem
35	InboxStoreTests	Enqueue_Overloaded_WritesNothing
36	InboxStoreTests	ElementHash_Is64LowerHex
37	InboxStoreTests	ElementHash_DependsOnKindAndTenant
38	InboxStoreTests	ElementHash_BadJson_Throws
39	InboxStoreTests	ClaimNext_ReturnsOldestDue
40	InboxStoreTests	ClaimNext_SameTimeOrderedById
41	InboxStoreTests	ClaimNext_SameCallInProcessing_IsSkipped
42	InboxStoreTests	ClaimNext_SameCallExpiredLease_DoesNotBlock
43	InboxStoreTests	ClaimNext_NothingDue_ReturnsNull
44	InboxStoreTests	ClaimNext_SkipsLockedProcessing
45	InboxStoreTests	ClaimNext_RecoversExpiredLease
46	InboxStoreTests	ClaimNext_ParallelCallers_NoDuplicates
47	InboxStoreTests	Complete_SetsDone
48	InboxStoreTests	Reschedule_ResetAttemptsFlag
49	InboxStoreTests	Fail_SetsFailed
50	InboxStoreTests	Transition_MissingId_Throws
51	InboxStoreTests	GetStats_CountsPendingOldestAndFailed
52	InboxStoreTests	RenewLease_ExtendsOnlyProcessing
53	InboxStoreTests	RenewLease_AfterReclaim_Fails
54	SourceStateStoreTests	RegisterIncomplete_First_SetsPending
55	SourceStateStoreTests	RegisterIncomplete_Schedule_1_5_15_60_60
56	SourceStateStoreTests	RegisterIncomplete_BecomesSuspendedAtLimit

№	Class	Method
57	SourceStateStoreTests	RegisterIncomplete_BelowLimit_StaysPending
58	SourceStateStoreTests	RegisterIncomplete_SuspendedRecheck_Reschedules
59	SourceStateStoreTests	RegisterIncomplete_BecomesExpiredAfterRetention
60	SourceStateStoreTests	RegisterIncomplete_AtExactRetention_NotExpired
61	SourceStateStoreTests	RegisterIncomplete_Expired_StaysExpired
62	SourceStateStoreTests	RegisterIncomplete_NewEpisodeAfterDone
63	SourceStateStoreTests	RegisterIncomplete_MissingInbox_RollsBack
64	SourceStateStoreTests	RecordRejection_G09_WritesRejectedAndState
65	SourceStateStoreTests	RecordRejection_KeepsMaxRejectedVersion
66	SourceStateStoreTests	RecordRejection_SameRowTwice_NotDuplicated
67	SourceStateStoreTests	RecordRejection_OtherHash_AddsRow
68	SourceStateStoreTests	RecordRejection_ClosesPendingEpisode
69	SourceStateStoreTests	RecordRejection_WithoutCallId_WritesOnlyRejected
70	SourceStateStoreTests	RecordRejection_MessageTruncatedTo1000
71	SourceStateStoreTests	RecordDiscrepancy_WritesAuditRow
72	SourceStateStoreTests	RecordDiscrepancy_NullAppliedHash_WritesJsonNull
73	SourceStateStoreTests	GetMany_ChunksOf500
74	SourceStateStoreTests	ListSuspendedDue_OrdersAndLimits
75	SourceStateStoreTests	GetCounts_GroupsByState
76	CdrApplierTests	Apply_G01_InsertsCdrStateOutbox
77	CdrApplierTests	Apply_G02_TwoSessions_FourOutboxRows
78	CdrApplierTests	Apply_G03_OverlappingSegments_OneCdr
79	CdrApplierTests	Apply_G04_Failed_StoresReason
80	CdrApplierTests	Apply_G05_NoParties_NoOutbox
81	CdrApplierTests	Apply_G06_PhoneAndUnknown_NoOutbox

№	Class	Method
82	CdrApplierTests	Apply_G10_KeepsSubSecondTime
83	CdrApplierTests	Apply_HigherVersion_UpdatesInPlace
84	CdrApplierTests	Apply_ReplaceSet_DeletesMissingSession
85	CdrApplierTests	Apply_ExistingSession_NoNewOutbox
86	CdrApplierTests	Apply_OldEndedCall_NoOutbox
87	CdrApplierTests	Apply_OutboxBoundary_ExactlyMaxAge
88	CdrApplierTests	Apply_SameCaller_AndCallee_OneRecipient
89	CdrApplierTests	Apply_OnlyUserParties_AreRecipients
90	CdrApplierTests	Apply_DecisionChanged_ChangesNothing
91	CdrApplierTests	Apply_SameVersionTwice_SecondIsDecisionChanged
92	CdrApplierTests	Apply_Failure_RollsBackEverything
93	CdrApplierTests	Apply_ClosesInboxInSameTransaction
94	CdrApplierTests	Apply_PreservesRejectedVersion
95	CdrApplierTests	Payload_Format_IsExact
96	CallProcessorTests	ChannelMessage_IsLoggedAndCompleted
97	CallProcessorTests	ReconcileWithoutCallId_RunsLookback
98	CallProcessorTests	CallRecordWithoutCallId_Fails
99	CallProcessorTests	G01_Applied
100	CallProcessorTests	SameVersion_Duplicate
101	CallProcessorTests	SameVersionOtherContent_Discrepancy
102	CallProcessorTests	HigherVersion_ReplacesCdr
103	CallProcessorTests	LowerVersion_Ignored
104	CallProcessorTests	OlderIncompleteVersion_DoesNotReopenDone
105	CallProcessorTests	SameVersionIncomplete_AfterApplied_Discrepancy
106	CallProcessorTests	G07_NoSessions_Pending

№	Class	Method
107	CallProcessorTests	G08_SessionEndMissing_Pending
108	CallProcessorTests	G08_ThenD03_Applied
109	CallProcessorTests	G09_InvalidSession_Rejected
110	CallProcessorTests	G09_Repeat_Duplicate
111	CallProcessorTests	G09_SourceHash_IsCanonical
112	CallProcessorTests	G09_NewerVersionValid_Applies
113	CallProcessorTests	BadJson_Malformed_RejectedWithRawHash
114	CallProcessorTests	BadJson_Repeated_NoSecondRejectedRow
115	CallProcessorTests	BadJson_FromGraphClientStub_IsRejectedNotFailed
116	CallProcessorTests	GraphNotFound_RegistersIncomplete
117	CallProcessorTests	GraphPagesIncomplete_RegistersIncomplete
118	CallProcessorTests	GraphNotFoundOrIncomplete_AlreadyApplied_Duplicate
119	CallProcessorTests	TransientGraphError_Reschedules
120	CallProcessorTests	TransientGraphError_RetryScheduleTable
121	CallProcessorTests	TransientError_ExhaustedAttempts_Fails
122	CallProcessorTests	PermanentGraphError_FailsImmediately
123	CallProcessorTests	UnexpectedException_Reschedules
124	CallProcessorTests	Cancellation_PropagatesAndKeepsProcessing
125	CallProcessorTests	DecisionChangedRepeatedly_ThreeTimes_Fails
126	CallProcessorTests	SessionLevelRejectionsNotWrittenWhileTemporaryReasonsPresent
127	CallProcessorTests	SuspendedRecheck_ExpiresAfterRetention
128	CallProcessorTests	ReprocessSameElement_AfterLeaseLoss_IsIdempotent
129	CallProcessorTests	ReconcileWithRange_UsesRawPeriod
130	CallProcessorTests	ReconcileBadRange_FailsNoLookback
131	ReconcilerTests	Reconcile_UnknownCall_IsEnqueued

№	Class	Method
132	ReconcilerTests	Reconcile_KnownSameVersion_NotEnqueued
133	ReconcilerTests	Reconcile_KnownHigherVersion_IsEnqueued
134	ReconcilerTests	Reconcile_VersionLowerThanRejected_NotEnqueued
135	ReconcilerTests	Reconcile_VersionNull_DoneState_IsEnqueued
136	ReconcilerTests	Reconcile_PendingSuspendedExpired_NotEnqueued
137	ReconcilerTests	Reconcile_Force_EnqueuesEveryState
138	ReconcilerTests	Reconcile_ForceFalse_SameVersionDone_NotEnqueued
139	ReconcilerTests	Reconcile_SecondRun_AddsNothing
140	ReconcilerTests	Reconcile_AfterFailedItem_AddsNewItem
141	ReconcilerTests	Reconcile_Overloaded_ReturnsZero
142	ReconcilerTests	Reconcile_GraphError_Propagates
143	ReconcilerTests	Lookback_UsesNowMinusLookback_AndSetsLastRun
144	ReconcilerTests	RecheckSuspended_EnqueuesDueRows
145	ReconcilerTests	RecheckSuspended_SameRowTwice_NotDuplicated
146	OutboxStoreTests	ClaimNext_ReturnsDueNew
147	OutboxStoreTests	ClaimNext_SkipsFutureAndSent
148	OutboxStoreTests	ClaimNext_ExpiredSending_BecomesFailedUnknown
149	OutboxStoreTests	ClaimNext_Parallel_NoDuplicates
150	OutboxStoreTests	MarkSent_SetsSentUtc
151	OutboxStoreTests	Reschedule_BackoffTable
152	OutboxStoreTests	Reschedule_AtMaxAttempts_Fails
153	OutboxStoreTests	Fail_SetsFailedWithoutBackoff
154	OutboxStoreTests	UniqueKey_PreventsSecondRowForSameRecipient
155	OutboxStoreTests	GetStats_CountsByState
156	OutboxStoreTests	BackoffFor_IndexIsClamped

№	Class	Method
157	OutboxSenderTests	RunOnce_NoItems_ReturnsFalse
158	OutboxSenderTests	RunOnce_Success_MarksSent
159	OutboxSenderTests	RunOnce_TransientError_Reschedules
160	OutboxSenderTests	RunOnce_TransientError_EightAttempts_Fails
161	OutboxSenderTests	RunOnce_PermanentError_FailsImmediately
162	OutboxSenderTests	RunOnce_OutcomeUnknown_FailsWithoutRetry
163	OutboxSenderTests	RunOnce_UnexpectedException_TreatedAsOutcomeUnknown
164	OutboxSenderTests	RunOnce_BadPayload_Fails
165	OutboxSenderTests	RunOnce_SendFailure_DoesNotTouchCdr
166	OutboxSenderTests	RunOnce_Cancellation_KeepsSending
167	GraphClientTests	GetCallRecord_Ok_ReturnsJson
168	GraphClientTests	GetCallRecord_404_ReturnsNotFound
169	GraphClientTests	GetCallRecord_EscapesCallId
170	GraphClientTests	GetCallRecord_MergesSessionPages
171	GraphClientTests	GetCallRecord_MergesSegmentPages
172	GraphClientTests	GetCallRecord_PageLimit_ReturnsPagesIncomplete
173	GraphClientTests	GetCallRecord_NextLinkOtherHost_Throws
174	GraphClientTests	GetCallRecord_BadJson_ReturnsMalformed
175	GraphClientTests	GetCallRecord_BadJsonOnNextPage_ReturnsMalformed
176	GraphClientTests	ListCallRecords_BadJson_Throws
177	GraphClientTests	Error_Classification
178	GraphClientTests	Error_Network_IsTransient
179	GraphClientTests	Error_Post_OutcomeUnknown
180	GraphClientTests	Error_Post_DefiniteRefusal_NotUnknown
181	GraphClientTests	Error_NonPost_NeverUnknown

№	Class	Method
182	GraphClientTests	Error_Timeout_IsTransient
183	GraphClientTests	Error_CallerCancellation_Propagates
184	GraphClientTests	Error_TokenFailure_IsPermanent
185	GraphClientTests	Error_Message_HasNoSecretsOrBody
186	GraphClientTests	Token_RequestedWithGraphScope
187	GraphClientTests	ListCallRecords_ParsesPages
188	GraphClientTests	ListCallRecords_Filter
189	GraphClientTests	ListCallRecords_SkipsItemWithoutId
190	GraphClientTests	ListCallRecords_PageLimit_Throws
191	GraphClientTests	ListSubscriptions_ParsesPages
192	GraphClientTests	ListSubscriptions_SkipsIncompleteItems
193	GraphClientTests	CreateSubscription_Body
194	GraphClientTests	RenewSubscription_Body
195	GraphClientTests	DeleteSubscription_404And204_AreSuccess
196	GraphClientTests	SendActivityNotification_Body
197	GraphClientTests	SendActivityNotification_403_IsPermanent
198	GraphResilienceTests	Retry_503_ThenOk_Succeeds
199	GraphResilienceTests	Retry_Exhausted_Returns5xx_AsTransient
200	GraphResilienceTests	Retry_429_HonorsRetryAfter
201	GraphResilienceTests	Retry_400_NotRetried
202	GraphResilienceTests	Retry_Post_503_NotRetried
203	GraphResilienceTests	Retry_Post_Timeout_NotRetried
204	GraphResilienceTests	Retry_Post_429_IsRetried
205	GraphResilienceTests	Retry_Patch_503_IsRetried
206	GraphResilienceTests	Retry_Delete_503_IsRetried

№	Class	Method
207	GraphResilienceTests	Retry_BackoffGrowsExponentially
208	GraphResilienceTests	Timeout_Attempt_IsTransient
209	GraphResilienceTests	CircuitBreaker_OpensAfterFailures
210	SubscriptionStoreTests	Upsert_InsertsAndReplaces
211	SubscriptionStoreTests	List_OrderedByCreated
212	SubscriptionStoreTests	Get_Missing_ReturnsNull
213	SubscriptionStoreTests	SetStatus_Changes_AndMissingIgnored
214	SubscriptionStoreTests	Delete_RemovesRow_AndMissingIgnored
215	SubscriptionStoreTests	HashClientState_IsSha256Lower64
216	SubscriptionStoreTests	Matches_ConstantTimeSemantics
217	SubscriptionStoreTests	ClientState_IsNeverStored
218	SubscriptionServiceTests	Ensure_NoRows_CreatesCallRecordsSubscription
219	SubscriptionServiceTests	Ensure_CreateOutcomeUnknown_NextEnsureAdoptsExisting
220	SubscriptionServiceTests	Ensure_DuplicatesInGraph_KeepsLatestDeletesOthers
221	SubscriptionServiceTests	Ensure_ForeignNotificationUrl_IsNotAdopted
222	SubscriptionServiceTests	Ensure_WithTeamAndChannel_CreatesBoth
223	SubscriptionServiceTests	Ensure_ActiveFarFromExpiry_DoesNothing
224	SubscriptionServiceTests	Ensure_LessThanHalfLeft_Renews
225	SubscriptionServiceTests	Ensure_RenewFails_CreatesNewAndDeletesOld
226	SubscriptionServiceTests	Ensure_ExpiredRow_IsReplaced
227	SubscriptionServiceTests	Ensure_CreateFails_ReturnsFalse_AndLogs
228	SubscriptionServiceTests	Ensure_ExtraRowsOfKind_AreRemoved
229	SubscriptionServiceTests	Lifecycle_ReauthorizationRequired_Renews
230	SubscriptionServiceTests	Lifecycle_SubscriptionRemoved_Recreates
231	SubscriptionServiceTests	Lifecycle_Missed_EnqueuesFullReconcile

№	Class	Method
232	SubscriptionServiceTests	Lifecycle_UnknownSubscription_OnlyLogs
233	SubscriptionServiceTests	Lifecycle_UnknownEvent_OnlyLogs
234	SubscriptionServiceTests	Lifecycle_GraphError_IsLoggedNotThrown
235	CdrReaderTests	Query_Interval_IsHalfOpen
236	CdrReaderTests	Query_OrderedByStartThenSeq
237	CdrReaderTests	Query_Paging_NoGapsNoRepeats
238	CdrReaderTests	Query_Paging_StableUnderInsert
239	CdrReaderTests	Query_LimitOutOfRange_Throws
240	CdrReaderTests	Query_FromAfterTo_Throws
241	CdrReaderTests	Query_BadCursor_Throws
242	CdrReaderTests	Query_NextCursor_IsDecodable
243	CdrReaderTests	Query_ReturnsCdrSeq
244	CdrReaderTests	GetStats_CountsDistinctCallsAndSumsDuration
245	CdrReaderTests	GetStats_EmptyWindow_ReturnsZeros
246	CdrReaderTests	GetStats_HalfOpenWindow
247	ErrorLogTests	Write_AddsRow
248	ErrorLogTests	Write_TruncatesMessageAndDetails
249	ErrorLogTests	Write_DatabaseFailure_DoesNotThrow
250	ErrorLogTests	Write_Cancellation_Propagates
251	ErrorLogTests	Write_NullDetails_Allowed
252	OptionsTests	Defaults_MatchSpecification
253	OptionsTests	RetryDelayFor_Table
254	OptionsTests	BackoffFor_Table
255	OptionsTests	Validate_InvalidValues
256	OptionsTests	ValidateOnStart_ListsViolations_WithoutValues

№	Class	Method
257	OptionsTests	AddTcriInfrastructure_ResolvesAllServices
258	OptionsTests	MigrateAsync_CreatesDirectory_AndWal
259	OptionsTests	Registration_UsesFakeTokenCredential
260	ScenarioTests	S01_FirstApply
261	ScenarioTests	S02_Duplicate
262	ScenarioTests	S03_Discrepancy
263	ScenarioTests	S04_HigherVersionReplaces
264	ScenarioTests	S05_LowerVersionIgnored
265	ScenarioTests	S06_ReplaceWithDeletion
266	ScenarioTests	S07_PendingThenSuspended
267	ScenarioTests	S08_InvalidSession_Rejected
268	ScenarioTests	S09_TransientErrors_ThenFailed
269	ScenarioTests	S10_LeaseRecovery
270	ScenarioTests	S11_DataCompleted_Later
271	ScenarioTests	S12_OutboxFailure
272	ScenarioTests	S13_SuspendedExpires
273	ScenarioTests	S14_RepeatedNotification_RefetchesGraph
274	ScenarioTests	S15_ReprocessForce
275	ScenarioTests	Restart_BetweenSteps_KeepsState
276	CursorTests	Encode_Decode_RoundTrip
277	CursorTests	Encode_KnownValue
278	CursorTests	Encode_IsUrlSafe_NoPadding
279	CursorTests	Encode_InvalidArguments_Throw
280	CursorTests	Decode_EmptyOrWhitespace_Throws
281	CursorTests	Decode_InvalidBase64_Throws

№	Class	Method
282	CursorTests	Decode_NoSeparatorOrExtraParts_Throws
283	CursorTests	Decode_NonCanonicalTime_Throws
284	CursorTests	Decode_BadSequence_Throws
285	CursorTests	Decode_InvalidUtf8_Throws

Appendix G. TRP-1.4 Integration Technical Record (Historical)

ID	Check / correction	Implemented in text	Execution confirmation
I-01	T0 for S01–S15=18:00 UTC, aligned with D03; separate unit tests may use 12:00 UTC	yes, §3.19/3.20	awaits .NET 8 testing
I-02	Only partial uq_inbox_new_hash, no uq_inbox_hash	yes, §§3.5/3.11/3.12	awaits DDL/EF model comparison
I-03	InboxEnqueueResult without Requeued; repeated notification after done/processing	yes, §§3.12/3.20	awaits .NET 8 testing
I-04	POST is not blindly retried without determining OutcomeUnknown	yes, §§3.14/3.20	verify against Graph contract
I-05	D04 uses G01 with same call_id and lower version	yes, §3.33, Appendix B	synthetic hash calculated; test required
I-06	NeedsFetch(s,r,force), including done + Version=NULL, and manual Reprocess	yes, §3.13.7/§3.19 S15	verify Graph list semantics on tenant
I-07	Complete inputs/outputs D01–D04	yes, Appendix B	verify all JSON
I-08	S14/S15 with row-by-row states	yes, §3.19	tests not yet executed
I-09	Named test register rather than arithmetic promises	yes, Appendix E	xUnit discovery pending
I-10	Synchronize original Infrastructure v0.2	**done in TRP-1.5: authentic MD v0.2 compared by SHA and sections**	comparison recorded in Appendix I; Appendix D sourced from authentic v0.2

The document remains a candidate for approval. No evidence establishes that actual code fully complies with this specification; SDK 8, Graph tenant, tests, and platform have not been run or changed.

Appendix H. TRP-1.5.1 Review Finding Closure and Readiness Boundaries

Item	Change	Normative text location	Check
A	RV-01: exact original v0.2 adopted, SHA-256 e9425fc7…	§§3.9–3.21, Appendices A–D; detailed comparison in I	primary-source comparison closed; targeted deviation log maintained; no claim of byte-for-byte identity for entire integrated edition

Item	Change	Normative text location	Check
B	Headings and test arithmetic recalculated from actual rows	§3.20, Appendix E	281 original + 4 new=285, 20 classes
C	Invalid reconcile.raw terminates with RECONCILE_BAD_RANGE without falling back to lookback	§3.13.3, CallProcessorTests, Appendix E	ReconcileBadRange_FailsNoLookback — one Theory method, three data sets
D	Duplicate raw reprocess while first item is new	§3.23.4, §3.29	DuplicateWhileNew_Collapsed; Host 93 methods
E	Diagrams include scenario names and no duplicated headers	§§3.0, 3.12–3.17, 3.26, 3.31	24 Mermaid in MD / 24 images in DOCX, selective PDF visual inspection
F	Teams SDK methods not compile-verified	§3.24, RV-02	explicitly marked pseudocode; compilation probe required before coding
G	Authentic Appendices A–D restored	after §5	fixtures D01–D04 and v0.2 change log included from original

H.1. TRP-1.5.1 Editorial Changes Following Independent Review

№	Finding	Correction adopted	Where to verify
1	PDF delivered with different SHA than message claimed	actual SHA-256 of delivered TRP-1.5 recorded: 9f87c1e2d690506c203af2e4522fe7b4c2a89510bf11e70ef80f5217c520253d; new SHAs calculated for three TRP-1.5.1 files	1.5.1 release checksum protocol
2	§3.21.1/3.21.2 called verbatim original but tables reworked	Appendix I marks "recalculated for 285 methods"; transformation of columns disclosed	Appendix I, §3.21.1–3.21.2 rows
3	Table I.1 omitted removal of obsolete Requeued	registered correction to original v0.2 internal contradiction in RecheckSuspendedAsync	§3.13.6, Appendix I.1
4	DR-16 was incorrectly referred for new owner decision	reclassified as reaffirmation of original due/force rule without separate decision	§3.37, DR-16
5	"41 JSON blocks" did not match number of marked blocks	control refined: 10 fenced Markdown blocks labeled json (validated separately); other JSON examples inside unlabeled blocks excluded from this count	TRP-1.5.1 machine validation report
6	Figure captions duplicated or detached; top of Fig. 4 clipped	one normative caption per figure, keep caption with figure, expanded top margin on Fig. 4	PDF/DOCX, figure atlas 1–24
Additionally	Release-file verification found that two SQL lines in a previously delivered MD had lost tenant_id filter	restored tenant_id={tenantId} in CDR query and statistics, preventing cross-tenant reads	§3.11.7; matches already stated Host contract

Status: TRP-1.5.1 — submitted for owner approval; NOT APPROVED. This detailed design does not prove working C# code. RV-02–RV-05, [P] decisions, and live-tenant checks remain open; the project has not been compiled or integration-tested.

Appendix I. RV-01 — Comparison of Authentic Infrastructure v0.2 and TRP-1.4

Authenticity control: file TCRI_TRP_Part2_v0.2.md, size 243,510 bytes, SHA-256 e9425fc7aae50113be4ecdf46eaf8c8393b5a734f328e9e925739d14d33cb8d3. Paragraphs and tables of §§3.9–3.21 were compared against the previous revision v1.4. In v1.5 the normative basis of these sections is the authentic v0.2 original; deviations from it are itemized below, and all other provisions are taken from the primary source without reconstruction.

Section	Original v0.2	TRP-1.4 before correction	TRP-1.5 decision
§3.9	124 lines; During Infrastructure design, a conflict was found in the Part 1 (3.2) tree: options classes InboxOption | text / tables differ; 104 lines | original v0.2 restored | | §3.10 | 32 lines; **3.10.1. Project and packages.**TeamsCallRecords.Infrastructure.csproj:Sdk="Microsoft.NET.Sdk",net8.0, references | close to original; 32 lines | original retained | | §3.11 | 1 lines; no data | close to original; 1 lines | original retained | | §3.11.1 | 13 lines; Static string-constant classes; values match CHECK constraints in §3.5. String literals | close to original; 13 lines | original retained | | §3.11.2 | 18 lines;public sealed class TcriDbContext : DbContext, constructorTcriDbContext(DbContextOptions op	text / tables differ; 19 lines	authentic v0.2 restored
§3.11.3	6 lines; public sealed class SqlitePragmaInterceptor : DbConnectionInterceptor. Overrides: ConnectionOpened(Db | close to original; 6 lines | original retained | | §3.11.4 | 18 lines; public interface IWriteGate | close to original; 18 lines | original retained | | §3.11.5 | 2 lines;public sealed class TcriDbContextFactory : IDesignTimeDbContextFactory;CreateDbContext(stri	close to original; 2 lines	original retained
§3.11.6	12 lines; Data/Migrations/InitialCreate.sql contains the Part 1 §3.5 DDL with amendment A1 from §3.9 (inbox table)	text / tables differ; 12 lines	authentic v0.2 restored
§3.11.7	21 lines; public sealed record CdrItem(long CdrSeq, Cdr Cdr);	editorially / algorithmically modified; 21 lines	original + traceable P-26/P-27 integration
§3.11.8	6 lines; public static class Cursor — opaque page pointer for CdrReader.QueryAsync and REST API (Part 3)	text / tables differ; 21 lines	authentic v0.2 restored
§3.12	1 line; no data	close to original; 1 line	original retained
§3.12.1	8 lines; public sealed record InboxNewItem(string Kind, string TenantId, string? CallId, string Raw);	text / tables differ; 8 lines	authentic v0.2 restored
§3.12.2	7 lines; internal static class InboxTransitions — item transitions inside an already open write transaction; used by processing components	close to original; 7 lines	original retained
§3.12.3	52 lines; InboxStore(IWriteGate gate, IDbContextFactory<TcriDbContext> factory, IOptions<InboxOptions> inbox, IOptions< | editorially / algorithmically changed; 11 lines | original + traceable integration P-26/P-27 | | §3.13 | 1 lines; no data | close to original; 1 lines | original retained | | §3.13.1 | 23 lines; public enum ProcessOutcome { Applied, IgnoredLower, Duplicate, Discrepancy, Rejected, Pending, Suspended, Expi | close to original; 23 lines | original retained | | §3.13.2 | 7 lines;ProcessingOptions:int Workers = 2,int LeaseSeconds = 120,string RetrySchedule = "1,5,15,60",int P	close to original; 7 lines	original retained

Section	Original v0.2	TRP-1.4 before correction	TRP-1.5 decision
§3.13.3	66 lines; public interface ICallProcessor { Task ProcessAsync(InboxItem item, CancellationToken ct); }	editorially / algorithmically modified; 39 lines	original + traceable P-26/P-27 integration
§3.13.4	24 lines; SourceStateStore(IWriteGate gate, IDbContextFactory<TcriDbContext> factory, IOptions<ProcessingOptions> proce | text / tables differ; 15 lines | original v0.2 restored | | §3.13.5 | 45 lines; public interface ICdrApplier { Task<ApplyResult> ApplyAsync(ApplyRequest request, CancellationToken ct); } | text / tables differ; 40 lines | original v0.2 restored | | §3.13.6 | 51 lines; public interface IReconciler | editorially / algorithmically changed; 16 lines | due/force rule restored from original | | §3.14 | 1 lines; no data | close to original; 1 lines | original retained | | §3.14.1 | 31 lines; public enum GraphFetchStatus { Ok, NotFound, PagesIncomplete, Malformed } | text / tables differ; 26 lines | original v0.2 restored | | §3.14.2 | 17 lines; / Property / Key / Default / | close to original; 17 lines | original retained | | §3.14.3 | 45 lines; public interface IGraphClient | text / tables differ; 44 lines | original v0.2 restored | | §3.14.4 | 9 lines; Registration (AddTcriGraphClient, 3.18) attaches toHttpClienthandlerAddResilienceHandlerwith policy | text / tables differ; 29 lines | original v0.2 restored | | §3.14.5 | 29 lines; Call-record paginated retrieval sequence — Figure 8. | text / tables differ; 3 lines | original v0.2 restored | | §3.15 | 1 lines; no data | close to original; 1 lines | original retained | | §3.15.1 | 8 lines; / Property / Key / Default / Purpose / | close to original; 8 lines | original retained | | §3.15.2 | 11 lines;SubscriptionStore(IWriteGate gate, IDbContextFactory factory); singleton. | close to original; 11 lines | original retained | | §3.15.3 | 57 lines;SubscriptionService(IGraphClient graph, ISubscriptionStore store, IInboxStore inbox, IErrorLog errors, IOptio	text / tables differ; 31 lines	authentic v0.2 restored
§3.16	1 line; no data	close to original; 1 line	original retained
§3.16.1	14 lines; public sealed record OutboxItem(long Id, string TenantId, string CallId, string SessionId, string Recipient, string Payload);	close to original; 14 lines	original retained
§3.16.2	8 lines; OutboxStore(IWriteGate gate, IDbContextFactory<TcriDbContext> factory, IOptions<OutboxOptions> options, TimeP | text / tables differ; 8 lines | original v0.2 restored | | §3.16.3 | 41 lines; public interface IOutboxSender { Task<bool> RunOnceAsync(CancellationToken ct); } | text / tables differ; 33 lines | original v0.2 restored | | §3.17 | 4 lines;IErrorLog:Task WriteAsync(string component, string message, string? details, CancellationToken ct).Erro	close to original; 4 lines	original retained
§3.18	1 line; no data	close to original; 1 line	original retained
§3.18.1	18 lines; public static class InfrastructureRegistration:	close to original; 18 lines	original retained
§3.18.2	8 lines; / Class / Validate() rules /	close to original; 8 lines	original retained
§3.18.3	2 lines; Options (validation) → MigrateAsync → SubscriptionService.EnsureAsync → queue workers → sender	close to original; 2 lines	original retained
§3.19	4 lines; scenarios specify expected database table states after processing. They are executed by ScenarioTests.	text / tables differ; 4 lines	authentic v0.2 restored

Section	Original v0.2	TRP-1.4 before correction	TRP-1.5 decision
§3.20	15 lines; Project tests/TeamsCallRecords.Infrastructure.Tests, net8.0; packages — 3.10.1. Run: dotnet test Team | editorially / algorithmically changed; 15 lines | original + traceable integration P-26/P-27 | | §3.20.1 | 10 lines; / Method / Input / Expected / K/F / | table/test counts differed; 9 lines | original named test table restored; +4 methods in affected classes | | §3.20.2 | 8 lines; / Method / Input / Expected / K/F / | table/test counts differed; 8 lines | original named test table restored; +4 methods in affected classes | | §3.20.3 | 7 lines; / Method / Input / Expected / K/F / | table/test counts differed; 7 lines | original named test table restored; +4 methods in affected classes | | §3.20.4 | 9 lines; / Method / Input / Expected / K/F / | close to original; 9 lines | original retained | | §3.20.5 | 27 lines; / Method / Input / Expected / K/F / | table/test counts differed; 26 lines | original named test table restored; +4 methods in affected classes | | §3.20.6 | 24 lines; / Method / Input / Expected / K/F / | table/test counts differed; 22 lines | original named test table restored; +4 methods in affected classes | | §3.20.7 | 22 lines; / Method / Input / Expected / K/F / | close to original; 22 lines | original retained | | §3.20.8 | 35 lines; / Method / Input / Expected / K/F / | table/test counts differed; 30 lines | original named test table restored; +4 methods in affected classes | | §3.20.9 | 17 lines; / Method / Input / Expected / K/F / | table/test counts differed; 15 lines | original named test table restored; +4 methods in affected classes | | §3.20.10 | 13 lines; / Method / Input / Expected / K/F / | table/test counts differed; 13 lines | original named test table restored; +4 methods in affected classes | | §3.20.11 | 12 lines; / Method / Input / Expected / K/F / | table/test counts differed; 11 lines | original named test table restored; +4 methods in affected classes | | §3.20.12 | 33 lines; / Method / Input / Expected / K/F / | table/test counts differed; 26 lines | original named test table restored; +4 methods in affected classes | | §3.20.13 | 14 lines; / Method / Input / Expected / K/F / | table/test counts differed; 10 lines | original named test table restored; +4 methods in affected classes | | §3.20.14 | 10 lines; / Method / Input / Expected / K/F / | close to original; 10 lines | original retained | | §3.20.15 | 19 lines; / Method / Input / Expected / K/F / | table/test counts differed; 16 lines | original named test table restored; +4 methods in affected classes | | §3.20.16 | 14 lines; / Method / Input / Expected / K/F / | table/test counts differed; 14 lines | original named test table restored; +4 methods in affected classes | | §3.20.17 | 7 lines; / Method / Input / Expected / K/F / | table/test counts differed; 7 lines | original named test table restored; +4 methods in affected classes | | §3.20.18 | 10 lines; / Method / Input / Expected / K/F / | close to original; 10 lines | original retained | | §3.20.19 | 18 lines; / Method / Input / Expected / K/F / | table/test counts differed; 16 lines | original named test table restored; +4 methods in affected classes | | §3.20.20 | 12 lines; / Method / Input / Expected / K/F / | table/test counts differed; 13 lines | original named test table restored; +4 methods in affected classes | | §3.21 | 2 lines; Table is built from column «K/F» sections 3.20. «Tests» — number of methods, where criterion is specified. Column «Remaining | close to original; 2 lines | original retained | | §3.21.1 | 17 lines; / Criterion / Content / Tests / Classes / Remaining outside Infrastructure / | text / tables differ; 17 lines | **recalculated for 285 methods**; columns «Content» and «Remaining outside Infrastructure» converted into summary of external checks; not a verbatim original | | §3.21.2 | 12 lines; / Function / Name / Tests / Classes / Remaining outside Infrastructure / | text / tables differ; 12 lines | **recalculated for 285 methods**; columns «Name» and «Remaining outside Infrastructure» omitted in summary format; not a verbatim original | | §S01 | 16 lines; **Initial state.** Database empty. In inboxenqueued item 1 (callRecord=call-id=cccccccc-0000-4000-8000-00	close to original; 16 lines	original retained

Section	Original v0.2	TRP-1.4 before correction	TRP-1.5 decision
§S02	3 lines; Action. At 18:05:00 item enqueued in inbox 2 (reconcile, call_id = cccccccc-0000-4000-80 | close to original; 3 lines | original retained | | §S03 | 6 lines; **Action.** At 18:10:00 item 3 (reconcile, version 4); Graph returnsD01(version 4, in first | close to original; 6 lines | original retained | | §S04 | 6 lines; **Action.** At 18:15:00 item 4 (reconcile, version 5); Graph returnsG11. | close to original; 6 lines | original retained | | §S05 | 5 lines; **Action.** At 18:20:00 item 5 (reconcile, version 4); Graph returnsG01. | text / tables differ; 3 lines | original v0.2 restored | | §S06 | 11 lines; **Action.** (1) At T0 processedcallRecordfor callcccccccc-0000-4000-8000-000000000002; Graph | close to original; 11 lines | original retained | | §S07 | 13 lines; **Initial state.** Item 1 (callRecord,call_id=cccccccc-0000-4000-8000-000000000007); Graph for each on | close to original; 13 lines | original retained | | §S08 | 9 lines; **Action.** Item 1 (callRecord,call_id=cccccccc-0000-4000-8000-000000000009); Graph returns	text / tables differ; 7 lines	authentic v0.2 restored
§S09	15 lines; Initial state. Item 1 (callRecord, call_id=cccccccc-0000-4000-8000-000000000001); FakeGraphClient	text / tables differ; 15 lines	authentic v0.2 restored
§S10	2 lines; Action. Item 1 (callRecord) claimed at T0 (attempts=1, locked_until_utc=18:02:00); processing not completed	close to original; 2 lines	original retained
§S11	4 lines; Action. (1) At T0 Graph returns G08 (version 2, session has no end, SESSION_DATES_MISSING). (2) At 18:0...	text / tables differ; 4 lines	authentic v0.2 restored
§S12	13 lines; Initial state. Outbox has one line 1 (state='new', attempts=0, next_attempt_utc=18:00:00); FakeGraphC | text / tables differ; 12 lines | original v0.2 restored | | §S13 | 4 lines; **Initial state.** State after S07:source_state(cccccccc-0000-4000-8000-000000000007) —state=’suspended	text / tables differ; 8 lines	authentic v0.2 restored
§S14	4 lines; Initial state. After S01, item 1 done, applied_version=4. Graph notification for same call arrives again	editorially / algorithmically modified; 11 lines	original + traceable P-26/P-27 integration
§S15	3 lines; Initial state. After S01 (done, applied_version=4). FakeGraphClient.ListCallRecordsAsync returns...	editorially / algorithmically modified; 9 lines	original + traceable P-26/P-27 integration

I.1. Material Deviations Retained

Change	Reason	Status
GetStatsAsync(tenantId,from,to,ct) and tenant filter	fix SQLite error and cross-tenant count	P-26; pending approval
RenewLeaseAsync with CAS and heartbeat	prevent double processing of long-running reprocess	P-27; pending approval
reconcile.raw with interval and RECONCILE_BAD_RANGE	manual range must not silently fall back to lookback	P-27; pending approval
RecheckSuspendedAsync: returns Added without Requeued (§3.13.6)	fixes internal contradiction in original v0.2: InboxEnqueueResult no longer contains Requeued but algorithm referenced it	corrected an error in the original source draft, not a new design decision

Change	Reason	Status
7 days for background job; HTTP 202 after enqueue	safe asynchronous API processing	P-27; pending approval
tenant-aware ix_inbox_call	align index and multi-tenant queue	P-26; pending approval
S14/S15 row by row	scenarios verifiable after late decisions	P-22/P-27; pending approval
4 new Infrastructure tests and 1 Host test	complete test coverage of changes	proposed tests, not executed
Ordinary NeedsFetch(pending/suspended)	**unchanged:** original "false" restored, retry on due; force=true unrestricted	primary source, no new decision

RV-01: the provenance comparison is complete; exact correspondence to v0.2 is not claimed due to registered later design corrections.